Large Language Models for Software Engineering

Theoretical foundations, methodologies, and practical architectures

Comprehensive master's course study guide

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)
Database and Data Mining Group (DBDMG)



Course instructors

Prof. Flavio Giobergia
Prof. Riccardo Coppola

Prepared by

Nicola Beccaceci

Academic Year 2025/2026

Contents

Preface iii

Notation and conventions v

I Foundations of large language models 1

1 Deep Learning Foundations 2
1.1 Historical Context and Theoretical Origins . . . 3
1.2 The Perceptron and Linear Separability . . . 6
1.3 Activation Functions and Non-Linearity . . . 10
1.4 Loss Functions and Optimization . . . 17
1.5 Computational Graphs and Backpropagation . . . 23

2 Recurrent Neural Networks 30
2.1 Sequential Data and Feed-Forward Limits . . . 31
2.2 RNN Architecture and Cell Dynamics . . . 34
2.3 Backpropagation Through Time . . . 38
2.4 Gradient Pathologies and Gated Units . . . 42
2.5 Sequence-to-Sequence and Encoder-Decoder Models . . . 47

3 Language Modeling Fundamentals 52
3.1 Probabilistic Language Modeling . . . 53
3.2 The Markov assumption, n-gram models, and operational inference . . . 56
3.3 Statistical Limits and the Neural Paradigm . . . 62

4 Word Embeddings and Word2Vec 67
4.1 The Word Representation Problem . . . 68
4.2 One-Hot Encoding and Its Limitations . . . 69
4.3 Word2Vec: CBOW and Skip-Gram . . . 74
4.4 FastText: subword character n-gram embeddings and limitations . . . 88
4.5 Training Dynamics and Semantic Geometry . . . 91

5 The Transformer Architecture 99
5.1 Tokenization and Subword Algorithms . . . 101
5.2 Positional Encodings . . . 109
5.3 The Attention Mechanism . . . 115
5.4 Attention Variants: Self, Masked, and Cross . . . 122
5.5 Transformer Blocks and Information Flow . . . 126

6 Transformer Variants and Decoding **134**
6.1 Encoder-Decoder Models . . . 134
6.2 Encoder-Only Models . . . 137
6.3 Decoder-Only Models . . . 142
6.4 Decoding and Sampling Strategies . . . 143

7 Pre-Training, Scaling Laws, and Model Evolution **150**
7.1 GPT-1: Generative Pre-Training . . . 150
7.2 GPT-2 and Empirical Scaling Laws . . . 153
7.3 GPT-3 and In-Context Learning . . . 157
7.4 Chinchilla and Compute-Optimal Scaling . . . 160
7.5 Open Foundation Models and Token Saturation . . . 164

8 Model Evaluation and Benchmarks **168**
8.1 Evaluation Metrics and Principles . . . 169
8.2 NLP Tasks and Evaluation Taxonomy . . . 188
8.3 Standardized Benchmark Suites . . . 197

9 Instruction Tuning and Alignment **203**
9.1 Inference-Time Alignment and Its Limitations . . . 204
9.2 Supervised Fine-Tuning and Instruction Alignment . . . 206
9.3 RLHF and Human Preference Modeling . . . 211
9.4 InstructGPT and Modern Alignment . . . 221

10 Efficient Fine-Tuning and Inference Optimization **226**
10.1 The Adaptation Bottleneck and PEFT Foundations . . . 226
10.2 Adapters, Low-Rank Adaptation, and Soft Prompts . . . 230
10.3 Quantization and Numerical Precision . . . 237
10.4 Knowledge Distillation and Serving Optimization . . . 242

Preface

Pedagogical vision

Large language models (LLMs) are increasingly used in software engineering tasks that involve code synthesis and repair, requirements analysis, and coordination among software agents. These applications rely on probabilistic models trained to represent and generate sequences of text and code. Understanding their use therefore requires both the mathematical foundations of language modeling and an account of how model behavior interacts with software development processes.

This guide accompanies the Master of Science course *Large Language Models for Software Engineering* at Politecnico di Torino, taught by Prof. Flavio Giobergia and Prof. Riccardo Coppola. The course is organized around two related areas:

1. **Part I: Foundations of large language models** — Develops the mathematical and architectural foundations of language modeling, from n-gram Markov chains and feedforward networks to distributed word embeddings, recurrent networks, and Transformers. The broader course syllabus also includes pre-training scaling laws, instruction tuning (RLHF and DPO), and parameter-efficient fine-tuning (LoRA and QLoRA).

2. **Part II: Large language models for software engineering** — Examines how model capabilities relate to the software development life cycle (SDLC), including prompt engineering, agent frameworks such as ReAct, automated test generation, software maintenance and refactoring, evaluation measures such as Pass@k and CodeBLEU, and security, intellectual property, and ethical considerations.

Visual callout structure

Conceptual interpretations, including attention as a differentiable lookup operation and the geometry of high-dimensional vector spaces, accompany the relevant mathematical development. Explanations also address common mathematical errors, such as using inconsistent logarithm bases when computing perplexity. Research context is cited in the text, including foundational work on Transformers and language-model scaling [1–3].

Examination framework and course organization

Assessment in the course consists of a written examination and a practical project. The stated examination format and course schedule are:

1. **Written Examination (18 Points)**:

    - The examination lasts 75 minutes and typically contains about 12 questions, combining multiple-choice items with open-ended technical or mathematical derivations.

- Its scope covers foundational and applied software-engineering topics; a score of at least 10/18 is required for a passing result.

2. **Practical Course Project (14 Points)**:
   - The project involves implementing an applied LLM-for-software-engineering system in Python and PyTorch, individually or in a small team.
   - The examination and project contribute 18 and 14 points, respectively, for a total of 32; a score above 30 earns *30 e Lode* (honors).
3. **Lecture and Lab Schedule**:
   - Lectures take place on Monday from 08:30 to 11:30 in Room 8 and cover theoretical foundations.
   - Laboratories take place on Wednesday from 11:30 to 13:00 in Room R1 and focus on practical workshops, coding, and PyTorch.

How to use this study guide

Each chapter develops its subject through explanatory prose, mathematical derivations, algorithmic procedures, and software-engineering applications. Readers can use the notation chapter as a reference and follow citations to the research that informs the discussion. Checking derivations and implementing core algorithms provide complementary ways to examine the behavior of the models.

Notation and conventions

Throughout this study guide, we adopt consistent mathematical, algorithmic, and typographic conventions aligned with standard deep learning and software engineering literature.

Linear algebra and tensor operations

Symbol	Type	Description
x, y, z	Scalar	Real-valued scalars or scalar components ($x \in \mathbb{R}$).
$\mathbf{x}, \mathbf{y}, \mathbf{h}$	Vector	Column vectors ($\mathbf{x} \in \mathbb{R}^d$). Lowercase bold font.
$\mathbf{x}^\top$	Transpose	Transpose of vector $\mathbf{x}$ (row vector $\mathbf{x}^\top \in \mathbb{R}^{1\times d}$).
$\mathbf{A}, \mathbf{W}, \mathbf{X}$	Matrix	Real matrices ($\mathbf{A} \in \mathbb{R}^{m\times n}$). Uppercase bold font.
$\mathbf{A}_{i,j}$ or a_{ij}	Scalar	Entry at row i and column j of matrix $\mathbf{A}$.
$\mathbf{I}$ or $\mathbf{I}_d$	Matrix	Identity matrix of dimension $d \times d$.
$\|\mathbf{x}\|_2$ or $\|\mathbf{x}\|$	Scalar	Euclidean (L_2) norm of vector $\mathbf{x}$, $\sqrt{\sum_i x_i^2}$.
$\|\mathbf{A}\|_F$	Scalar	Frobenius norm of matrix $\mathbf{A}$, $\sqrt{\sum_{i,j} A_{i,j}^2}$.
$\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u}^\top \mathbf{v}$	Scalar	Standard dot (inner) product between vectors $\mathbf{u}$ and $\mathbf{v}$.
$\odot$	Operator	Element-wise (Hadamard) product between tensors of identical shape.
$\otimes$	Operator	Outer product or Kronecker tensor product.

Table 1: Linear algebra and tensor notation summary.

Probability and information theory

Neural networks, transformers, and software engineering

Symbol	Type	Description
$\mathbb{P}(A)$	Scalar	Probability of event A ($\mathbb{P} : \mathcal{F} \to [0, 1]$).
$\mathbb{P}(X = x)$	Scalar	Probability mass function for discrete random variable X.
$\mathbb{P}(Y \mid X)$	Distribution	Conditional probability distribution of Y given X.
$\mathbb{E}_{x\sim P}[f(x)]$	Scalar	Expected value of function $f(x)$ under distribution P.
$\mathbb{V}\text{ar}(X)$	Scalar	Variance of random variable X, $\mathbb{E}[(X - \mathbb{E}[X])^2]$.
$\mathcal{H}(P)$	Scalar	Shannon entropy of distribution P, $-\sum_x P(x) \log P(x)$.
$\mathcal{H}(P, Q)$	Scalar	Cross-entropy between target distribution P and model Q.
$D_{\mathrm{KL}}(P \parallel Q)$	Scalar	Kullback–Leibler (KL) divergence from Q to P.
$\mathrm{PPL}(W)$	Scalar	Perplexity of word sequence W under a language model.

Table 2: Probability and information theoretic notation.

Symbol	Type	Description
$\mathcal{V}$	Set	Discrete vocabulary of tokens; $\lvert\mathcal{V}\rvert = V$ denotes vocabulary size.
T or T	Integer	Sequence length (number of input or output tokens).
d_{model}	Integer	Hidden embedding dimensionality of the Transformer model.
d_{ff}	Integer	Dimensionality of the intermediate Feed-Forward Network layer.
d_k, d_v	Integer	Dimensionality of query/key and value projections per head.
h	Integer	Number of parallel attention heads ($h \cdot d_k = d_{\text{model}}$).
$\sigma(z)$	Function	Standard logistic sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$.
$\mathrm{softmax}(\mathbf{z})$	Function	Softmax normalized exponential vector operator.
$\mathrm{ReLU}(z)$	Function	Rectified Linear Unit activation: $\max(0, z)$.
$\mathrm{GELU}(z)$	Function	Gaussian Error Linear Unit activation.
$\mathcal{L}$ or $\mathcal{L}$	Scalar	Scalar objective / loss function minimized during optimization.
Pass@k	Metric	Probability that at least one of k generated code samples passes unit tests.
CodeBLEU	Metric	Multi-aspect code evaluation metric combining BLEU, AST, and data flow.

Table 3: Model architecture parameters and SE metrics.

Part I

Foundations of large language models

CHAPTER 1

Deep Learning Foundations

The fundamental challenge of artificial intelligence resides in discovering computational mechanisms capable of mapping high-dimensional, unstructured sensory and symbolic observations into structured semantic representations. Classical machine learning systems approached this objective by decoupling feature engineering from statistical inference: domain experts manually designed deterministic transformation pipelines to extract handcrafted numerical descriptors, which were subsequently supplied to shallow linear classifiers or regression models. While computationally tractable in controlled environments, this paradigm proved fragile when confronted with the continuous variability, high dimensionality, and compositional complexity of natural language and perceptual signals.

Deep learning fundamentally alters this architectural philosophy by adopting a connectionist, data-driven formulation. Rather than relying on human engineering to prescribe fixed representational primitives, deep neural networks learn hierarchical feature representations directly from raw data through the end-to-end composition of parameterized, differentiable mathematical functions. Each successive layer computes an abstract geometric transformation of its input, constructing intermediate latent representations that disentangle the underlying factors of variation.

This chapter establishes the theoretical, geometric, and algorithmic foundations of deep feed-forward architectures. We trace the lineage of neural computation from early biological abstractions and the single-layer perceptron, formalize the geometric barriers imposed by linear decision boundaries, derive the space-folding mechanics of non-linear activation functions, establish the statistical grounding of task-specific loss functions under maximum likelihood estimation, and formulate reverse-mode automatic differentiation on computational graphs as the universal optimization engine for deep learning.

1.1 Historical Context and Theoretical Origins

Understanding the structural necessity of deep neural networks requires examining why classical algorithmic paradigms and shallow statistical models proved insufficient for complex perceptual and linguistic tasks. The progression from symbolic reasoning to modern representation learning illustrates how computational and geometric constraints shaped modern neural architectures.

1.1.1 From symbolic artificial intelligence to connectionism

For several decades following the Dartmouth workshop of 1956, artificial intelligence research was dominated by symbolic systems, colloquially termed Good Old-Fashioned Artificial Intelligence (GOFAI) [4]. This paradigm operated on the physical symbol system hypothesis, positing that intelligent behavior could be instantiated through the formal manipulation of discrete, explicit symbols according to hand-engineered logical rules. Knowledge was encoded using production systems, semantic networks, and first-order predicate calculus, where inference engines evaluated deterministic truth values over closed ontologies.

While symbolic systems demonstrated notable competence in closed, deterministic micro-worlds such as chess or theorem proving, they encountered severe operational bottlenecks when deployed in real-world environments characterized by noise, perceptual ambiguity, and continuous variations. Hand-crafted rule bases were inherently brittle: any scenario outside the codified axioms triggered catastrophic failure, and resolving contradictory evidence in large rule graphs became computationally intractable. More critically, symbolic systems lacked an intrinsic metric of semantic similarity; the discrete symbols `cat` and `feline` shared no mathematical relationship in coordinate space, precluding continuous generalization.

In response to these limitations, the connectionist paradigm emerged, proposing that cognitive processing is realized through distributed computation over vast networks of simple, interconnected processing units [5–7]. Rather than storing knowledge in localized, explicit symbolic rules, connectionist networks encode knowledge in the continuous, distributed pattern of synaptic connection strengths (weights) across the entire system. Representation and computation are inherently continuous and differentiable, enabling models to interpolate smoothly across input spaces, tolerate corrupt or incomplete inputs through graceful degradation, and acquire representations directly from empirical data through statistical learning.

1.1.2 The curse of dimensionality and high-dimensional geometry

Early statistical approaches to non-linear pattern recognition attempted to circumvent the limitations of linear models by projecting input vectors $\mathbf{x} \in \mathbb{R}^D$ into higher-dimensional feature spaces via fixed basis functions $\phi(\mathbf{x}) : \mathbb{R}^D \to \mathbb{R}^M$. A standard linear model was then evaluated on the transformed representations:

$$y(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) + w_0, \tag{1.1}$$

where $\mathbf{w} \in \mathbb{R}^M$ denotes the parameter vector and w_0 represents the scalar bias [8]. Although this formulation preserves linear parameter optimization, it suffers directly from the curse of dimensionality when scaling to high-dimensional input spaces.

Consider constructing a complete polynomial basis of order P over an input dimension D. The total number of independent polynomial basis functions required scales asymptotically as:

$$M = \binom{D+P}{P} = \mathcal{O}\left(D^P\right). \tag{1.2}$$

For an input space of moderate dimensionality, such as $D = 100$, a third-order polynomial expansion ($P = 3$) demands $M \approx 1.7 \times 10^5$ parameters, while a fifth-order polynomial ($P = 5$) requires $M \approx 7.5 \times 10^7$ parameters. When processing natural images or tokenized textual contexts where $D \geq 10^3$, the parameter count exceeds any feasible computational or sample budget.

The curse of dimensionality manifests with equal severity in non-parametric and partitioning methods. Suppose the input domain $[0, 1]^D$ is partitioned into regular grid cells by dividing each dimension into K intervals. The total number of hypercubical cells is:

$$N_{\text{cells}} = K^D. \tag{1.3}$$

If $D = 50$ and each axis is divided into only $K = 10$ bins, the volume is partitioned into 10^{50} distinct cells. Because any realistic dataset contains $N \ll 10^{50}$ observations, the overwhelming majority of cells will remain completely unpopulated, causing localized neighborhood estimators to fail through severe sample sparsity.

Furthermore, the geometry of high-dimensional Euclidean space exhibits counter-intuitive properties that undermine distance-based algorithms [8]. Consider a hypersphere of radius $r = 1$ in D dimensions. The volume of the hypersphere is proportional to r^D. The fraction of this volume lying within a thin spherical shell of thickness $\epsilon \in (0, 1)$ near the outer surface is given by:

$$\frac{V_D(1) - V_D(1 - \epsilon)}{V_D(1)} = 1 - (1 - \epsilon)^D. \tag{1.4}$$

As the dimensionality $D \to \infty$, the term $(1 - \epsilon)^D \to 0$, which causes the volume fraction to converge asymptotically to 1 for any non-zero ϵ:

$$\lim_{D \to \infty} \left[1 - (1 - \epsilon)^D\right] = 1. \tag{1.5}$$

Consequently, in high-dimensional spaces, nearly all the volume of a hypersphere is concentrated in an infinitesimally thin boundary shell. A complementary phenomenon occurs with multivariate Gaussian distributions $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$: transforming to spherical polar coordinates and integrating over directional variables reveals that the radial probability density $p(r) \propto r^{D-1}e^{-r^2/(2\sigma^2)}$ concentrates entirely within a narrow spherical shell of radius $r \approx \sigma\sqrt{D}$. Data points in high dimensions do not cluster around the mean; they reside along an equatorial hypersphere, causing pairwise Euclidean distances between random observations to concentrate tightly around a single constant value.

1.1.3 The manifold hypothesis and the expressive power of depth

The realization that high-dimensional data points become mutually equidistant under isotropic assumptions led to the formulation of the manifold hypothesis [8, 9]. The hypothesis states that natural high-dimensional datasets (such as natural language sentences, audio waveforms, or natural images) do not populate the ambient Euclidean space $\mathbb{R}^D$ uniformly; rather, they concentrate in the immediate vicinity of a low-dimensional, non-linear embedded sub-manifold $\mathcal{M} \subset \mathbb{R}^D$ having intrinsic dimension $d \ll D$.

Fixed basis function models cannot exploit this intrinsic low-dimensional structure because their basis functions $\phi_j(\mathbf{x})$ are chosen prior to observing the data distribution. To model arbitrary functions over the ambient space, fixed basis methods must tile the entire D-dimensional volume uniformly, falling victim to the exponential scaling of K^D. Conversely, artificial neural networks resolve this impasse by replacing fixed basis functions with parameterized, data-dependent basis functions that are optimized jointly to conform to the geometry of the data manifold.

The capacity of neural networks to parameterize complex manifolds depends critically on network depth. Consider a feed-forward network utilizing Rectified Linear Unit (ReLU)

activation functions. Because the ReLU function is piecewise linear, any network constructed from ReLU activations computes a continuous, piecewise linear function that partitions the input domain $\mathbb{R}^{d_{\text{in}}}$ into a collection of convex polyhedral regions, within each of which the model evaluates a distinct affine mapping [10, 11].

For a shallow network comprising a single hidden layer with K hidden units and a scalar input ($d_{\text{in}} = 1$), the model can construct at most $K + 1$ linear regions, parameterized by $3K + 1$ weights and biases:

$$y = \phi_0 + \sum_{j=1}^{K} \phi_j \max\left(0, \theta_{j0} + \theta_{j1} x\right). \tag{1.6}$$

To increase the number of linear regions in a shallow architecture, one must expand the hidden width linearly: achieving $O(K)$ linear regions requires $O(K)$ parameters.

In sharp contrast, stacking multiple non-linear layers allows deeper layers to compute piecewise linear compositions of the piecewise linear functions evaluated by preceding layers. Each successive layer folds the input space, replicating the linear partitions constructed by earlier stages. Formalized by Montúfar et al. [11], a deep neural network with L hidden layers, each possessing K hidden units, can partition the input space $\mathbb{R}^{d_{\text{in}}}$ into a number of distinct linear regions that scales exponentially with depth:

$$N_{\text{regions}} \geq \left(\left\lfloor \frac{K}{d_{\text{in}}} \right\rfloor \right)^{(L-1)d_{\text{in}}} K^{d_{\text{in}}} = O\left(K^{L \cdot d_{\text{in}}}\right), \tag{1.7}$$

while the total parameter count grows only linearly with depth and quadratically with width:

$$N_{\text{parameters}} = O\left(L \cdot K^2\right). \tag{1.8}$$

This exponential disparity in expressive efficiency provides the fundamental mathematical justification for deep architectures: deep composition generates exponentially complex, multi-scale decision boundaries using a polynomial parameter budget, enabling neural networks to model intricate manifold topologies that remain inaccessible to shallow models.

1.2 The Perceptron and Linear Separability

The conceptual building block of modern neural architectures is the artificial perceptron. To understand why modern deep architectures are structured as hierarchical non-linear networks, one must first analyze the computational properties and geometric limitations of this foundational linear operator.

1.2.1 The artificial perceptron

The mathematical formulation of the artificial neuron originated in the simplified physiological model of biological neurons proposed by Warren McCulloch and Walter Pitts in 1943 [5], which Frank Rosenblatt subsequently formalized and operationalized as the perceptron learning model in 1958 [6]. The biological abstraction captures three primary mechanisms: dendrites receive incoming electrochemical impulses, the soma integrates these inputs, and the axon fires an action potential if the accumulated potential surpasses an excitation threshold.

Formally, consider an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$. Each input component x_i is scaled by an associated scalar weight $w_i \in \mathbb{R}$, which quantifies the synaptic efficacy of that specific connection. The neuron aggregates these weighted contributions and adds an intrinsic scalar parameter termed the bias, $b \in \mathbb{R}$, yielding a scalar pre-activation value $z \in \mathbb{R}$:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b, \tag{1.9}$$

where $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$ represents the weight vector. The pre-activation value is subsequently processed by an activation function $f : \mathbb{R} \to \mathcal{Y}$ to produce the final scalar output y:

$$y = f(z) = f\left(\mathbf{w}^\top \mathbf{x} + b\right). \tag{1.10}$$

In Rosenblatt's classical formulation, the activation function is instantiated as the discontinuous Heaviside step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.11}$$

producing a binary decision variable indicating class membership.

To streamline the algebraic notation, the formulation can be expressed homogeneously by augmenting the input vector with a dummy feature $x_0 = 1$ and defining an augmented weight vector $\tilde{\mathbf{w}} = [w_0, w_1, \ldots, w_n]^\top \in \mathbb{R}^{n+1}$ where $w_0 = b$. Under this homogeneous convention, Equation (1.9) simplifies to a single inner product:

$$z = \sum_{i=0}^{n} w_i x_i = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}, \tag{1.12}$$

where $\tilde{\mathbf{x}} = [1, x_1, \ldots, x_n]^\top \in \mathbb{R}^{n+1}$.

1.2.2 The geometry of decision hyperplanes

The operational behavior of the artificial perceptron is governed by the geometric properties of its decision boundary in the Euclidean input space $\mathbb{R}^n$. The decision boundary corresponds to the locus of points where the pre-activation vanishes identically:

$$\mathcal{H} = \left\{ \mathbf{x} \in \mathbb{R}^n \;\middle|\; \mathbf{w}^\top \mathbf{x} + b = 0 \right\}. \tag{1.13}$$

In an n-dimensional space, Equation (1.13) defines an $(n-1)$-dimensional affine subspace termed a hyperplane. In a two-dimensional input space ($n = 2$), the hyperplane reduces to a standard straight line:

$$w_1 x_1 + w_2 x_2 + w_0 = 0 \iff x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2}, \tag{1.14}$$

assuming $w_2 \neq 0$.

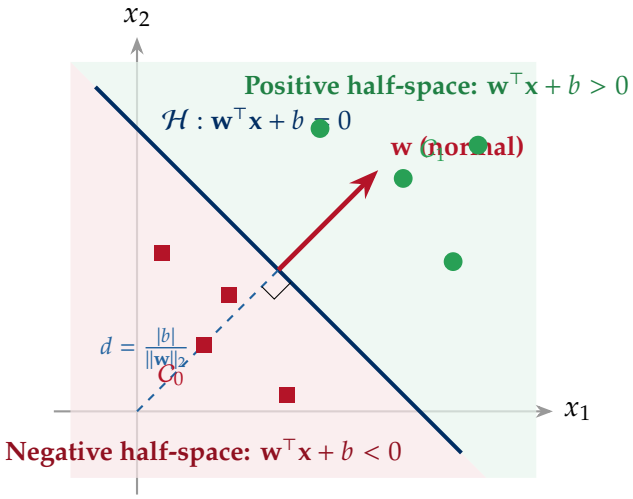


Figure 1.1: Geometric partitioning of a two-dimensional input space by an artificial perceptron decision hyperplane $\mathcal{H}$. The parameter vector $\mathbf{w}$ is strictly orthogonal to the decision boundary, while the scalar bias b determines the orthogonal translation of the hyperplane relative to the origin.

The geometric orientation and placement of the hyperplane are governed by two mathematical properties:

(i) **Orthogonality of the weight vector**: The parameter vector $\mathbf{w}$ is strictly orthogonal to every vector lying within the hyperplane $\mathcal{H}$. To verify this, consider two arbitrary points $\mathbf{x}_A, \mathbf{x}_B \in \mathcal{H}$. By definition of the hyperplane:

$$\mathbf{w}^\top \mathbf{x}_A + b = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_B + b = 0. \tag{1.15}$$

Subtracting the two equalities yields:

$$\mathbf{w}^\top (\mathbf{x}_A - \mathbf{x}_B) = 0. \tag{1.16}$$

Because the difference vector $\mathbf{v} = \mathbf{x}_A - \mathbf{x}_B$ is parallel to the hyperplane and its inner product with $\mathbf{w}$ vanishes, $\mathbf{w}$ is normal to the decision boundary.

(ii) **Orthogonal distance to the boundary**: The signed orthogonal distance $d(\mathbf{x}, \mathcal{H})$ from an arbitrary point $\mathbf{x} \in \mathbb{R}^n$ to the hyperplane $\mathcal{H}$ is obtained by projecting the displacement vector $(\mathbf{x} - \mathbf{x}_p)$, where $\mathbf{x}_p \in \mathcal{H}$, onto the unit normal vector $\frac{\mathbf{w}}{\|\mathbf{w}\|_2}$:

$$d(\mathbf{x}, \mathcal{H}) = \frac{\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_p)}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{x}_p}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|_2}. \tag{1.17}$$

Evaluating this expression at the origin ($\mathbf{x} = \mathbf{0}$) yields the perpendicular distance from the coordinate origin to the hyperplane:

$$d_{\text{origin}} = \frac{|b|}{\|\mathbf{w}\|_2}. \tag{1.18}$$

The perceptron partitions the input space into two half-spaces: a positive half-space where $\mathbf{w}^\top \mathbf{x} + b > 0$ (yielding $y = 1$) and a negative half-space where $\mathbf{w}^\top \mathbf{x} + b < 0$ (yielding $y = 0$).

1.2.3 The linear stacking collapse proof

Given the limitations of a single linear decision boundary, an intuitive proposal involves cascading multiple perceptrons into a multi-layer hierarchical network, where intermediate units supply inputs to subsequent stages. However, if the activation functions between successive layers remain purely linear (that is, $f(z) = z$), the multi-layer architecture collapses algebraically into an ordinary single-layer linear model.

Proof of linear stacking collapse. Consider a multi-layer feed-forward network comprising L fully connected linear layers. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector. The first hidden layer computes an affine transformation parameterized by weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_0 \times d_1}$ and bias vector $\mathbf{b}_1 \in \mathbb{R}^{d_1}$:

$$\mathbf{h}_1 = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1. \tag{1.19}$$

The second hidden layer processes $\mathbf{h}_1$ linearly via $\mathbf{W}_2 \in \mathbb{R}^{d_1 \times d_2}$ and $\mathbf{b}_2 \in \mathbb{R}^{d_2}$:

$$\mathbf{h}_2 = \mathbf{W}_2^\top \mathbf{h}_1 + \mathbf{b}_2 = \mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2. \tag{1.20}$$

Expanding this expression by distributing the matrix multiplication yields:

$$\mathbf{h}_2 = (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left(\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2\right). \tag{1.21}$$

By defining an effective composite weight matrix $\mathbf{W}' \in \mathbb{R}^{d_0 \times d_2}$ and an effective composite bias vector $\mathbf{b}' \in \mathbb{R}^{d_2}$:

$$\mathbf{W}' = \mathbf{W}_1 \mathbf{W}_2 \quad \text{and} \quad \mathbf{b}' = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2, \tag{1.22}$$

Equation (1.21) simplifies to:

$$\mathbf{h}_2 = \mathbf{W}'^\top \mathbf{x} + \mathbf{b}'. \tag{1.23}$$

By mathematical induction, compounding L successive linear layers yields:

$$\mathbf{y} = \mathbf{W}_{\text{eff}}^\top \mathbf{x} + \mathbf{b}_{\text{eff}}, \quad \text{where} \quad \mathbf{W}_{\text{eff}} = \prod_{l=1}^{L} \mathbf{W}_l. \tag{1.24}$$

Because the product of L linear operators is itself a linear operator, stacking arbitrary numbers of purely linear layers provides zero additional expressive capacity over a single affine layer. Consequently, depth without non-linearity is functionally vacuous.

1.2.4 The XOR linear inseparability barrier

The catastrophic consequence of the linear restriction was formally exposed by Marvin Minsky and Seymour Papert in their seminal 1969 monograph [12], which demonstrated that a single-layer perceptron is incapable of computing the basic exclusive-OR (XOR) logic function.

The XOR truth table maps two binary inputs $x_1, x_2 \in \{0, 1\}$ to a scalar output $y \in \{0, 1\}$ according to:

$$\text{XOR}(0,0) = 0, \quad \text{XOR}(0,1) = 1, \quad \text{XOR}(1,0) = 1, \quad \text{XOR}(1,1) = 0. \tag{1.25}$$

For a perceptron with threshold activation $f(z) = \mathbf{1}_{z>0}$ to compute this function, there must exist real parameters $w_1, w_2, b \in \mathbb{R}$ satisfying four simultaneous linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_1(0) + w_2(0) + b \leq 0 \iff b \leq 0, \tag{1.26}$$
$$(x_1 = 0, x_2 = 1) \implies w_1(0) + w_2(1) + b > 0 \iff w_2 + b > 0, \tag{1.27}$$
$$(x_1 = 1, x_2 = 0) \implies w_1(1) + w_2(0) + b > 0 \iff w_1 + b > 0, \tag{1.28}$$
$$(x_1 = 1, x_2 = 1) \implies w_1(1) + w_2(1) + b \leq 0 \iff w_1 + w_2 + b \leq 0. \tag{1.29}$$

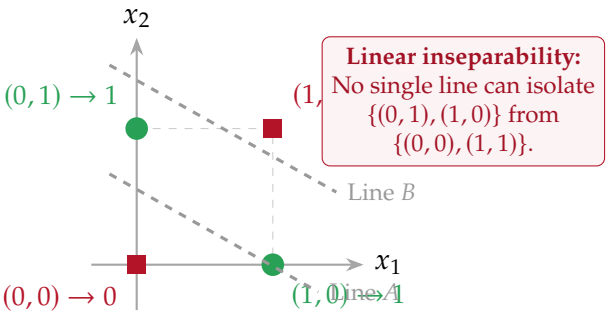


Figure 1.2: The XOR linear inseparability barrier. The two input classes alternate diagonally across the vertices of the unit square, preventing any single linear hyperplane from isolating Class 1 from Class 0.

Proof of algebraic contradiction. Summing the strict inequalities (1.27) and (1.28) yields:

$$(w_2 + b) + (w_1 + b) > 0 \iff w_1 + w_2 + 2b > 0. \tag{1.30}$$

Conversely, summing inequalities (1.26) and (1.29) produces:

$$(w_1 + w_2 + b) + b \leq 0 + 0 \iff w_1 + w_2 + 2b \leq 0. \tag{1.31}$$

Inequality (1.30) and Inequality (1.31) are mutually contradictory: the scalar quantity $w_1 + w_2 + 2b$ cannot simultaneously be strictly positive and non-positive ($0 < w_1 + w_2 + 2b \leq 0$). Hence, no parameters $\{w_1, w_2, b\}$ exist that solve the system. Because the input points alternate diagonally across the vertices of the unit square, the dataset is linearly inseparable, establishing an insurmountable barrier for purely linear units.

1.3 Activation Functions and Non-Linearity

To overcome the linear stacking collapse and solve non-linearly separable problems such as XOR, neural networks must interleave affine transformations with point-wise non-linear mapping functions. These mappings, termed activation functions, provide the mathematical mechanism that enables multi-layer networks to approximate arbitrary functional relationships.

1.3.1 Coordinate space folding and the resolution of XOR

Non-linear activation functions resolve the linear inseparability barrier by non-linearly warping and folding the input coordinate space. When input points are projected through an intermediate non-linear layer, the underlying geometric distances are altered such that points belonging to different classes, initially entangled in the original space, become linearly separable in the transformed hidden space [9].

To illustrate this mechanism concretely, we examine the analytical solution to the XOR problem using a two-layer Multi-Layer Perceptron (MLP) equipped with Rectified Linear Unit (ReLU) activations [9]:

$$\mathbf{h} = g\left(\mathbf{W}^\top \mathbf{x} + \mathbf{c}\right), \quad y = \mathbf{w}^\top \mathbf{h} + b, \tag{1.32}$$

where $g(z) = \max(0, z)$ applies element-wise. Let the network parameters be configured as:

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0. \tag{1.33}$$

The hidden layer pre-activation vector is given by:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{c} = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}. \tag{1.34}$$

Applying the non-linear ReLU activation $g(\mathbf{z})$ evaluates the hidden representations $\mathbf{h} = [h_1, h_2]^\top$ for all four input patterns:

$$\mathbf{x} = [0,0]^\top \implies \mathbf{z} = [0,-1]^\top \implies \mathbf{h} = [\max(0,0), \max(0,-1)]^\top = [0,0]^\top, \tag{1.35}$$
$$\mathbf{x} = [0,1]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.36}$$
$$\mathbf{x} = [1,0]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.37}$$
$$\mathbf{x} = [1,1]^\top \implies \mathbf{z} = [2,1]^\top \implies \mathbf{h} = [\max(0,2), \max(0,1)]^\top = [2,1]^\top. \tag{1.38}$$

The geometric effect of the non-linearity is readily visible: the inputs $(0,1)$ and $(1,0)$, which occupied disjoint coordinates in the original input space, are mapped to the identical latent coordinate $\mathbf{h} = [1,0]^\top$.

The output layer computes a linear combination of the hidden coordinates:

$$y = \mathbf{w}^\top \mathbf{h} + b = 1 \cdot h_1 - 2 \cdot h_2 + 0. \tag{1.39}$$

Evaluating the output on each transformed latent point yields:

$$\mathbf{x} = [0,0]^\top \implies y = 1(0) - 2(0) = 0, \tag{1.40}$$
$$\mathbf{x} = [0,1]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.41}$$
$$\mathbf{x} = [1,0]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.42}$$
$$\mathbf{x} = [1,1]^\top \implies y = 1(2) - 2(1) = 0. \tag{1.43}$$

The model reproduces the XOR truth table with exact precision. By collapsing $(0,1)$ and $(1,0)$ onto a shared latent location while projecting $(1,1)$ away along the second dimension ($h_2 = 1$), the non-linear activation folded the coordinate space, transforming a linearly inseparable problem into a linearly separable representation.

1.3.2 The universal approximation theorem

The theoretical expressive power conferred by non-linear hidden units is formalized by the Universal Approximation Theorem (UAT), proved independently by George Cybenko (1989) for sigmoidal activations [13] and Kurt Hornik, Maxwell Stinchcombe, and Halbert White (1989) for general non-linearities [14].

Theorem 1.1: *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, bounded, non-constant activation function. Let $I_n = [0,1]^n$ denote the compact n-dimensional unit hypercube, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n. Then, for any target function $f \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exists an integer $M \in \mathbb{N}$, a set of real coefficients $v_j \in \mathbb{R}$, weight vectors $\mathbf{w}_j \in \mathbb{R}^n$, and biases $b_j \in \mathbb{R}$ such that the finite sum:

$$F(\mathbf{x}) = \sum_{j=1}^{M} v_j \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right) \tag{1.44}$$

satisfies:

$$\sup_{\mathbf{x} \in I_n} \left| F(\mathbf{x}) - f(\mathbf{x}) \right| < \epsilon. \tag{1.45}$$

In functional analysis terms, the family of single-hidden-layer feed-forward networks is dense in $C(I_n)$ with respect to the uniform supremum metric.

Constructive intuition via localized bump functions. The mathematical mechanism underlying Cybenko's proof can be understood through the constructive superposition of localized rectangular "bump" functions. Consider a one-dimensional input $x \in \mathbb{R}$ and a standard logistic sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$. By taking the difference between two sigmoids displaced by a distance $\Delta = b - a$ and scaled by an arbitrary steepness parameter $w \gg 0$, one constructs a localized pulse or bump function:

$$B(x; a, b, w) = \sigma(w(x - a)) - \sigma(w(x - b)). \tag{1.46}$$

As the scaling factor $w \to \infty$, the sigmoid transitions approximate ideal Heaviside steps, causing $B(x)$ to converge to a rectangular gate function that equals 1 on the interval (a, b) and vanishes outside:

$$\lim_{w \to \infty} B(x; a, b, w) = \begin{cases} 1 & \text{if } x \in (a, b), \\ 0 & \text{otherwise.} \end{cases} \tag{1.47}$$

In an n-dimensional domain, intersecting $2n$ such sigmoidal half-spaces constructs an n-dimensional hypercubical localized bump function that isolates a localized neighborhood of volume ϵ^n. By scaling the amplitude of each bump to match the local value of the target function $f(\mathbf{x})$ and tiling the compact domain I_n with a finite collection of such bumps, the network constructs a piecewise-constant Riemann approximation of $f(\mathbf{x})$. Because continuous functions on compact domains are uniformly continuous, taking sufficiently fine partitions guarantees that the approximation error remains strictly bounded below ϵ.

While the Universal Approximation Theorem provides a foundational mathematical guarantee, two critical caveats govern its practical interpretation:

(i) **Existence versus learnability**: The theorem is purely an existence proof. It guarantees that a configuration of parameters exists that approximates f, but provides no assurance that gradient-based optimization algorithms (such as gradient descent) will successfully discover those parameters from finite, noisy training samples.

(ii) **Width versus depth efficiency**: The theorem guarantees approximation using a single hidden layer, but the required hidden width M may scale exponentially with the input dimension n. As established in Section 1.1.3, deep hierarchical composition constructs multi-scale piecewise linear partitions using polynomial parameter budgets, rendering deep architectures exponentially more parameter-efficient than shallow networks of equivalent capacity.

1.3.3 Taxonomy of activation functions

The choice of non-linear activation function directly influences gradient flow, convergence velocity, and representational capacity across deep networks. Table 1.1 summarizes the mathematical formulations, analytical derivatives, output ranges, and primary failure modes of the canonical activation functions.

Table 1.1: Comparative taxonomy of modern neural activation functions.

Function	Formulation $\sigma(z)$	Derivative $\frac{d\sigma}{dz}$	Range	Zero-Centered	Primary Failure Mode
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$(0,1)$	No	Severe gradient saturation; zig-zag updates
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$1-\tanh^2(z)$	$(-1,1)$	Yes	Gradient saturation at extreme inputs
ReLU	$\max(0,z)$	$\mathbf{1}_{z>0}$	$[0,\infty)$	No	"Dying ReLU" (inactive negative half-plane)
Leaky ReLU	$\max(\alpha z, z)$	$\mathbf{1}_{z>0}+\alpha\mathbf{1}_{z\leq 0}$	$(-\infty,\infty)$	Partial	Hyperparameter sensitivity of slope α
GELU	$z\Phi(z)$	$\Phi(z)+z\phi(z)$	$(-0.17,\infty)$	Partial	Higher transcendental computational cost
SwiGLU	$\text{Swish}(\mathbf{xW})\otimes(\mathbf{xV})$	Gated Jacobian	$(-\infty,\infty)$	Partial	Increases parameter overhead by 1.5×
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$S_i(\delta_{ij}-S_j)$	$(0,1)$	No	Sensitive to logit scale; $O(\lvert\mathcal{V}\rvert)$ cost

Logistic sigmoid

The logistic sigmoid function squashes the entire real line into a bounded interval:

$$\sigma(z)=\frac{1}{1+e^{-z}}=\frac{e^z}{e^z+1}. \tag{1.48}$$

Its first derivative can be written directly in terms of its function values:

$$\frac{d\sigma}{dz}=\frac{e^{-z}}{(1+e^{-z})^2}=\left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right)=\sigma(z)\left(1-\sigma(z)\right). \tag{1.49}$$

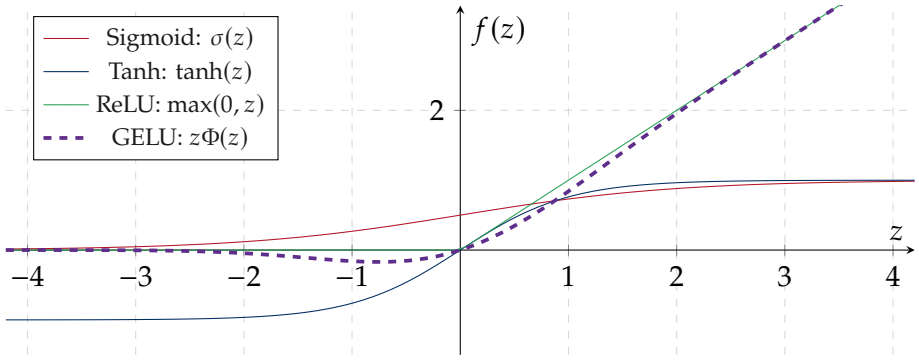


Figure 1.3: Comparison of primary activation functions across the operational domain $z \in [-4, 4]$.

Because the output is bounded in $(0, 1)$, the sigmoid function maps real logits to valid Bernoulli probabilities, making it the canonical choice for binary classification output layers.

However, when employed within deep hidden layers, the sigmoid function exhibits two severe pathologies:

(i) **Gradient saturation**: The maximum value of the derivative occurs at the origin, where $\sigma'(0) = 0.5(1 - 0.5) = 0.25$. When the magnitude of the pre-activation becomes moderately large ($|z| > 4$), the derivative rapidly decays to zero ($\sigma'(z) \approx 0$). During backpropagation, error gradients are scaled by this factor; compounding multiple values bounded above by 0.25 causes the backpropagated gradient to vanish exponentially with depth.

(ii) **Non-zero-centered outputs**: Because $\sigma(z) \in (0, 1)$, the activations entering subsequent layers are strictly positive ($a_i > 0$). In a linear transformation $z = \sum_i w_i a_i + b$, the gradient of the loss with respect to weight w_i is $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} a_i$. Because all $a_i > 0$, the partial derivatives for all incoming weights to a unit share the exact sign of $\frac{\partial \mathcal{L}}{\partial z}$. Consequently, parameter updates can only move simultaneously in all-positive or all-negative directions, inducing inefficient zig-zag optimization trajectories.

Hyperbolic tangent

The hyperbolic tangent (tanh) function resolves the non-zero-centered limitation of the sigmoid by mapping inputs into the symmetric, zero-centered interval $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1. \tag{1.50}$$

Its first derivative evaluates to:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z). \tag{1.51}$$

At the origin, the derivative evaluates to $\tanh'(0) = 1.0$, which is four times larger than that of the sigmoid function, facilitating stronger initial gradient propagation. Furthermore, because outputs have an expected value near zero under symmetric weight initializations, the covariance shifts that produce zig-zag gradient dynamics are mitigated. Nonetheless, the function remains susceptible to saturation plateaus: when $|z| \geq 3$, $|\tanh(z)| \approx 1$ and $\tanh'(z) \to 0$, causing gradients to vanish in deep architectures.

Rectified linear unit

The Rectified Linear Unit (ReLU), popularized in deep learning by Glorot and Bengio [15], introduces a non-saturating threshold operator:

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0, \\ 0 & \text{if } z \leq 0. \end{cases} \tag{1.52}$$

Its derivative is piecewise constant:

$$\frac{d\mathrm{ReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.53}$$

with the sub-gradient convention setting the derivative to 0 (or 0.5) at the non-differentiable point $z = 0$.

The advantages of ReLU over sigmoidal activations are structural:

(i) **Non-saturating gradient flow**: For positive pre-activations ($z > 0$), the derivative is identically equal to 1.0. The gradient passes through the unit without attenuation, mitigating the vanishing gradient problem across deep feed-forward and convolutional networks.

(ii) **Computational efficiency**: Evaluating $\max(0, z)$ requires only a comparison and conditional assignment, avoiding expensive floating-point exponentials.

(iii) **Induced sparsity**: When $z \leq 0$, the unit outputs exactly zero. In large networks, a significant proportion of units are inactive for any given input, yielding sparse representations that improve interpretability and computational economy.

The dying ReLU pathology. Despite these advantages, ReLU networks suffer from the "dying ReLU" problem [10]. If a large gradient update shifts a neuron's incoming weights and bias such that its pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ is negative for all training examples, the unit outputs zero and its derivative evaluates to zero across the entire dataset:

$$\forall \mathbf{x} \in \mathcal{D}_{\text{train}}, \quad z(\mathbf{x}) < 0 \implies \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}}{\partial z}\mathbf{x} = \mathbf{0}. \tag{1.54}$$

Once a unit enters this regime, gradient descent cannot provide non-zero updates to alter its incoming weights, permanently deactivating the neuron.

Leaky rectified linear unit and variants

To eliminate the dead state of standard ReLUs, the Leaky ReLU assigns a small, fixed positive slope $\alpha > 0$ to the negative half-plane:

$$\mathrm{LReLU}(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0, \\ \alpha z & \text{if } z \leq 0, \end{cases} \tag{1.55}$$

typically parameterized with $\alpha = 0.01$. Its derivative evaluates to:

$$\frac{d\mathrm{LReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ \alpha & \text{if } z \leq 0. \end{cases} \tag{1.56}$$

Because the gradient remains non-zero ($\alpha > 0$) for negative inputs, neurons receive corrective gradients even when inactive, preventing permanent deactivation. In Parametric ReLU (PReLU), the slope α is treated as a learnable parameter optimized jointly via backpropagation.

Gaussian error linear unit

The Gaussian Error Linear Unit (GELU), formulated by Hendrycks and Gimpel in 2016 [16], serves as the standard activation function across modern foundation models, including BERT, GPT-2, and GPT-3. GELU integrates stochastic regularization directly into a deterministic activation function by weighting inputs by their probability under a standard Gaussian cumulative distribution:

$$\text{GELU}(z) = z\Phi(z) = zP(X \leq z), \quad \text{where} \quad X \sim \mathcal{N}(0,1). \tag{1.57}$$

The cumulative normal distribution $\Phi(z)$ is defined by the error function erf:

$$\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-\frac{t^2}{2}} dt = \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right]. \tag{1.58}$$

For efficient hardware execution, GELU is frequently evaluated using a fast cubic approximation:

$$\text{GELU}(z) \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{1.59}$$

Unlike ReLU, GELU is non-monotonic, smooth, and continuously differentiable everywhere. For negative inputs, GELU exhibits a small curvature dip with a local minimum at $z \approx -0.7517$ where $f(z) \approx -0.170$, before smoothly approaching zero as $z \to -\infty$. This smooth transition allows small negative activations to convey gradient information while suppressing large negative outliers.

SwiGLU and gated linear units

Modern Large Language Models (such as LLaMA, Mistral, and PaLM) employ Gated Linear Unit (GLU) variants within their feed-forward blocks, specifically SwiGLU [17]. A standard GLU computes the element-wise product of two linear transformations, one of which is modulated by a gating activation:

$$\text{GLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}) = (\mathbf{x}\mathbf{W} + \mathbf{b}) \otimes \sigma\left(\mathbf{x}\mathbf{V} + \mathbf{c}\right), \tag{1.60}$$

where $\otimes$ denotes the element-wise Hadamard product. In SwiGLU, the logistic sigmoid gate is replaced by the Swish activation ($\text{Swish}_\beta(z) = z\sigma(\beta z)$ with $\beta = 1$):

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}) \otimes \text{Swish}\left(\mathbf{x}\mathbf{V}\right). \tag{1.61}$$

This bilinear multiplicative interaction enables hidden representations to dynamically modulate individual feature channels, yielding superior convergence stability and empirical language modeling performance, offset by a 1.5× parameter expansion within the intermediate MLP projection.

The softmax operator

When neural network outputs represent categorical probability distributions over C mutually exclusive classes, the scalar unconstrained pre-activation logits $\mathbf{z} = [z_1, z_2, \ldots, z_C]^\top \in \mathbb{R}^C$ must be mapped onto the probability simplex:

$$\Delta^{C-1} = \left\{ \mathbf{p} \in \mathbb{R}^C \;\middle|\; p_i \geq 0, \; \sum_{i=1}^{C} p_i = 1 \right\}. \tag{1.62}$$

This transformation is performed by the Softmax operator:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}. \tag{1.63}$$

Because the exponential function is strictly positive ($e^{z_i} > 0$), all output probabilities are strictly positive, and normalizing by the partition function $\sum_j e^{z_j}$ guarantees unit sum.

To compute the Jacobian matrix of the Softmax operator, we evaluate the partial derivative of output $\hat{y}_i$ with respect to logit z_k:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{1.64}$$

Applying the quotient rule, we distinguish two cases. When $i = k$:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i}\sum_j e^{z_j} - e^{z_i}e^{z_i}}{\left(\sum_j e^{z_j}\right)^2} = \hat{y}_i - \hat{y}_i^2 = \hat{y}_i(1 - \hat{y}_i). \tag{1.65}$$

When $i \neq k$:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{0 \cdot \sum_j e^{z_j} - e^{z_i}e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = -\hat{y}_i\hat{y}_k. \tag{1.66}$$

Combining both cases using the Kronecker delta δ_{ik} yields the compact Jacobian formulation:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \hat{y}_i\left(\delta_{ik} - \hat{y}_k\right). \tag{1.67}$$

Numerical stability via shift invariance. Direct floating-point evaluation of Equation (1.63) causes numerical overflow if $z_i \gg 0$ (e.g., $e^{1000} \to \infty$ yielding `NaN`) or underflow if all $z_i \ll 0$. The Softmax operator satisfies the shift-invariance property: for any arbitrary scalar constant $c \in \mathbb{R}$:

$$\text{softmax}(\mathbf{z} + c)_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}} = \text{softmax}(\mathbf{z})_i. \tag{1.68}$$

In practical implementations, numerical stability is guaranteed by choosing $c = -\max_k z_k$. The largest exponentiated term evaluates to $e^0 = 1.0$, which prevents overflow while ensuring the denominator contains at least one non-vanishing term.

1.4 Loss Functions and Optimization

To train a parameterized neural network, one must define a scalar criterion that quantifies the discrepancy between model predictions and empirical ground truth. This objective, termed the loss function, provides the supervisory signal that guides parameter optimization.

1.4.1 The operational role of loss functions

Formally, let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote a training dataset sampled from an underlying data-generating distribution $P_{\text{data}}(\mathbf{x}, y)$. A neural network parameterized by $\theta \in \mathbb{R}^P$ evaluates a forward prediction mapping $\hat{y}_i = f(\mathbf{x}_i; \theta)$. The loss function $\mathcal{L}(\hat{y}_i, y_i) \in \mathbb{R}_{\geq 0}$ assigns a non-negative scalar penalty to prediction errors.

Under the framework of Empirical Risk Minimization (ERM), the training objective is to discover a parameter configuration θ^* that minimizes the empirical risk across the observed sample:

$$\theta^* = \arg\min_\theta \mathcal{R}_{\text{emp}}(\theta) = \arg\min_\theta \left[\frac{1}{N} \sum_{i=1}^N \mathcal{L}\left(f(\mathbf{x}_i; \theta), y_i\right) \right]. \tag{1.69}$$

The loss function serves as the sole mathematical feedback channel through which parameter errors are communicated to optimization algorithms. Without a well-defined scalar loss, the system possesses no objective measure of performance and cannot formulate directional gradient vectors for parameter improvement.

1.4.2 Task-conditioned output topologies

A fundamental principle of statistical machine learning is that a single universal loss function cannot exist. Different tasks operate across distinct mathematical domains:

(i) **Continuous regression**: Target variables span the unconstrained real line $y \in \mathbb{R}$ or vector spaces $\mathbb{R}^K$.

(ii) **Binary classification**: Targets represent dichotomous categorical indicators $y \in \{0, 1\}$.

(iii) **Multi-class classification**: Targets represent discrete class memberships $y \in \{1, \ldots, C\}$, geometrically residing on the probability simplex Δ^{C-1}.

To design mathematically principled loss functions across these diverse regimes, Simon Prince [10] formulates the Maximum Likelihood Estimation (MLE) recipe:

Step 1: Define an appropriate conditional probability distribution $P(y \mid \mathbf{x}; \phi)$ over the target space, parameterized by vector ϕ.

Step 2: Configure the neural network to output the parameters $\phi = f(\mathbf{x}; \theta)$ of that chosen distribution (e.g., predicting mean μ for a Gaussian, or probability p for a Bernoulli).

Step 3: Train the network to maximize the log-likelihood of the observed dataset, or equivalently, minimize the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\sum_{i=1}^N \log P\left(y_i \,\middle|\, \mathbf{x}_i; \theta\right). \tag{1.70}$$

This recipe ensures that the loss function is mathematically consistent with the output geometry and noise distribution of the underlying task.

1.4.3 Task-specific loss taxonomy and statistical derivations

Applying the maximum likelihood recipe across canonical distributions produces the standard loss taxonomy employed in modern neural networks.

Regression via Gaussian noise models

In continuous regression tasks, we assume the true target $y \in \mathbb{R}$ differs from the model's deterministic prediction $\hat{y} = f(\mathbf{x}; \theta)$ by additive zero-mean Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y = f(\mathbf{x}; \theta) + \epsilon \implies P(y \mid \mathbf{x}; \theta, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x}; \theta))^2}{2\sigma^2}\right). \tag{1.71}$$

Evaluating the negative log-likelihood over N independently and identically distributed observations yields:

$$\begin{aligned} \mathcal{L}_{\text{NLL}}(\theta) &= -\sum_{i=1}^{N} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \theta))^2}{2\sigma^2}\right)\right] \\ &= \frac{N}{2} \log\left(2\pi\sigma^2\right) + \frac{1}{2\sigma^2} \sum_{i=1}^{N} \left(y_i - f(\mathbf{x}_i; \theta)\right)^2. \end{aligned} \tag{1.72}$$

Assuming homoscedastic noise (σ^2 constant), discarding constant terms, and scaling by $\frac{1}{N}$ recovers the standard Mean Squared Error (MSE) objective:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \hat{y}_i\right)^2. \tag{1.73}$$

If the noise distribution is modeled instead as Laplacian noise $\epsilon \sim \text{Laplace}(0, b)$ with density $p(\epsilon) = \frac{1}{2b} \exp(-|\epsilon|/b)$, minimizing the negative log-likelihood recovers the Mean Absolute Error (MAE) or L_1 loss:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} \left|y_i - \hat{y}_i\right|, \tag{1.74}$$

which exhibits enhanced robustness to dataset outliers because penalties scale linearly rather than quadratically.

Binary classification via Bernoulli distributions

For binary classification ($y \in \{0, 1\}$), the target is modeled as a Bernoulli random variable conditioned on the input:

$$P(y \mid \mathbf{x}) = \hat{y}^y \left(1 - \hat{y}\right)^{1-y}, \quad \text{where} \quad \hat{y} = \sigma(z) \in (0, 1). \tag{1.75}$$

Evaluating the negative log-likelihood of this distribution yields the Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\right]. \tag{1.76}$$

The expression functions as a ground-truth selector:

(i) When ground-truth $y_i = 1$, the second term $(1 - y_i)$ vanishes identically, reducing the penalty to $-\log \hat{y}_i$. The loss approaches 0 as $\hat{y}_i \to 1$, but penalizes incorrect predictions with an asymptote approaching $+\infty$ as $\hat{y}_i \to 0$.

(ii) When ground-truth $y_i = 0$, the first term vanishes, reducing the penalty to $-\log(1 - \hat{y}_i)$, which penalizes predictions approaching $\hat{y}_i \to 1$ with infinite cost.

Multi-class classification via categorical distributions

In multi-class classification over C mutually exclusive categories ($y \in \{1, \ldots, C\}$), targets are represented as one-hot vectors $\mathbf{y} \in \{0,1\}^C$ where $\sum_{k=1}^{C} y_k = 1$. The network predicts a categorical distribution $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$. The conditional likelihood is given by the multinomial density:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{C} \hat{y}_k^{y_k}. \tag{1.77}$$

Minimizing the negative log-likelihood yields the Categorical Cross-Entropy (CCE) loss:

$$\mathcal{L}_{\text{CCE}} = -\sum_{k=1}^{C} y_k \log \hat{y}_k = -\log \hat{y}_c, \tag{1.78}$$

where $c \in \{1, \ldots, C\}$ denotes the index of the true positive class ($y_c = 1$).

Analytical proof of MSE classification failure

A common pedagogical error involves applying Mean Squared Error to classification tasks equipped with sigmoidal outputs. To demonstrate why this combination fails, we evaluate the gradient of the loss with respect to the pre-activation logit z for a binary classifier ($\hat{y} = \sigma(z)$) under both MSE and BCE.

Case 1: MSE logit gradient. The MSE loss for a single observation is $\mathcal{L}_{\text{MSE}} = (y - \hat{y})^2$. Applying the chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} = -2\left(y - \hat{y}\right)\sigma'(z) = -2\left(y - \hat{y}\right)\hat{y}\left(1 - \hat{y}\right). \tag{1.79}$$

Now, evaluate the gradient when the model is confident and wrong: suppose the true label is $y = 1$, but the network currently produces a strongly negative logit $z = -5$, such that $\hat{y} = \sigma(-5) \approx 0.0067$. The residual error $(y - \hat{y}) = 1 - 0.0067 = 0.9933$ is maximal. However, the derivative evaluates to:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -2(0.9933) \cdot (0.0067)(0.9933) \approx -0.0132 \approx 0. \tag{1.80}$$

Because the sigmoid derivative saturates ($\sigma'(z) \to 0$ as $|z| \gg 0$), the gradient vanishes despite maximal prediction error. The parameter updates stall, preventing gradient descent from correcting catastrophic misclassifications.

Case 2: BCE logit gradient. Evaluating the logit gradient under Binary Cross-Entropy:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} \\
&= \left[-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right]\hat{y}\left(1-\hat{y}\right) \\
&= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})}\hat{y}\left(1-\hat{y}\right) \\
&= -y + y\hat{y} + \hat{y} - y\hat{y} \\
&= \hat{y} - y.
\end{aligned} \tag{1.81}$$

Under cross-entropy, the saturating term $\hat{y}(1-\hat{y})$ in the denominator of $\frac{\partial \mathcal{L}}{\partial \hat{y}}$ cancels the identical term in the sigmoid derivative $\frac{d\hat{y}}{dz}$. The resulting gradient evaluates to the linear residual error

$\hat{y} - y$. If $y = 1$ and $\hat{y} \approx 0.0067$, the gradient evaluates to $\hat{y} - y = 0.0067 - 1 = -0.9933$, providing a strong, unattenuated corrective signal.

Furthermore, analyzing the second derivative demonstrates that $\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2}$ flips signs across logit space, creating non-convex regions and saddle plateaus [9]. Conversely, for BCE:

$$\frac{\partial^2 \mathcal{L}_{\text{BCE}}}{\partial z^2} = \frac{d}{dz}\left(\hat{y} - y\right) = \sigma'(z) = \hat{y}\left(1 - \hat{y}\right) > 0, \quad \forall z \in \mathbb{R}. \tag{1.82}$$

The strictly positive second derivative proves that Binary Cross-Entropy is strictly convex with respect to the output logits, guaranteeing that no non-convex plateaus impede convergence.

1.4.4 The analytical intractability of direct root finding

In classical linear regression with quadratic loss, the parameter optimization problem admits an exact, closed-form analytical solution. Consider an input data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and target vector $\mathbf{y} \in \mathbb{R}^N$. The quadratic loss function is:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 . \tag{1.83}$$

To minimize this loss, one computes the gradient with respect to $\mathbf{w}$ and sets it equal to the zero vector:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}. \tag{1.84}$$

This condition reduces to the system of linear equations:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}, \tag{1.85}$$

which, assuming $\mathbf{X}^\top \mathbf{X}$ is non-singular, yields the Ordinary Least Squares (OLS) normal equations:

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}. \tag{1.86}$$

In sharp contrast, setting $\nabla_\theta \mathcal{L}(\theta) = \mathbf{0}$ fails completely for deep neural networks. In multi-layer architectures, the model output is constructed through nested compositions of affine transformations and non-linear activation functions:

$$\hat{y} = f(\mathbf{x}; \theta) = \mathbf{W}_L^\top \sigma\left(\mathbf{W}_{L-1}^\top \dots \sigma\left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_{L-1}\right) + \mathbf{b}_L. \tag{1.87}$$

When differentiated, the parameters $\theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L$ appear inside complex transcendental functions (exponentials, piecewise steps, products). Setting the resulting gradient equations to zero yields a coupled system of non-linear transcendental algebraic equations with millions of unknowns:

$$\nabla_\theta \mathcal{L}(\theta) = \sum_{i=1}^{N} \frac{\partial \mathcal{L}_i}{\partial f_i} \nabla_\theta f(\mathbf{x}_i; \theta) = \mathbf{0}. \tag{1.88}$$

This system possesses no closed-form analytical root. Moreover, the loss surface $\mathcal{L}(\theta)$ is highly non-convex, characterized by complex landscapes with multiple local minima, degenerate saddle points, and ill-conditioned ravines. Consequently, deep neural networks cannot be solved analytically; their optimization requires iterative, numerical gradient-based methods.

1.4.5 Gradient descent and modern optimization heuristics

Iterative numerical optimization computes successive parameter updates by evaluating local first-order gradient vectors.

Vanilla gradient descent

The foundational iterative algorithm is Gradient Descent (GD). At each iteration t, the algorithm updates parameter vector θ_t by taking a step proportional to the negative gradient of the empirical risk:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t), \tag{1.89}$$

where $\alpha > 0$ denotes the learning rate hyperparameter.

While intuitive, vanilla gradient descent encounters three severe operational pathologies:

(i) **Sensitivity to learning rate**: If α is chosen too small, parameter updates crawl inefficiently, requiring prohibitive computational time. Conversely, if α exceeds the reciprocal of the maximum eigenvalue of the Hessian ($\alpha > \frac{2}{\lambda_{\max}}$), the updates overshoot valleys and diverge.

(ii) **Oscillations in ill-conditioned ravines**: When the loss surface exhibits high condition numbers (where curvature along one axis is drastically steeper than along another, $\lambda_{\max} \gg \lambda_{\min}$), gradient descent oscillates violently across the steep walls while making negligible progress along the flat base.

(iii) **Stagnation at saddle points**: In high-dimensional optimization landscapes, local minima are rare; the dominant topological features are saddle points where the gradient vanishes ($\nabla_\theta \mathcal{L} \approx \mathbf{0}$) but some directions exhibit negative curvature. Vanilla GD decelerates severely in the vicinity of saddle plateaus.

Momentum

To dampen high-frequency oscillations across steep ravine walls and accelerate descent along flat valleys, Polyak (1964) introduced the momentum method [9]. Momentum simulates physical inertia by accumulating an exponentially decaying moving average of past gradients into a velocity vector $\mathbf{v}_t$:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t), \tag{1.90}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_t, \tag{1.91}$$

where $\beta \in [0, 1)$ represents the momentum decay coefficient (typically $\beta = 0.9$). Along directions where the gradient repeatedly reverses sign, the oscillating terms cancel out in the running average, dampening instability. Along directions where the gradient consistently points in the same direction, the velocity accumulates to an effective magnitude $\frac{1}{1-\beta}$ times larger than the instantaneous gradient, accelerating convergence through plateaus.

Adam: Adaptive Moment Estimation

In deep architectures and large language models, different parameters operate at widely disparate scales and receive gradients of vastly differing frequencies (e.g., embeddings of rare tokens receive sparse, large updates). Adaptive learning rate algorithms assign individual learning rates to each parameter.

The universal optimization standard across foundation models is Adam (Adaptive Moment Estimation), introduced by Kingma and Ba in 2014 [18]. Adam tracks both the exponentially decaying average of past gradients (first raw moment) and the exponentially decaying average of squared gradients (second uncentered moment):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{1.92}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{1.93}$$

where $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t)$, and $\mathbf{g}_t^2$ denotes the element-wise Hadamard square $\mathbf{g}_t \odot \mathbf{g}_t$. The decay hyperparameters are standardly initialized to $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

Because $\mathbf{m}_0$ and $\mathbf{v}_0$ are initialized to the zero vector $\mathbf{0}$, the moment vectors are biased toward zero, particularly during initial training steps when β_1 and β_2 are close to 1. To correct for this initialization artifact, Adam applies analytical bias corrections:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \tag{1.94}$$

The parameter update is then evaluated coordinate-wise:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t, \tag{1.95}$$

where $\epsilon \approx 10^{-8}$ prevents division by zero. Adam dynamically rescales updates: parameters with large historical variance receive smaller, dampened steps, while parameters with sparse, small gradients receive amplified steps, ensuring robust optimization dynamics across diverse neural layers.

1.5 Computational Graphs and Backpropagation

To execute gradient-based optimization algorithms such as Adam, the learning system must compute the partial derivatives of the scalar loss function with respect to every parameter in the network ($\nabla_\theta \mathcal{L}$). This section establishes the graph-theoretic formalism of computational graphs, derives the reverse-mode automatic differentiation algorithm, and synthesizes these mechanisms into a complete multi-layer feed-forward network.

1.5.1 Methods of gradient evaluation

As formalized by Christopher Bishop and Hugh Bishop (2024, Ch. 8) [8], there exist four distinct methodological approaches to computing derivatives in numerical systems:

(i) **Analytical differentiation**: The manual derivation of derivative expressions by hand using pencil and paper calculus. While mathematically exact, this approach is rigid, labor-intensive, and prone to human algebraic errors. Furthermore, modifying network architecture requires completely re-deriving the analytical formulas from scratch.

(ii) **Numerical differentiation (finite differences)**: The numerical approximation of partial derivatives via perturbation formulas:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon \mathbf{e}_i) - \mathcal{L}(\theta)}{\epsilon}, \tag{1.96}$$

where $\mathbf{e}_i$ denotes the i-th standard basis vector, and $\epsilon \approx 10^{-5}$. While universally applicable to black-box functions, numerical differentiation is computationally prohibitive: evaluating the gradient across a model with D parameters requires $D + 1$ complete forward passes, yielding an asymptotic computational complexity of $O(D)$. For modern foundation models where $D \geq 10^9$, computing a single gradient step would require over one billion forward passes. Furthermore, finite differences suffer from numerical roundoff and truncation errors.

(iii) **Symbolic differentiation**: Computer algebra systems (such as Mathematica or SymPy) construct symbolic mathematical expressions for derivatives using calculus rules. While exact, symbolic differentiation treats mathematical expressions as abstract trees and suffers from exponential *expression swell*: redundant subexpressions are duplicated repeatedly across the expression tree, resulting in massive computational overhead. Moreover, symbolic engines cannot easily differentiate algorithms containing procedural control flow (loops, branches, and conditional recursion).

(iv) **Automatic differentiation**: The algorithmic evaluation of exact numerical derivatives by decomposing complex functions into sequences of elementary mathematical primitives whose derivatives are known. Rather than manipulating symbolic expressions, automatic differentiation evaluates numerical derivatives directly during program execution, providing exact gradients at machine precision with computational cost proportional to the forward function evaluation.

1.5.2 Computational graphs and reverse-mode automatic differentiation

Modern deep learning frameworks implement automatic differentiation by representing neural network execution as a Directed Acyclic Graph (DAG) $G = (\mathcal{V}, \mathcal{E})$ [8, 9].

(i) The set of vertices $\mathcal{V}$ represents computational operations (such as $+$, $\times$, exp, tanh) or intermediate tensor variables.

(ii) The set of directed edges $\mathcal{E}$ represents the directional flow of tensor values, where a directed edge $(u, v) \in \mathcal{E}$ indicates that node v directly depends on the value computed by node u.

Forward evaluation corresponds to traversing the graph according to a topological sort: any node v can be evaluated once all its incoming parent nodes parents(v) have completed execution.

Reverse-mode automatic differentiation (Backpropagation). To compute gradients, reverse-mode automatic differentiation traverses the computational graph in reverse topological order, starting from the final scalar loss $\mathcal{L}$ and propagating gradient signals backward toward the input and parameter leaves.

Following the adjoint notation introduced by Linnainmaa (1970) and Rumelhart et al. (1986) [7], we define the adjoint $\bar{u}$ of any intermediate node u as the partial derivative of the final scalar loss with respect to u:

$$\bar{u} \equiv \frac{\partial \mathcal{L}}{\partial u}. \tag{1.97}$$

The multivariable chain rule of calculus states that if a variable u influences the loss $\mathcal{L}$ through multiple downstream dependent nodes $v \in \text{children}(u)$, the total sensitivity $\bar{u}$ is obtained by summing the contributions transmitted through all outgoing edges:

$$\bar{u} = \sum_{v \in \text{children}(u)} \bar{v} \frac{\partial v}{\partial u} = \sum_{v \in \text{children}(u)} \frac{\partial \mathcal{L}}{\partial v} \frac{\partial v}{\partial u}. \tag{1.98}$$

The computational efficiency of reverse-mode automatic differentiation is profound. Consider a function mapping D parameters to a single scalar loss: $f : \mathbb{R}^D \to \mathbb{R}$.

(i) **Forward-mode automatic differentiation** computes directional derivatives (tangents $\dot{v} = \frac{\partial v}{\partial \theta_i}$) forward from the inputs. Computing the gradient with respect to all D parameters requires D distinct forward sweeps, scaling as $O(D)$.

(ii) **Reverse-mode automatic differentiation** initiates a single adjoint evaluation at the scalar loss ($\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$) and sweeps backward across the graph in reverse topological order. All D partial derivatives are evaluated simultaneously in a single backward pass.

The time complexity of the reverse sweep is bounded by $O(|\mathcal{V}| + |\mathcal{E}|)$, which is approximately equal to twice the computational cost of the forward pass ($T_{\text{backward}} \approx 2 \times T_{\text{forward}}$), completely independent of the parameter dimension D. This fundamental efficiency renders backpropagation the engine of deep learning.

1.5.3 Worked arithmetic calculation of backpropagation

To demystify the abstract multivariable calculus of reverse-mode autodiff, we reconstruct the exact step-by-step arithmetic trace demonstrated in the course lecture slides.

Consider a single training observation $(x, y) \in \mathbb{R} \times \mathbb{R}$. The model computes a prediction $\hat{y}$ through the product of two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$:

$$\hat{y} = \theta_1 \theta_2 x. \tag{1.99}$$

The optimization objective is the scalar squared error loss:

$$\mathcal{L} = (\hat{y} - y)^2 = (\theta_1 \theta_2 x - y)^2. \tag{1.100}$$

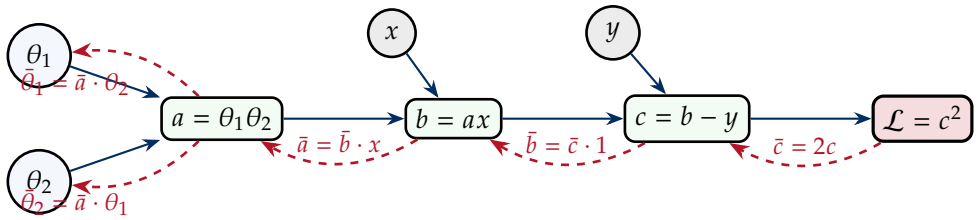


Figure 1.4: Computational graph representation for the lecture worked example $\mathcal{L} = (\theta_1\theta_2 x - y)^2$. Solid blue arrows indicate forward topological execution; dashed red arrows trace the reverse-mode adjoint propagation.

Forward evaluation trace. The computational graph decomposes this loss into a sequence of four elementary intermediate operations:

$$\text{Node } a = \theta_1\theta_2, \tag{1.101}$$
$$\text{Node } b = a \cdot x = \theta_1\theta_2 x, \tag{1.102}$$
$$\text{Node } c = b - y = \theta_1\theta_2 x - y, \tag{1.103}$$
$$\text{Node } \mathcal{L} = c^2 = \left(\theta_1\theta_2 x - y\right)^2. \tag{1.104}$$

Backward adjoint propagation trace. In the reverse pass, we evaluate the adjoints $\bar{u} = \frac{\partial \mathcal{L}}{\partial u}$ in reverse topological order:

Step 1: **Initialize root adjoint**: At the root node, the derivative of the loss with respect to itself is identically unity:

$$\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1. \tag{1.105}$$

Step 2: **Adjoint of node** c: Differentiating $\mathcal{L} = c^2$ with respect to c:

$$\bar{c} = \frac{\partial \mathcal{L}}{\partial c} = \bar{\mathcal{L}}\frac{\partial \mathcal{L}}{\partial c} = 1 \cdot (2c) = 2\left(\theta_1\theta_2 x - y\right). \tag{1.106}$$

Step 3: **Adjoint of node** b: Differentiating $c = b - y$ with respect to b:

$$\bar{b} = \frac{\partial \mathcal{L}}{\partial b} = \bar{c}\frac{\partial c}{\partial b} = \bar{c} \cdot 1 = 2\left(\theta_1\theta_2 x - y\right). \tag{1.107}$$

Step 4: **Adjoint of node** a: Differentiating $b = a \cdot x$ with respect to a:

$$\bar{a} = \frac{\partial \mathcal{L}}{\partial a} = \bar{b}\frac{\partial b}{\partial a} = \bar{b} \cdot x = \left[2\left(\theta_1\theta_2 x - y\right)\right] x = 2x\left(\theta_1\theta_2 x - y\right). \tag{1.108}$$

Step 5: **Adjoint of parameter** θ_1: Differentiating $a = \theta_1\theta_2$ with respect to θ_1:

$$\bar{\theta}_1 = \frac{\partial \mathcal{L}}{\partial \theta_1} = \bar{a}\frac{\partial a}{\partial \theta_1} = \bar{a} \cdot \theta_2 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_2 = 2\theta_2 x\left(\theta_1\theta_2 x - y\right). \tag{1.109}$$

Step 6: **Adjoint of parameter** θ_2: Differentiating $a = \theta_1\theta_2$ with respect to θ_2:

$$\bar{\theta}_2 = \frac{\partial \mathcal{L}}{\partial \theta_2} = \bar{a}\frac{\partial a}{\partial \theta_2} = \bar{a} \cdot \theta_1 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_1 = 2\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.110}$$

With the exact partial derivatives computed, parameter updates under gradient descent evaluate to:

$$\theta_1 \leftarrow \theta_1 - \alpha\bar{\theta}_1 = \theta_1 - 2\alpha\theta_2 x\left(\theta_1\theta_2 x - y\right), \tag{1.111}$$

$$\theta_2 \leftarrow \theta_2 - \alpha\bar{\theta}_2 = \theta_2 - 2\alpha\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.112}$$

This minimal worked example illustrates the operational universality of reverse-mode automatic differentiation: regardless of model depth or topological branching, the global gradient is resolved into localized, modular partial derivative multiplications evaluated backward along computational edges.

1.5.4 The complete multi-layer feed-forward network

We now synthesize all established primitives—linear projections, non-linear activation functions, task-specific loss criteria, and reverse-mode automatic differentiation—into a unified, fully vectorized multi-layer Feed-Forward Network (FFN).

Vectorized layer equations

Consider an L-layer network. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector, and define $\mathbf{a}^{(0)} \equiv \mathbf{x}$. For each layer $l \in \{1, 2, \ldots, L\}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)\top}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}\left(\mathbf{z}^{(l)}\right), \tag{1.113}$$

where:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l} \quad \text{denotes the weight transformation matrix for layer } l, \tag{1.114}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the scalar bias vector for layer } l, \tag{1.115}$$

$$\mathbf{z}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the pre-activation logit vector,} \tag{1.116}$$

$$\mathbf{a}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the post-activation latent feature representation,} \tag{1.117}$$

$$f^{(l)}(\cdot) \quad \text{denotes the element-wise non-linear activation function.} \tag{1.118}$$

The final output vector $\hat{\mathbf{y}} = \mathbf{a}^{(L)} \in \mathbb{R}^{d_L}$ produces predictions tailored to task topology (e.g., continuous predictions via linear activations for regression, or probability distributions via Softmax for classification).

Vectorized backpropagation equations

Let $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}^*)$ denote the scalar loss evaluated on ground truth $\mathbf{y}^*$. In vectorized matrix calculus, we define the layer error vector $\boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}$ as the gradient of the loss with respect to the pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \tag{1.119}$$

Output layer error $\boldsymbol{\delta}^{(L)}$. Applying the chain rule to the output layer:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot f'^{(L)}\left(\mathbf{z}^{(L)}\right), \tag{1.120}$$

where $\odot$ denotes the element-wise Hadamard product. As proven in Equation (1.81), when pairing cross-entropy loss with Softmax or Sigmoid activations, this expression simplifies directly to the residual difference:

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}^*. \tag{1.121}$$

Hidden layer error recursion $\boldsymbol{\delta}^{(l)}$. For any internal hidden layer $l \in \{1, \dots, L-1\}$, the error vector $\boldsymbol{\delta}^{(l)}$ is evaluated recursively backward from layer $l+1$:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \right)^{\top} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}} = \left(\mathbf{W}^{(l+1)} \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)} \left(\mathbf{z}^{(l)} \right). \tag{1.122}$$

Equation (1.122) demonstrates the structural symmetry of backpropagation: the error signal $\boldsymbol{\delta}^{(l+1)}$ is projected backward through the transpose of the forward weight matrix $\mathbf{W}^{(l+1)}$ and subsequently modulated by the local derivative of the activation function $f'^{(l)}(\mathbf{z}^{(l)})$.

Parameter gradient evaluation. Once the error vectors $\boldsymbol{\delta}^{(l)}$ are evaluated, the gradients of the loss with respect to the weight matrices and bias vectors are obtained via outer products:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \mathbf{a}^{(l-1)} \left(\boldsymbol{\delta}^{(l)} \right)^{\top} \in \mathbb{R}^{d_{l-1} \times d_l}, \tag{1.123}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}. \tag{1.124}$$

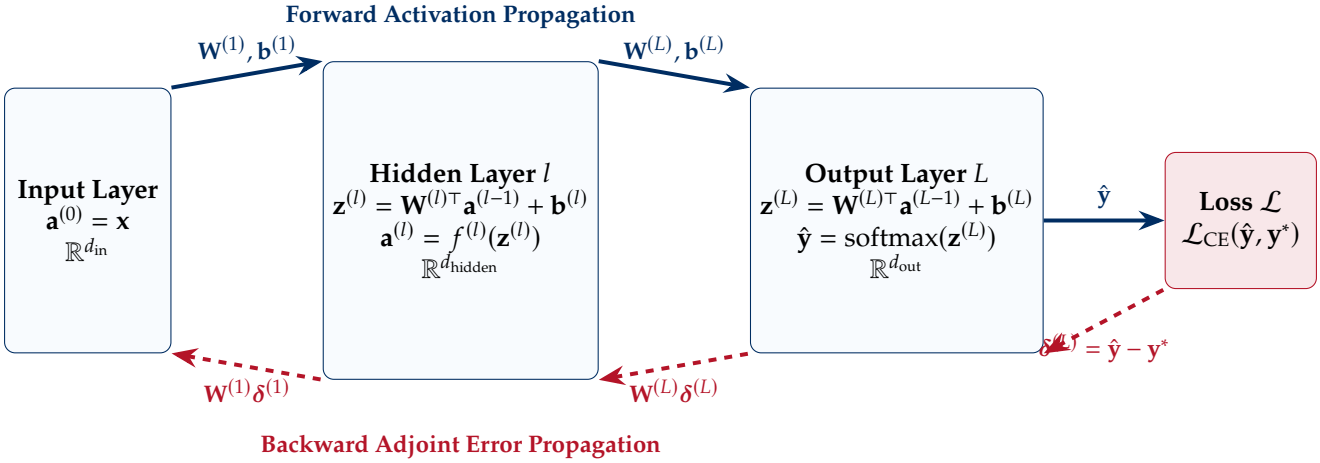


Figure 1.5: Vectorized dataflow schematic across a multi-layer Feed-Forward Network. Solid blue lines trace forward activations; dashed red lines trace the reverse-mode propagation of error vectors $\boldsymbol{\delta}^{(l)}$.

Structural limitations of feed-forward networks for sequence processing

While the multi-layer feed-forward network provides a universal framework for static pattern recognition, its fundamental architectural assumptions break down when deployed on sequential data, such as natural language text, source code, and time series [8, 9]. Specifically, standard feed-forward networks exhibit three structural failure modes:

(i) **Fixed-dimensional input constraints**: An FFN requires an input vector of strictly fixed dimension d_{in}. However, natural language utterances and software engineering artifacts vary arbitrarily in length. Forcing variable-length sentences into a fixed vector through zero-padding or truncation either wastes computation on empty tokens or discards critical semantic context.

(ii) **Absence of temporal memory**: Feed-forward layers operate statelessly: each input vector $\mathbf{x}$ is processed in total isolation from preceding observations. The network possesses no persistent internal hidden state to store linguistic dependencies, resolve pronouns, or maintain scope across long programmatic contexts.

(iii) **Inability to share parameters across sequence positions**: In a standard linear layer $\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$, each input feature is assigned a unique, position-dependent weight. If an informative linguistic motif (such as the keyword `function` or the phrase *"not found"*) appears at token position $t = 1$, the network cannot recognize that identical motif at token position $t = 15$ without separately learning an independent set of weights for those coordinates.

Overcoming these limitations requires an architecture equipped with persistent internal states and position-invariant parameter sharing. This fundamental representational shift directly motivates the transition to Recurrent Neural Networks, explored in Chapter 2.

Chapter 2

Recurrent Neural Networks

The foundational feed-forward architectures established in the preceding chapter operate as static spatial mappings between fixed-dimensional Euclidean vector spaces. In a conventional multi-layer perceptron, an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is mapped deterministically to an output vector $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ through a sequence of parameterized affine transformations interleaved with coordinate-wise non-linearities. While this formulation provides universal function approximation under compact domains, its topological contract assumes that every input observation arrives as a self-contained, isolated point of fixed dimensionality, devoid of internal temporal ordering or dynamic duration.

Real-world perceptual and symbolic processes—most notably natural language text, acoustic speech signals, software execution traces, and biological macromolecules—fundamentally violate this static spatial premise. Natural sequences exhibit variable lengths across observations, temporal dependencies where the interpretation of a token depends intrinsically on antecedent context, and translation-invariant motifs that retain identical semantic meaning regardless of the temporal index at which they appear. Applying static feed-forward networks to such sequential signals requires ad-hoc zero-padding, rigid context window truncation, or the redundant allocation of localized parameters across spatial coordinates, inducing severe sample inefficiencies and representational brittleness.

This chapter establishes the theoretical, algorithmic, and architectural foundations of Recurrent Neural Networks (RNNs). We begin by formalizing the structural failure modes of feed-forward networks on sequential data, motivating the necessity of persistent internal memory and position-invariant parameter sharing. We derive the mathematical mechanics of recurrent state transitions, unroll the network across time into an equivalent directed acyclic graph, and formulate the exact gradient accumulation dynamics of Backpropagation Through Time (BPTT). We then analyze the analytical pathologies of vanilla recurrence—specifically the exponential decay and explosion of temporal Jacobians—and examine how gated recurrent architectures, such as the Long Short-Term Memory network and the Gated Recurrent Unit, enforce constant error propagation. Finally, we explore the sequence-to-sequence encoder-decoder paradigm and demonstrate how the fundamental information bottleneck of compressing variable-length sequences into a single static context vector directly motivates the Attention mechanism.

2.1 Sequential Data and Feed-Forward Limits

Understanding the emergence of recurrent architectures requires analyzing why standard feed-forward networks, despite their universal approximation capabilities, are structurally ill-suited for sequence processing. The limitations of feed-forward networks stem from their rigid dimensional contracts, their localized parameter binding, and their inability to model temporal dynamics without severe compromises.

2.1.1 Fixed dimensionality constraints on inputs and outputs

A standard fully connected neural layer computes an affine transformation followed by an activation function:

$$\mathbf{h} = \sigma\left(\mathbf{W}\mathbf{x} + \mathbf{b}\right), \tag{2.1}$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ denotes the input vector, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ represents the weight parameter matrix, and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector [9]. The algebraic definition of matrix multiplication requires that the column dimension of $\mathbf{W}$ strictly matches the dimensionality of the input vector $\mathbf{x}$. Consequently, any neural network constructed from feed-forward layers imposes an unyielding contract: every input vector must possess exactly d_{in} coordinates, and every output prediction must produce exactly d_{out} values.

In structured machine learning domains, such as tabular databases, this fixed dimensional contract is entirely appropriate because each coordinate of $\mathbf{x}$ corresponds to a specific, predefined feature column. However, unstructured perceptual and symbolic signals are intrinsically variable in extent. In natural language processing, a text sample may span a single-word query, a five-token sentence, or a document containing thousands of words. Similarly, raw acoustic recordings vary in duration from fractions of a second to multiple hours, and computer vision images vary across capture devices in height and width before standardization.

When practitioners attempt to process variable-length sequences using static feed-forward networks, they are forced to define an arbitrary upper bound on sequence length, denoted $T_{\max}$, and enforce dimensional uniformity via artificial zero-padding or heuristic truncation. If an observation sequence has length $T < T_{\max}$, the remaining $T_{\max} - T$ slots are populated with synthetic null tokens; conversely, if $T > T_{\max}$, the sequence is truncated, discarding all information residing beyond the arbitrary horizon. This operational compromise introduces substantial structural deficiencies:

1. **Computational waste**: Zero-padded dimensions must undergo full matrix multiplications throughout all layers, consuming memory bandwidth and floating-point operations on non-informative null values.

2. **Information loss**: Rigid truncation destroys long-range context and truncates syntactic clauses that are essential for accurate semantic inference.

3. **Spurious gradient signals**: Unless complex masking logic is manually applied at every layer, artificial padding tokens contribute synthetic activations that skew batch statistics and destabilize optimization.

The dimensional rigidity of the output layer is equally problematic. In sequential tasks such as syntactic parsing, semantic segmentation, or machine translation, the target output sequence is not a static scalar or fixed-size vector; it is itself an ordered sequence whose length T_{out} varies dynamically and cannot be determined prior to observing the input signal. A feed-forward network with a fixed output dimension d_{out} is mathematically incapable of generating variable-length sequences without enforcing artificial output horizons.

2.1.2 Lack of position invariance and weight localization

The second fundamental pathology of feed-forward networks is their spatial and temporal parameter localization. In a dense layer, each scalar weight W_{ji} is dedicated exclusively to the connection between input coordinate i and hidden neuron j. This architecture binds each parameter to an absolute coordinate index in the input vector space [8].

To conceptualize the consequences of parameter localization, consider an image classification task evaluated on flattened pixel vectors. Suppose an image contains the visual motif of a dog located in the upper-left quadrant of the frame. The feed-forward network activates the subset of weights associated with those specific coordinate indices to detect the object. If an identical dog appears in the lower-right quadrant of another image, the corresponding visual features activate an entirely disjoint set of pixel coordinates, corresponding to completely different columns of $\mathbf{W}$. Because the network lacks parameter sharing across spatial positions, it cannot transfer what it learned about dogs in the upper-left corner to dogs appearing in the lower-right corner. The model is forced to independently rediscover and parameterize the visual concept of a dog at every discrete spatial coordinate.

When applied to sequential text processing, this parameter localization leads to severe sample inefficiency. Consider a character-level capitalization task where an input character sequence must be mapped to its correctly cased variant. In natural language, an opening quotation mark " serves as a structural syntactic cue indicating that the immediately following character must be capitalized as the start of a dialogue utterance. In a feed-forward network where a sequence of T characters is concatenated into a single input vector:

$$\mathbf{x} = \left[\mathbf{x}_1^\top, \mathbf{x}_2^\top, \ldots, \mathbf{x}_T^\top\right]^\top \in \mathbb{R}^{T \cdot d_c}, \tag{2.2}$$

the network must learn separate, independent weight parameters to detect quotation marks at position $t = 1$, position $t = 2$, position $t = 3$, and every subsequent temporal offset up to T. If the training corpus contains numerous examples of dialogue starting at index 1 but none starting at index 14, the model will fail to capitalize a dialogue quote appearing at index 14, despite possessing the relevant knowledge at index 1.

Because identical linguistic and acoustic motifs occur at arbitrary temporal offsets, a sequence processing model must satisfy *temporal translation equivariance*: shifting an input pattern in time should produce an identical, equivalently shifted representation without requiring separate parameters. A feed-forward network completely lacks this inductive bias, demanding a parameter budget that scales quadratically with sequence length:

$$N_{\text{parameters}} = O\left(T \cdot d_{\text{in}} \cdot d_{\text{hidden}}\right), \tag{2.3}$$

which becomes computationally intractable for long textual contexts and renders generalization to unseen sequence positions virtually impossible.

2.1.3 The structural requirement for dynamic temporal processing

The failure of feed-forward architectures establishes the formal requirements for an authentic sequential learning paradigm. Mathematically, an ordered sequence is defined as a tuple:

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T), \quad \mathbf{x}_t \in \mathbb{R}^{d_x}, \quad T \in \mathbb{N}^+, \tag{2.4}$$

where the temporal index t specifies chronological or symbolic ordering, and the total length T varies across observations.

To process such sequences effectively, an architecture must satisfy three primary design imperatives:

1. **Length-agnostic ingestion**: The model must operate on sequences of arbitrary length T through an iterative transition operator whose parametric specification is strictly independent of T.

2. **Position-invariant parameter sharing**: The transformation applied at time step t must utilize the exact same parameter tensors as the transformation applied at time step t', enabling structural motifs learned at one temporal position to generalize across all positions.

3. **Contextual persistence and state propagation**: The representation computed at step t must not depend solely on the isolated token $\mathbf{x}_t$, but must integrate a synthesized historical record of all prior observations $(\mathbf{x}_1, \ldots, \mathbf{x}_{t-1})$.

These three requirements directly motivate the architecture of Recurrent Neural Networks, which replace the static, one-shot feed-forward mapping with a dynamical system governed by an internal recurrent state.

2.2 RNN Architecture and Cell Dynamics

Recurrent Neural Networks address the structural limitations of feed-forward networks by introducing an internal state that persists across temporal steps. Instead of consuming an entire sequence simultaneously, an RNN ingests one token at a time, continuously updating its internal memory to synthesize past observations with current sensory input.

2.2.1 The recurrent intuition and persistent hidden state

The fundamental conceptual mechanism of a recurrent neural network can be understood through the pedagogical analogy of a human reader taking running notes while reading a document. A human reader does not attempt to ingest an entire book in a single visual fixation; rather, the reader scans the text sequentially, word by word. As each new word is read, the reader does not discard previous knowledge, nor do they carry the verbatim text of every preceding page in immediate working memory. Instead, the reader maintains a continuous mental summary—a running piece of paper—recording essential context, active subjects, unresolved clauses, and grammatical state. When a new word is encountered, the reader combines their current mental notes with the incoming word, updates the notes, and optionally generates an interpretation or expectation of upcoming text.

In a Recurrent Neural Network, this mental notepad is formalized as the *hidden state vector*, denoted $\mathbf{h}_t \in \mathbb{R}^{d_h}$. The hidden state serves as an internal, distributed latent representation that encodes the historical trajectory of the sequence up to time step t [19]. At each discrete time step $t \in \{1, \dots, T\}$, the recurrent unit operates as an operator characterized by a dual-input, dual-output interface:

$$\text{Inputs at step } t: \quad \mathbf{x}_t \in \mathbb{R}^{d_x} \text{ (current sequence element)}, \quad \mathbf{h}_{t-1} \in \mathbb{R}^{d_h} \text{ (previous state)}, \tag{2.5}$$

$$\text{Outputs at step } t: \quad \hat{\mathbf{y}}_t \in \mathbb{R}^{d_y} \text{ (step prediction)}, \quad \mathbf{h}_t \in \mathbb{R}^{d_h} \text{ (updated state)}. \tag{2.6}$$

Before any sequence elements are observed (at step $t = 0$), the network must initialize its internal memory. In the absence of prior context, the initial state is typically defined as a vector of zeros:

$$\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^{d_h}, \tag{2.7}$$

representing a clean, unwritten slate. As sequence tokens $\mathbf{x}_1, \mathbf{x}_2, \dots$ arrive, the recurrent transition function modifies $\mathbf{h}_t$ at every step, allowing downstream predictions $\hat{\mathbf{y}}_t$ to be conditioned on the complete historical context $\mathbf{x}_{\leq t}$.

2.2.2 Time unfolding and unrolled computational graphs

The recurrent relationship is traditionally depicted in two equivalent representational forms: the compact *folded* diagram and the explicit *unfolded* computational graph.

In the folded representation (Figure 2.1a), the network is illustrated as a single processing unit possessing a directed cyclic feedback loop: the hidden state output $\mathbf{h}_t$ is fed directly back into the cell to serve as the state input for the subsequent step. While algebraically concise, cyclic graphs cannot be directly evaluated by standard automatic differentiation engines, which require acyclic computation paths to establish reverse topological orderings.

To resolve this cyclic dependency, the recurrent execution is *unfolded in time* (Figure 2.1b). Unfolding expands the recurrent unit across the temporal sequence of length T, replicating the cell into a chain of T interconnected feed-forward units. In this unfolded representation, the cyclic feedback loop is mapped onto a sequence of forward-directed horizontal edges connecting unit $t-1$ to unit t. The resulting structure forms a classical Directed Acyclic Graph (DAG), enabling standard forward inference and reverse-mode automatic differentiation across the temporal chain.

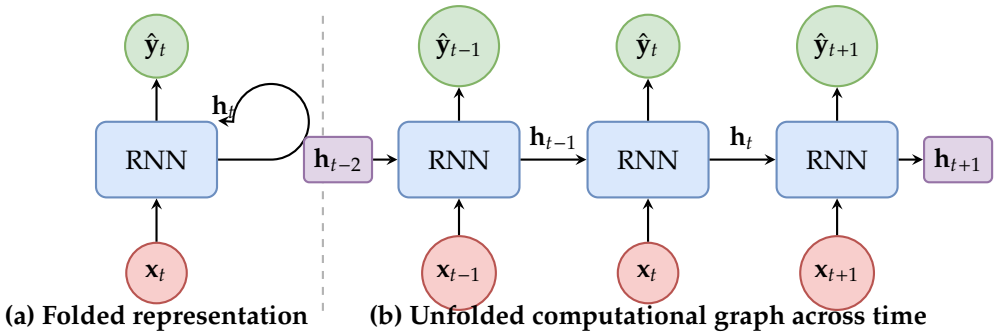


Figure 2.1: Comparison between the compact folded representation of a recurrent neural network with a self-referential temporal feedback loop and its unfolded computational Directed Acyclic Graph (DAG) across consecutive time steps.

To observe how time unfolding coordinates contextual reasoning, consider tracking the character-level casing problem introduced earlier. Suppose the model receives the sequential lowercase stream `['"', 'y', 'o', 'u']`:

1. **Step 1 ($t = 1$)**: The model ingests $\mathbf{x}_1 =$ `'"'` and initial state $\mathbf{h}_0 = \mathbf{0}$. The network emits $\hat{\mathbf{y}}_1 =$ `'"'` and transitions to state $\mathbf{h}_1$. Internally, the parameter transformations configure $\mathbf{h}_1$ to encode the semantic feature: *"an opening quotation mark has occurred; dialogue has begun; the subsequent token must be capitalized"*.

2. **Step 2 ($t = 2$)**: The model ingests $\mathbf{x}_2 =$ `'y'` alongside $\mathbf{h}_1$. Driven by the contextual indicator stored in $\mathbf{h}_1$, the output projection produces an uppercase token $\hat{\mathbf{y}}_2 =$ `'Y'`. Concurrently, the state updates to $\mathbf{h}_2$, encoding: *"the initial dialogue letter has been capitalized; subsequent letters in the current token should revert to lowercase"*.

3. **Step 3 ($t = 3$)**: The model ingests $\mathbf{x}_3 =$ `'o'` alongside $\mathbf{h}_2$. Consulting $\mathbf{h}_2$, the model emits lowercase $\hat{\mathbf{y}}_3 =$ `'o'` and maintains a state reflecting active word continuation.

Through this sequential unfolding, the network resolves local contextual ambiguities dynamically, utilizing the persistent state vector to propagate structural syntactic memory across successive steps.

2.2.3 Internal cell mechanics and state-space transformations

Having examined the external input-output behavior of the recurrent unit, we now formulate the internal algebraic operations governing the Elman recurrent cell [19]. The interior of an Elman cell consists of three parameterized affine projections and two non-linear mapping stages.

The mathematical transformation unfolds in four explicit stages:

1. **Input projection**: The input vector $\mathbf{x}_t \in \mathbb{R}^{d_x}$ is projected into the hidden state space via the input-to-hidden parameter matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$:

$$\mathbf{u}_t = \mathbf{W}_{xh}\mathbf{x}_t \in \mathbb{R}^{d_h}. \tag{2.8}$$

   This operation aligns the dimensionality of the incoming observation with the hidden representation space, mapping disparate input modalities (such as one-hot character vectors or continuous embeddings) into a standardized coordinate frame.

2. **Recurrent state transformation**: The previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$ is transformed through the hidden-to-hidden recurrent transition matrix $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$:

$$\mathbf{v}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}. \tag{2.9}$$

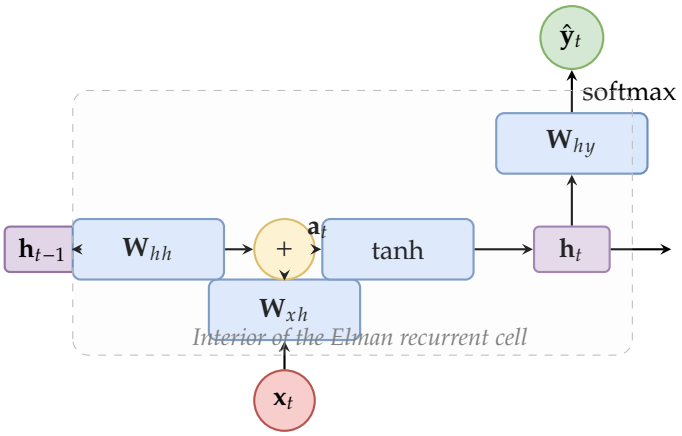


Figure 2.2: Detailed architectural anatomy of the Elman recurrent cell, illustrating the affine projections of the incoming token $\mathbf{x}_t$ and antecedent state $\mathbf{h}_{t-1}$, the linear state aggregation node, the hyperbolic tangent activation function, and the output classification head.

This transformation rotates and scales the historical memory representation, filtering and projecting antecedent information into coordinates compatible with the incoming input.

3. **Linear aggregation and activation**: The projected input $\mathbf{u}_t$ and projected antecedent state $\mathbf{v}_t$ are summed coordinate-wise alongside a learnable hidden bias vector $\mathbf{b}_h \in \mathbb{R}^{d_h}$, yielding the pre-activation state vector $\mathbf{a}_t \in \mathbb{R}^{d_h}$:

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h. \tag{2.10}$$

The pre-activation vector is then mapped through an element-wise non-linear activation function, canonically chosen as the hyperbolic tangent (tanh):

$$\mathbf{h}_t = \tanh(\mathbf{a}_t) = \frac{\exp(\mathbf{a}_t) - \exp(-\mathbf{a}_t)}{\exp(\mathbf{a}_t) + \exp(-\mathbf{a}_t)} \in (-1, 1)^{d_h}. \tag{2.11}$$

The hyperbolic tangent squashes state activations into the symmetric bounded interval $(-1, 1)$, preventing unbounded numerical growth across successive temporal steps while centering activations around zero.

4. **Output projection and distribution**: If the task demands a token prediction at time step t, the updated state $\mathbf{h}_t$ is projected into the output target space via the hidden-to-output matrix $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ and bias $\mathbf{b}_y \in \mathbb{R}^{d_y}$:

$$\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^{d_y}, \tag{2.12}$$

where $\mathbf{z}_t$ represents the unnormalized logit vector. For categorical prediction over a discrete vocabulary $\mathcal{V}$ with dimension $d_y = |\mathcal{V}|$, normalized class probabilities are computed via the softmax operator:

$$\hat{\mathbf{y}}_t = \operatorname{softmax}(\mathbf{z}_t) = \frac{\exp(\mathbf{z}_t)}{\sum_{k=1}^{d_y} \exp(z_{t,k})} \in \Delta^{d_y - 1}. \tag{2.13}$$

To maintain rigorous mathematical clarity, Table 2.1 summarizes the explicit shapes and functional roles of all parameters and representations comprising the Elman recurrent architecture.

Mathematical Symbol	Tensor Shape	Space / Domain	Functional Role
$\mathbf{x}_t$	$d_x \times 1$	$\mathbb{R}^{d_x}$	Input sequence token at step t
$\mathbf{h}_t$	$d_h \times 1$	$(-1, 1)^{d_h}$	Recurrent latent state vector at step t
$\hat{\mathbf{y}}_t$	$d_y \times 1$	$\Delta^{d_y - 1}$	Predicted probability distribution at step t
$\mathbf{W}_{xh}$	$d_h \times d_x$	$\mathbb{R}^{d_h \times d_x}$	Input-to-hidden projection matrix
$\mathbf{W}_{hh}$	$d_h \times d_h$	$\mathbb{R}^{d_h \times d_h}$	Hidden-to-hidden recurrent transition matrix
$\mathbf{W}_{hy}$	$d_y \times d_h$	$\mathbb{R}^{d_y \times d_h}$	Hidden-to-output projection matrix
$\mathbf{b}_h$	$d_h \times 1$	$\mathbb{R}^{d_h}$	Hidden pre-activation bias vector
$\mathbf{b}_y$	$d_y \times 1$	$\mathbb{R}^{d_y}$	Output logit bias vector

Table 2.1: Explicit tensor dimensions, mathematical domains, and functional roles of representations and learnable parameters in the standard Elman recurrent cell.

2.3 Backpropagation Through Time

The computational elegance and representational power of Recurrent Neural Networks arise from two structural principles: the strict sharing of parameter tensors across all sequence coordinates, and the recursive backpropagation of error signals through the unfolded computational graph.

2.3.1 The weight-sharing paradigm and position invariance

In standard feed-forward networks, processing an input sequence of length T requires distinct weight matrices for each temporal offset, causing parameter count to explode linearly with sequence duration. In sharp contrast, a recurrent neural network enforces *parameter tying*: the exact same weight matrices and bias vectors are reused at every time step:

$$\mathbf{W}_{xh}^{(1)} = \mathbf{W}_{xh}^{(2)} = \cdots = \mathbf{W}_{xh}^{(T)} = \mathbf{W}_{xh}, \quad \mathbf{W}_{hh}^{(1)} = \mathbf{W}_{hh}^{(2)} = \cdots = \mathbf{W}_{hh}^{(T)} = \mathbf{W}_{hh}. \tag{2.14}$$

This architectural invariance yields two decisive advantages:

1. **Position-invariant inference**: Because the transition function is parameterized identically across time, the network applies uniform operational logic to input tokens regardless of their absolute temporal offset. An opening quotation mark encountered at $t = 1$, $t = 25$, or $t = 100$ activates the exact same input projection $\mathbf{W}_{xh}$ and recurrent transition $\mathbf{W}_{hh}$, triggering identical state updates and eliminating the need to independently learn position-specific rules.

2. **Length-invariant parameter complexity**: The total number of learnable parameters in an Elman RNN is:
$$N_{\text{parameters}} = d_h \cdot d_x + d_h^2 + d_h + d_y \cdot d_h + d_y. \tag{2.15}$$
Crucially, Equation 2.15 contains no term depending on sequence length T. The model can process a sequence of 5 tokens or 500 tokens using the exact same parameter budget, enabling deployment on sequences of arbitrary and varying lengths.

2.3.2 Backpropagation through time (BPTT) formulation

Because the computational graph of an RNN unrolls across time into an interconnected chain of operations sharing identical parameters, computing the gradients of the training objective requires a specialized formulation of reverse-mode automatic differentiation termed *Backpropagation Through Time* (BPTT) [20].

Consider a training sequence of length T paired with target ground-truth one-hot vectors $\mathbf{y}_1^*, \ldots, \mathbf{y}_T^*$. At each time step t, the network outputs a predicted probability distribution $\hat{\mathbf{y}}_t$ and evaluates a token-level categorical cross-entropy loss:

$$\mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right) = -\sum_{k=1}^{d_y} y_{t,k}^* \log \hat{y}_{t,k}. \tag{2.16}$$

The total scalar sequence loss $\mathcal{L}$ is defined as the temporal summation over all token losses:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right). \tag{2.17}$$

To update the parameters via gradient descent, we must evaluate the partial derivatives of $\mathcal{L}$ with respect to the shared weight matrices: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}}$, and $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}}$.

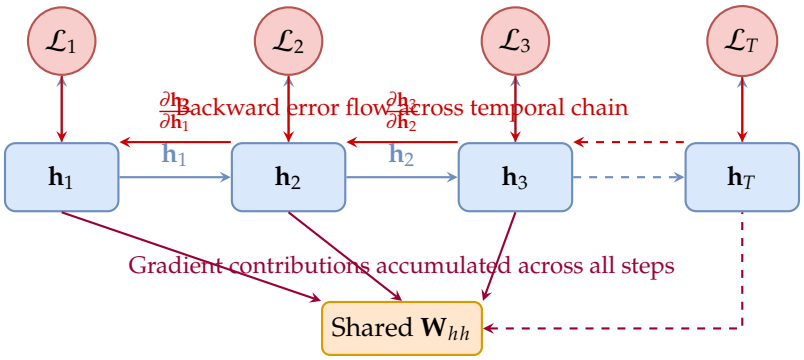


Figure 2.3: The computational flow of Backpropagation Through Time (BPTT). Forward activations propagate chronologically (blue arrows), generating per-step losses. Error signals propagate backward along the temporal chain (red arrows), and local gradient contributions are accumulated into the shared parameter tensor (purple arrows).

The output projection gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$ depends only on current state activations. By applying the standard cross-entropy softmax gradient derived in Chapter 1:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \left(\hat{\mathbf{y}}_t - \mathbf{y}_t^*\right) \mathbf{h}_t^\top. \tag{2.18}$$

In contrast, evaluating the gradient with respect to the recurrent matrix $\mathbf{W}_{hh}$ requires accounting for the multivariable chain rule across all ancestral steps. Because $\mathbf{W}_{hh}$ was used to compute $\mathbf{h}_t$, which in turn depended on $\mathbf{h}_{t-1}$, which depended on $\mathbf{h}_{t-2}$, a perturbation in $\mathbf{W}_{hh}$ affects the loss $\mathcal{L}_t$ through multiple distinct pathways across time.

Derivation of the recurrent gradient accumulation. To formalize this temporal dependence, let $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denote the *direct* partial derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$, treating antecedent state $\mathbf{h}_{k-1}$ as a constant:

$$\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{h}_{k-1}^\top, \tag{2.19}$$

where $\otimes$ denotes the tensor outer product. The total derivative of step loss $\mathcal{L}_t$ with respect to $\mathbf{W}_{hh}$ sums over all temporal paths from step $k = 1$ up to $k = t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{2.20}$$

where $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the temporal Jacobian matrix describing how hidden activations at step k propagate forward to influence hidden activations at step t.

Summing over all sequence time steps $t \in \{1, \ldots, T\}$, the complete gradient with respect to $\mathbf{W}_{hh}$ evaluates to:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}. \tag{2.21}$$

By identical reasoning, the gradient with respect to the input projection matrix $\mathbf{W}_{xh}$ accumulates across time as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}}, \quad \text{where} \quad \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{x}_k^\top. \tag{2.22}$$

Equation 2.21 embodies the core operational mechanism of BPTT: during training, the network first performs a forward pass from $t = 1$ to T, computing and caching all states $\mathbf{h}_t$ and

per-step losses $\mathcal{L}_t$. Subsequently, the backward pass propagates error vectors backward through time, accumulating gradient contributions from every historical step into the shared parameter tensors.

2.3.3 Empirical training dynamics: the Sherlock Holmes casing benchmark

To ground the mathematical formulation of BPTT in empirical practice, we analyze the training dynamics of a character-level Elman recurrent network tasked with sentence case correction. The experimental framework directly mirrors the diagnostic benchmark evaluated during course lectures.

Experimental setup. A vanilla Elman RNN is trained on textual excerpts drawn from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*. The input sequence consists of raw sentences converted entirely to lowercase, represented as a stream of 256-dimensional one-hot vectors corresponding to standard ASCII characters ($d_x = 256$). The network utilizes a hidden state dimension $d_h = 32$, initialized with $\mathbf{h}_0 = \mathbf{0}$, and projects to a 256-dimensional categorical output distribution ($d_y = 256$). Optimization is conducted over 10 epochs using the Adam optimizer [18] minimizing token-level cross-entropy loss.

The global training loss trajectory exhibits a characteristic progression: starting from an initial loss $\mathcal{L} \approx 3.0$, the loss drops to 2.0 after 2 epochs, reaches 1.72 at epoch 4, drops sharply to 0.95 at epoch 6, and asymptotically plateaus around 0.85 by epoch 10. Inspecting the qualitative outputs generated for a held-out test utterance reveals five distinct developmental stages of sequential pattern acquisition:

$$\text{Input text:} \quad \texttt{"it may be so, mr. holmes"} \tag{2.23}$$

Epoch	Loss $\mathcal{L}$	Predicted Character Sequence Output
0	3.00	`\x00 ...\x00` (repeated null padding bytes)
1	2.45	`eeee eeee eeee eeee eeee eeee`
3	1.72	`ie mae be so, mr.  holmes`
5	0.95	`"it may be so, mr.  holmes"`
7	0.88	`"It may be so, mr.  Holmes"`
10	0.84	`"It may be so, mr.  Holmes"`
Target	—	`"It may be so, Mr.  Holmes"`

Table 2.2: Empirical evolution of character-level casing predictions produced by a 32-dimensional Elman RNN across 10 training epochs on an excerpt from *The Adventures of Sherlock Holmes*. The progression illustrates the acquisition of padding bias, unigram frequency priors, whitespace structure, identity copying, and local punctuation-driven capitalization.

Deconstructing the progression recorded in Table 2.2 reveals how gradient descent navigates the sequential hypothesis space:

1. **Epoch 0 (Null-byte padding bias)**: The uninitialized model produces repeated null bytes (`\x00`). In training batches where variable-length strings are aligned using zero-padding, the null character exhibits high empirical prevalence. The optimizer initially exploits this zero-order marginal distribution to achieve trivial loss reduction.

2. **Epoch 1 (Unigram letter frequency prior)**: As backpropagation penalizes null outputs on non-padding tokens, the network discovers the marginal unigram frequency distribution of the English language. It repeatedly predicts the character `'e'`—the most statistically frequent English character—interleaved with periodic whitespace tokens.

3. **Epoch 3 (Whitespace and lexical boundaries)**: The model masters word boundary segmentation. It reliably predicts space characters following character clusters, establishing basic lexical cadence even while specific character identities remain noisy.

4. **Epoch 5 (Identity copying and the steep loss drop)**: The loss experiences a dramatic decline from 1.72 to 0.95. The recurrent network discovers the global structural invariant of the task: *most output characters are identical to their input counterparts*. By configuring the composite transformation $\mathbf{W}_{hy} \tanh(\mathbf{W}_{xh}\mathbf{x}_t) \approx \mathbf{x}_t$, the network implements an identity bypass, echoing the input string verbatim.

5. **Epoch 7 (Local syntactic rule discovery)**: The network learns fine-grained sequential rules. Having observed that opening quotation marks precede dialogue, the state $\mathbf{h}_t$ transitions to capitalize the subsequent character, transforming `"it` into `"It`. Furthermore, it learns that the character sequence `holmes` constitutes a proper noun, correctly capitalizing the `'H'` in `Holmes`.

The causal horizon failure. Despite reaching training convergence by Epoch 10, the model exhibits a persistent failure: it fails to capitalize the title `mr.`, leaving it as lowercase `mr. Holmes`. This failure exposes the fundamental theoretical limitation of strictly causal, one-to-one recurrent processing. When the network ingests the character `'m'` at time step $t = 15$, its decision to output `'M'` or `'m'` must be made strictly on the basis of antecedent observations $\mathbf{x}_{\leq 15}$ = `'"it may be so, '`. At this point in time, the network has not yet observed the upcoming characters `'r'`, `'.'`, or `'holmes'`. In general English prose, words beginning with the letter `'m'` (such as *may, much, more, my, must*) are lowercase in over 95% of occurrences. Because the causal one-to-one RNN is prohibited from observing future tokens, Bayes' optimal decision rule under cross-entropy forces the network to predict lowercase `'m'`.

Resolving this structural ambiguity requires an architecture that can inspect the entire sentence—including upcoming future tokens—before committing to an output classification.

2.4 Gradient Pathologies and Gated Units

While the Elman recurrent network resolves the dimensional and parameter-sharing limitations of feed-forward models, it suffers from severe analytical and computational pathologies that impede learning across long temporal spans.

2.4.1 The vanishing and exploding gradient problem

The primary mathematical pathology governing vanilla recurrent architectures is the vanishing and exploding gradient phenomenon [21, 22]. To understand its origin, we examine the temporal Jacobian matrix connecting hidden state $\mathbf{h}_k$ to hidden state $\mathbf{h}_t$ in Equation 2.21.

Applying the multivariate chain rule across $t-k$ intermediate recurrent steps yields a continuous product of Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{2.24}$$

Recalling the hidden state transition equation $\mathbf{h}_j = \tanh(\mathbf{W}_{xh}\mathbf{x}_j + \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h)$, the single-step temporal Jacobian evaluates to:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top, \tag{2.25}$$

where $\operatorname{diag}(1-\mathbf{h}_j^2) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the first derivatives of the element-wise hyperbolic tangent function, $\tanh'(a) = 1 - \tanh^2(a)$.

Substituting Equation 2.25 back into the chain product of Equation 2.24 produces:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top. \tag{2.26}$$

The matrix product in Equation 2.26 contains $t-k$ successive matrix multiplications of the transposed recurrent weight matrix $\mathbf{W}_{hh}^\top$, modulated at each step by the activation derivatives.

Let $\lambda_{\max}$ denote the largest eigenvalue (spectral radius) of the recurrent weight matrix $\mathbf{W}_{hh}$. Because the derivative of hyperbolic tangent is strictly bounded such that $0 < 1 - h_{j,m}^2 \leq 1$, the asymptotic magnitude of the temporal Jacobian is bounded by:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \left(\gamma \left\| \mathbf{W}_{hh}^\top \right\|\right)^{t-k}, \quad \text{where } \gamma \leq 1. \tag{2.27}$$

This analytical bound establishes two opposing failure modes:

1. **Vanishing gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| < 1$, or if hidden activations operate in saturating regimes where $1 - h_j^2 \ll 1$, the product decays exponentially toward zero as temporal distance $t-k$ increases:

$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = 0. \tag{2.28}$$

   To appreciate the speed of this decay, consider a sequence of moderate length where $t-k = 50$, and suppose the effective scaling factor is 0.90. Over 50 time steps, the gradient scaling factor diminishes to:

$$0.90^{50} \approx 0.00515, \tag{2.29}$$

attenuating the error signal by more than two orders of magnitude. For $t - k = 100$, the scaling factor drops to $0.90^{100} \approx 2.65 \times 10^{-5}$. Consequently, error signals computed at time step t exert virtually zero influence on parameter updates at distant antecedent steps $k \ll t$. The network becomes incapable of learning long-range temporal dependencies.

2. **Exploding gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| > 1$ and activations remain in the linear regime near zero ($\gamma \approx 1$), the matrix product grows exponentially with temporal distance:
$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \infty. \tag{2.30}$$
Exploding gradients cause parameter updates $\Delta\mathbf{W} = -\eta\nabla_{\mathbf{W}}\mathcal{L}$ to become astronomically large, catapulting parameters across the loss surface into regions of numerical overflow (`NaN`) and destroying previously acquired representations.

While exploding gradients can be mitigated using the heuristic engineering technique of *gradient clipping by global norm* [22]:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\|_2 \le v, \\ v\frac{\mathbf{g}}{\|\mathbf{g}\|_2} & \text{if } \|\mathbf{g}\|_2 > v, \end{cases} \tag{2.31}$$

where $v > 0$ defines the maximum allowable gradient norm, gradient clipping cannot resolve vanishing gradients. When error gradients vanish to machine precision zero, no simple rescaling can restore the lost directional signal.

2.4.2 Long-term dependency decay and computational sequentiality

The mathematical vanishing of gradients manifests behaviorally as the loss of long-term temporal memory. Because the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ has a strictly finite coordinate dimensionality, it must compress everything observed from step 1 to step t into a fixed-size vector. At each step, Equation 2.11 subjects the state to non-linear transformation and vector addition with incoming tokens:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right). \tag{2.32}$$

In practice, new observations continuously overwrite the coordinates of $\mathbf{h}_t$. Unless specific architectural constraints prevent overwriting, information concerning tokens observed dozens of steps in the past decays exponentially, leaving the hidden state dominated almost entirely by the most recent 5 to 10 tokens.

To illustrate this failure mode, consider the diagnostic sequence evaluated in lecture:

Prompt: `"The mouse was getting chased by the cat through the big house.` (2.33)

`After a long chase, the cat finally managed to catch the [ ]"` (2.34)

To correctly predict the target noun `"mouse"` at the sequence terminus, a language model must retain the identity of the original grammatical patient (`"mouse"`) across an intervening clause spanning more than 20 tokens. In a standard Elman RNN, the intervening tokens (*"through the big house. After a long chase, the cat finally managed to catch the"*) trigger repeated state transitions that overwrite the coordinates storing `"mouse"`. Consequently, when the model predicts the final token, its state reflects recent terms such as *"catch"*, prompting the generation of semantically plausible but contextually disconnected completions, such as `"ball"` or `"tail"`, while failing to recover the true referent.

Beyond memory degradation, vanilla RNNs suffer from a profound hardware inefficiency: *computational sequentiality*. By definition, evaluating state $\mathbf{h}_t$ requires antecedent state $\mathbf{h}_{t-1}$ as an

explicit mathematical input. Consequently, a recurrent neural network cannot compute step t until step $t-1$ has fully terminated:

$$\mathbf{h}_1 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \cdots \longrightarrow \mathbf{h}_T. \tag{2.35}$$

This temporal serialization creates a strict dependency chain along the sequence dimension, precluding the parallel evaluation of sequence tokens on modern massively parallel hardware accelerators (GPUs and TPUs). While batch instances can be processed in parallel across sequences, training runtime along the sequence dimension scales strictly as $O(T)$, severely limiting the feasibility of scaling vanilla RNNs to long document contexts.

2.4.3 Gated recurrent units: LSTM and GRU

To overcome the vanishing gradient pathology and prevent the continuous overwriting of historical context, Hochreiter and Schmidhuber introduced the *Long Short-Term Memory* (LSTM) network in 1997 [23].

The foundational architectural innovation of the LSTM is the decoupling of memory into two distinct operational streams:

1. **Cell state ($\mathbf{c}_t \in \mathbb{R}^{d_h}$)**: Functions as a dedicated, linear long-term memory trunk (colloquially termed the "conveyor belt").
2. **Hidden state ($\mathbf{h}_t \in \mathbb{R}^{d_h}$)**: Serves as the short-term, task-facing output representation used for immediate predictions and gate conditioning.

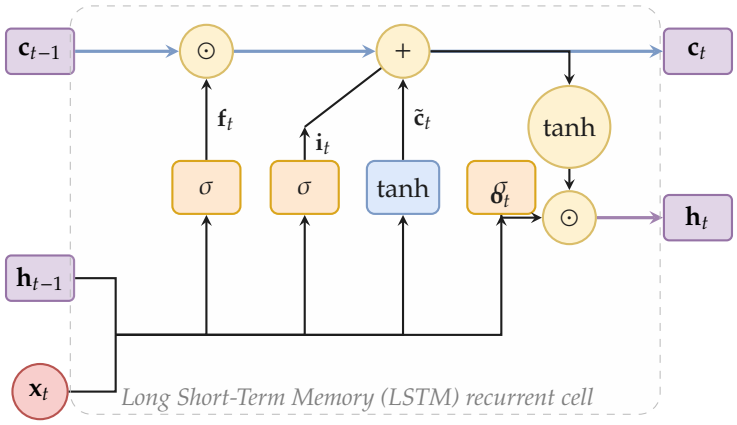


Figure 2.4: Detailed architectural schematic of the Long Short-Term Memory (LSTM) cell, illustrating the linear cell-state trunk ($\mathbf{c}_t$), the multiplicative forget gate ($\mathbf{f}_t$), input gate ($\mathbf{i}_t$), candidate state ($\tilde{\mathbf{c}}_t$), and output gate ($\mathbf{o}_t$).

The interaction between cell memory and incoming data is governed by three non-linear gates, parameterized using logistic sigmoid functions $\sigma(z) = (1 + e^{-z})^{-1} \in [0, 1]$. Because the sigmoid output ranges strictly between 0 and 1, gate activations act as coordinate-wise attenuation coefficients: a value of 0 blocks information transmission entirely, while a value of 1 permits complete transmission.

The algebraic execution of the LSTM unfolds in five sequential stages:

1. **Forget gate**: Determines what fraction of the antecedent cell state $\mathbf{c}_{t-1}$ should be discarded:

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_f\right) \in [0, 1]^{d_h}, \tag{2.36}$$

where $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_x}$ denotes vector concatenation, $\mathbf{W}_f \in \mathbb{R}^{d_h \times (d_h + d_x)}$ is the forget weight matrix, and $\mathbf{b}_f \in \mathbb{R}^{d_h}$ is the forget bias.

2. **Input gate and candidate memory**: Regulates what new information should be added to the persistent cell state:

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_i\right) \in [0,1]^{d_h}, \tag{2.37}$$

$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{2.38}$$

   where $\mathbf{i}_t$ controls write access, and $\tilde{\mathbf{c}}_t$ creates a new candidate vector of bounded additions.

3. **Cell state update (The Constant Error Carousel)**: The new cell state $\mathbf{c}_t$ is computed via linear combination:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{2.39}$$

   where $\odot$ denotes element-wise (Hadamard) vector multiplication.

4. **Output gate and hidden state**: Computes the visible output state by filtering the updated cell state:

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_o\right) \in [0,1]^{d_h}, \tag{2.40}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right) \in (-1,1)^{d_h}. \tag{2.41}$$

Proof of the Constant Error Carousel (CEC). To understand why the LSTM eliminates the vanishing gradient pathology, we differentiate the cell state update (Equation 2.39) with respect to the antecedent cell state $\mathbf{c}_{t-1}$:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}\left(\mathbf{f}_t\right) + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \odot \mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \odot \tilde{\mathbf{c}}_t + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}. \tag{2.42}$$

Along the primary diagonal path, the gradient is dominated by the direct linear transmission term:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \operatorname{diag}\left(\mathbf{f}_t\right). \tag{2.43}$$

Across an extended temporal interval from step k to step t, the accumulated error gradient evaluates to:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(\mathbf{f}_j\right). \tag{2.44}$$

Crucially, Equation 2.44 does not involve repeated multiplications of a weight matrix $\mathbf{W}_{hh}^{\top}$, nor does it involve saturating activation derivatives $\tanh'$. If the network learns to saturate the forget gates near one ($\mathbf{f}_j \approx \mathbf{1}$), the gradient product evaluates to the identity matrix:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \mathbf{I} = \mathbf{I}. \tag{2.45}$$

Under this condition, error gradients propagate backward across arbitrary temporal distances without exponential attenuation, creating a constant error highway (the Constant Error Carousel) that enables the acquisition of long-range dependencies spanning hundreds of steps.

The Gated Recurrent Unit (GRU). In 2014, Cho et al. introduced the Gated Recurrent Unit [24], offering a streamlined alternative to the LSTM that merges cell state and hidden state into a single representation $\mathbf{h}_t$, controlled by two gates:

$$\text{Reset gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_r\right) \in [0,1]^{d_h}, \tag{2.46}$$

$$\text{Update gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_z\right) \in [0,1]^{d_h}, \tag{2.47}$$

$$\text{Candidate state:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h\left[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_h\right) \in (-1,1)^{d_h}, \tag{2.48}$$

$$\text{State interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{2.49}$$

The update gate $\mathbf{z}_t$ acts as an adaptive convex interpolator between past memory $\mathbf{h}_{t-1}$ and candidate update $\tilde{\mathbf{h}}_t$, performing the combined roles of the LSTM's forget and input gates while reducing parameter count by approximately 25%.

Architectural Property	**Vanilla Elman RNN**	**LSTM**	**GRU**
Separate cell memory ($\mathbf{c}_t$)	No	Yes	No
Number of gating sub-units	0	3 ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$)	2 ($\mathbf{r}_t, \mathbf{z}_t$)
Parameter count scaling	$O\left(d_h^2 + d_h d_x\right)$	$O\left(4(d_h^2 + d_h d_x)\right)$	$O\left(3(d_h^2 + d_h d_x)\right)$
Gradient propagation highway	None (exponential decay)	Linear additive trunk	Linear convex interpolation
Computational parallelizability	Strictly serialized	Strictly serialized	Strictly serialized

Table 2.3: Architectural comparison between the vanilla Elman RNN, the Long Short-Term Memory (LSTM) network, and the Gated Recurrent Unit (GRU).

As summarized in Table 2.3, while gated architectures resolve the analytical vanishing gradient problem through additive linear memory trunks, they remain fundamentally serialized along the sequence dimension, preserving the computational bottleneck that prevents efficient parallel scaling.

2.5 Sequence-to-Sequence and Encoder-Decoder Models

All recurrent configurations examined thus far operate under a *one-to-one mapping paradigm*, where each incoming token $\mathbf{x}_t$ immediately triggers the emission of a corresponding prediction $\hat{\mathbf{y}}_t$. While suitable for synchronous sequence tagging, this formulation fails when applied to general sequence-to-sequence transformations.

2.5.1 Limitations of one-to-one sequence mapping

Synchronous one-to-one sequence processing exhibits two fatal structural failure modes when applied to complex linguistic tasks such as machine translation, summarization, or dialogue:

1. **Sequence length disparity ($T_x \neq T_y$)**: In natural language translation, source and target sentences rarely contain the same number of words. Consider translating an English utterance into Italian:

$$\text{English source:} \quad \texttt{[It] [is] [indeed] [sunny] [today]} \quad (T_x = 5 \text{ tokens}), \tag{2.50}$$

$$\text{Italian target:} \quad \texttt{[Effettivamente] [oggi] [è] [soleggiato]} \quad (T_y = 4 \text{ tokens}). \tag{2.51}$$

   Under a one-to-one mapping architecture, emitting 4 Italian words requires ingesting exactly 4 English tokens, leaving the 5th token unaligned. If the target sequence is longer than the source ($T_y > T_x$), the recurrent unit exhausts its input tokens and has no valid data to ingest for subsequent steps.

2. **Non-monotonic alignment and lack of future context**: Even if sequence lengths happen to match, the semantic alignment between languages is non-monotonic and structurally inverted. In the example above, the initial Italian word `"Effettivamente"` translates the English adverb `"indeed"`, which occupies the *third* position in the source sentence. Under causal one-to-one processing, the network must generate the first output token after observing only `"It"`, long before the conditioning token `"indeed"` has entered the model.

Table 2.4 illustrates this structural misalignment, demonstrating why causal synchronous emission cannot resolve cross-lingual translation.

Step t	Source Token $\mathbf{x}_t$	Target Token y_t^*	Semantic Alignment	Causal Feasibility
1	`It`	`Effettivamente`	Maps to $\mathbf{x}_3$ (`indeed`)	**No** (future context required)
2	`is`	`oggi`	Maps to $\mathbf{x}_5$ (`today`)	**No** (future context required)
3	`indeed`	`è`	Maps to $\mathbf{x}_2$ (`is`)	**No** (inverted syntax)
4	`sunny`	`soleggiato`	Maps to $\mathbf{x}_4$ (`sunny`)	Partially (local match)
5	`today`	$\emptyset$	No target token	**No** (length mismatch)

Table 2.4: Cross-lingual sequence alignment between English source and Italian target sentences, illustrating the failure of causal one-to-one recurrent mapping due to non-monotonic word order and length disparity.

2.5.2 The encoder-decoder paradigm

To resolve the constraints of one-to-one mapping, Sutskever et al. [25] and Cho et al. [24] decoupled sequence processing into two specialized recurrent networks: an *Encoder* and a *Decoder*.

The operational workflow of the Seq2Seq architecture (Figure 2.5) decouples sequence encoding from generation through three distinct phases:

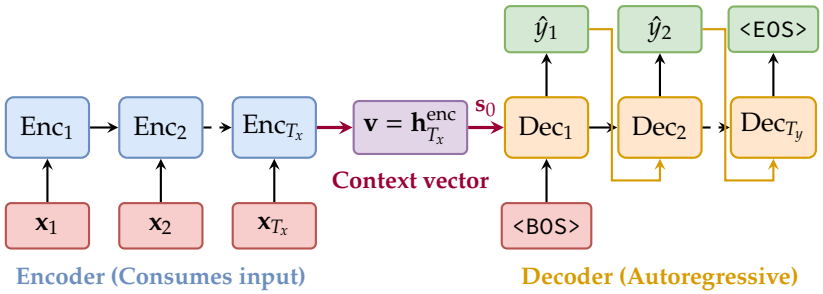


Figure 2.5: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder ingests the full source sequence of length T_x, compressing it into a single context vector $\mathbf{v}$. The Decoder is initialized with $\mathbf{v}$ and generates target tokens autoregressively until emitting the end-of-sequence delimiter <EOS>.

1. **Sequence encoding**: The Encoder RNN sequentially ingests the entire source sequence $\mathbf{x}_1, \ldots, \mathbf{x}_{T_x}$ without generating any output tokens. At the final source token T_x, the encoder's terminal hidden state is extracted to serve as the *context vector*:

$$\mathbf{v} = \mathbf{h}^{\text{enc}}_{T_x} \in \mathbb{R}^{d_h}. \tag{2.52}$$

   The vector $\mathbf{v}$ represents a compressed semantic summary of the entire source utterance.

2. **Decoder state initialization**: The Decoder RNN initializes its internal state using the context vector:

$$\mathbf{s}_0 = \mathbf{v}, \tag{2.53}$$

   thereby conditioning all subsequent generation steps on the global context of the input.

3. **Autoregressive generation**: Generation commences by feeding a synthetic start-of-sequence token, denoted <BOS> (Beginning of Sequence), into the decoder:

$$\mathbf{s}_1 = \text{DecoderRNN}\left(\mathbf{s}_0, \mathbf{e}_{\texttt{<BOS>}}\right). \tag{2.54}$$

   The decoder projects state $\mathbf{s}_1$ to emit a categorical probability distribution over target vocabulary $\mathcal{V}_y$, sampling or greedily selecting token $\hat{y}_1$. In the subsequent step, the previously generated token $\hat{y}_1$ is fed back as the input token to compute $\mathbf{s}_2$:

$$\mathbf{s}_t = \text{DecoderRNN}\left(\mathbf{s}_{t-1}, \mathbf{e}_{\hat{y}_{t-1}}\right). \tag{2.55}$$

   Generation terminates when the decoder emits a dedicated end-of-sequence token, denoted <EOS> (End of Sequence), allowing target length T_y to be determined dynamically by the model.

Mathematically, the encoder-decoder framework models the conditional probability of target sequence $Y = (y_1, \ldots, y_{T_y})$ given source sequence $X = (\mathbf{x}_1, \ldots, \mathbf{x}_{T_x})$ via the exact probability chain rule:

$$P(Y \mid X) = \prod_{t=1}^{T_y} P\left(y_t \mid y_1, \ldots, y_{t-1}, \mathbf{v}\right). \tag{2.56}$$

2.5.3 Empirical validation: revised Sherlock Holmes casing

To evaluate how the encoder-decoder paradigm resolves the causal horizon failure identified in Section 2.3.3, we revisit the character casing benchmark on the ambiguous test utterance:

$$\text{Input text:} \quad \texttt{"indeed iam sherlock holmes,"} \tag{2.57}$$

$$\text{Target text:} \quad \texttt{"Indeed I am Sherlock Holmes,"} \tag{2.58}$$

Under the one-to-one causal RNN evaluated in Section 2.3.3, the model committed four distinct capitalization errors:

1. **The initial token `'i'` in `indeed`**: Capitalized correctly only because it immediately followed an opening quote.
2. **The isolated pronoun `'i'` in `iam`**: Remained lowercase (`'i'`) because the causal model, seeing only antecedent tokens `"indeed "`, could not determine whether `'i'` represented the personal pronoun *I* or the initial letter of common words such as *in*, *is*, *it*, *if*.
3. **The initial `'s'` in `sherlock`**: Remained lowercase because `"iam "` is followed by lowercase words in over 90% of training contexts.
4. **The initial `'h'` in `holmes`**: Capitalized correctly only through local bigram memorization following `sherlock`.

When the exact same sequence is evaluated using a Sequence-to-Sequence Encoder-Decoder architecture, the model resolves all four tokens without ambiguity:

$$\text{Seq2Seq Output:} \quad \texttt{"Indeed I am Sherlock Holmes,"} \tag{2.59}$$

The conceptual basis for this success resides in the timing of representation formation. In the Seq2Seq paradigm, the Encoder ingests the entire sentence `"indeed iam sherlock holmes,"` before the Decoder generates a single character. When the Decoder initiates generation from $\mathbf{s}_0 = \mathbf{v}$, the context vector $\mathbf{v}$ already encodes the presence of the proper noun `sherlock holmes` and the pronoun `i`. With full bidirectional information synthesized into the context vector, the decoder generates correctly capitalized characters from the very first token, resolving ambiguities that are mathematically unsolvable under causal one-to-one processing.

2.5.4 The information bottleneck and the prelude to attention

Despite the empirical triumph of the Encoder-Decoder paradigm on short sentences, the architecture encounters an inescapable theoretical barrier: the *fixed-vector information bottleneck* [26].

In the standard Seq2Seq formulation, the Encoder must compress all semantic, syntactic, stylistic, and relational information present in a source sequence of arbitrary length T_x into a single, fixed-dimensional Euclidean vector $\mathbf{v} \in \mathbb{R}^{d_h}$. Mathematically, this transformation represents a severe dimensionality reduction mapping:

$$f_{\text{enc}} : \mathbb{R}^{T_x \times d_x} \longrightarrow \mathbb{R}^{d_h}. \tag{2.60}$$

For a short sentence of 5 words, compressing the input into $d_h = 512$ dimensions preserves sufficient fidelity for accurate reconstruction. However, as sequence length increases to 20, 50, or 100 words, the amount of information in the source sequence grows linearly with T_x, while the capacity of the context vector $\mathbf{v}$ remains strictly constant at d_h.

Consequently, the context vector becomes saturated. As the encoder processes later words in a long document, earlier clauses are progressively overwritten and attenuated by the recurrent transition operator. When the decoder begins generating target tokens, the context vector $\mathbf{v}$ retains vivid representations of the final few source tokens, but retains only degraded, blurred traces of earlier tokens.

Empirical evaluations by Cho et al. [24] and Bahdanau et al. [26] demonstrated that the translation performance of standard RNN encoder-decoder models plummets catastrophically when sentence length exceeds 15 to 20 words. Even when employing multi-layer LSTMs with thousands of hidden units, compressing an arbitrary-length document into a single static vector remains an insurmountable information-theoretic bottleneck.

We are therefore confronted with the dual architectural crises of recurrent sequence modeling:

1. **The computational crisis**: The strict sequential recurrence $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ serializes training along the sequence dimension, precluding GPU hardware parallelization and enforcing an $O(T)$ training wall-clock runtime.
2. **The representational crisis**: Compressing variable-length context into a single static state vector $\mathbf{v} \in \mathbb{R}^{d_h}$ destroys long-range dependencies and forces decoders to reconstruct sentences from lossy, blurred summaries.

Resolving these dual crises requires a fundamental paradigm shift: replacing static vector compression with dynamic, query-conditioned associative retrieval. Rather than forcing a single vector to summarize an entire sequence, the model should maintain a collection of all intermediate encoder states and allow the decoder to dynamically "attend" to the most relevant source positions at each step of generation. This foundational breakthrough—the Attention Mechanism—forms the conceptual bridge that dismantles the recurrent paradigm and underpins the modern Transformer architecture.

Chapter 3

Language Modeling Fundamentals

The preceding chapters established the computational foundations of deep learning and analyzed recurrent neural networks as dynamical systems capable of processing variable-length sequences. However, before deploying neural architectures to process or generate human text, a foundational theoretical question must be resolved: what does it mean, in formal mathematical terms, to model a natural language? Rather than treating text as arbitrary strings of alphanumeric characters or hand-crafted symbolic parse trees, modern computational linguistics conceptualizes language as a discrete stochastic process governed by an underlying, non-uniform probability distribution over sequential utterances.

The goal of language modeling is to formulate computational mechanisms capable of distinguishing between probable, semantically coherent utterances and implausible, grammatically corrupted sequences. Historically, this objective originated in the pioneering work of Claude Shannon and the engineering impasses of early speech recognition, where deterministic grammars proved fundamentally inadequate to capture the pervasive ambiguity and continuous variability of natural human expression. Early statistical approaches addressed this challenge through Markovian assumptions and discrete frequency estimation over local n-gram tuples, establishing the foundational paradigms of maximum likelihood training, likelihood evaluation, and autoregressive generation.

This chapter develops the theoretical, statistical, and algorithmic principles of language modeling. We begin by examining the historical imperatives that motivated probabilistic language modeling and formalize the joint distribution over word sequences via the probability chain rule. We then formulate the Markov assumption and n-gram models, trace parameter estimation via Maximum Likelihood Estimation on a concrete corpus, and systematically analyze the dual inference modes: sequence likelihood evaluation and autoregressive sequence generation. Finally, we dissect the fundamental limitations of statistical counting models—specifically combinatorial data sparsity, context horizon truncation, and the lack of semantic generalization—examine the smoothing mechanism as a mitigation strategy, and establish the structural necessity of transitioning to continuous distributed word embeddings.

3.1 Probabilistic Language Modeling

Understanding modern foundation architectures requires first examining how natural language was formalized as a statistical object. The formulation of language modeling emerged not from abstract linguistic taxonomy, but from concrete engineering challenges in telecommunications and signal processing that required machines to quantify the plausibility of textual hypotheses.

3.1.1 Historical motivation: the urge to model natural language

The mathematical formalization of language modeling originated in Claude Shannon's landmark 1948 treatise, *A Mathematical Theory of Communication* [27]. Shannon sought to quantify the fundamental limits of data compression and transmission across noisy telecommunication channels. Rather than viewing text as static philosophical prose, Shannon conceptualized natural language as a stochastic process generating discrete sequences of symbols chosen from a finite alphabet according to stationary statistical probabilities. By measuring the entropy of printed English through letter-level and word-level approximation experiments, Shannon demonstrated that natural text possesses substantial statistical redundancy, implying that preceding context exerts powerful probabilistic constraints on upcoming symbols.

In the decades that followed, the imperative to build automated language models transitioned from theoretical information theory to critical real-world signal processing domains:

1. **Acoustic speech recognition (ASR)**: In automated speech transcription, an input acoustic waveform must be mapped to a written sequence of words. However, human phonetics exhibits pervasive phonetic ambiguity: homophones and continuous connected speech yield acoustic signals that correspond equally well to multiple disparate word sequences. For example, the acoustic sequence corresponding to the phonetic sequence `/rek-uhg-nyze-beech/` can be transcribed as either *"recognize speech"* or *"wreck a nice beach"*. An acoustic sensor analyzing frequency spectra alone cannot reliably resolve this ambiguity. To achieve human-level accuracy, an ASR pipeline must factorize the problem into two complementary components:

$$W^* = \arg\max_W P(W \mid A) = \arg\max_W \frac{P(A \mid W)P(W)}{P(A)} = \arg\max_W P(A \mid W)P(W), \tag{3.1}$$

   where $P(A \mid W)$ denotes the acoustic likelihood model (how likely the sound A is given words W), and $P(W)$ represents the language model prior. Because the phrase *"recognize speech"* occurs frequently in technical discourse while *"wreck a nice beach"* is exceptionally rare, the prior enforces $P(\text{"recognize speech"}) \gg P(\text{"wreck a nice beach"})$, allowing the system to effortlessly resolve phonetic noise.

2. **Machine translation (MT) and optical character recognition (OCR)**: In machine translation, transferring syntactic structures between disparate languages rarely proceeds via word-for-word substitution. An automated translator generating candidate lexical mappings requires a statistical prior over the target language to select phrase orderings that sound natural and grammatically fluent. Similarly, in optical character recognition, noisy visual scans that render ambiguous characters (such as distinguishing `'l'`, `'1'`, and `'I'`) are disambiguated by evaluating which lexical candidate forms a coherent word sequence within the sentence.

These engineering requirements triggered a philosophical pivot in computational linguistics. Early artificial intelligence research, guided by Chomskyan generative grammars [28], attempted to model natural language through deterministic, handcrafted formal rule sets that partitioned sentences strictly into grammatically valid or invalid categories. However, human language constantly breaks rigid grammatical rules through idioms, colloquialisms, poetry, and colloquial

speech while remaining completely intelligible. Statistical language modeling replaced this binary grammaticality dichotomy with a continuous, probabilistic spectrum of plausibility.

3.1.2 The language model task: formal probability distribution

Formally, let $\mathcal{V}$ denote a finite vocabulary comprising all discrete token primitives recognized by the system, with cardinality $|\mathcal{V}|$. An arbitrary natural language utterance W is defined as an ordered tuple of T tokens:

$$W = (w_1, w_2, \ldots, w_T), \quad w_t \in \mathcal{V}, \quad T \in \mathbb{N}^+. \tag{3.2}$$

The set of all possible finite sequences formed from vocabulary $\mathcal{V}$ is given by the Kleene closure $\mathcal{V}^* = \bigcup_{k=1}^{\infty} \mathcal{V}^k$.

A *Language Model* (LM) is a mathematical function that assigns a joint probability $P(W)$ to every possible sequence $W \in \mathcal{V}^*$, satisfying the standard Kolmogorov probability axioms:

$$P(W) \geq 0 \quad \forall W \in \mathcal{V}^*, \quad \text{and} \quad \sum_{W \in \mathcal{V}^*} P(W) = 1. \tag{3.3}$$

The probability distribution $P(W)$ can be conceptualized as defining a continuous density landscape across the discrete space of possible word combinations. In this space:

- **High-probability regions**: Encompass plausible, fluent, and syntactically well-formed sentences that frequently occur in human communication:

$$P(\texttt{"This is a reasonable sentence"}) \approx 10^{-1}. \tag{3.4}$$

- **Low-probability regions**: Encompass syntactically well-formed but semantically anomalous or bizarre sequences that violate common semantic expectations:

$$P(\texttt{"This is a purple sentence"}) \approx 10^{-3}. \tag{3.5}$$

- **Near-zero probability regions**: Encompass ungrammatical, disjoint, or corrupted strings of symbols that violate the structural syntax of the language:

$$P(\texttt{"This are a reasonable sentence"}) \approx 10^{-5}. \tag{3.6}$$

Because the combinatorial space of arbitrary symbol permutations is astronomically large, the overwhelming majority of sequences in $\mathcal{V}^*$ are ungrammatical noise. Consequently, natural human language occupies an extremely narrow, low-dimensional probability manifold within $\mathcal{V}^*$. The fundamental objective of language modeling is to accurately estimate this non-uniform probability distribution from finite empirical observations.

3.1.3 Factoring joint probabilities via the chain rule

Directly estimating the joint probability $P(w_1, w_2, \ldots, w_T)$ over an entire sentence simultaneously is computationally intractable because the number of possible complete sentences grows exponentially with sequence length T. To render this estimation feasible, we apply the general *chain rule of probability* (or Kolmogorov multiplication law), which factors any joint distribution into a product of conditional distributions:

$$\begin{aligned} P(W) &= P(w_1, w_2, \ldots, w_T) \\ &= P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1, w_2) \cdots P(w_T \mid w_1, \ldots, w_{T-1}) \\ &= \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{T} P(w_t \mid w_{<t}), \end{aligned} \tag{3.7}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ denotes the sequence prefix comprising all tokens preceding step t, with the base case defined as $P(w_1 \mid w_{<1}) = P(w_1)$.

Equation (3.7) provides a profound conceptual and operational shift. It demonstrates that the problem of assigning a joint probability to an entire sentence is mathematically identical to the sequential task of predicting the conditional probability distribution of the single next word w_t given the observed historical context $w_{<t}$.

This formulation directly aligns language modeling with the classical *Cloze* (fill-in-the-blank) evaluation methodology in cognitive linguistics:

$$\text{Prompt 1:} \quad \texttt{"This is a reasonable [ ]"} \implies P\left(w_t = \texttt{"sentence"} \mid w_{<t}\right), \tag{3.8}$$

$$\text{Prompt 2:} \quad \texttt{"The black cat sat on the [ ]"} \implies P\left(w_t = \texttt{"mat"} \mid w_{<t}\right). \tag{3.9}$$

By framing language modeling as sequential next-token prediction, we can construct models that process text incrementally from left to right, evaluating sentence plausibility and generating synthetic continuations through successive conditional probability evaluations.

3.2 The Markov assumption, n-gram models, and operational inference

While the exact chain rule decomposition in Equation (3.7) provides a mathematically sound probabilistic factorization, it cannot be implemented directly in practice using raw tabular frequencies. As the sequence length t increases, the conditioning history $w_{<t} = (w_1, \ldots, w_{t-1})$ expands monotonically, rendering each historical context effectively unique. To construct tractable statistical models from empirical corpora, we must introduce principled independence assumptions that truncate the conditioning context.

3.2.1 The Markov assumption and combinatorial state-space reduction

To understand why an unconstrained history is computationally intractable, consider tracking the exact sequence of preceding tokens. At position t, the conditioning history $h_t = (w_1, \ldots, w_{t-1})$ belongs to the Cartesian product $\mathcal{V}^{t-1}$. The cardinality of the conditioning state space at step t evaluates to:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1}. \tag{3.10}$$

For a modest vocabulary of $|\mathcal{V}| = 10{,}000$ words, tracking histories of length $t = 4$ requires estimating probabilities across $10{,}000^3 = 10^{12}$ distinct conditioning states. For a realistic vocabulary of $|\mathcal{V}| = 50{,}000$ at step $t = 10$, the state space expands to $|\mathcal{V}|^9 \approx 1.95 \times 10^{42}$ states. Because no empirical text corpus contains more than 10^{10} to 10^{12} total tokens, the vast majority of prefixes will occur zero times in training data, rendering empirical frequency ratios completely undefined.

To make parameter estimation mathematically tractable, statistical language modeling applies the *Markov assumption* [29]. In his foundational 1948 treatise on information theory [27], Claude Shannon demonstrated that modeling natural language as a discrete stochastic process with bounded memory yields increasingly coherent approximations as the conditioning order increases.

Definition 3.1: *Markov assumption of order $n-1$*

An n-gram language model posits that the conditional probability of the current word w_t depends exclusively on the immediately preceding $n-1$ words, remaining conditionally independent of all earlier discourse history:

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{3.11}$$

The integer $n \geq 1$ dictates the order of the statistical model:

- **Unigram model ($n = 1$, Order 0)**: Completely memoryless. Each word is treated as statistically independent of past and future tokens:

$$P(w_1, \ldots, w_T) \approx \prod_{t=1}^{T} P(w_t). \tag{3.12}$$

  A unigram model ignores all syntactic structure and word order, treating the sequence as an unordered bag of words.

- **Bigram model ($n = 2$, Order 1)**: First-order Markov chain. Conditioning depends exclusively on the single preceding token w_{t-1}:

$$P(w_1, \ldots, w_T) \approx P(w_1) \prod_{t=2}^{T} P(w_t \mid w_{t-1}). \tag{3.13}$$

- **Trigram model ($n = 3$, Order 2)**: Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$P(w_1, \ldots, w_T) \approx P(w_1) P(w_2 \mid w_1) \prod_{t=3}^{T} P(w_t \mid w_{t-2}, w_{t-1}). \tag{3.14}$$

The primary mathematical virtue of the Markov assumption is that it collapses an exponentially expanding state space into a stationary cardinality that remains constant across all sequence positions t. Under an n-gram model, the conditioning context is always restricted to $n-1$ words, yielding a fixed state-space cardinality:

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1}. \tag{3.15}$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires a conditioning table containing:

$$N_{\text{parameters}} = |\mathcal{V}|^{n-1} \times (|\mathcal{V}| - 1) \approx |\mathcal{V}|^n, \tag{3.16}$$

free parameters. Table 3.1 contrasts the combinatorial explosion of the unconstrained historical state space with fixed n-gram Markov orders for representative vocabulary sizes.

Model / Order	Effective Context	Formula	States ($\lvert\mathcal{V}\rvert = 10^4$)	States ($\lvert\mathcal{V}\rvert = 5 \times 10^4$)
Unconstrained ($t = 2$)	1 previous token	$\lvert\mathcal{V}\rvert^1$	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	$\lvert\mathcal{V}\rvert^2$	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	$\lvert\mathcal{V}\rvert^4$	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	$\lvert\mathcal{V}\rvert^9$	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	$\lvert\mathcal{V}\rvert^{19}$	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$\lvert\mathcal{V}\rvert^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	$\lvert\mathcal{V}\rvert^1$	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	$\lvert\mathcal{V}\rvert^2$	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	$\lvert\mathcal{V}\rvert^3$	1.0×10^{12}	1.25×10^{14}

Table 3.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t in unconstrained conditioning histories versus fixed n-gram Markov orders for vocabularies $|\mathcal{V}| = 10{,}000$ and $|\mathcal{V}| = 50{,}000$.

As demonstrated in Table 3.1, while the unconstrained state space exceeds 10^{76} states by position $t = 20$ (surpassing the total number of atoms in the observable universe), a bigram model maintains a fixed, stationary state space of 50,000 conditioning rows. Truncating the context horizon makes statistical modeling computationally viable, enabling direct parameter estimation from observed text counts.

3.2.2 The worked transition matrix and parameter estimation

By truncating the conditioning horizon to a small constant n, the parameters of an n-gram model can be estimated directly from a reference corpus using *Maximum Likelihood Estimation* (MLE). Under MLE, the parameter values are chosen to maximize the likelihood of the observed training text, which equates to computing empirical frequency ratios.

For a bigram model ($n = 2$), the conditional probability of token w_t given predecessor w_{t-1} is evaluated by counting the joint occurrences of the bigram (w_{t-1}, w_t) and dividing by the

unigram frequency of the prefix w_{t-1}:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)} = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}, \tag{3.17}$$

where $C(\cdot)$ denotes the count of exact occurrences in the training corpus.

Worked pedagogical example. To ground this formulation in concrete numbers, we analyze the exact toy corpus evaluated during course lectures. Consider a tiny reference corpus C consisting of two sentences:

$$\text{Sentence 1:} \quad \texttt{"The cat chased the mouse happily."} \tag{3.18}$$

$$\text{Sentence 2:} \quad \texttt{"The mouse ate the cheese."} \tag{3.19}$$

Extracting all unique lexical tokens yields a vocabulary of cardinality $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\texttt{ate}, \texttt{cat}, \texttt{chased}, \texttt{cheese}, \texttt{happily}, \texttt{mouse}, \texttt{the}\}. \tag{3.20}$$

We construct the empirical transition matrix by scanning all adjacent bigram pairs across the corpus. Scanning Sentence 1 yields the bigram sequence:

$$(\texttt{the}, \texttt{cat}),\ (\texttt{cat}, \texttt{chased}),\ (\texttt{chased}, \texttt{the}),\ (\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{happily}). \tag{3.21}$$

Scanning Sentence 2 yields:

$$(\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{ate}),\ (\texttt{ate}, \texttt{the}),\ (\texttt{the}, \texttt{cheese}). \tag{3.22}$$

Summing these occurrences yields the absolute frequency matrix $C(w_{t-1}, w_t)$. Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w_t \in \mathcal{V}} P(w_t \mid w_{t-1}) = 1.0$ for all non-terminal rows, resulting in the complete bigram transition matrix presented in Table 3.2.

$w_{t-1} \setminus w_t$	**ate**	**cat**	**chased**	**cheese**	**happily**	**mouse**	**the**
ate	0	0	0	0	0	0	1.00 (1/1)
cat	0	0	1.00 (1/1)	0	0	0	0
chased	0	0	0	0	0	0	1.00 (1/1)
cheese	0	0	0	0	0	0	0
happily	0	0	0	0	0	0	0
mouse	0.50 (1/2)	0	0	0	0.50 (1/2)	0	0
the	0	0.25 (1/4)	0	0.25 (1/4)	0	0.50 (2/4)	0

Table 3.2: Empirical Maximum Likelihood Estimation (MLE) bigram transition probability matrix computed from the lecture reference corpus. Each entry (i, j) represents the conditional probability $P(w_j \mid w_i) = C(w_i, w_j)/C(w_i)$. Rows for terminal words (`cheese`, `happily`) sum to zero in the absence of explicit sentence boundary delimiters.

Inspecting Table 3.2 illustrates the statistical profile learned by the model:

- Following the word `"the"`, the model distributes probability mass across three nouns: $P(\texttt{mouse} \mid \texttt{the}) = 0.50$, $P(\texttt{cat} \mid \texttt{the}) = 0.25$, and $P(\texttt{cheese} \mid \texttt{the}) = 0.25$.
- Following the word `"mouse"`, the model assigns equal probability to the verb `"ate"` and the adverb `"happily"`: $P(\texttt{ate} \mid \texttt{mouse}) = 0.50$, $P(\texttt{happily} \mid \texttt{mouse}) = 0.50$.
- Deterministic transitions emerge where a word was observed with only a single successor: $P(\texttt{chased} \mid \texttt{cat}) = 1.00$, $P(\texttt{the} \mid \texttt{chased}) = 1.00$, and $P(\texttt{the} \mid \texttt{ate}) = 1.00$.

Once the transition matrix is populated and normalized, it functions as a complete, trained statistical language model. We can deploy this model in two fundamental operational modes: sequence likelihood evaluation and autoregressive text generation.

3.2.3 Inference: evaluating sentence likelihood

The first core inference task consists in evaluating the likelihood of a candidate word sequence. Given an unobserved test sentence, the language model computes its joint probability by taking the product of the corresponding transition probabilities from Table 3.2.

Worked numerical evaluation. Suppose we deploy the model to evaluate the probability of a candidate sentence that did not appear in the training corpus:

$$W_{\text{test}} = \texttt{"The cat chased the cheese."} \tag{3.23}$$

Applying the bigram factorization from Equation (3.13):

$$P\left(W_{\text{test}}\right) = P\left(\texttt{the}\right) \cdot P\left(\texttt{cat} \mid \texttt{the}\right) \cdot P\left(\texttt{chased} \mid \texttt{cat}\right) \cdot P\left(\texttt{the} \mid \texttt{chased}\right) \cdot P\left(\texttt{cheese} \mid \texttt{the}\right). \tag{3.24}$$

Assuming the unigram prior for the sentence-initial token is $P(\texttt{the}) = 1.0$ (since all training sentences in corpus C begin with `"The"`), we substitute the empirical conditional probabilities from Table 3.2:

$$\begin{aligned} P\left(W_{\text{test}}\right) &= 1.0 \times \frac{1}{4} \times 1.0 \times 1.0 \times \frac{1}{4} \\ &= 1.0 \times 0.25 \times 1.0 \times 1.0 \times 0.25 = \frac{1}{16} = 0.0625. \end{aligned} \tag{3.25}$$

The model assigns a non-zero probability ($P = 0.0625$) to *"The cat chased the cheese"*. Crucially, this demonstrates that statistical language models can assign valid probability mass to novel combinations of words that were never observed verbatim during training, provided that all constituent transitions exist in the learned transition graph.

Numerical stability via log-likelihood. In practical applications involving sentences of realistic length ($T \geq 20$), multiplying dozens of conditional probabilities bounded within $[0, 1]$ leads directly to numerical underflow:

$$P(W) = \prod_{t=1}^{T} P\left(w_t \mid w_{t-1}\right) < 10^{-308}, \tag{3.26}$$

exceeding the precision of standard 64-bit IEEE double-precision floating-point arithmetic. To guarantee numerical stability, all probability computations are evaluated in the logarithmic domain:

$$\log P(W) = \log P\left(w_1\right) + \sum_{t=2}^{T} \log P\left(w_t \mid w_{t-1}\right). \tag{3.27}$$

Operating in log-space transforms unstable products of small fractions into stable sums of negative real numbers, preserving full analytical fidelity across sequences of arbitrary length.

3.2.4 Inference: autoregressive text generation and decoding dynamics

The second operational mode of a language model is *autoregressive text generation*. In this regime, the model does not score an existing sentence; instead, it synthesizes new text token by token. At each step t, the model conditions on the antecedent token w_{t-1}, retrieves the conditional categorical distribution $P(\cdot \mid w_{t-1})$ from its transition matrix, samples a token w_t, appends w_t to the generated sequence, and feeds w_t back into the model to condition the subsequent step:

$$w_t \sim P\left(\cdot \mid w_{t-1}\right). \tag{3.28}$$

Step-by-step lecture generation trace. We trace the complete generative execution evaluated in lecture, initiated with the prompt token $w_1 = \texttt{"The"}$:

1. **Step 1**: Prompt w_1 = "The". The model queries row "the" in Table 3.2, obtaining the categorical distribution:

$$P(\texttt{cat} \mid \texttt{the}) = 0.25, \quad P(\texttt{cheese} \mid \texttt{the}) = 0.25, \quad P(\texttt{mouse} \mid \texttt{the}) = 0.50. \tag{3.29}$$

   Sampling stochastically from this distribution selects the candidate w_2 = "mouse".

$$\text{Current sequence:} \quad \texttt{"The mouse"} \tag{3.30}$$

2. **Step 2**: Preceding token w_2 = "mouse". The model queries row "mouse":

$$P(\texttt{ate} \mid \texttt{mouse}) = 0.50, \quad P(\texttt{happily} \mid \texttt{mouse}) = 0.50. \tag{3.31}$$

   Sampling from this distribution selects w_3 = "ate".

$$\text{Current sequence:} \quad \texttt{"The mouse ate"} \tag{3.32}$$

3. **Step 3**: Preceding token w_3 = "ate". The model queries row "ate":

$$P(\texttt{the} \mid \texttt{ate}) = 1.00. \tag{3.33}$$

   The transition is fully deterministic, yielding w_4 = "the".

$$\text{Current sequence:} \quad \texttt{"The mouse ate the"} \tag{3.34}$$

4. **Step 4**: Preceding token w_4 = "the". The model again queries row "the". Although the model previously generated "mouse" at step 2, the bigram Markov assumption possesses zero memory of steps 1 to 3. Sampling from the identical distribution selects w_5 = "mouse".

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse"} \tag{3.35}$$

5. **Step 5**: Preceding token w_5 = "mouse". Consulting row "mouse", the model samples w_6 = "happily".

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse happily"} \tag{3.36}$$

6. **Step 6**: Preceding token w_6 = "happily". Row "happily" contains strictly zero counts across all columns ($\sum_w C(\texttt{happily}, w) = 0$). In the transition graph, "happily" represents an *absorbing terminal state*. The model has no valid successor tokens to sample, and generation halts.

The final generated utterance is:

$$\texttt{"The mouse ate the mouse happily."} \tag{3.37}$$

This sentence never existed in the training corpus; it was synthesized dynamically through iterative sampling along the stochastic transition graph.

Decoding strategies and pathological dynamics. The generation trace highlights two critical operational aspects of autoregressive generation:

1. **Absorbing terminal states and explicit boundary modeling**: In the toy example, generation halted because the terminal word "happily" had zero outgoing transitions. In rigorous natural language processing, stopping conditions are formalized by augmenting the vocabulary with dedicated boundary tokens: <BOS> (Beginning of Sentence) and <EOS> (End of Sentence). Training sentences are padded as <BOS> $w_1 \ldots w_T$ <EOS>. The model learns explicit transition probabilities $P(\texttt{<EOS>} \mid w_t)$, allowing the network to decide autonomously when an utterance is complete.

2. **Greedy decoding versus stochastic sampling and cyclic limit cycles**: Instead of sampling stochastically, an alternative decoding strategy is *greedy decoding*, where the model deterministically selects the single token with maximum conditional probability:

$$w_t^* = \arg\max_{w \in \mathcal{V}} P\left(w \mid w_{t-1}\right). \tag{3.38}$$

However, when greedy decoding is applied to finite transition matrices, the model frequently falls into *cyclic limit cycles*. In Table 3.2, starting from `"the"`, the greedy choice is w_2 = `"mouse"` ($P = 0.50$). If ties are broken in favor of `"ate"`, the sequence proceeds `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"` $\rightarrow$ `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"`..., trapping the model in an infinite periodic loop. This pathology persists in modern foundation models, necessitating advanced decoding heuristics such as temperature scaling, top-k sampling, and nucleus (top-p) truncation.

3.3 Statistical Limits and the Neural Paradigm

Despite their mathematical simplicity and clear probabilistic interpretations, statistical n-gram models suffer from severe architectural and theoretical limitations that fundamentally restrict their scalability and utility in modern natural language processing. In this section, we examine the three primary limitations of statistical language models—combinatorial data sparsity, context horizon truncation, and the absence of semantic generalization—analyze smoothing as an intuitive mitigation strategy, and establish the necessity of continuous neural representations.

3.3.1 Combinatorial data sparsity and the zero-frequency collapse

The most prominent limitation of n-gram models is the curse of combinatorial data sparsity. An n-gram model over a realistic vocabulary $\mathcal{V}$ requires maintaining a contingency table whose parameter count scales exponentially with the window size n:

$$N_{\text{parameters}} = |\mathcal{V}|^n. \tag{3.39}$$

For a standard vocabulary of size $|\mathcal{V}| = 50{,}000$, the parameter requirements scale aggressively with n:

- **Bigram ($n = 2$)**: Requires $|\mathcal{V}|^2 = 2.5 \times 10^9$ parameters (requiring $\approx$ 10 GB of memory under 32-bit floats).
- **Trigram ($n = 3$)**: Requires $|\mathcal{V}|^3 = 1.25 \times 10^{14}$ parameters (requiring $\approx$ 500 TB of storage).
- **5-gram ($n = 5$)**: Requires $|\mathcal{V}|^5 = 3.125 \times 10^{23}$ parameters, far exceeding the physical storage capacity of global computing infrastructure.

Because human language allows words to combine in virtually boundless configurations, no finite corpus can observe more than a microscopic fraction of all valid n-grams. In any empirical training set, the vast majority of legitimate word combinations have an observed count of zero:

$$C(w_{t-n+1}, \ldots, w_t) = 0. \tag{3.40}$$

Under Maximum Likelihood Estimation, if a specific bigram (w_{t-1}, w_t) was never seen during training, its assigned conditional probability is identically zero:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{0}{C(w_{t-1})} = 0. \tag{3.41}$$

This leads directly to the *zero-frequency collapse*: when evaluating a candidate test sentence $W = (w_1, \ldots, w_T)$, if even a single constituent bigram is unobserved in the training corpus, the entire multiplicative chain collapses to zero:

$$P(W) = \prod_{t=1}^{T} P(w_t \mid w_{t-1}) = 0. \tag{3.42}$$

In log-space, $\log P(W) = \log 0 = -\infty$. Consequently, a statistical model that encounters a single unobserved word combination declares the entire sequence impossible, regardless of how fluent or probable the remaining tokens are.

3.3.2 Context horizon truncation and the loss of long-range dependencies

The second structural limitation arises from the Markov assumption itself. By conditioning the prediction of w_t strictly on the previous $n-1$ tokens:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}), \tag{3.43}$$

the model discards all contextual information residing beyond this immediate historical window. In natural language, however, syntactic agreement and semantic coherence frequently span dozens or hundreds of tokens.

Consider long-range subject-verb agreement across intervening subordinate clauses:

*"The **teacher** who guided the students across several challenging semesters **was** recognized."*

Under a bigram or trigram model ($n \leq 3$), when predicting the auxiliary verb, the conditioning context consists exclusively of the preceding tokens *"challenging semesters"*. Because *"semesters"* is plural, an n-gram model assigns high probability to the plural auxiliary *"were"*, completely blind to the singular subject *"teacher"* introduced nine tokens earlier.

Furthermore, discourse-level narrative and topic consistency operate over long time horizons. Human communication requires persistent thematic memory: statements made several paragraphs prior constrain the interpretation of pronouns, references, and stylistic choices. Because scaling n beyond 4 or 5 is computationally intractable due to exponential parameter explosion, n-gram models cannot maintain long-term narrative coherence.

3.3.3 Lack of semantic generalization and orthogonal representations

The third fundamental limitation is the complete absence of semantic generalization. In statistical contingency tables, words are represented as atomic, discrete symbolic entities. Mathematically, this corresponds to representing each vocabulary item w_i as a discrete one-hot standard basis vector $\mathbf{e}_i \in \{0,1\}^{|\mathcal{V}|}$. In this discrete representation space, any pair of distinct words w_i and w_j ($i \neq j$) is strictly orthogonal:

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}. \tag{3.44}$$

Every word is equidistant and geometrically independent of every other word. Consequently, the model possesses no intrinsic notion of lexical similarity or semantic relatedness:

1. **Failure to transfer knowledge across synonyms**: Suppose the training corpus contains multiple instances of the sentence *"The cat chased the mouse"*. The model learns high transition probabilities for $P(\text{chased} \mid \text{cat})$ and $P(\text{mouse} \mid \text{chased})$. If the model is subsequently tasked with evaluating the semantically identical sentence *"The feline pursued the rodent"*, it treats the words *"feline"*, *"pursued"*, and *"rodent"* as entirely novel, unrelated symbols. It cannot transfer the statistical knowledge acquired from *"cat"* to its near-synonym *"feline"*.

2. **Polysemy neglect**: Discrete tables assign a single conditional probability distribution to each lexical token, conflating distinct word senses. A polysemous word such as *"bank"* (denoting either a financial institution or a riverbank) is treated as a single uniform entry. The model cannot adapt the internal representation of the word based on the semantic domain established by the surrounding discourse.

Because each lexical entry occupies an isolated, independent row in the transition matrix, statistical n-gram models exhibit zero parameter sharing across semantically related contexts.

3.3.4 Mitigating zero frequencies: the smoothing mechanism

To prevent the catastrophic zero-frequency collapse caused by unobserved n-grams, statistical natural language processing introduced *smoothing* mechanisms.

The general intuition of smoothing. The core principle of smoothing is probability mass reallocation: the model shaves off a small portion of probability mass from frequently observed events and redistributes this reserved mass to unobserved events. By ensuring that every valid vocabulary token receives a strictly positive probability ($P(w_t \mid w_{t-1}) > 0$), smoothing guarantees that candidate sentences never collapse to zero probability when evaluated on unseen text.

Basic implementation: Laplace (Add-1) smoothing. The most fundamental and intuitive implementation of smoothing is *Laplace smoothing* (also known as Add-1 smoothing). The algorithm modifies the Maximum Likelihood Estimation formula by artificially incrementing the observed count of every possible n-gram by 1. For a bigram model, the smoothed conditional probability is defined as:

$$P_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + 1}{\sum_{w \in \mathcal{V}} (C(w_{t-1}, w) + 1)} = \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + |\mathcal{V}|}. \tag{3.45}$$

By adding 1 to the numerator, any unobserved bigram ($C(w_{t-1}, w_t) = 0$) receives a non-zero probability:

$$P_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{1}{C(w_{t-1}) + |\mathcal{V}|} > 0. \tag{3.46}$$

To preserve the axioms of probability such that the conditional distribution over next tokens sums to 1:

$$\sum_{w_t \in \mathcal{V}} P_{\text{Laplace}}(w_t \mid w_{t-1}) = \sum_{w_t \in \mathcal{V}} \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + |\mathcal{V}|} = \frac{C(w_{t-1}) + |\mathcal{V}|}{C(w_{t-1}) + |\mathcal{V}|} = 1.0, \tag{3.47}$$

the denominator must be incremented by the vocabulary size $|\mathcal{V}|$.

Limitations of basic smoothing. While Laplace smoothing mathematically resolves the zero-frequency collapse, it introduces a severe structural distortion when applied to realistic vocabularies. Suppose $|\mathcal{V}| = 50{,}000$ and an observed context word appears $C(w_{t-1}) = 100$ times. The denominator in Equation (3.45) increases from 100 to $100 + 50{,}000 = 50{,}100$. An observed bigram that appeared 5 times in the corpus has its estimated probability suppressed from $5/100 = 0.05$ to $6/50{,}100 \approx 0.00012$, attenuating empirical evidence by over 99%. Laplace smoothing steals excessive probability mass from genuine linguistic patterns and diffuses it across thousands of unobserved, syntactically invalid combinations.

3.3.5 The architectural transition to continuous neural representations

The limitations examined in this section demonstrate that smoothing merely patches zero-frequency counts without addressing the root cause of the failure. Statistical n-gram models remain fundamentally constrained by:

1. Exponential parameter growth $O(|\mathcal{V}|^n)$, which enforces severely truncated context horizons ($n \leq 3$).

2. Discrete symbolic representations, which prevent parameter sharing across synonymous or semantically related words.

Furthermore, while quantitative evaluation metrics such as Perplexity and Cross-Entropy are essential for benchmarking language models systematically (and are analyzed comprehensively in Chapter 8), no mathematical metric can overcome the intrinsic capacity ceiling of tabular n-grams.

Resolving these fundamental challenges requires a paradigm shift away from discrete, tabular frequency counts toward continuous, parameterized neural networks [30]. In this modern framework, words are projected into continuous, low-dimensional Euclidean vector spaces $\mathbb{R}^d$ ($d \ll |\mathcal{V}|$), where geometric proximity directly encodes semantic similarity. By framing language modeling as a continuous classification task parameterized by neural architectures, models can simultaneously maintain extended context histories and share statistical strength across related lexical concepts. This foundational transition—from discrete orthogonal indices to continuous distributed representations—is explored in depth in Chapter 4.

CHAPTER 4

Word Embeddings and Word2Vec

The preceding chapters established the computational foundations of deep learning, sequence modeling through recurrent state transitions, and the probabilistic principles of language models. However, deploying neural architectures on natural language requires resolving a fundamental representation problem: mapping discrete symbolic tokens into continuous numerical inputs on which gradient backpropagation can operate.

Tokens in written language and source code are discrete, categorical symbols. Assigning each word an isolated standard basis vector produces extreme sparsity and pairwise orthogonality, preventing neural networks from sharing statistical evidence across related concepts. Dense word embeddings resolve this representational impasse by mapping tokens into continuous, low-dimensional Euclidean vector spaces where geometric proximity reflects semantic and functional similarity.

This chapter systematically analyzes the theoretical, algorithmic, and geometric foundations of continuous word representations. We begin by examining why words require continuous vector representations and analyze the baseline one-hot encoding alongside its structural failure modes. We then formulate the Word2Vec framework, examining the Continuous Bag-of-Words (CBOW) and Skip-Gram objectives, the softmax computational bottleneck, and negative sampling approximations. Next, we analyze FastText, which extends word vectors to subword character n-grams to eliminate the out-of-vocabulary barrier and capture morphological regularities. Finally, we dissect how language models update embedding matrices through self-supervised training and explore the emergent geometric properties of continuous semantic spaces, including clustering, metric distances, and linear relational analogies.

4.1 The Word Representation Problem

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as `"compiler"`, `"optimizer"`, `"gradient"`, and keywords such as `"class"`, `"def"`, `"return"`.

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An L-layer deep neural network $\mathcal{F}(\mathbf{x};\Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x};\Theta) = \sigma_L\Big(\mathbf{W}_L\,\sigma_{L-1}\big(\cdots\sigma_1(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)\cdots\big)+\mathbf{b}_L\Big), \tag{4.1}$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta\,\nabla_\Theta \mathcal{L}(\Theta). \tag{4.2}$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial \mathcal{L}}{\partial \texttt{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add `"cat"` to `"dog"`, nor compute an inner product between `"while"` and `"for"`.

To bridge this gap between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

Definition 4.1: *Continuous word embedding*

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \tag{4.3}$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle\cdot,\cdot\rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30{,}000$ to upwards of $V = 1{,}000{,}000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

4.2 One-Hot Encoding and Its Limitations

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a V-dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0, \dots, 0}_{i-1 \text{ zeros}}, 1, \underbrace{0, \dots, 0}_{V-i \text{ zeros}}]^\top \in \{0,1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \tag{4.4}$$

For example, consider a canonical toy vocabulary of $V = 5$ tokens:

$$\mathcal{V} = \{\texttt{"dog"}, \texttt{"cat"}, \texttt{"fish"}, \texttt{"pen"}, \texttt{"pencil"}\}. \tag{4.5}$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \tag{4.6}$$

4.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In the foundational guide *Build a Large Language Model from Scratch* [31], it is proven that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

Theorem 4.1: *Embedding lookup equivalence*

Let $\mathcal{V}$ be a vocabulary of size V, and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0,1\}^V$.

Column-vector orientation (standard basis representation)

Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ be the embedding matrix whose k-th column corresponds to the d-dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & & | \end{bmatrix}. \tag{4.7}$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the i-th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}}\mathbf{e}_i = \sum_{k=1}^{V} (\mathbf{e}_i)_k (\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k \neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \tag{4.8}$$

Row-vector orientation (PyTorch convention)

Let $\mathbf{W} \in \mathbb{R}^{V \times d}$ be an embedding table whose k-th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1 \times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the i-th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top \mathbf{W} = \sum_{k=1}^{V} (\mathbf{e}_i)_k \mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k \neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1 \times d}. \tag{4.9}$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}}\mathbf{e}_i \equiv \texttt{lookup}(\mathbf{W}_{\text{in}}, i). \tag{4.10}$$

4.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ and table indexing $\texttt{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs**: Evaluating the matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V-1) \cdot d$ additions. In a standard language model with vocabulary $V = 50{,}257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:
$$50{,}257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \tag{4.11}$$
Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \ldots, V-1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:
$$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \texttt{sizeof}(\text{float32}), \tag{4.12}$$
retrieving the d-dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction)**: Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1{,}024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1{,}024, 50{,}257)$. Storing this tensor in single-precision floating point (`float32`, 4 bytes per element) would require:
$$\text{VRAM}_{\text{one-hot}} = 64 \times 1{,}024 \times 50{,}257 \times 4 \text{ bytes} \approx 13{,}175{,}095{,}296 \text{ bytes} \approx 13.18\,\text{GB of VRAM,} \tag{4.13}$$
simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1{,}024)$, requiring:
$$\text{VRAM}_{\text{indices}} = 64 \times 1{,}024 \times 8 \text{ bytes} = 524{,}288 \text{ bytes} \approx 524\,\text{KB of VRAM.} \tag{4.14}$$
This constitutes a memory bandwidth reduction factor of $\frac{13.18\,\text{GB}}{524\,\text{KB}} \approx \mathbf{25{,}128}\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation**: Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \tag{4.15}$$
This gradient matrix is massive ($V \times d$) and strictly sparse: only row i contains non-zero values, while the remaining $V-1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the i-th row of $\mathbf{W}$.

4.2.3 The 5 structural failure modes of one-hot representations

The lookup-table implementation makes input projection computationally efficient, but one-hot vectors do not encode semantic relationships between words. They have five limitations:

Extreme sparsity and curse of dimensionality

As vocabulary size scales to web scale ($V \geq 50{,}000$ to 10^6), one-hot vectors become very large sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

Metric collapse and semantic orthogonality

Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \tag{4.16}$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i,j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \tag{4.17}$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\texttt{"pen"}, \texttt{"pencil"}) = d_E(\texttt{"pen"}, \texttt{"crocodile"}) = \sqrt{2}. \tag{4.18}$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.

Zero gradient parameter sharing during backpropagation

Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token w_i produces gradients exclusively for the parameters associated with index i:

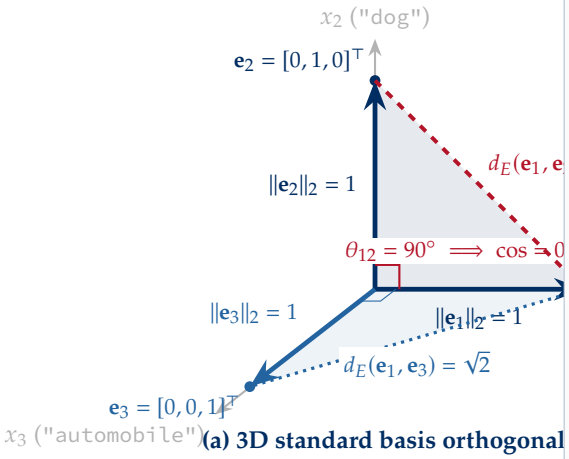
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = \mathbf{0}, \quad \forall k \neq i. \tag{4.19}$$

If a model updates its weights upon observing the sentence *"The feline climbed the tree"*, the parameters associated with `"feline"` receive gradient updates, but the parameters for `"cat"`, `"kitten"`, or `"leopard"` remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

Simplex confinement and absence of latent manifolds

One-hot vectors are confined strictly to the V extreme vertices of the standard $(V-1)$-dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^{V} x_i = 1, \; x_i \geq 0 \right\}. \tag{4.20}$$



(a) 3D standard basis orthogonal

Pairwise metric equality matrices:

• **Cosine similarity matrix** (Gramian $\mathbf{E}^\top\mathbf{E} = \mathbf{I}_V$):

$$\mathbf{S}_{\cos} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 1 & \mathbf{0} & \mathbf{0} \\ \text{dog} & \mathbf{0} & 1 & \mathbf{0} \\ \text{car} & \mathbf{0} & \mathbf{0} & 1 \end{pmatrix}$$

• **Euclidean distance matrix** ($\mathbf{D}_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$\mathbf{D}_{L_2} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 0 & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} \\ \text{dog} & \mathbf{\sqrt{2}} & 0 & \mathbf{\sqrt{2}} \\ \text{car} & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} & 0 \end{pmatrix}$$

The geometric pathology:
dist("cat", "dog") = dist("cat", "car") = $\sqrt{2}$
cos("cat", "dog") = cos("cat", "car") = 0

(b) Semantic metric collapse

All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.

Figure 4.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In V-dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|\mathbf{e}_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin $\mathbf{0}$, yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90°$). (b) The resulting Gramian matrix is the identity $\mathbf{I}_V$, treating related concepts like `"cat"` and `"dog"` with the exact same geometric distance as totally unrelated concepts like `"cat"` and `"automobile"`.

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

Software engineering identifier polymorphism

In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants**: `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms**: `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents**: Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like `";"`.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\texttt{get_user_by_id}}, \mathbf{e}_{\texttt{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{LinkedList}}) = d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{";"}}) = \sqrt{2}. \tag{4.21}$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

4.3 Word2Vec: CBOW and Skip-Gram

To overcome the orthogonality and isolation of local representations, modern deep learning adopts *distributed representations*, a concept formalized in connectionist neural networks [7]. In 2013, Mikolov et al. [32, 33] introduced **Word2Vec**, a family of computationally efficient log-linear architectures designed to learn continuous word representations from web-scale text corpora. In this section, we examine the distributional hypothesis underlying distributed representations, analyze the Continuous Bag-of-Words (CBOW) and Skip-Gram architectures, derive the negative sampling objective to scale beyond the full softmax bottleneck, and examine the structural limitations of static word vectors.

4.3.1 The distributional hypothesis and distributed representations

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept formalized in connectionist neural networks in 1986 [7].

Definition 4.2: *Local vs. distributed representations*

- **Local Representation**: Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation**: Each concept is represented by a continuous pattern of activation across all d dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics in 1954 [34] and popularized in 1957 [35]:

> *"You shall know a word by the company it keeps."*
> — Distributional Hypothesis [35]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: "I used a **pencil** to write the essay."
Sentence 2: "You used my **pen** to write a letter."

Even if a machine has never encountered a dictionary definition of `"pencil"` or `"pen"`, it observes that both tokens co-occur with identical context words: `"used"`, `"write"`, `"essay"`, `"letter"`. Conversely, an unrelated word such as `"crocodile"` or `"refrigerator"` almost never occurs in the context of `"used a [...] to write"`.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting

surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

4.3.2 Continuous Bag-of-Words (CBOW) architecture

In 2013, **Word2Vec** was introduced by Mikolov et al. [32, 33] as a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token w_t given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}\} \setminus \{w_t\}. \tag{4.22}$$

Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token k when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token k when acting as the target.

Let $\mathbf{e} \in \{0, 1\}^V$ denote the multi-hot context vector indicating the presence of words in C_t:

$$e_k = \sum_{j \in \{-C, \dots, C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j} = w_k\}}. \tag{4.23}$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C} \mathbf{W}_{\text{in}} \mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \tag{4.24}$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \tag{4.25}$$

As emphasized in geometric visual analyses [36], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos\theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.26}$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h}). \tag{4.27}$$

Figure 4.2 illustrates this computational flow.

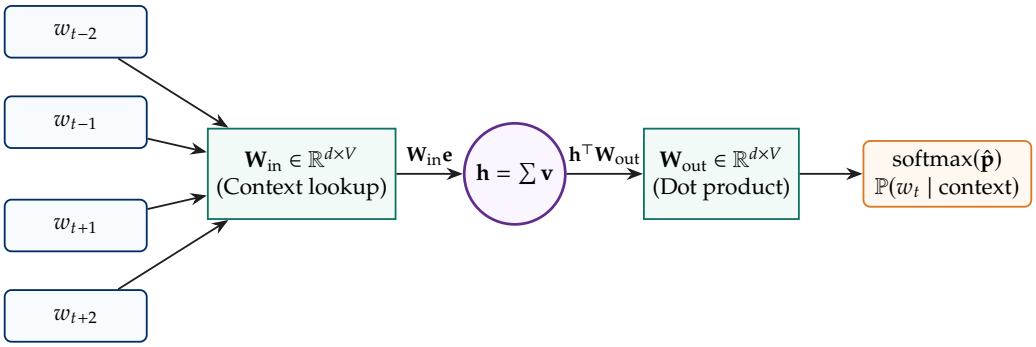


Figure 4.2: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \ldots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t.

Comparison with earlier neural language models: the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior in 2003 [30]. However, that architecture had two major differences:

1. **Strict Causal Prediction**: In [30], language was modeled autoregressively, predicting w_t strictly from past tokens $w_{t-n+1}, \ldots, w_{t-1}$. In Word2Vec, the window was opened symmetrically to both left and right context tokens ($w_{t-C}, \ldots, w_{t+C}$), doubling the statistical signal.

2. **Elimination of the Non-Linear Hidden Layer**: In [30], concatenated context embeddings were passed through a non-linear dense hidden layer $\tanh(\mathbf{W}_h\mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($O(C \cdot d \cdot d_h)$). In Word2Vec, the non-linear activation was stripped away entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

Strengths and representational bottlenecks of CBOW

Strengths of CBOW

- **Rapid Training Convergence**: Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.

- **Robust Representations for Frequent Words**: For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

Representational bottlenecks of CBOW

- **Loss of Sequential Word Order (Syntactic Blindness)**: Vector summation in Equation (4.24) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing *"the dog bit the man"* from *"the man bit the dog"*.

- **Rare Word Dilution**: Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., *"the **polymorphism** in the"*), the rare token's embedding is diluted by the dominant stopwords: $\frac{1}{2C}\mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic word order. To learn high-fidelity representations for infrequent words, inverting the predictive formulation yields the Continuous Skip-Gram architecture.

4.3.3 Continuous Skip-Gram architecture

To preserve the distinct gradient signal of rare words, the **Continuous Skip-Gram** architecture was introduced in [32]. Instead of aggregating context words to predict a single center token, Skip-Gram **inverts the predictive direction**: given a single center target word w_t, the model independently predicts each surrounding context word within window radius C.

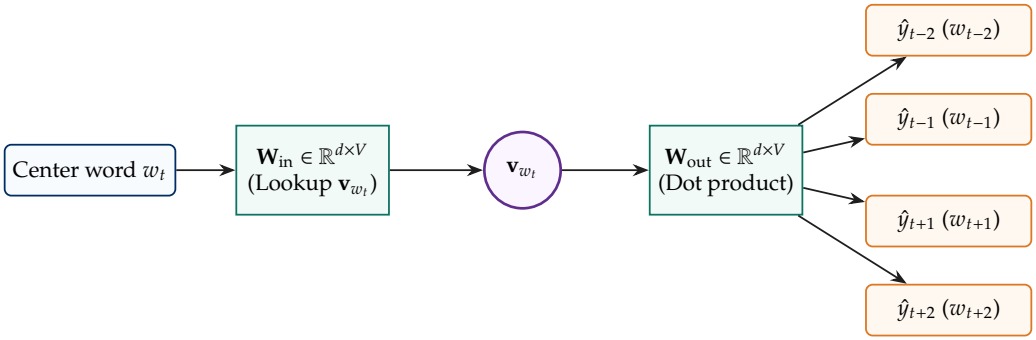


Figure 4.3: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

Definition 4.3: *Skip-gram objective function*

Given a training sequence of T words $w_1, w_2, \ldots, w_T$ and a context window of half-width C, the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} \mid w_t). \tag{4.28}$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O \mid w_I) = \frac{\exp\left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right)}, \tag{4.29}$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence (w_I, w_O) generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as `"refactoring"` or `"polymorphism"`) appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left(\mathbf{u}_{w_O} - \sum_{k=1}^{V} \mathbb{P}(w_k \mid w_I) \mathbf{u}_k \right). \tag{4.30}$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and

nuanced semantic distinctions than CBOW.

The full softmax computational bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an prohibitive computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (4.29):

$$Z(w_I) = \sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right). \tag{4.31}$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token w in the entire vocabulary $\mathcal{V}$**.

- **Computational Complexity per Token**: Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost**: For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \tag{4.32}$$

- **Gradient Synchronization Bottleneck**: In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \ldots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

4.3.4 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, two distinct scaling methods were formulated in [33]: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

Hierarchical softmax ($O(\log_2 V)$)

First developed in 2005 [37] and adapted in [33], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology**: The V vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification**: For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) =$ root to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters**: Each internal node $j \in \{1, \ldots, L(w)-1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.
- **Binary Branching Probability**: At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \frac{1}{1 + \exp\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right)}. \tag{4.33}$$

  The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1 - \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \sigma\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right). \tag{4.34}$$

- **Path Probability Product**: The conditional probability of reaching leaf word w is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma\left(\text{sign}(w, j) \cdot \mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right), \tag{4.35}$$

  where $\text{sign}(w, j) = +1$ if the path branches left at node j, and -1 if it branches right.

Because probabilities sum to 1 at each binary branching step ($\sigma(z) + \sigma(-z) = 1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w \in \mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \tag{4.36}$$

No global partition function $Z(w_I)$ ever needs to be computed.

Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w) - 1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $O(\log_2 V)$. For a vocabulary of $V = 100{,}000$:

$$\log_2(100{,}000) \approx 16.6 \implies \frac{100{,}000}{17} \approx \mathbf{5{,}880}\times \text{ acceleration}. \tag{4.37}$$

Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $O(\log_2 V)$, an even more efficient and empirically superior alternative was formulated in [33]: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over V mutually exclusive categories, SGNS reformulates the problem as $1 + K$ **independent binary logistic classifications**:

1. **Positive Sample ($y = 1$)**: The true center-context pair (w_c, w_o) observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples ($y = 0$)**: K noise words $\{w_{n_1}, w_{n_2}, \ldots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

Definition 4.4: *Skip-gram negative sampling objective*

For an observed center-context token pair (w_c, w_o) and K negative samples $\{w_{n_1}, \ldots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma\left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}\right), \tag{4.38}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

Analytical gradient derivations for SGNS

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{4.39}$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \tag{4.40}$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \tag{4.41}$$

Gradient with respect to positive target vector

The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (4.38):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\ &= -\left[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})\right] \frac{\partial (\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\ &= \left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{v}_{w_c}. \end{aligned} \tag{4.42}$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1)\mathbf{v}_{w_c}. \tag{4.43}$$

Gradient with respect to negative noise vector

Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (4.38):

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= -\left[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\ &= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{v}_{w_c}. \end{aligned} \tag{4.44}$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k}\, \mathbf{v}_{w_c}. \tag{4.45}$$

Gradient with respect to center word vector

The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all K negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^{K} \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^{K} \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}}. \end{aligned} \tag{4.46}$$

Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta \left(1 - \hat{y}_{\text{pos}}\right) \mathbf{u}_{w_o} - \eta \sum_{k=1}^{K} \hat{y}_{\text{neg},k}\, \mathbf{u}_{w_{n_k}}. \tag{4.47}$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force**: Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force**: For every negative sample k, $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (4.47) is $O(K \cdot d)$, which is **entirely independent of vocabulary size** V. In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

Theoretical justification of the $3/4$ unigram noise exponent

How should negative samples be drawn? In [33], the noise distribution $P_n(w)$ is defined as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \tag{4.48}$$

where $f(w)$ denotes the raw unigram occurrence frequency of token w in the corpus.

Theoretical justification of the 3/4 unigram noise exponent

1. **Failure of Raw Frequency ($\alpha = 1.0$)**: Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords (`"the"`, `"of"`, `"and"`, `"to"`) account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from `"the"`, while rare content words (`"polymorphism"`, `"compiler"`) have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling ($\alpha = 0.0$)**: Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.

3. **The Sub-linear Compromise ($\alpha = 0.75$)**: The exponent $3/4$ acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Consider a frequent stopword w_{freq} with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., 10^6 occurrences in 10^8 words) and a rare technical term w_{rare} with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in 10^8 words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10{,}000. \tag{4.49}$$

  The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}}\right)^{0.75} = (10{,}000)^{0.75} = (10^4)^{3/4} = 10^3 = 1{,}000. \tag{4.50}$$

By applying the $3/4$ power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10{,}000}{1{,}000} = \mathbf{10}\times. \tag{4.51}$$

This $10\times$ boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

Global count matrix factorization: GloVe

Around the same time, **GloVe (Global Vectors for Word Representation)** was introduced in [38]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where X_{ij} counts the number of times word j appears in the context window of word i.

GloVe's core insight is that the semantic distinction between two words i and j relative to a probe word k is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k \mid i)}{P(k \mid j)}. \tag{4.52}$$

Table 4.1 reproduces the seminal demonstration from [38] comparing i = `"ice"` and j = `"steam"` across probe words k.

Table 4.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [38].

Probability & Ratio	k = `"solid"`	k = `"gas"`	k = `"water"`	k = `"fashion"`
$P(k \mid$ `"ice"`$)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid$ `"steam"`$)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid$ `"ice"`$)/P(k \mid$ `"steam"`$)$	**8.9**	**0.085**	1.36	0.96

When k = `"solid"` is strongly related to `"ice"` but not `"steam"`, the ratio is large ($8.9 \gg 1$). When k = `"gas"` is related to `"steam"` but not `"ice"`, the ratio is tiny ($0.085 \ll 1$). When k = `"water"` is equally related to both (or k = `"fashion"` is related to neither), the ratio hovers near 1.0.

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{4.53}$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

4.3.5 Limitations of the Word2Vec model family

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \ldots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time**: In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., `"recusive"`), and inflected stems.
- **Mapping to `<UNK>`**: When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{oov}} \longmapsto \texttt{<UNK>} \implies \mathbf{v}_{w_{\text{oov}}} = \mathbf{v}_{\texttt{<UNK>}}. \tag{4.54}$$

  This discards all semantic identity. To Word2Vec, a typo like `"algorithmm"` is as completely devoid of meaning as an alien string.

Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words share sub-morphemic stems, prefixes, and suffixes:

Natural language: `"develop"`, `"developer"`, `"developing"`, `"undeveloped"`,
Software identifiers: `"parseFile"`, `"parseTokens"`, `"parseTree"`.

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{\text{in}}$. The model possesses zero architectural mechanism to share parameters across words containing the common root `"develop"` or `"parse"`. If `"undeveloped"` occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token `"develop"`.

Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector $\mathbf{v}_w \in \mathbb{R}^d$ to each word string** w, regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

Definition 4.5: *Polysemy collapse*

Let word w possess M distinct semantic senses $\mathcal{S}_1, \ldots, \mathcal{S}_M$ with respective corpus mixture proportions $\pi_1, \ldots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^{M} \pi_m \mathbf{v}_{\mathcal{S}_m}. \tag{4.55}$$

For example, consider the polysemous word `"bat"`:

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."
Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{\text{bat}}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- `"Python"`: A biological serpent vs. a high-level dynamic programming language.
- `"Spring"`: A season of the year vs. a popular Java enterprise application framework.
- `"rust"`: Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps `"Rust"` to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character n-grams, FastText establishes a morphologically aware embedding paradigm.

4.4 FastText: subword character n-gram embeddings and limitations

To conquer the OOV barrier and enable morphological parameter sharing, **FastText** was introduced by Bojanowski et al. [39].

4.4.1 Character n-gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character n-grams**.

1. **Boundary Enclosure**: To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word w is bracketed by special boundary symbols `"<"` and `">"`:
$$\tilde{w} = \texttt{"<"} \circ w \circ \texttt{">"}. \tag{4.56}$$
This ensures that the prefix sequence `"<re"` (as in `"<refactor>"`) is mathematically distinct from the internal substring `"re"` (as in `"<where>"`).

2. **Subword Extraction**: All character n-grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

Subword decomposition worked example

Consider the word `"where"` decomposed into character trigrams ($n = 3$):

$$\texttt{"<where>"} \implies \mathcal{G}_{\texttt{where}} = \Big\{\texttt{"<wh"}, \texttt{"whe"}, \texttt{"her"}, \texttt{"ere"}, \texttt{"re>"}, \texttt{"<where>"}\Big\}. \tag{4.57}$$

4.4.2 Vector sum composition

Each subword n-gram g is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word w is defined as the **sum of its constituent subword vectors**:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \tag{4.58}$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word w and context word c becomes:

$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \tag{4.59}$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

4.4.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens**: When an unseen word w_{oov} appears at test time (e.g., a compound identifier like `"refactorable"` or a code typo like `"recusive"`), even if the full token was never seen during training, its constituent character n-grams (`"<re"`, `"ref"`, `"fact"`, `"able>"`) were updated across other vocabulary words. FastText

synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:

$$\mathbf{v}_{w_{\text{oov}}} = \sum_{g \in \mathcal{G}_{w_{\text{oov}}}} \mathbf{z}_g. \tag{4.60}$$

The OOV barrier is completely eliminated.

2. **Morphological Parameter Sharing**: In natural language, prefixes like `"un-"`, suffixes like `"-tion"`, and inflections like `"-ing"` accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\texttt{"parseXmlDocument"} \quad \text{and} \quad \texttt{"parseJsonDocument"} \tag{4.61}$$

share the character subwords for `"parse"` and `"Document"`. The model immediately recognizes that both identifiers describe parsing operations on structured documents.

Memory bounding via the Fowler-Noll-Vo hash trick

Because the universe of all character n-grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \tag{4.62}$$

Colliding n-grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

4.4.4 Connection to modern subword tokenizers: BPE and WordPiece

As highlighted in pedagogical breakdowns of language models [40], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** [41, 42]: Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** [43]: Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character n-gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character n-grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

4.4.5 Limitations of FastText

While FastText successfully addresses the out-of-vocabulary barrier and morphological blindness through character n-gram composition, it possesses distinct architectural and computational trade-offs:

1. **Memory footprint and parameter explosion**: Representing all character n-grams ($n \in [3, 6]$) expands the parameter inventory from $|\mathcal{V}|$ word vectors to millions of distinct subword strings. While the Fowler-Noll-Vo hash trick caps table size to a fixed number of buckets (e.g., 2×10^6), hash collisions unavoidably introduce acoustic and morphological noise between unrelated subwords.

2. **Heuristic subword decomposition**: FastText decomposes words via fixed-length sliding windows without considering empirical language statistics or morphological boundaries. Highly frequent subwords receive identical representational capacity as rare character tuples.

3. **Absence of dynamic contextualization**: Crucially, FastText remains a *static* embedding model. Once the subword vectors are summed to produce $\mathbf{v}_w$, the word's representation remains permanently fixed regardless of the syntactic or semantic context in which it appears. Polysemous words (e.g., *"bat"* as an animal versus baseball equipment) continue to collapse into a single invariant vector representation.

4.5 Training Dynamics and Semantic Geometry

Having formalized the mathematical objectives of continuous word embeddings, we now examine the mechanistic and geometric dynamics of how these representations are acquired during training. Specifically, we investigate how language models update dual embedding matrices via gradient backpropagation, how repeated contextual prediction yields semantic clustering, and how vector spaces spontaneously organize into linear relational geometries.

4.5.1 How language models update embedding matrices

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of T running words $C = (w_1, w_2, \ldots, w_T)$, we shift a sliding context window of half-width C across the text. At each step t, the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}). \tag{4.63}$$

This self-supervised prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices**: Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.

2. **Context Lookup and Aggregation**: For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).

3. **Output Candidate Projection**: Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \tag{4.64}$$

4. **Probability Normalization**: Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^{V} \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.65}$$

5. **Loss Evaluation**: Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left(\frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \tag{4.66}$$

6. **Backpropagation Error Adjoints**: Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h}\, \boldsymbol{\delta}^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \boldsymbol{\delta} \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L})\, \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \tag{4.67}$$

7. **Stochastic Gradient Descent Parameter Update**: Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \qquad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \tag{4.68}$$

Figure 4.4 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

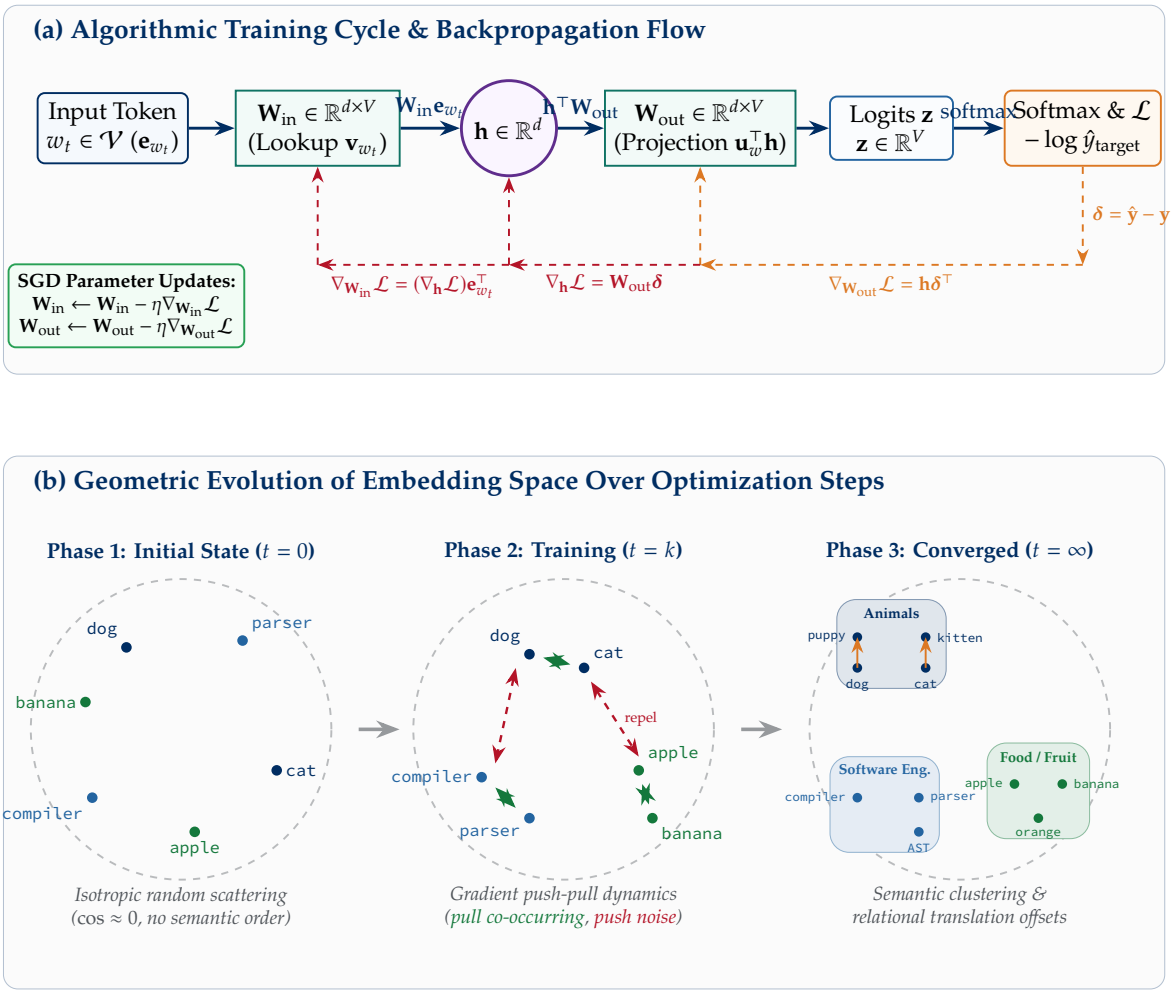


Figure 4.4: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 4.4, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—`"dog"`, `"compiler"`, `"apple"`—are scattered isotropically across the unit sphere with zero structural order.

- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., *"The compiler parsed the syntax tree"*), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

Initialization dynamics and the hockey-stick loss curve

In educational analyses of neural language models and lookup tables [44], it is demonstrated that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \tag{4.69}$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \tag{4.70}$$

For a vocabulary of $V = 50{,}000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50{,}000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological "hockey-stick" loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

4.5.2 Emergent vector space geometry: clustering and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items**: `"table"`, `"chair"`, `"desk"`, `"sofa"`.
- **Mammals**: `"dog"`, `"cat"`, `"tiger"`, `"elephant"`.
- **Birds**: `"eagle"`, `"sparrow"`, `"parrot"`, `"pigeon"`.

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \tag{4.71}$$

Country-to-capital linear offsets

For instance, subtracting the vector for `"Berlin"` from `"Germany"` produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \tag{4.72}$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \tag{4.73}$$

The gender-royalty parallelogram

The canonical demonstration established in [33] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{4.74}$$

Figure 4.5 illustrates this parallelogram in two dimensions.

Software engineering relational analogies

In code representation learning (e.g., CodeBERT [45], OpenAI Codex embeddings [46]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \tag{4.75}$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \tag{4.76}$$

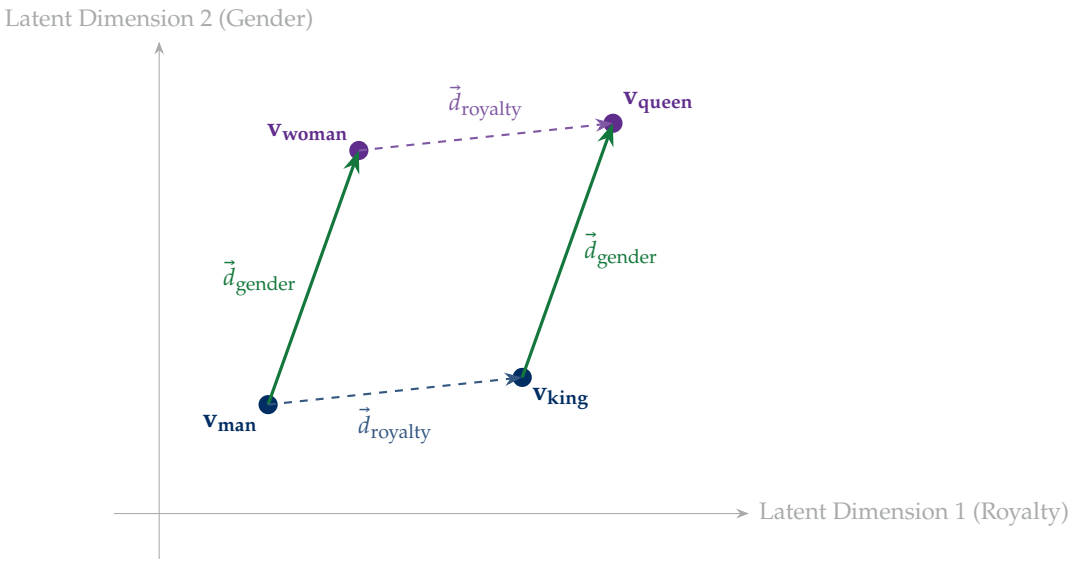


Figure 4.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

1. **Euclidean (L_2) Distance**:

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}. \tag{4.77}$$

2. **Dot Product (Inner Product)**:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta. \tag{4.78}$$

3. **Cosine Distance**:

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \tag{4.79}$$

Theorem 4.2: *Monotonic equivalence under L_2 normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \tag{4.80}$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

Why cosine similarity dominates metric evaluations

As demonstrated in educational analyses [40], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization ($d < 50$)**: Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization ($d \gg 10{,}000$)**: The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

Theorem 4.3: *Johnson-Lindenstrauss Lemma*

Given $0 < \epsilon < 1$, any set of N points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \tag{4.81}$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1-\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1+\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2. \tag{4.82}$$

For a vocabulary of $N = 100{,}000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

4.5.3 The representational ceiling and the bridge to the Transformer architecture

The geometric and relational capabilities analyzed in this chapter establish continuous word embeddings as the foundation of modern natural language processing. However, static embedding models—from one-hot baselines to Word2Vec and FastText—all share an insurmountable architectural ceiling: **context invariance**. Each word $w \in \mathcal{V}$ is assigned an invariant vector $\mathbf{v}_w \in \mathbb{R}^d$ prior to sequence processing. In natural language, however, true meaning is profoundly contextual: the interpretation of a word is dynamically shaped by the entirety of its surrounding discourse.

Overcoming this static ceiling requires neural architectures capable of synthesizing *contextualized embeddings* on the fly, where the representation of each token dynamically shifts based on its interactions with every other token in the sequence. Constructing such representations requires two foundational technologies:

1. **Discrete subword tokenization**: Replacing heuristic character n-grams with statistical algorithms such as Byte Pair Encoding (BPE), which build compact subword vocabularies from raw byte streams.

2. **Dynamic attention mechanisms**: Replacing static vector summation with self-attention operators that compute pairwise token affinities across the sequence, coupled with positional encodings to inject sequential order.

These core technologies form the architecture of the Transformer, which is analyzed in depth in Chapter 5.

CHAPTER 5

The Transformer Architecture

Chapters 1 through 4 developed the foundations of neural representation learning, recurrent sequence transitions, statistical language modeling, and dense vector embeddings. Through these developments, language models acquired the mathematical capacity to represent discrete tokens in continuous semantic vector spaces and to optimize parameterized neural transitions using gradient backpropagation.

Despite these breakthroughs, sequence-to-sequence modeling remained constrained by the architectural paradigm established by recurrent networks. Recurrent sequence modeling relies on the fundamental transition step $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, which enforces an unbroken, strictly sequential computational chain. This dependency imposes three prohibitive operational bottlenecks:

1. **The sequential execution barrier**: Computing $\mathbf{h}_t$ strictly requires the prior memory write of $\mathbf{h}_{t-1}$. Consequently, an input sequence of length L demands $O(L)$ serialized operational steps, preventing modern hardware accelerators (GPUs and TPUs) from exploiting parallel tensor arithmetic across the temporal dimension.

2. **The fixed-vector information compression bottleneck**: In classical sequence-to-sequence encoder-decoder networks, an encoder compresses an input sequence of arbitrary length L into a single, fixed-size latent vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$. As sequence length expands across hundreds or thousands of tokens, compressing intricate semantic, syntactic, and structural dependencies into a finite-dimensional vector induces severe information dissipation and contextual amnesia.

3. **Long-range gradient decay**: Information and backpropagation error signals propagating between distant tokens must traverse an unbroken chain of L recurrent transitions. As proven by the recurrent Jacobian product $\prod_{k=t+1}^{L} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$ in Chapter 2, this path length leads to exponential gradient decay or explosion, severely limiting long-range dependency modeling.

To resolve this impasse, Vaswani et al. introduced the **Transformer** [1]. The Transformer completely eliminates recurrent transitions, replacing recurrence with multi-head attention mechanisms. Under this paradigm, every token in a sequence communicates directly with every other token across an $O(1)$ path length in parallel, while maintaining uncompressed sequence representations $\mathbf{H} \in \mathbb{R}^{L \times d_{\text{model}}}$.

This chapter presents a comprehensive, first-principles analysis of the Transformer architecture. Section 5.1 establishes the foundations of tokenization, comparing character-, word-, and subword-level granularities, deriving the Byte Pair Encoding (BPE) algorithm with empirical Zipfian compression dynamics, and detailing operational special tokens. Section 5.2 examines positional representations, formalizing the permutation equivariance of self-attention, developing sinusoidal and learned absolute positional encodings, and formulating Shaw et al.'s relative positional representations. Section 5.3 derives the Attention primitive from classical dictionary lookup to continuous scaled dot-product attention, complete with geometric projections, variance

scaling proofs, and an exact numerical walkthrough. Section 5.4 analyzes the three fundamental attention types: bidirectional encoder self-attention, causal masked decoder self-attention, and cross-attention. Finally, Section 5.5 dissects Multi-Head Attention, Feed-Forward Networks, Residual Connections, and Layer Normalization, culminating in an end-to-end translation journey tracing tokens through the full architecture.

5.1 Tokenization and Subword Algorithms

5.1.1 Why tokenization is mandatory

Natural language text in written documents and software engineering source code is stored as continuous, unsegmented sequences of characters encoded in digital formats such as ASCII or UTF-8. Conversely, modern neural network architectures and language models operate exclusively on dense, continuous numerical tensors $\mathbf{x} \in \mathbb{R}^d$ through multivariable matrix operations and gradient-based backpropagation.

Tokenization establishes the fundamental discrete segmentation function $\mathcal{T} : \mathcal{S} \to \{t_1, t_2, \ldots, t_N\}$ that partitions a raw input string into discrete, atomic symbolic units known as **tokens**. Without tokenization, neural architectures have no mechanism to parse continuous character sequences into structured units amenable to vocabulary mapping and continuous vector projection.

Choosing the granular unit of tokenization represents a critical architectural trade-off that directly dictates the operational characteristics of the entire language model. Specifically, tokenization establishes a fundamental tension between **vocabulary size** $|\mathcal{V}|$ and **sequence length** N:

$$|\mathcal{V}| \uparrow \implies N \downarrow \quad \text{and} \quad |\mathcal{V}| \downarrow \implies N \uparrow . \tag{5.1}$$

Because Transformer self-attention incurs quadratic computational and memory complexity $O(N^2)$ with respect to sequence length, an inflated sequence length imposes severe hardware burdens. Conversely, an inflated vocabulary size expands the memory footprint of the token embedding matrix $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ and the final classification head $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, while rendering rare words severely under-trained. To navigate this landscape, natural language processing historically explored three primary tokenization paradigms: character-level, word-level, and subword-level tokenization.

5.1.2 Character-level tokenization

Character-level tokenization segments input text into individual typographic characters or single raw bytes. Under this regime, every letter, digit, whitespace symbol, and punctuation mark is treated as an isolated, atomic token.

Consider the illustrative sentence from Arthur Conan Doyle's *The Adventure of the Copper Beeches*:

```
"Mrs.  Rucastle was downstairs, so I had an admirable opportunity."
```

Under character-level tokenization, this sequence decomposes into an extensive array of 68 individual character tokens:

```
['M','r','s','.',' ','R','u','c','a','s','t','l','e',' ',...,'y','.'].
```
(5.2)

Advantages of character-level tokenization

1. **Compact vocabulary**: The base vocabulary size is strictly bounded by the alphabet of the language. For standard English ASCII text, $|\mathcal{V}| \approx 100$ characters; even across multilingual Unicode alphabets, the vocabulary rarely exceeds a few hundred entries.
2. **Zero out-of-vocabulary (OOV) failure**: Because any conceivable word is merely an arrangement of characters from the base alphabet, character-level models never encounter unknown words. Every rare technical term, neologism, or novel software variable can be ingested without falling back to an uninformative unknown token placeholder.
3. **Robustness to orthographic noise and typos**: Minor misspellings (e.g., `"recieved"` instead of `"received"`) alter only one or two characters, allowing the model to leverage morphological similarity and maintain representational stability.

Disadvantages of character-level tokenization

1. **Sequence length explosion**: Sentences expand into lengthy sequences that are 4 to 6 times longer than their word-level counterparts. In Transformer models where self-attention memory and computation scale as $O(N^2)$, quadrupling sequence length increases computational overhead by a factor of 16, rendering long-context modeling computationally prohibitive.

2. **Weak semantic signal per token**: Individual characters carry negligible semantic meaning in isolation. The letter `'e'` or `'t'` provides almost zero context regarding the semantic topic of a document. Consequently, the model must dedicate multiple neural layers merely to composing characters into words before learning higher-level syntactic and semantic relationships.

3. **Representational inefficiency for common words**: Frequent, high-utility words such as `"the"`, `"and"`, or `"is"` must be redundantly reconstructed character-by-character at every occurrence, wasting valuable attention capacity.

5.1.3 Word-level tokenization

Word-level tokenization segments text along whitespace boundaries and orthographic punctuation marks, assigning an individual token identifier to each complete, autonomous word.

Applying word-level tokenization to our running example produces a compact sequence of 11 distinct tokens:

```
["Mrs.","Rucastle","was","downstairs",",","so","I","had","an","admirable","opportunity","."].
```
(5.3)

Advantages of word-level tokenization

1. **Direct semantic correspondence**: Words are the primary lexical units of human language. Each token vector directly encapsulates semantic and syntactic attributes (e.g., entity types, verb tenses, nominal properties), enabling models to learn grammatical relationships rapidly.

2. **Compact sequence length**: Because each token represents an entire word, input sequence lengths remain short ($N \approx \frac{1}{5}N_{\text{char}}$). This property preserves the quadratic compute budget of self-attention, enabling models to process extensive documents within moderate memory footprints.

3. **Interpretability and conceptual transparency**: Attention weights and intermediate representations map directly onto human-comprehensible concepts and lexical dependencies.

Disadvantages of word-level tokenization

1. **Severe out-of-vocabulary (OOV) bottleneck**: Any word not present in the pre-defined training vocabulary $\mathcal{V}$ cannot be represented, and must be mapped to the generic unknown token `<UNK>`. In our running example, the proper noun `"Rucastle"` is an uncommon surname; in a standard corpus, it would collapse into `<UNK>`, irreversibly discarding its identity.

2. **Vocabulary size explosion**: Natural languages exhibit rich vocabularies that follow Zipf's law. Capturing comprehensive coverage across broad domains requires vocabularies spanning hundreds of thousands or millions of words ($|\mathcal{V}| \geq 500{,}000$). This inflates the

parameter count of the embedding and output projection matrices, consuming gigabytes of accelerator memory.

3. **Under-training of rare words**: In large vocabularies, the long tail of rare words appears only a handful of times across billions of training tokens. Consequently, their corresponding embedding vectors receive infrequent gradient updates and fail to converge to robust semantic representations.

4. **Morphological blindness**: Word-level tokenization treats inflections and derivations as completely unrelated entities. The tokens `"teach"`, `"teacher"`, `"teaching"`, and `"teaches"` are assigned distinct indices and independent rows in the embedding table, sharing zero statistical parameters despite their shared lexical root.

5.1.4 Subword-level tokenization

Subword-level tokenization represents an optimal hybrid paradigm that synthesizes the strengths of word-level and character-level representations while systematically eliminating their core failure modes. Under this approach, frequent words are preserved as complete single tokens, while rare words, inflected variants, and compound terms are decomposed into smaller, morphologically meaningful subword chunks or base characters.

Applying subword tokenization to our running example decomposes rare or morphologically complex terms while preserving common lexical units:

```
["Mrs.","Ru","cast","le","was","downstairs",",","so","I","had","an","admirable","opportunity"
```
(5.4)

Here, the uncommon surname `"Rucastle"` is losslessly segmented into known subwords `["Ru", "cast", "le"]`, allowing the model to process the word without collapsing into `<UNK>` and without inflating the overall sequence length.

Advantages of subword-level tokenization

1. **Optimal vocabulary-length equilibrium**: Subword tokenizers operate with manageable vocabulary sizes (typically $|\mathcal{V}| \in [32{,}000, 100{,}000]$) while maintaining sequence lengths only slightly longer than word-level tokenization.

2. **Complete resolution of the OOV barrier**: Because the base vocabulary contains all fundamental characters (or raw bytes), any novel, unseen word can always be segmented into its constituent characters as a fallback. The model achieves 100% lexical coverage without ever needing an `<UNK>` token.

3. **Morphological parameter sharing**: Subword decomposition inherently captures shared prefixes, roots, and suffixes. For example, `"unhappiness"` decomposes into `["un", "happi", "ness"]`, allowing the negation prefix `"un-"` and noun suffix `"-ness"` to share statistical evidence across thousands of words.

4. **High compute and data efficiency**: Frequent words are processed in a single step, while computational capacity is dynamically allocated to dissecting rarer, more complex lexical structures.

Disadvantages of subword-level tokenization

1. **Non-intuitive segmentations**: Data-driven statistical subword algorithms occasionally create subword boundaries that do not align with human linguistic morphology, producing non-intuitive segmentations.

2. **Pre-tokenization pipeline dependency**: Subword tokenizers must be trained empirically on large representative text corpora prior to model pretraining, and tokenization models cannot be easily modified post-training without retraining the entire language model.

Table 5.1 summarizes the trade-offs across these three tokenization regimes.

Table 5.1: Systematic comparison across tokenization paradigms.

Property	Character-Level	Word-Level	Subword-Level (BPE)
Vocabulary size	Extremely small ($\lvert\mathcal{V}\rvert \approx 10^2$)	Massive ($\lvert\mathcal{V}\rvert \geq 10^5 - 10^6$)	Balanced ($\lvert\mathcal{V}\rvert \approx 3 \times 10^4 - 10^5$)
Sequence length	Longest ($4\times - 6\times$ words)	Shortest (1 token / word)	Moderate ($1.2\times - 1.4\times$ words)
OOV vulnerability	Zero (universal coverage)	Acute (frequent `<UNK>`)	Zero (character/byte fallback)
Morphological sharing	None (too granular)	None (isolated entries)	Extensive (shared subwords)
Attention compute cost	Highest ($O(N_{\text{char}}^2)$)	Lowest ($O(N_{\text{word}}^2)$)	Balanced ($O(N_{\text{subword}}^2)$)
Modern deployment	Specialized character models	Deprecated in modern LLMs	Standard across all modern LLMs

5.1.5 Byte Pair Encoding (BPE)

To construct an optimal subword vocabulary empirically from data without relying on handcrafted linguistic rules, modern Large Language Models deploy **Byte Pair Encoding (BPE)**. Originally invented by Philip Gage in 1994 as a statistical data compression algorithm [47], BPE was adapted to neural natural language processing by Sennrich, Haddow, and Birch in 2016 for neural machine translation [41].

Desiderata of the BPE algorithm

The fundamental design goals of BPE for natural language modeling are:

1. **Data-driven subword discovery**: Automatically identify high-utility subwords from raw corpus statistics without requiring language-specific grammatical rules or morphological dictionaries.

2. **Frequency-conditioned segmentation**: Map frequent character sequences (common words and frequent grammatical affixes) into single atomic tokens, while reserving multi-token decompositions for rare words.

3. **Explicit vocabulary budgeting**: Allow model designers to specify an exact target vocabulary budget V_{target} that satisfies memory and computational constraints.

4. **Lossless character/byte fallback**: Guarantee that any arbitrary sequence of text can be represented using base symbols without information loss.

The 5-step BPE training algorithm

The BPE vocabulary construction algorithm operates through an iterative greedy bottom-up merge procedure:

1. **Initialization**: Initialize the base vocabulary $\mathcal{V}_0$ with all individual base characters occurring across the training corpus $\mathcal{C}$. To represent whitespace losslessly, whitespace is either converted into an explicit symbol (such as '␣' or 'Ġ') or preserved directly via byte-level encoding. Every word in the training corpus is split into a contiguous sequence of individual characters.

2. **Adjacent pair frequency counting**: Scan the entire tokenized corpus and compute the frequency histogram of all contiguous adjacent token pairs (p_1, p_2):

$$\text{Freq}(p_1, p_2) = \sum_{w \in \mathcal{C}} \text{Count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{5.5}$$

3. **Greedy merge candidate selection**: Identify the most frequent adjacent pair (F_1, F_2) across the corpus:

$$(F_1, F_2) = \arg\max_{(p_1, p_2)} \text{Freq}(p_1, p_2). \tag{5.6}$$

   Assign this pair a new, unique merged token identifier $T = F_1 \circ F_2$, and log the merge rule $(F_1, F_2) \mapsto T$ into an ordered merge table $\mathcal{M}$.

4. **In-place corpus contraction**: Replace every adjacent occurrence of (F_1, F_2) across the entire corpus with the new symbol T, and expand the vocabulary:

$$\mathcal{V}_{k+1} = \mathcal{V}_k \cup \{T\}. \tag{5.7}$$

5. **Iterative repetition**: Repeat Steps 2 through 4 until reaching the target vocabulary budget $|\mathcal{V}| = V_{\text{target}}$ or until no pair frequency exceeds a minimum threshold.

Byte-level BPE: eliminating unknown tokens via UTF-8

In standard character-level BPE, multilingual corpora spanning non-Latin scripts (e.g., Cyrillic, Chinese, Arabic, mathematical symbols, emojis) can expand the base alphabet into thousands of unique Unicode characters. If an unseen Unicode character appears during inference, standard BPE still fails and outputs `<UNK>`.

To achieve universal lexical coverage, Radford et al. [42] introduced **Byte-Level BPE** for GPT-2. Byte-level BPE initializes the base alphabet not with Unicode characters, but with the **256 possible values of a raw byte** (0x00 through 0xFF). Because the UTF-8 standard encodes all 1,114,112 Unicode code points using variable-length sequences of 1 to 4 bytes, initializing with 256 byte tokens guarantees that *any digital file or textual string can be encoded without ever encountering an unknown token*:

$$\mathcal{V}_{\text{base}} = \{0\text{x}00, 0\text{x}01, \ldots, 0\text{x}FF\}, \quad |\mathcal{V}_{\text{base}}| = 256. \tag{5.8}$$

Byte-level BPE preserves ASCII characters directly as single bytes while naturally merging multi-byte UTF-8 sequences into frequent subwords during training.

Worked walkthrough: the slide example on "would a woodchuck chuck wood"

To trace the mechanical operation of BPE, consider the exact worked example presented in the lecture slides, operating on the tongue-twister corpus:

```
"would a woodchuck chuck wood"
```

Let whitespace be explicitly represented by the boundary symbol '␣'.
Step 1: Initial character encoding.
The raw corpus decomposes into 28 initial character tokens:

|w|o|u|l|d|␣|a|␣|w|o|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|w|o|o|d|

The initial base vocabulary comprises 10 unique character symbols:

$$\mathcal{V}_0 = \{\text{'␣','a','c','d','h','k','l','o','u','w'}\}, \quad |\mathcal{V}_0| = 10. \tag{5.9}$$

Step 2: Adjacent pair frequency counting.
Scanning the 28 contiguous tokens across the corpus produces the initial pair frequency histogram:

- ('w','o') : 3 (occurring in "would", "woodchuck", and "wood").
- ('␣','w') : 2, ('o','o') : 2, ('o','d') : 2.
- ('c','h') : 2, ('h','u') : 2, ('u','c') : 2, ('c','k') : 2, ('k','␣') : 2.
- All other pairs (such as ('o','u'), ('u','l'), ('l','d'), ('␣','a'), ('a','␣'), ('d','c'), ('␣','c')) appear exactly once (1).

Step 3 & 4: Merge 1.
The most frequent pair is ('w','o') with frequency 3. We create the merged token "wo" and update the vocabulary: $\mathcal{V}_1 = \mathcal{V}_0 \cup \{\text{"wo"}\}$. Replacing all occurrences of ('w','o') in the corpus yields:

|wo|u|l|d|␣|a|␣|wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|wo|o|d|

The sequence length contracts from 28 to 25 tokens.
Subsequent Merge Iterations.
At iteration 2, multiple pairs share the maximum frequency of 2. Following the slide progression:

- **Merge 2**: Pair ('␣',"wo") (frequency 2) merges into "␣wo". Corpus: |wo|u|l|d|␣|a|␣wo|o|d|c|h|u|c|k (23 tokens).
- **Merge 3**: Pair ("␣wo",'o') (frequency 2) merges into "␣woo". Corpus: |wo|u|l|d|␣|a|␣woo|d|c|h|u|c| (21 tokens).
- **Merge 4**: Pair ("␣woo",'d') (frequency 2) merges into "␣wood". Corpus: |wo|u|l|d|␣|a|␣wood|c|h|u|c (19 tokens).
- **Merge 5**: Pair ('c','h') (frequency 2) merges into "ch". Corpus: |wo|u|l|d|␣|a|␣wood|ch|u|c|k|␣|ch (17 tokens).
- **Merge 6**: Pair ("ch",'u') (frequency 2) merges into "chu". Corpus: |wo|u|l|d|␣|a|␣wood|chu|c|k|␣|c (15 tokens).
- **Merge 7**: Pair ("chu",'c') (frequency 2) merges into "chuc". Corpus: |wo|u|l|d|␣|a|␣wood|chuc|k|␣| (13 tokens).
- **Merge 8**: Pair ("chuc",'k') (frequency 2) merges into "chuck". Corpus: |wo|u|l|d|␣|a|␣wood|chuck|␣ (11 tokens).
- **Merge 9**: Pair ("wo",'u') merges into "wou". Corpus: |wou|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood| (10 tokens).

Table 5.2 documents the progressive contraction of sequence length alongside the expansion of the subword vocabulary across these merge steps.

Table 5.2: BPE merge progression on the corpus "would a woodchuck chuck wood".

Iter.	Merged Pair	New Token	Segmented Corpus State	Tokens
0	—	—	|w|o|u|l|d|␣|a|␣|w|o|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|w|o|o|d|	28
1	('w','o')	"wo"	|wo|u|l|d|␣|a|␣|wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣|wo|o|d|	25
2	('␣',"wo")	"␣wo"	|wo|u|l|d|␣|a|␣wo|o|d|c|h|u|c|k|␣|c|h|u|c|k|␣wo|o|d|	23
3	("␣wo",'o')	"␣woo"	|wo|u|l|d|␣|a|␣woo|d|c|h|u|c|k|␣|c|h|u|c|k|␣woo|d|	21
4	("␣woo",'d')	"␣wood"	|wo|u|l|d|␣|a|␣wood|c|h|u|c|k|␣|c|h|u|c|k|␣wood|	19
5	('c','h')	"ch"	|wo|u|l|d|␣|a|␣wood|ch|u|c|k|␣|ch|u|c|k|␣wood|	17
6	("ch",'u')	"chu"	|wo|u|l|d|␣|a|␣wood|chu|c|k|␣|chu|c|k|␣wood|	15
7	("chu",'c')	"chuc"	|wo|u|l|d|␣|a|␣wood|chuc|k|␣|chuc|k|␣wood|	13
8	("chuc",'k')	"chuck"	|wo|u|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood|	11
9	("wo",'u')	"wou"	|wou|l|d|␣|a|␣wood|chuck|␣|chuck|␣wood|	10

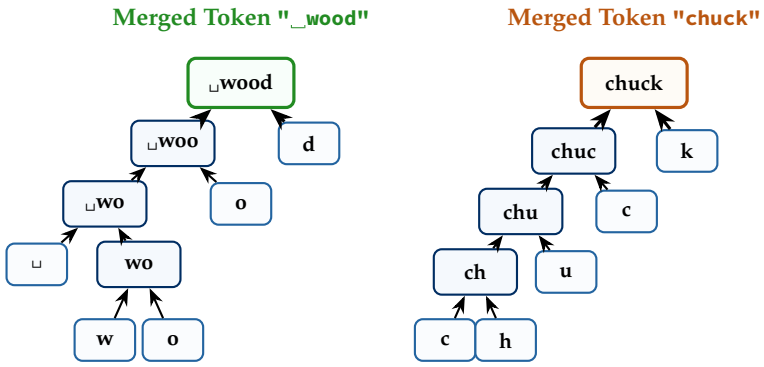


Figure 5.1: Hierarchical BPE merge tree representing the iterative composition of subwords into atomic tokens "␣wood" and "chuck" across the slide example.

Empirical compression dynamics and Zipf's law

As established empirically by Sennrich et al. [41] and emphasized in Slide 28, the number of tokens required to represent a corpus exhibits an exponential decay as a function of the number of merge iterations.

This behavior is a direct manifestation of **Zipf's law**: a small fraction of character combinations account for the vast majority of textual occurrences. Consequently, the first few thousand merges achieve dramatic text compression, collapsing frequent grammatical particles, affixes, and ubiquitous root words into single tokens. Beyond 30,000 to 50,000 merge operations, the marginal compression benefit flattens significantly, as subsequent merges capture increasingly rare and specialized domain words. In foundation models, vocabulary budgets typically range from 32,000 (e.g., LLaMA) to 100,000+ (e.g., GPT-4), striking an empirical balance between sequence length compression and embedding table memory overhead.

5.1.6 Special tokens in modern language models

In addition to linguistic subwords, neural sequence models require explicit structural control markers. These operational signals are designated as **special tokens**. Special tokens occupy dedicated indices in the vocabulary $\mathcal{V}$ and provide non-linguistic conditioning signals during training and autoregressive generation:

1. **<BOS> / <s> (Beginning of Sequence)**: Prepended to the input sequence to indicate the starting boundary of a text stream. In encoder-decoder and decoder-only generation, feeding <BOS> into the decoder serves as the initial prompt to trigger the generation of the very first output token when no prior context exists.

2. **<EOS> / </s> (End of Sequence)**: Appended to the end of target sentences. During autoregressive inference, generating <EOS> acts as the stopping criterion that halts generation, preventing unbounded repetition.

3. **<PAD> (Padding Token)**: When grouping sequences of variable lengths into a mini-batch tensor $\mathbf{X} \in \mathbb{R}^{B \times L_{\max}}$, shorter sequences are padded with <PAD> tokens to achieve rectangular tensor dimensions. Crucially, padding positions are masked out during self-attention so that real tokens do not attend to artificial padding.

4. **<UNK> (Unknown Token)**: Serves as a placeholder for words or characters absent from discrete vocabularies. In modern byte-level BPE models, <UNK> is rendered obsolete because any character can be decomposed into base UTF-8 bytes.

5. **<CLS> (Classification Token)**: Introduced in BERT [48], this token is prepended to the input sequence. Because its bidirectional attention representation pools contextual signals across all positions, the final hidden state of <CLS> is extracted as the sequence-level semantic summary for classification tasks.

6. **<SEP> (Separator Token)**: Demarcates boundaries between distinct segments within a single input (e.g., separating premise and hypothesis in natural language inference, or context and question in reading comprehension).

7. **<MASK> (Mask Token)**: Used during masked language model pretraining (e.g., BERT) to replace a subset of input tokens (typically 15%), forcing the bidirectional encoder to predict the original identity of the masked token from surrounding bidirectional context.

5.2 Positional Encodings

5.2.1 Motivation: permutation equivariance of self-attention

In Recurrent Neural Networks, sequential order is inherently baked into the temporal transition equation: $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$. By construction, token $\mathbf{x}_t$ cannot be processed until token $\mathbf{x}_{t-1}$ has updated the hidden state. In Convolutional Neural Networks, sequential order is preserved through localized 1D sliding receptive fields.

In stark contrast, the core mathematical operator of the Transformer—**Self-Attention**—possesses zero intrinsic knowledge of sequential or temporal order. As established in Theorem 5.1, self-attention is strictly **permutation-equivariant**.

Theorem 5.1: *Permutation equivariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L \times d}$ be an input sequence of L token vectors, and let $\mathbf{P} \in \{0,1\}^{L \times L}$ be an arbitrary permutation matrix satisfying $\mathbf{P}^\top \mathbf{P} = \mathbf{I}$. Under standard scaled dot-product attention:

$$\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V, \tag{5.10}$$

the operator is equivariant under input permutations:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{5.11}$$

Proof. Let $\mathbf{X}' = \mathbf{P}\mathbf{X}$. The linearly projected query and key matrices become $\mathbf{Q}' = \mathbf{P}\mathbf{X}\mathbf{W}_Q$ and $\mathbf{K}' = \mathbf{P}\mathbf{X}\mathbf{W}_K$. The unscaled inner-product score matrix evaluates to:

$$\mathbf{Q}'(\mathbf{K}')^\top = (\mathbf{P}\mathbf{X}\mathbf{W}_Q)(\mathbf{P}\mathbf{X}\mathbf{W}_K)^\top = \mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top. \tag{5.12}$$

Because the softmax operator evaluates independently along each row, applying a permutation matrix $\mathbf{P}$ on the left permutes the rows, while $\mathbf{P}^\top$ on the right permutes the columns:

$$\text{softmax}\left(\frac{\mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top. \tag{5.13}$$

Multiplying by the value projection $\mathbf{V}' = \mathbf{P}\mathbf{X}\mathbf{W}_V$:

$$\begin{aligned}\texttt{Attention}(\mathbf{P}\mathbf{X}) &= \left[\mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top\right](\mathbf{P}\mathbf{X}\mathbf{W}_V) \\ &= \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)(\mathbf{P}^\top\mathbf{P})\mathbf{X}\mathbf{W}_V \\ &= \mathbf{P}\,\texttt{Attention}(\mathbf{X}),\end{aligned} \tag{5.14}$$

where the identity $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ collapses the internal permutation. ■ □

Theorem 5.1 demonstrates that if the input sequence is permuted, the resulting representations undergo the exact same permutation with identical values. Self-attention operates on the sequence as an **unordered multiset** (a "bag of vectors").

As highlighted in Slide 32, this property creates an immediate syntactic dilemma. Consider two sentences composed of the exact same lexical units:

$$\begin{aligned}&\textbf{Input 1}\text{: } \textit{"I ate an apple"} \quad \Longrightarrow \quad \{\mathbf{x}_{\text{I}}, \mathbf{x}_{\text{ate}}, \mathbf{x}_{\text{an}}, \mathbf{x}_{\text{apple}}\}, \\ &\textbf{Input 2}\text{: } \textit{"an apple, I ate"} \quad \Longrightarrow \quad \{\mathbf{x}_{\text{an}}, \mathbf{x}_{\text{apple}}, \mathbf{x}_{\text{I}}, \mathbf{x}_{\text{ate}}\}.\end{aligned}$$

Without positional information, both sentences generate identical multiset representations, completely conflating grammatical subject, verb, and direct object. To enable Transformers to understand grammatical syntax, sequential order must be injected into the representations explicitly.

5.2.2 Taxonomy of positional representations

Positional representations can be categorized systematically across two independent architectural axes:

1. **Fixed vs. Learned**:
   - *Fixed Encodings*: Fully deterministic mathematical functions of position coordinates, requiring zero trainable parameters (e.g., Sinusoidal Encodings [1]).
   - *Learned Embeddings*: Trainable parameter matrices $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$ optimized via gradient descent alongside token embeddings (e.g., GPT-2 [42], BERT [48]).
2. **Absolute vs. Relative**:
   - *Absolute Positional Embeddings*: Assign an isolated vector to each discrete absolute coordinate $pos \in \{0, 1, \ldots, L-1\}$, additively combined with the token embedding prior to the first layer: $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$.
   - *Relative Positional Embeddings*: Parameterize the relative distance $j - i$ between tokens, injecting positional bias directly into attention logits during attention computation (e.g., Shaw et al. [49], RoPE [50], ALiBi [51]).

Table 5.3 summarizes this two-dimensional classification.

Table 5.3: Taxonomy of positional representations across modern architectures.

Paradigms	Fixed (Deterministic)	Learned (Data-Driven)
Absolute	**Sinusoidal Encodings** [1] sin/cos harmonic geometric frequencies	**Learned Absolute Embeddings** [42, 48] Trainable lookup table $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$
Relative	**Rotary Embeddings (RoPE)** [50] Orthogonal 2D rotation of queries/keys	**Shaw et al. Encodings** [49] Trainable relative bias matrix $\mathbf{S}_{\text{rel}}$ in attention

5.2.3 Absolute positional embeddings

Sinusoidal positional encodings: formulation and geometric clock intuition

In the landmark paper *Attention Is All You Need* [1], Vaswani et al. introduced fixed sinusoidal positional encodings. For a token at integer sequence position $pos \in \{0, 1, \ldots, L-1\}$ and feature dimension index $i \in \{0, 1, \ldots, \frac{d_{\text{model}}}{2} - 1\}$, the positional vector $\mathbf{p}_{pos} = \mathsf{PE}_{(pos,:)} \in \mathbb{R}^{d_{\text{model}}}$ is defined by:

$$\mathsf{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \tag{5.15}$$

$$\mathsf{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right). \tag{5.16}$$

The Geometric Clock Analogy.
To understand the geometric intuition underlying this formula, consider an analog clock with multiple hands rotating at different speeds. In a multi-dial clock:

- **Low-dimension channels ($i \to 0$)**: Oscillate at high angular frequencies $\omega_0 = 1.0$, corresponding to a rapid *seconds hand*. Wavelength $\lambda_0 = 2\pi \approx 6.28$ tokens. This rapid oscillation disambiguates immediate neighbors and local syntactic relationships.
- **High-dimension channels ($i \to \frac{d_{\text{model}}}{2} - 1$)**: Oscillate at low angular frequencies $\omega \to \frac{1}{10000}$, corresponding to a slow *hours hand*. Wavelength $\lambda_{\max} = 2\pi \cdot 10000 \approx 62{,}832$ tokens. This slow progression captures global sequence context and document flow.

Pairing sin and cos across dimension pairs $(2i, 2i+1)$ eliminates 1D wave aliasing. Because $\sin(\theta) = \sin(\pi - \theta)$, observing height along a single sine wave creates directional ambiguity (e.g., 2:00 and 10:00 produce identical vertical coordinates). By pairing sine and cosine, the vector $(\sin\theta, \cos\theta)$ traces a unique, unambiguous point along the unit circle S^1 in 2D Euclidean space.

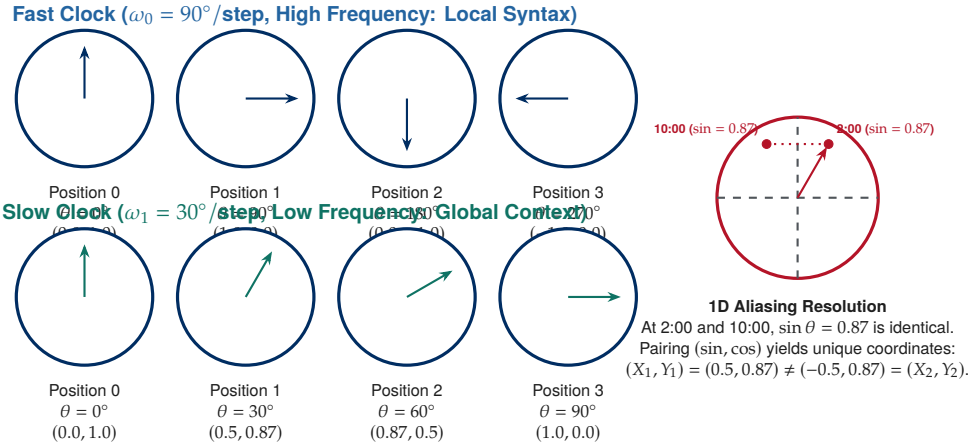


Figure 5.2: The geometric clock intuition for positional encodings. Each frequency channel pair acts as a rotating phasor. Fast clocks disambiguate immediate neighbors, while slow clocks track long-range sequences. Pairing sin and cos prevents 1D wave aliasing.

Numerical Calculations from Slide 35.
To illustrate the exact arithmetic, consider sequence position $pos = 1$ (the second token, under 0-indexed coordinates) with $d_{\text{model}} = 768$:

1. **Lowest channel ($i = 0$, even index $2i = 0$)**:

$$\text{PE}_{(1,0)} = \sin\left(\frac{1}{10000^{0/768}}\right) = \sin(1) \approx 0.8415. \tag{5.17}$$

2. **Lowest odd channel ($i = 0$, odd index $2i + 1 = 1$)**:

$$\text{PE}_{(1,1)} = \cos\left(\frac{1}{10000^{0/768}}\right) = \cos(1) \approx 0.5403. \tag{5.18}$$

3. **Intermediate channel ($i = 2$, even index $2i = 4$)**:

$$\text{PE}_{(1,4)} = \sin\left(\frac{1}{10000^{4/768}}\right) = \sin\left(\frac{1}{10000^{0.005208}}\right) \approx \sin\left(\frac{1}{1.0490}\right) = \sin(0.9533) \approx 0.8153. \tag{5.19}$$

These exact values (0.8415, 0.5403, 0.8153) reproduce the values in Slide 35.
Theoretical Properties: Periodicity, Linear Shift, and Similarity.
Sinusoidal encodings exhibit three pivotal theoretical properties:

1. **Positional Uniqueness**: The constant base 10000 guarantees that within the first $2\pi \cdot 10000 \approx 62{,}832$ positions, the slowest frequency does not complete a single cycle, ensuring that every position vector in operational memory is globally unique.

2. **The Linear Relative Shift Property**: For any fixed spatial offset $k \in \mathbb{Z}$, the positional vector at position $pos + k$ can be computed as a direct linear transformation of the vector at position pos:

   Theorem 5.2: *Linear relative transformation property*

   For any frequency $\omega_i = 10000^{-2i/d_{\text{model}}}$ and spatial offset $k \in \mathbb{Z}$, there exists an orthogonal rotation matrix $\mathbf{M}_{i,k} \in \mathbb{R}^{2\times 2}$ independent of pos such that:

   $$\begin{bmatrix} \mathsf{PE}_{(pos+k,2i)} \\ \mathsf{PE}_{(pos+k,2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \mathsf{PE}_{(pos,2i)} \\ \mathsf{PE}_{(pos,2i+1)} \end{bmatrix}. \tag{5.20}$$

   Proof. Applying the standard trigonometric angle addition identities:

   $$\sin(\omega_i(pos + k)) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k), \tag{5.21}$$
   $$\cos(\omega_i(pos + k)) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k). \tag{5.22}$$

   Expressing this system in matrix notation yields Equation (5.20). Because rotation matrix $\mathbf{M}_{i,k}$ depends strictly on offset k and is independent of pos, linear attention layers ($\mathbf{W}_Q, \mathbf{W}_K$) can easily learn to attend to relative shifts. ■ □

3. **Cosine Similarity Preservation Across Distance**: Computing the inner product between positional vectors at positions p_1 and p_2 evaluates to:

   $$\mathbf{p}_{p_1}^{(i)} \cdot \mathbf{p}_{p_2}^{(i)} = \sin(\omega_i p_1)\sin(\omega_i p_2) + \cos(\omega_i p_1)\cos(\omega_i p_2) = \cos(\omega_i(p_1 - p_2)). \tag{5.23}$$

   The similarity depends strictly on relative displacement $|p_1 - p_2|$. As verified in Slides 38 and 39, the cosine similarity Gram matrix exhibits a bright main diagonal (cos = 1) with smooth monotonic decay as distance increases.

Advantages and Disadvantages of Sinusoidal Encodings.
Pros: Zero trainable parameters; theoretically capable of generalizing to arbitrary sequence lengths; provable linear shift properties.
Cons: Rigid, hand-crafted inductive bias; cannot adapt empirically to complex non-linear document structures or software syntax trees.

Learned absolute positional embeddings

An alternative paradigm, deployed in GPT-2, GPT-3, and BERT [2, 42, 48], treats positional encodings as trainable embedding parameters stored in a matrix:

$$\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}. \tag{5.24}$$

Each integer coordinate $pos \in \{0, 1, \ldots, L_{\max} - 1\}$ indexes row $\mathbf{p}_{pos} = (\mathbf{E}_{\text{pos}})_{pos,:} \in \mathbb{R}^{d_{\text{model}}}$, updated via gradient descent alongside token embeddings.

Advantages and Disadvantages of Learned Embeddings.
Pros: Highly flexible; automatically learns task-specific positional priors directly from data; empirical evaluations in [1] showed identical BLEU scores to sinusoidal encodings.
Cons: Hard context cutoff at $L_{\max}$. If an input sequence exceeds $L_{\max}$ tokens during inference, the model has no embedding vector for position $L_{\max}$, failing completely without architectural modifications or complex position interpolation techniques.

5.2.4 Relative positional embeddings (Shaw et al.)

In natural language and software engineering source code, the precise absolute coordinate of a token is frequently less important than its **relative displacement** with respect to other tokens. Whether an active verb appears at index 12 or index 84, its grammatical subject is typically found 1 or 2 tokens earlier.

To incorporate this inductive bias directly into the attention mechanism, Shaw et al. [49] formulated **Relative Positional Representations**. Instead of adding absolute vectors to input token embeddings, relative positional embeddings inject pairwise distance information directly into the scaled dot-product attention logits (Slides 78–79):

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top + \mathbf{S}_{\text{rel}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{5.25}$$

where $\mathbf{S}_{\text{rel}} \in \mathbb{R}^{L\times L}$ is a relative position bias matrix. For query position i and key position j, the entry is defined by:

$$S_{\text{rel}}[i, j] = \mathbf{q}_i^\top \mathbf{a}^K_{j-i}, \tag{5.26}$$

where $\mathbf{a}^K_{j-i} \in \mathbb{R}^{d_k}$ is a learned vector representing the relative distance $\delta = j - i$.

To bound parameter count, relative positions are clipped to a maximum horizon $k_{\max}$:

$$\text{clip}(\delta, k_{\max}) = \max(-k_{\max}, \min(k_{\max}, \delta)). \tag{5.27}$$

Table 5.4 reproduces the relative coordinate matrix from Slide 78 across a 6-token sequence.

Table 5.4: Relative displacement matrix $\delta = j - i$ between queries (rows i) and keys (columns j).

Query \ Key	Key 1	Key 2	Key 3	Key 4	Key 5	Key 6
Query 1	0	+1	+2	+3	+4	+5
Query 2	−1	0	+1	+2	+3	+4
Query 3	−2	−1	0	+1	+2	+3
Query 4	−3	−2	−1	0	+1	+2
Query 5	−4	−3	−2	−1	0	+1
Query 6	−5	−4	−3	−2	−1	0

Asymmetry: why relative embeddings fit self-attention but not cross-attention

As emphasized in Slide 79, relative positional embeddings are mathematically natural for **self-attention**, but fundamentally problematic for **cross-attention**:

- In self-attention, queries and keys reside in the same physical sequence. Therefore, spatial difference $\delta = j - i$ has an unambiguous physical and topological meaning (e.g., "2 tokens before the query").
- In encoder-decoder cross-attention, queries originate from the target language sequence (the decoder) while keys originate from the source language sequence (the encoder).

Because linguistic word orders diverge substantially between languages (for example, English Subject-Verb-Object versus Japanese Subject-Object-Verb), there is no monotonic 1-to-1 linear correspondence between source and target indices. Computing a scalar distance $j_{\text{source}} - i_{\text{target}}$ has no meaningful linguistic or topological interpretation.

Consequently, cross-attention must rely on absolute representations or content-based addressing, whereas self-attention can deploy relative representations effectively.

5.3 The Attention Mechanism

5.3.1 Motivation: dynamic context routing vs. static aggregation

As established in Chapter 4, static word embeddings map discrete words to fixed vector coordinates $\mathbf{w} \in \mathbb{R}^d$. However, natural language is inherently polysemous and context-dependent: the noun *"bank"* occupies radically different semantic realms in *"the bank approved the mortgage"* versus *"the grassy bank of the river"*.

Prior to the Transformer, recurrent neural networks attempted to resolve context by recursively updating a hidden state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$. Yet in sequence-to-sequence encoder-decoder networks, compressing an entire input sequence into a single static context vector $\mathbf{h}_L$ induces an information compression bottleneck that destroys fine-grained details on long sequences.

The **Attention Mechanism** overcomes both limitations simultaneously. Rather than compressing context into a single static vector, attention allows every token representation to be dynamically synthesized at run-time as an input-dependent weighted combination of all other tokens in the sequence. Each token's representation is contextualized by dynamically routing information from the most relevant words in its surrounding environment.

5.3.2 The associative memory progression: dictionary lookup to continuous attention

To understand the mathematical mechanics of attention from first principles, it is instructive to trace its conceptual evolution from classical computer science data structures (Slides 48–52).

1. Classical dictionary lookup (discrete hash table)

In standard computer systems, a dictionary, associative array, or hash map stores information as a collection of M discrete key-value pairs:

$$\mathcal{D} = \{(k_1, v_1), (k_2, v_2), \ldots, (k_M, v_M)\}. \tag{5.28}$$

When a discrete search query q is submitted, the hash map evaluates an exact binary match against all keys:

$$\text{Lookup}(q, \mathcal{D}) = \begin{cases} v_m & \text{if } \exists\, m \text{ such that } q = k_m, \\ \texttt{None} & \text{otherwise.} \end{cases} \tag{5.29}$$

This classical retrieval is binary (0 or 1) and non-differentiable: if query q differs from key k_m by a single character or bit, the match fails completely.

2. Continuous dot-product matching (soft alignment)

Attention generalizes hash-table retrieval to continuous vector spaces:

- A **Query** vector $\mathbf{q} \in \mathbb{R}^{d_k}$ represents the search criterion: *"What information am I seeking?"*
- A set of M **Key** vectors $\mathbf{k}_j \in \mathbb{R}^{d_k}$ represent content labels: *"What features do I describe?"*
- A set of M **Value** vectors $\mathbf{v}_j \in \mathbb{R}^{d_v}$ represent the informative payload: *"What content do I supply?"*

Instead of checking binary equality $q = k_j$, we evaluate a continuous compatibility score $s_j \in \mathbb{R}$ using vector inner products (Slide 50):

$$s_j = \mathbf{q}^\top \mathbf{k}_j. \tag{5.30}$$

The dot product quantifies the directional alignment and projection of the query onto the key in the latent feature space.

3. Softmax probability normalization

Because raw inner products $s_j \in (-\infty, +\infty)$ are unbounded, we map them onto the probability simplex using the softmax function (Slide 52):

$$\alpha_j = \frac{\exp(s_j)}{\sum_{l=1}^{M} \exp(s_l)}, \quad \text{satisfying} \quad \alpha_j \in [0,1] \quad \text{and} \quad \sum_{j=1}^{M} \alpha_j = 1. \tag{5.31}$$

The scalar α_j represents the fractional attention weight allocated by query $\mathbf{q}$ to key $\mathbf{k}_j$.

4. Contextual value aggregation

Finally, the retrieved contextual vector $\mathbf{z}$ is computed as the weighted convex combination of all value vectors (Slide 50):

$$\mathbf{z} = \sum_{j=1}^{M} \alpha_j \mathbf{v}_j \in \mathbb{R}^{d_v}. \tag{5.32}$$

Because vector inner products, exponentials, and summations are smooth, differentiable operators, the entire retrieval mechanism can be trained end-to-end via gradient backpropagation.

5.3.3 Visual and geometric intuition: 3Blue1Brown projection perspective

To build a deep geometric intuition for self-attention, we examine the visual perspective popularized by Grant Sanderson (3Blue1Brown) [52].

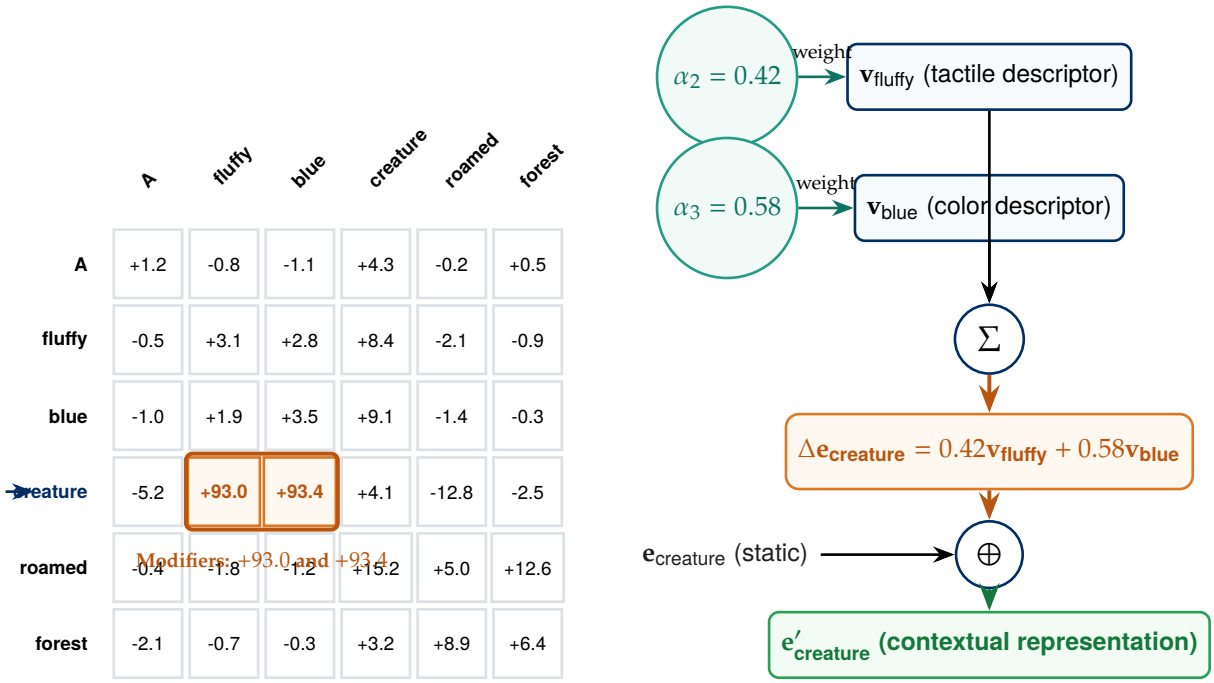
Consider tokens residing in a high-dimensional continuous embedding space $\mathbb{R}^{d_{\text{model}}}$. Initially, each token begins as a static context-free embedding vector $\mathbf{x}_i$. To allow tokens to communicate, the model introduces three learned linear transformation matrices:

$$\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \tag{5.33}$$

Geometrically, these matrices act as specialized feature lenses:

1. **Queries ($\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$)**: Represent a directional question or request formulated by token i. For example, an active verb projects a query asking: *"Where is my direct object noun?"*
2. **Keys ($\mathbf{k}_j = \mathbf{x}_j \mathbf{W}_K$)**: Represent feature advertisements broadcast by token j. A noun projects a key advertising: *"I am a direct object noun preceding you."*
3. **Inner Product ($\mathbf{q}_i^\top \mathbf{k}_j$)**: Geometrically computes the dot product between the query vector and the key vector. When a query and a key point in aligned directions in feature space ($\theta \approx 0^\circ$), their inner product is large and positive, indicating high semantic or syntactic affinity.
4. **Values ($\mathbf{v}_j = \mathbf{x}_j \mathbf{W}_V$)**: Represent the substantive informational payload that token j transmits to any token that attends to it.
5. **Contextual Update ($\Delta \mathbf{x}_i$)**: The weighted sum of values $\sum_j \alpha_{ij} \mathbf{v}_j$ produces a directional displacement vector. When added back into the residual stream, it physically displaces token i's coordinate in embedding space toward the specialized manifold region representing its contextualized meaning.

Figure 5.3 illustrates this geometric dynamic on the phrase *"A fluffy blue creature roamed the verdant forest"*.



(a) Query-Key compatibility matrix ($\mathbf{QK}^\top$) (b) Value matrix weighting and vector update ($\Delta\mathbf{e}$)

Figure 5.3: Geometric attention dynamics and matrix transformations in 3Blue1Brown's pedagogical breakdown [52]. (a) Pairwise Query-Key dot products evaluate compatibility across all token pairs; the Query for *"creature"* strongly matches the Keys for descriptive modifiers *"fluffy"* (+93.0) and *"blue"* (+93.4). (b) The attention table routes the corresponding Value vectors: after softmax normalization (0.42 and 0.58), the weighted sum produces a displacement vector $\Delta\mathbf{E}_4 = 0.42\mathbf{V}_2 + 0.58\mathbf{V}_3$, shifting the representation into the specialized manifold coordinates of a fluffy blue creature.

5.3.4 Scaled Dot-Product Attention and theoretical variance derivation

Packing all N queries, M keys, and M values into matrices $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} \in \mathbb{R}^{M \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{M \times d_v}$ yields the canonical matrix formulation of **Scaled Dot-Product Attention** [1] (Slide 55):

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{5.34}$$

where $\mathbf{M} \in \{0, -\infty\}^{N \times M}$ is an optional masking matrix, and softmax is applied row-wise.

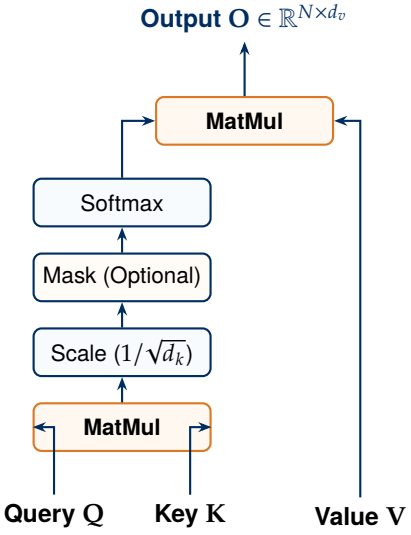


Figure 5.4: Computational dataflow of Scaled Dot-Product Attention [1].

Theoretical derivation: why the $\frac{1}{\sqrt{d_k}}$ scaling factor is mandatory

A central mathematical innovation of Vaswani et al. is the scaling factor $\frac{1}{\sqrt{d_k}}$. To prove why this scaling is mathematically indispensable, we analyze the statistical behavior of inner products in high dimensions.

Theorem 5.3: *Variance of high-dimensional dot products and softmax gradient vanishing*

Assume the components of query vector $\mathbf{q} = [q_1, \ldots, q_{d_k}]^\top$ and key vector $\mathbf{k} = [k_1, \ldots, k_{d_k}]^\top$ are independent and identically distributed random variables with mean zero and unit variance:

$$\mathbb{E}[q_m] = \mathbb{E}[k_m] = 0, \quad \mathbb{V}\text{ar}(q_m) = \mathbb{V}\text{ar}(k_m) = 1, \quad \forall m \in \{1, 2, \ldots, d_k\}. \tag{5.35}$$

Then the dot product $s = \mathbf{q}^\top \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$ has mean zero and variance d_k:

$$\mathbb{E}[s] = 0, \quad \mathbb{V}\text{ar}(s) = d_k. \tag{5.36}$$

Furthermore, as $d_k \to \infty$, the unscaled scores saturate the softmax operator, causing backpropagation gradients to vanish exponentially:

$$\lim_{|s| \to \infty} \frac{\partial \, \text{softmax}_i(\mathbf{s})}{\partial s_j} = 0. \tag{5.37}$$

Scaling by $\frac{1}{\sqrt{d_k}}$ preserves unit variance $\mathbb{V}\text{ar}\left(\frac{s}{\sqrt{d_k}}\right) = 1$ independently of dimension d_k, ensuring healthy, non-vanishing gradient flow.

Proof. By the linearity of expectation and independence:

$$\mathbb{E}[s] = \mathbb{E}\left[\sum_{m=1}^{d_k} q_m k_m\right] = \sum_{m=1}^{d_k} \mathbb{E}[q_m]\,\mathbb{E}[k_m] = \sum_{m=1}^{d_k} 0 \cdot 0 = 0. \tag{5.38}$$

For the variance, since the individual product terms $z_m = q_m k_m$ are mutually independent:

$$\mathbb{V}\text{ar}(s) = \mathbb{V}\text{ar}\left(\sum_{m=1}^{d_k} q_m k_m\right) = \sum_{m=1}^{d_k} \mathbb{V}\text{ar}(q_m k_m). \tag{5.39}$$

Using the identity $\mathbb{V}\text{ar}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$:

$$\mathbb{V}\text{ar}(q_m k_m) = \mathbb{E}[q_m^2 k_m^2] - (\mathbb{E}[q_m k_m])^2 = \mathbb{E}[q_m^2]\,\mathbb{E}[k_m^2] - 0 = (1)(1) = 1. \tag{5.40}$$

Summing across all d_k dimensions:

$$\mathbb{V}\text{ar}(s) = \sum_{m=1}^{d_k} 1 = d_k. \tag{5.41}$$

The standard deviation of the raw dot product is therefore $\sigma_s = \sqrt{d_k}$. For a typical projection dimension $d_k = 64$, standard deviation is $\sigma_s = 8$. Scores frequently take values in $[-24, +24]$.

Now consider the derivative of the softmax function $\sigma_i = \text{softmax}_i(\mathbf{s})$:

$$\frac{\partial \sigma_i}{\partial s_j} = \sigma_i(\delta_{ij} - \sigma_j). \tag{5.42}$$

When the inputs $\mathbf{s}$ have large magnitudes, the largest element $s_{\max}$ dominates the exponent sum:

$$\sigma_{\max} = \frac{\exp(s_{\max})}{\sum_l \exp(s_l)} \approx 1, \quad \sigma_j \approx 0 \quad (\forall j \neq \max). \tag{5.43}$$

Evaluating the Jacobian in Equation (5.42) under this saturation regime:

$$\text{For } i = j = \max: \quad \frac{\partial \sigma_{\max}}{\partial s_{\max}} = \sigma_{\max}(1 - \sigma_{\max}) \approx 1(1-1) = 0, \tag{5.44}$$

$$\text{For } i \neq j: \quad \frac{\partial \sigma_i}{\partial s_j} = -\sigma_i \sigma_j \approx -(0)(0) = 0. \tag{5.45}$$

All partial derivatives vanish completely. The model enters a plateau where gradient backpropagation cannot adjust the projection weights $\mathbf{W}_Q$ and $\mathbf{W}_K$.

Dividing by the scalar $\sqrt{d_k}$ normalizes the variance:

$$\mathbb{V}\text{ar}\left(\frac{s}{\sqrt{d_k}}\right) = \frac{1}{(\sqrt{d_k})^2}\,\mathbb{V}\text{ar}(s) = \frac{1}{d_k}(d_k) = 1. \tag{5.46}$$

Regardless of how large d_k is chosen, the variance of the scaled attention scores $\tilde{s} = \frac{s}{\sqrt{d_k}}$ remains strictly unitary, maintaining the logits within the linear, steep gradient regime of the softmax function. ■ □

5.3.5 Complete numerical worked example: reproducing the lecture calculations

To illustrate the matrix operations with complete numerical transparency, we reproduce the step-by-step worked example from Slides 57 through 65, tracking every calculation down to four decimal places.

Consider an input sequence of $N = 3$ tokens in a 2-dimensional latent embedding space ($d_{\text{model}} = 2$), with projection dimensions $d_k = d_v = 2$:

$$\mathbf{X} = \begin{bmatrix} 0.1 & 0.5 \\ 0.1 & 0.2 \\ 0.9 & 0.9 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{W}_Q = \begin{bmatrix} 0.9 & 0.9 \\ 0.5 & 0.3 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 1.0 & 0.9 \\ 1.0 & 0.6 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 0.8 & 0.9 \\ 0.9 & 1.0 \end{bmatrix}. \tag{5.47}$$

Step 1: Linear projection into Queries, Keys, and Values.
Multiplying $\mathbf{X}$ by each projection matrix yields:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q = \begin{bmatrix} 0.1(0.9) + 0.5(0.5) & 0.1(0.9) + 0.5(0.3) \\ 0.1(0.9) + 0.2(0.5) & 0.1(0.9) + 0.2(0.3) \\ 0.9(0.9) + 0.9(0.5) & 0.9(0.9) + 0.9(0.3) \end{bmatrix} = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix}, \tag{5.48}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K = \begin{bmatrix} 0.1(1.0) + 0.5(1.0) & 0.1(0.9) + 0.5(0.6) \\ 0.1(1.0) + 0.2(1.0) & 0.1(0.9) + 0.2(0.6) \\ 0.9(1.0) + 0.9(1.0) & 0.9(0.9) + 0.9(0.6) \end{bmatrix} = \begin{bmatrix} 0.60 & 0.39 \\ 0.30 & 0.21 \\ 1.80 & 1.35 \end{bmatrix}, \tag{5.49}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V = \begin{bmatrix} 0.1(0.8) + 0.5(0.9) & 0.1(0.9) + 0.5(1.0) \\ 0.1(0.8) + 0.2(0.9) & 0.1(0.9) + 0.2(1.0) \\ 0.9(0.8) + 0.9(0.9) & 0.9(0.9) + 0.9(1.0) \end{bmatrix} = \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix}. \tag{5.50}$$

Step 2: Unscaled dot-product computation.
Evaluating the matrix multiplication $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{3\times 3}$:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix} \begin{bmatrix} 0.60 & 0.30 & 1.80 \\ 0.39 & 0.21 & 1.35 \end{bmatrix} = \begin{bmatrix} 0.2976 & 0.1524 & 0.9360 \\ 0.1725 & 0.0885 & 0.5445 \\ 1.1772 & 0.6048 & 3.7260 \end{bmatrix}. \tag{5.51}$$

Focusing on the first token ($\mathbf{q}_1 = [0.34, 0.24]$), the individual dot products evaluate to (Slides 61–63):

$$s_{1,1} = \mathbf{q}_1^\top \mathbf{k}_1 = 0.34(0.60) + 0.24(0.39) = 0.2040 + 0.0936 = 0.2976, \tag{5.52}$$

$$s_{1,2} = \mathbf{q}_1^\top \mathbf{k}_2 = 0.34(0.30) + 0.24(0.21) = 0.1020 + 0.0504 = 0.1524, \tag{5.53}$$

$$s_{1,3} = \mathbf{q}_1^\top \mathbf{k}_3 = 0.34(1.80) + 0.24(1.35) = 0.6120 + 0.3240 = 0.9360. \tag{5.54}$$

Step 3: Scaling by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.7071$.
Dividing row 1 by $\sqrt{2}$ yields the scaled scores (Slide 64):

$$\tilde{s}_{1,1} = \frac{0.2976}{\sqrt{2}} \approx 0.2104, \tag{5.55}$$

$$\tilde{s}_{1,2} = \frac{0.1524}{\sqrt{2}} \approx 0.1078, \tag{5.56}$$

$$\tilde{s}_{1,3} = \frac{0.9360}{\sqrt{2}} \approx 0.6619. \tag{5.57}$$

Step 4: Softmax probability normalization.
Exponentiating and normalizing row 1 across the three keys:

$$\exp(0.2104) \approx 1.2342, \quad \exp(0.1078) \approx 1.1138, \quad \exp(0.6619) \approx 1.9385, \tag{5.58}$$

$$\text{Partition Sum } Z_1 = 1.2342 + 1.1138 + 1.9385 = 4.2865, \tag{5.59}$$

$$\alpha_{1,1} = \frac{1.2342}{4.2865} \approx 0.2879, \tag{5.60}$$

$$\alpha_{1,2} = \frac{1.1138}{4.2865} \approx 0.2598, \tag{5.61}$$

$$\alpha_{1,3} = \frac{1.9385}{4.2865} \approx 0.4522. \tag{5.62}$$

Notice that $\sum_{j=1}^{3} \alpha_{1,j} = 0.2879 + 0.2598 + 0.4522 = 0.9999 \approx 1.0000$.
Step 5: Weighted value aggregation.
The contextual output vector $\mathbf{o}_1$ for token 1 is computed as the linear combination of values (Slide 65):

$$\begin{aligned} \mathbf{o}_1 &= \alpha_{1,1}\mathbf{v}_1 + \alpha_{1,2}\mathbf{v}_2 + \alpha_{1,3}\mathbf{v}_3 \\ &= 0.2879[0.53, 0.59]^\top + 0.2598[0.26, 0.29]^\top + 0.4522[1.53, 1.71]^\top \\ &= [0.1526, 0.1699]^\top + [0.0675, 0.0753]^\top + [0.6919, 0.7733]^\top \\ &\approx [0.9120, 1.0185]^\top \approx [0.91, 1.02]^\top. \end{aligned} \tag{5.63}$$

Evaluating all three rows simultaneously via matrix multiplication $\mathbf{O} = \mathbf{AV} \in \mathbb{R}^{3\times 2}$ yields:

$$\mathbf{O} = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix} \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 & 1.02 \\ 0.85 & 0.95 \\ 1.29 & 1.44 \end{bmatrix}. \tag{5.64}$$

This reproduces the exact tensor calculations displayed in the lecture slides.

5.4 Attention Variants: Self, Masked, and Cross

5.4.1 Architectural motivation for diverse attention types

Within the full Transformer architecture, attention is not applied in a monolithic, uniform configuration. Depending on whether a sublayer is performing **representation extraction**, **causal sequence generation**, or **cross-lingual conditioning**, the visibility of tokens and the sources of queries, keys, and values must satisfy distinct structural constraints:

1. **Representation Learning (Encoder)**: Requires an unconstrained bidirectional receptive field where every token can freely inspect its complete surrounding linguistic context (both left and right).
2. **Autoregressive Generation (Decoder)**: Mandates strict causal isolation, guaranteeing that the model cannot peek at future tokens during generation or training.
3. **Sequence-to-Sequence Grounding (Encoder-Decoder)**: Requires a cross-attention bridge that allows target-language decoder tokens to query and extract information dynamically from the source-language encoder memory.

Figure 5.5 illustrates the structural distinction across these three attention mechanisms.

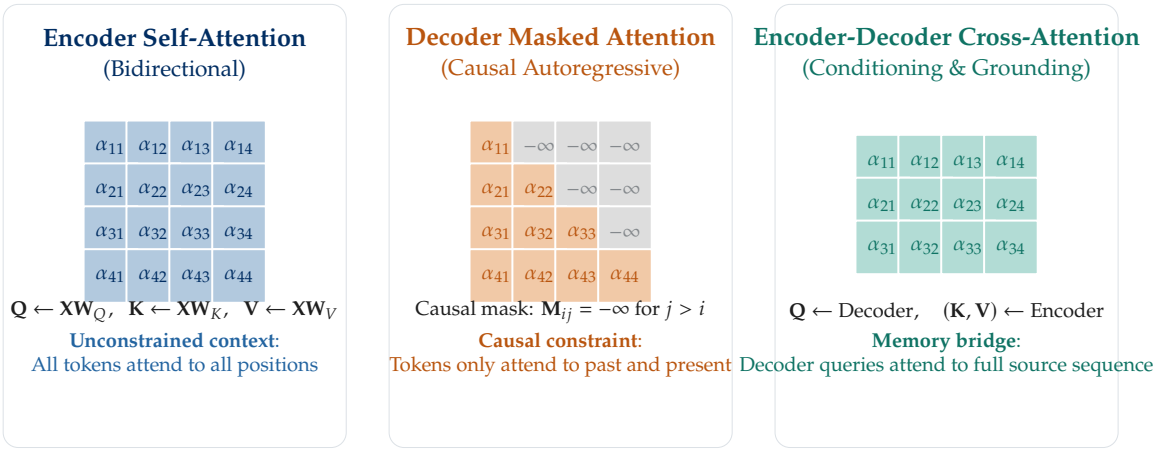


Figure 5.5: The triad of attention mechanisms across the Transformer architecture: (a) Bidirectional Encoder Self-Attention, (b) Causal Decoder Masked Self-Attention, and (c) Encoder-Decoder Cross-Attention.

5.4.2 Bidirectional Encoder Self-Attention

General idea

In the Transformer Encoder, the primary objective is producing uncompressed, deeply contextualized representations of the input sequence. Because the encoder does not perform autoregressive generation, there is no risk of future information leakage. Therefore, every token is granted an unconstrained, bidirectional receptive field: token i can attend to tokens in the past ($j < i$), the present ($j = i$), and the future ($j > i$).

Mathematical formulation

Queries, keys, and values all derive from the exact same input tensor $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ via linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{5.65}$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. No masking matrix is applied ($\mathbf{M} = \mathbf{0}$). The bidirectional attention operator evaluates:

$$\mathbf{A}_{\text{enc}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right), \quad \mathbf{O}_{\text{enc}} = \mathbf{A}_{\text{enc}}\mathbf{V} \in \mathbb{R}^{L_{\text{enc}} \times d_v}. \tag{5.66}$$

Every element $\alpha_{ij} > 0$, enabling full all-to-all information exchange.

Concrete natural language example: polysemy disambiguation

Consider the sentence: *"The bank of the river was overflowing."* The word *"bank"* is polysemous (financial institution vs. river embankment). When computing the representation for *"bank"* (index 2), an unconstrained bidirectional encoder computes queries that match against the key of the future token *"river"* (index 5). The resulting high attention weight $\alpha_{2,5}$ routes the value features of *"river"* into the output representation of *"bank"*, resolving its meaning into a geological riverbank.

5.4.3 Decoder Masked Causal Self-Attention

General idea

In autoregressive text generation, the probability distribution of the next token y_t depends strictly on previously generated tokens: $P(y_t \mid y_1, \ldots, y_{t-1})$. During training, modern hardware processes all target tokens simultaneously in a single forward pass using **Teacher Forcing**. However, if token t were allowed to attend to tokens $t+1, \ldots, T$, it would trivially copy the target token, destroying the generative learning task.

To prevent future information leakage while preserving full parallel training efficiency, the decoder applies **Causal Masking**.

Mathematical formulation

Causal masking adds an upper-triangular mask matrix $\mathbf{M} \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{dec}}}$ to the unscaled query-key logits prior to the softmax operator (Slide 70):

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \quad \text{(past and present tokens: valid)}, \\ -\infty & \text{if } j > i \quad \text{(future tokens: masked)}. \end{cases} \tag{5.67}$$

The masked self-attention operator evaluates:

$$\mathbf{A}_{\text{dec}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right), \quad \mathbf{O}_{\text{dec}} = \mathbf{A}_{\text{dec}}\mathbf{V} \in \mathbb{R}^{L_{\text{dec}} \times d_v}. \tag{5.68}$$

Because $\exp(-\infty) = 0$, applying the softmax operator along row i yields:

$$\alpha_{ij} = \frac{\exp(s_{ij} + M_{ij})}{\sum_{l=1}^{L_{\text{dec}}} \exp(s_{il} + M_{il})} = \begin{cases} \frac{\exp(s_{ij})}{\sum_{l=1}^{i} \exp(s_{il})} & \text{for } j \le i, \\ 0 & \text{for } j > i. \end{cases} \tag{5.69}$$

Future positions receive strictly zero attention weight. As noted in Slide 70, computing all pairwise dot products via matrix multiplication and masking invalid entries is dramatically faster on GPUs than executing sequential loops.

Concrete worked example: autoregressive generation under causal masking

Consider an autoregressive target sequence of $T = 3$ tokens: `["<BOS>", "The", "cat"]`. Suppose scaled query-key logits evaluate to:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} = \begin{bmatrix} 1.2 & 0.8 & 0.4 \\ 0.5 & 1.5 & 1.1 \\ 0.3 & 0.9 & 2.1 \end{bmatrix}. \tag{5.70}$$

Adding the causal mask $\mathbf{M}$ produces:

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M} = \begin{bmatrix} 1.2 & -\infty & -\infty \\ 0.5 & 1.5 & -\infty \\ 0.3 & 0.9 & 2.1 \end{bmatrix}. \tag{5.71}$$

Evaluating the row-wise softmax yields the strictly lower-triangular attention matrix:

$$\mathbf{A}_{\text{masked}} = \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 \\ 0.2689 & 0.7311 & 0.0000 \\ 0.1171 & 0.2133 & 0.6696 \end{bmatrix}. \tag{5.72}$$

Token 1 (`"<BOS>"`) attends exclusively to itself with weight 1.0. Token 2 (`"The"`) attends to `"<BOS>"` (0.27) and itself (0.73), but is strictly isolated from `"cat"` (0.0). Causal autoregressive integrity is rigorously preserved.

5.4.4 Encoder-Decoder Cross-Attention

General idea

Encoder-Decoder Cross-Attention acts as the cognitive bridge linking the target-generation decoder to the source-encoding encoder (Slide 71). It enables the decoder to dynamically query the full source sequence representation at each step of text generation.

Mathematical formulation and asymmetric dimensions

In cross-attention, the inputs are strictly decoupled:

- **Queries (Q)**: Linearly projected from the decoder's prior sublayer representations $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{L_{\text{dec}} \times d_{\text{model}}}$:
$$\mathbf{Q} = \mathbf{H}_{\text{dec}}\mathbf{W}_Q \in \mathbb{R}^{L_{\text{dec}} \times d_k}. \tag{5.73}$$
- **Keys (K) and Values (V)**: Linearly projected from the terminal representations of the encoder stack $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$:
$$\mathbf{K} = \mathbf{H}_{\text{enc}}\mathbf{W}_K \in \mathbb{R}^{L_{\text{enc}} \times d_k}, \quad \mathbf{V} = \mathbf{H}_{\text{enc}}\mathbf{W}_V \in \mathbb{R}^{L_{\text{enc}} \times d_v}. \tag{5.74}$$

The cross-attention operator evaluates:

$$\mathbf{A}_{\text{cross}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{enc}}}, \quad \mathbf{O}_{\text{cross}} = \mathbf{A}_{\text{cross}}\mathbf{V} \in \mathbb{R}^{L_{\text{dec}} \times d_v}. \tag{5.75}$$

Critical Dimensional and Masking Properties:

1. **Asymmetric sequence lengths**: The source sequence length L_{enc} and target sequence length L_{dec} do not need to be equal ($L_{\text{dec}} \neq L_{\text{enc}}$). The query-key product produces an $L_{\text{dec}} \times L_{\text{enc}}$ alignment matrix, which multiplies $\mathbf{V} \in \mathbb{R}^{L_{\text{enc}} \times d_v}$ to yield an output tensor matching the decoder's length $\mathbb{R}^{L_{\text{dec}} \times d_v}$.
2. **Zero causal masking**: Because the encoder has already completely processed the source sequence, the entire source text is available. The decoder can attend across all encoder positions without causal masking (Slide 71).

Concrete worked example: cross-lingual machine translation

Consider translating English into French:

$$\textbf{Source}: \texttt{["The", "black", "cat"]}\ (L_{\text{enc}} = 3) \implies \textbf{Target}: \texttt{["Le", "chat", "noir"]}\ (L_{\text{dec}} = 3).$$

When the decoder is positioned at token 2 (`"chat"`), its query vector $\mathbf{q}_{\text{chat}}$ searches for the corresponding source entity. The query computes inner products against the encoder keys:

$$\mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{The}} = 0.5, \quad \mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{black}} = 1.2, \quad \mathbf{q}_{\text{chat}}^{\top}\mathbf{k}_{\text{cat}} = 6.8. \tag{5.76}$$

Applying softmax allocates over 99% of the attention weight to the source key $\mathbf{k}_{\text{cat}}$. Cross-attention dynamically extracts the value payload $\mathbf{v}_{\text{cat}}$, allowing the decoder to generate `"chat"` conditioned directly on the source noun.

5.5 Transformer Blocks and Information Flow

5.5.1 Multi-Head Attention (MHA)

Motivation: why a single attention head is insufficient

A single scaled dot-product attention head computes a single weighted average over values for each token. However, in natural language syntax and semantics, a single word simultaneously participates in multiple, distinct linguistic relationships (Slide 72).

Consider the word *"making"* in the sentence *"The programmer was making rapid progress on the compiler"*. The token *"making"* simultaneously participates in:

1. **Syntactic subject agreement**: Connecting back to the subject *"programmer"*.
2. **Direct object selection**: Connecting forward to the nominal object *"progress"*.
3. **Adverbial modification**: Linking with the adjective modifier *"rapid"*.
4. **Prepositional attachment**: Grounding into the prepositional phrase *"on the compiler"*.

If forced through a single attention head, all these disparate relational vectors are blended together into a single partition sum, diluting specialized information.

Multi-Head Attention resolves this representational bottleneck by projecting queries, keys, and values into h distinct, lower-dimensional representation subspaces in parallel (Slides 72–73). Each attention head is parameterized by its own projection matrices, allowing individual heads to specialize in different linguistic patterns (e.g., Head 1 tracks syntax, Head 2 resolves coreference, Head 3 models semantic associations).

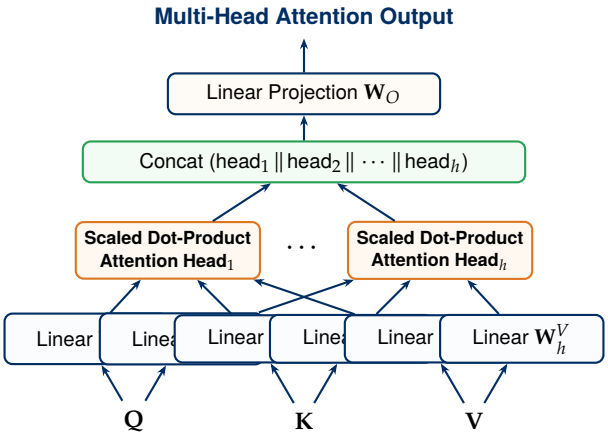


Figure 5.6: Architecture of Multi-Head Attention (MHA) [1]. Queries, Keys, and Values are projected into h parallel representation subspaces, evaluated independently via Scaled Dot-Product Attention, concatenated, and projected through $\mathbf{W}_O$.

Mathematical formulation

Let h denote the number of parallel attention heads. In the canonical architecture, projection dimensions are scaled such that $d_k = d_v = d_{\text{model}}/h$. For example, with $d_{\text{model}} = 512$ and $h = 8$, each head operates on subspace dimension $d_k = 64$.

For each head $i \in \{1, 2, \ldots, h\}$, projections are parameterized by learned matrices:

$$\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \tag{5.77}$$

Each individual head evaluates scaled dot-product attention in its designated subspace:

$$\text{head}_i = \texttt{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right) \in \mathbb{R}^{L \times d_v}. \tag{5.78}$$

The outputs of all h heads are concatenated along the feature dimension and projected back to the model dimension d_{model} using an output projection matrix $\mathbf{W}_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$:

$$\texttt{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)\,\mathbf{W}_O. \tag{5.79}$$

Parameter and computational invariance

Crucially, Multi-Head Attention incurs the **exact same computational and parameter cost** as a single full-dimensional attention head:

- In a single head operating on d_{model}: The projection parameter count is $3\times(d_{\text{model}}\times d_{\text{model}}) = 3d_{\text{model}}^2$.
- In Multi-Head Attention with h heads of dimension $d_k = d_{\text{model}}/h$: Each head requires $3 \times (d_{\text{model}} \times \frac{d_{\text{model}}}{h}) = \frac{3d_{\text{model}}^2}{h}$ parameters. Summing across all h heads yields: $h \times \frac{3d_{\text{model}}^2}{h} = 3d_{\text{model}}^2$. Adding the output projection $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ (d_{model}^2 parameters) produces identical parameter footprints.

Due to reduced subspace dimensions ($d_k = d_{\text{model}}/h$), total compute time is comparable to single-head attention while achieving vastly superior representational diversity.

5.5.2 Position-wise Feed-Forward Networks (FFN)

Architecture and non-linear expansion

While attention mechanisms model horizontal relationships across different sequence positions, they are linear weighted combinations of values. To introduce deep non-linear transformations and feature interactions, every Transformer layer incorporates a position-wise **Feed-Forward Network** (FFN) applied identically and independently to each token vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\texttt{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2, \tag{5.80}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$.

In the original Transformer [1], the inner hidden dimension is expanded by a factor of 4:

$$d_{\text{ff}} = 4d_{\text{model}}. \tag{5.81}$$

For example, in the base model ($d_{\text{model}} = 512$), the intermediate dimension expands to $d_{\text{ff}} = 2048$ before contracting back to 512.

The FFN as a key-value associative memory

Recent theoretical investigations [53] demonstrate that Transformer FFN layers operate as **key-value associative memories** storing factual knowledge acquired during pretraining:

- The first layer $\mathbf{W}_1$ acts as a key detector: each intermediate neuron activates when a token's contextual state matches specific conceptual, semantic, or encyclopedic patterns.
- The non-linear activation (ReLU or GELU) enforces selective firing.
- The second layer $\mathbf{W}_2$ acts as a value provider, retrieving and injecting factual attributes and token updates into the residual stream.

The FFN layer accounts for approximately two-thirds of the total parameter count in a standard Transformer block.

5.5.3 Residual connections (skip connections)

In deep architectures stacking multiple layers ($N = 6$ to 96), backpropagating gradients through consecutive non-linear matrix transformations leads to rapid gradient decay or explosion. To resolve this, Vaswani et al. adopted **Residual Connections** [54] around every sublayer (Slide 75):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}), \tag{5.82}$$

where $\mathcal{F}(\mathbf{x})$ represents the Multi-Head Attention or Feed-Forward transformation.

Mathematical gradient analysis: the identity superhighway

To prove why residual connections prevent gradient extinction, we analyze the Jacobian of the residual update during backpropagation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\mathbf{I} + \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}. \tag{5.83}$$

The presence of the identity matrix $\mathbf{I}$ establishes an unobstructed **gradient superhighway**. Even if the Jacobian of the learned sublayer vanishes entirely ($\frac{\partial \mathcal{F}}{\partial \mathbf{x}} \to \mathbf{0}$), the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ propagates directly back to earlier layers without attenuation. This identity term enables the successful training of models spanning dozens or hundreds of stacked layers.

5.5.4 Layer Normalization

To ensure numerical stability and prevent activation values from exploding across deep networks, each sublayer is paired with **Layer Normalization** [55] (Slide 76).

Formulation and comparison with Batch Normalization

Unlike Batch Normalization (which computes statistics across the batch dimension for each feature, making it sensitive to batch size and variable sequence padding), Layer Normalization computes mean and variance statistics across the feature channels for each token vector independently:

Definition 5.1: *Layer Normalization*

For a token vector $\mathbf{x} = [x_1, x_2, \ldots, x_{d_{\text{model}}}]^\top \in \mathbb{R}^{d_{\text{model}}}$, Layer Normalization evaluates:

$$\mu = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} x_j, \quad \sigma^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (x_j - \mu)^2, \tag{5.84}$$

$$\texttt{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \tag{5.85}$$

where $\epsilon > 0$ is a small constant (e.g., 10^{-5}) added for numerical stability, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_{\text{model}}}$ are learnable affine gain and bias parameters initialized to $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$.

Because statistics μ and σ^2 depend strictly on the individual token vector and are independent of batch size or neighboring tokens, Layer Normalization operates identically during training and evaluation, providing exceptional stability for variable-length natural language sequences.

Architectural placement: Post-LN vs. Pre-LN

- **Post-LN (Original Transformer [1])**: Normalization is applied *after* the residual sum:

$$\mathbf{y} = \texttt{LayerNorm}(\mathbf{x} + \mathcal{F}(\mathbf{x})). \tag{5.86}$$

  While mathematically elegant, Post-LN models suffer from vanishing gradients in early layers, requiring delicate learning rate warm-up schedules.

- **Pre-LN (Modern Standard [42, 56])**: Normalization is applied to the input *before* the sublayer, preserving an unbroken identity shortcut:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\left(\texttt{LayerNorm}(\mathbf{x})\right). \tag{5.87}$$

  Pre-LN significantly stabilizes gradient flow, enabling deeper stacking without warm-up instability.

5.5.5 End-to-end Transformer assembly and the translation journey

Full architectural assembly

Assembling these core blocks produces the canonical Encoder-Decoder Transformer architecture (Figure 5.7). The Encoder consists of a stack of $N = 6$ identical layers, each containing a Bidirectional Multi-Head Self-Attention sublayer and an FFN sublayer. The Decoder consists of a stack of $N = 6$ identical layers, each containing a Causal Masked Multi-Head Self-Attention sublayer, an Encoder-Decoder Multi-Head Cross-Attention sublayer, and an FFN sublayer.

Step-by-step translation journey: English to Italian

To understand how information traverses the entire architecture, consider translating the English sentence *"The bank of the river"* into Italian: *"La riva del fiume"*.
Phase 1: Source Sequence Ingestion in the Encoder.

1. **Tokenization**: The source text decomposes into $S = 5$ tokens:

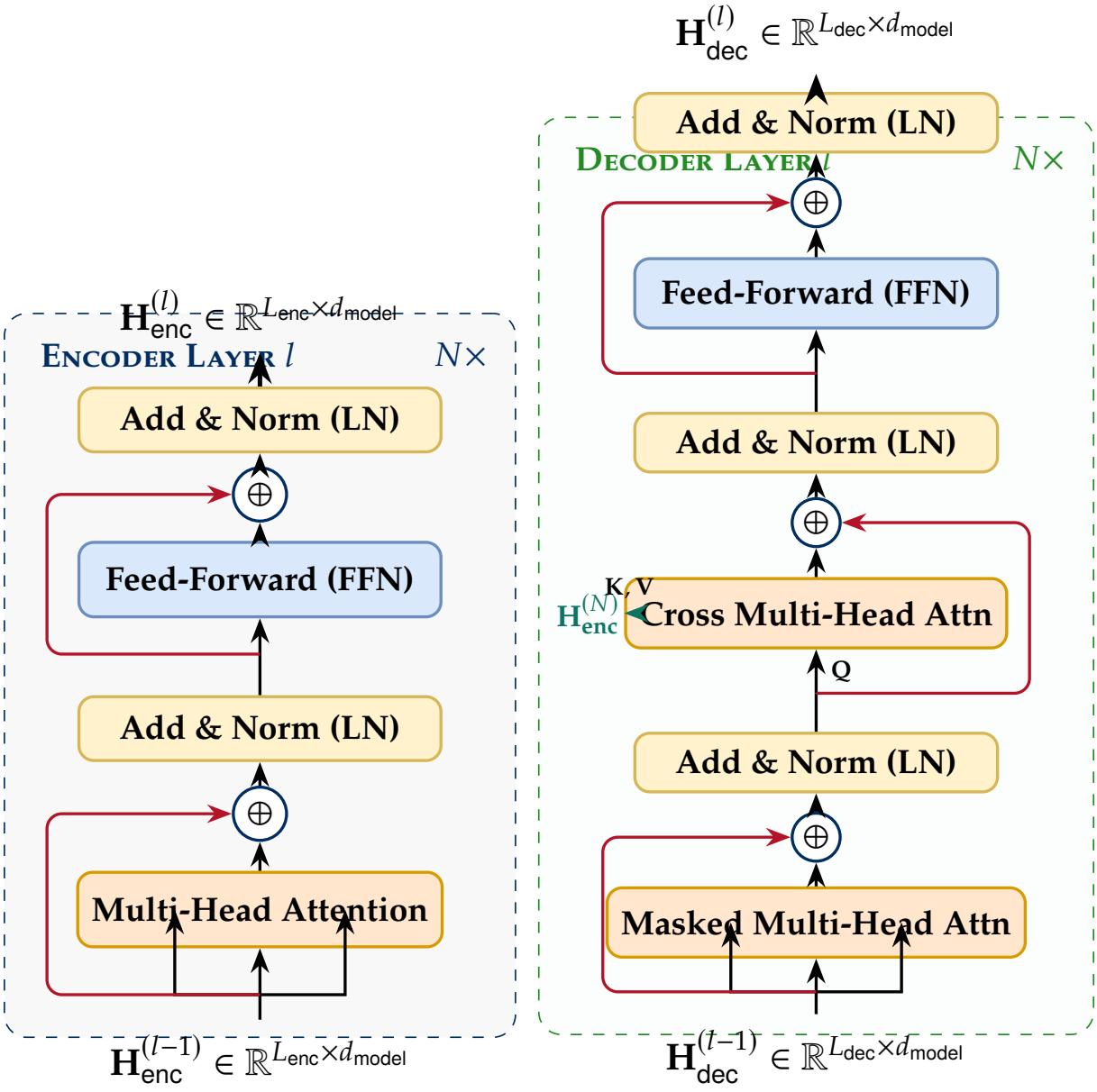
$$X_{\text{src}} = [\texttt{"The"}, \texttt{"bank"}, \texttt{"of"}, \texttt{"the"}, \texttt{"river"}]. \tag{5.88}$$

2. **Embedding and Positional Injection**: Each token indexes the embedding matrix $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, and sinusoidal positional encodings are additively combined:

$$\mathbf{H}_{\text{enc}}^{(0)} = \mathbf{E}_{\text{tok}} + \mathbf{PE} \in \mathbb{R}^{5 \times d_{\text{model}}}. \tag{5.89}$$

3. **Encoder Layers (1 to $N = 6$)**:
   - *Bidirectional Self-Attention*: Every token attends to all 5 tokens. For example, token `"bank"` computes high attention with `"river"`, absorbing geographical context.
   - *Add & Norm*: Residual skip connection adds the input $\mathbf{H}_{\text{enc}}^{(l-1)}$, followed by LayerNorm.
   - *FFN & Add & Norm*: Non-linear expansion to $4d_{\text{model}}$ and contraction back to d_{model}, followed by residual addition and LayerNorm.

4. **Terminal Encoder Memory**: After 6 layers, the encoder outputs the continuous memory tensor $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{5 \times d_{\text{model}}}$. Unlike RNNs, this is *not* a single summary vector, but an uncompressed sequence of 5 deeply contextualized vectors.



(a) Encoder layer stack ($N\times$)

(b) Decoder layer stack ($N\times$)

Figure 5.7: Complete structural assembly of the Transformer architecture [1].

Phase 2: Autoregressive Target Decoding (Inference).
At test time, the target sequence is generated autoregressively, token by token (Slides 8–9):

1. **Step 1: Generating the first target token ("La")**:
   - The decoder input is the single special token: ["<BOS>"].
   - *Masked Self-Attention*: "<BOS>" attends only to itself.
   - *Cross-Attention*: The query from "<BOS>" queries the 5 keys from encoder memory $\mathbf{H}_{\text{enc}}$, attending to "The" and the global topic "river".
   - *FFN and Output Projection*: The representation passes through the FFN, projects via $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}_{\text{target}}| \times d_{\text{model}}}$ into vocabulary logits, and softmax samples token 1: "La".
2. **Step 2: Generating the second target token ("riva")**:
   - The decoder input expands to: ["<BOS>", "La"].
   - *Masked Self-Attention*: Token "La" attends to "<BOS>" and itself, but is prevented from seeing future positions.
   - *Cross-Attention*: Token "La" queries the encoder keys. Because the model must translate "bank", and the encoder memory for "bank" is contextualized by "river", cross-attention aligns "La" with "bank", extracting the value representation for geological riverbank.
   - The classification head projects the state and emits "riva".
3. **Step 3 to 5: Sequential completion**: The decoder continues iteratively:

$$\texttt{["<BOS>","La","riva"]} \implies \texttt{"del"}, \tag{5.90}$$

$$\texttt{["<BOS>","La","riva","del"]} \implies \texttt{"fiume"}, \tag{5.91}$$

$$\texttt{["<BOS>","La","riva","del","fiume"]} \implies \texttt{"<EOS>"}. \tag{5.92}$$

   When "<EOS>" is emitted, the generation process halts.

Parallel training vs. autoregressive inference

As highlighted in Slide 80, a profound architectural strength of the Transformer is the duality between training and inference:

- **Training Phase (Teacher Forcing)**: The full target sentence is known in advance. By feeding the entire ground-truth sequence shifted right into the decoder simultaneously and applying the causal mask $\mathbf{M}$, all T target token probabilities $P(y_t \mid y_{<t})$ and cross-entropy losses are computed concurrently in a single forward pass ($O(1)$ sequential steps). Training across billions of tokens scales with massive GPU parallelism.
- **Inference Phase**: Autoregressive generation cannot know the future. Tokens must be generated sequentially in T forward steps ($O(T)$ sequential operations), each step appending the newly sampled token to the prompt context.

5.5.6 Computational complexity and the quadratic attention barrier

Table 5.5 compares the asymptotic complexity of Self-Attention, Recurrent, and Convolutional layers across sequence length L and representation dimension d.

1. **Parallel Execution**: Computing the attention matrix $\mathbf{QK}^\top$ and weighted sum $\mathbf{AV}$ involves dense matrix multiplications that execute in a single parallel step ($O(1)$ sequential operations) on GPU tensor cores, shattering the recurrent execution wall.

Table 5.5: Asymptotic complexity across layer paradigms per layer [1].

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(L^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(L \cdot d^2)$	$O(L)$	$O(L)$
Convolutional	$O(k \cdot L \cdot d^2)$	$O(1)$	$O(\log_k(L))$

2. **Constant Path Length**: The distance between any arbitrary pair of tokens (i, j) is strictly $O(1)$, eliminating long-range gradient decay.

3. **The Quadratic Memory Barrier**: Computing and storing the attention matrix $\mathbf{A} \in \mathbb{R}^{L \times L}$ incurs quadratic time and memory complexity $O(L^2 \cdot d)$. While optimal for moderate sequence lengths ($L \leq 2048$), expanding context lengths to tens of thousands of tokens encounters the quadratic memory wall, motivating modern sparse, flash, and linear attention architectures.

CHAPTER 6

Transformer Variants and Decoding

Chapter 5 established the foundational mathematics of the canonical Transformer, detailing how multi-head self-attention, cross-attention, and position-wise feed-forward networks overcome the sequential bottlenecks of recurrent architectures. While that framework was originally formulated as a dual-stack encoder-decoder for bilingual machine translation, subsequent deep learning research recognized that the encoder and decoder perform distinct representational functions that can be decoupled and specialized for different NLP tasks.

This structural divergence produced three principal architectural paradigms: unified encoder-decoder models conditioned on arbitrary task prefixes, encoder-only models leveraging unconstrained bidirectional context for sequence comprehension, and decoder-only models employing causal autoregression for open-ended generative synthesis. Concurrently, generating coherent natural text from autoregressive distributions revealed fundamental limitations in classical maximization decoding algorithms, necessitating the development of probabilistic truncation heuristics.

This chapter provides an exhaustive theoretical and empirical examination of Transformer architectural specializations and decoding mechanics. We begin by analyzing the multi-task text-to-text paradigm embodied by T5, dissecting its prefix-conditioning mechanism and dual-stack computational bottlenecks. We then deconstruct encoder-only representations through BERT, formalizing its tripartite input embedding structure, self-supervised pre-training objectives, emergent attention head specializations, and downstream task adaptation heads. Next, we formalize the decoder-only causal architecture, deriving its exact autoregressive probability factorization, causal masking mechanics, and computational rationale for modern large language models. Finally, we dissect the decoding spectrum, formulating deterministic and stochastic sampling strategies and evaluating their failure modes and empirical behaviors through rigorous mathematical formulations and controlled generation benchmarks.

6.1 Encoder-Decoder Models

The canonical Transformer proposed by Vaswani et al. [1] was engineered specifically for sequence-to-sequence machine translation, mapping an input sequence in a source language to a corresponding sequence in a target language. In that original paradigm, model weights were strictly specialized for an isolated task and language pair. The text-to-text transfer framework introduced by Raffel et al. [57] fundamentally reimagined this dual-stack topology, proving that a single shared parameterization could execute heterogeneous natural language processing tasks by unifying their input-output formats into a shared string-to-string interface.

6.1.1 The unified text-to-text framework and T5

Prior to the introduction of unified text-to-text architectures, empirical natural language processing relied on fragmented task-specific design conventions. Researchers typically affixed

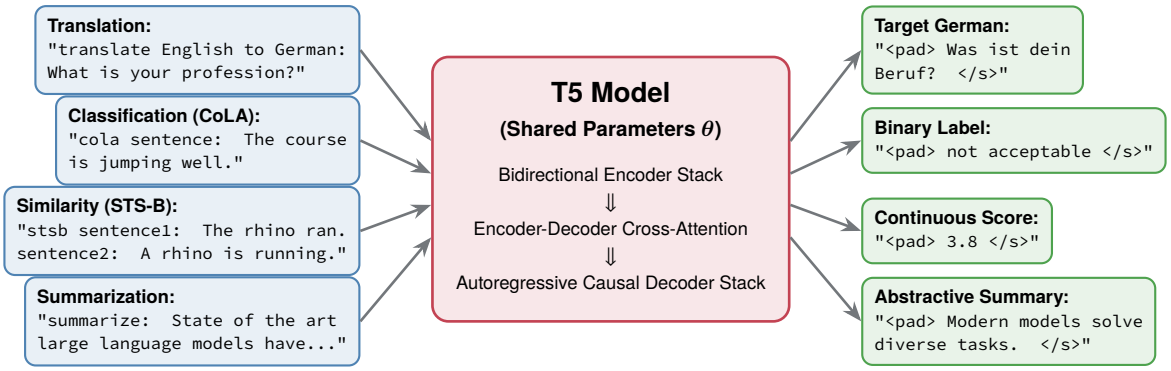


Figure 6.1: The T5 unified text-to-text framework [57]. Heterogeneous downstream tasks are cast into a single uniform string-to-string format conditioned by explicit task prefixes, utilizing identical loss functions and decoding routines.

specialized classification layers atop an encoder for sentiment analysis, token-level linear classifiers for named entity recognition, regression heads for semantic textual similarity, or specialized sequence-to-sequence decoders for summarization and translation. Each downstream objective demanded a bespoke loss formulation, architectural head modification, and dedicated optimization schedule.

The Text-to-Text Transfer Transformer (T5) introduced by Raffel et al. [57] resolved this fragmentation by reframing every natural language objective as a text generation problem. Formally, let $\mathcal{V}^*$ denote the set of all arbitrary-length token sequences formed from vocabulary $\mathcal{V}$. Regardless of whether the underlying target is continuous regression, categorical classification, span extraction, or sequence translation, both the model input $\mathbf{x} \in \mathcal{V}^*$ and output $\mathbf{y} \in \mathcal{V}^*$ are represented exclusively as plain text strings:

$$f_{\boldsymbol{\theta}} : \mathcal{V}^* \to \mathcal{V}^*, \quad \mathbf{y} = f_{\boldsymbol{\theta}}(\mathbf{x}). \tag{6.1}$$

To enable a single invariant set of network parameters $\boldsymbol{\theta}$ to distinguish between competing functional objectives, T5 introduces *task-specific conditioning prefixes*. The user or training pipeline prepends a discrete textual indicator to the source sequence specifying the operational transformation desired. Figure 6.1 illustrates this unified conditioning paradigm across four disparate NLP tasks.

The mathematical elegance of this formulation lies in the universal applicability of the standard autoregressive maximum-likelihood objective. For any dataset $\mathcal{D} = \{(\mathbf{x}^{(m)}, \mathbf{y}^{(m)})\}_{m=1}^{M}$, the network is trained using token-level teacher-forced cross-entropy:

$$\mathcal{L}_{\text{seq2seq}}(\boldsymbol{\theta}) = -\sum_{m=1}^{M} \sum_{t=1}^{T_m} \log P\left(y_t^{(m)} \mid y_{<t}^{(m)}, \mathbf{x}^{(m)}; \boldsymbol{\theta}\right), \tag{6.2}$$

where $y_t^{(m)}$ represents the t-th target token of sequence m and T_m is its length. Continuous regression tasks such as STS-B are handled by requiring the model to emit decimal digit tokens (e.g., `"3.8"`), which are parsed back to floating-point values at inference. Binary classification tasks emit string literals (`"acceptable"` or `"not acceptable"`).

Crucially, as emphasized in course lectures, this prefix-conditioning paradigm must not be confused with modern instruction tuning. In T5, the model does not possess open-ended natural language intent comprehension. The prefix strings are rigid, memorized syntactic conditioning triggers encountered during multi-task pre-training. If an unobserved or perturbed phrasing is supplied (for example, substituting `"can you translate from English to German:"` in place of `"translate English to German:"`), the conditioning signal degrades significantly because the underlying weights merely map memorized prompt templates to target output manifolds.

6.1.2 Architectural modifications and pre-training in T5

Beyond its operational text-to-text unification, T5 introduced key architectural deviations from the original 2017 Transformer specification to improve optimization stability and representation quality.

First, T5 dispenses with fixed absolute sinusoidal positional encodings $\mathbf{p}_i \in \mathbb{R}^{d_{\text{model}}}$. Instead, it adopts a parameterized *relative position bias*. Under this formulation, the scalar pre-softmax attention score between query token i and key token j is augmented by a learnable scalar offset $b_{i,j}$:

$$A_{i,j} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + b_{i,j}, \tag{6.3}$$

where $b_{i,j}$ depends strictly on the relative displacement $(i - j)$. To bound the parameter footprint while retaining long-range sensitivity, displacements are mapped into discrete logarithmic buckets. Small relative distances ($|i - j| < 8$) receive exact dedicated bias parameters, whereas larger offsets are compressed logarithmically up to a maximum context window. This structure enforces translation invariance along sequence positions and enables extrapolation across varying sequence lengths.

Second, T5 simplifies the normalization topology by utilizing a modified root-mean-square normalization where the activation vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ is normalized strictly by its second moment without subtracting the mean:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d_{\text{model}}} \sum_{k=1}^{d_{\text{model}}} x_k^2 + \epsilon}} \odot \gamma, \tag{6.4}$$

where $\gamma \in \mathbb{R}^{d_{\text{model}}}$ is a learned scaling vector and $\epsilon > 0$ guarantees numerical stability. Normalization is placed on the input paths of each sub-block (*Pre-LN*), stabilizing backpropagation gradients through residual connections.

Third, for self-supervised pre-training over the Colossal Clean Crawled Corpus (C4), T5 introduced a compute-efficient *span corruption* objective. Rather than masking individual isolated tokens, contiguous spans of text (averaging 3 tokens in length, totaling 15% of the sequence) are replaced by unique sentinel tokens $\langle \text{extra_id_0} \rangle, \langle \text{extra_id_1} \rangle, \ldots$. The target sequence $\mathbf{y}$ consists exclusively of these sentinel tokens followed by their excised original text. By omitting uncorrupted tokens from the decoder targets, T5 drastically reduces the decoder sequence length during pre-training, yielding substantial computational savings over standard sequence reconstruction.

6.1.3 Trade-offs and computational bottlenecks of dual-stack models

Despite the elegance of the text-to-text abstraction, full encoder-decoder architectures impose structural and computational overheads that ultimately motivated researchers to explore single-stack variants.

1. **Parameter Redundancy in Separate Stacks**: An encoder-decoder network maintains two complete Transformer stacks. For an L-layer architecture with hidden dimension d_{model} and feed-forward dimension d_{ff}, the model instantiates L bidirectional encoder blocks and L causal decoder blocks. Furthermore, every decoder layer must maintain an additional multi-head cross-attention mechanism with parameter projection matrices $\mathbf{W}^Q_{\text{cross}}, \mathbf{W}^K_{\text{cross}}, \mathbf{W}^V_{\text{cross}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ and $\mathbf{W}^O_{\text{cross}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. When evaluated against a single-stack architecture of equivalent total parameter volume, parameter capacity is split between encoding and decoding stages.

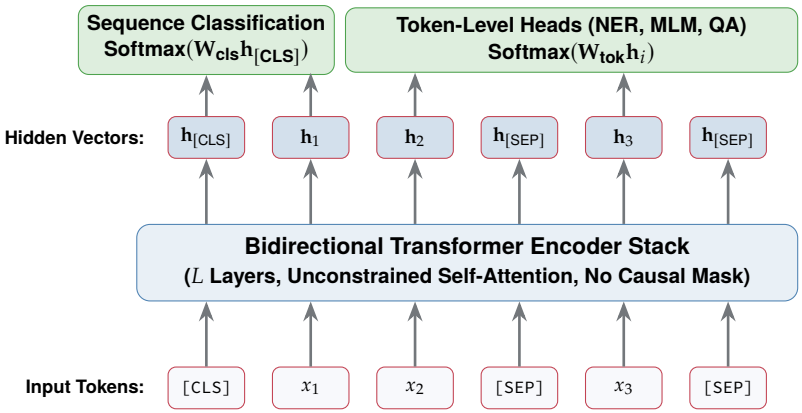


Figure 6.2: The Encoder-only architecture (BERT) [48]. By removing the decoder, bidirectional self-attention enables every token representation to condition simultaneously on the entire context. Sequence-level tasks extract the aggregate vector $\mathbf{h}_{[\text{CLS}]}$, while token-level tagging and span tasks branch directly from individual hidden vectors $\mathbf{h}_i$.

2. **Cross-Attention Memory and Latency Footprint**: In the decoding phase, generating each output token requires computing attention across the entire encoded representation. For an input sequence of length S and an output sequence of length T, cross-attention requires materializing and storing key-value tensors $\mathbf{K}_{\text{cross}}, \mathbf{V}_{\text{cross}} \in \mathbb{R}^{S \times d_k}$ across all heads and layers. The cumulative cross-attention computational complexity scales as $O(L \cdot S \cdot T \cdot d_{\text{model}})$. In latency-critical deployment scenarios, repeatedly attending over static encoder states creates high memory bandwidth demands.

3. **Asymmetric Computational Allocation**: Natural language tasks exhibit marked structural asymmetry. In pure comprehension problems (e.g., text categorization, document retrieval, grammatical acceptability), generating an autoregressive sequence of tokens via a decoder stack is strictly superfluous; a direct pooling of the encoded states provides sufficient representation. Conversely, in open-ended generative tasks (e.g., creative writing, dialogue, code synthesis), there is no distinct "source" sequence to encode; the prompt is merely the initial conditioning segment of a single unified sequence.

These structural trade-offs led to an architectural fork: removing the autoregressive decoder produced bidirectional *encoder-only* representations, whereas removing the encoder and cross-attention produced causal *decoder-only* generative architectures.

6.2 Encoder-Only Models

When an application demands deep semantic understanding, entity tagging, syntactic parsing, or document classification rather than open-ended text synthesis, the autoregressive decoder constitutes unnecessary computational overhead. By excising the decoder stack entirely and eliminating the causal triangular attention mask, the model gains unconstrained bidirectional context: every token at every layer attends simultaneously to all prior, concurrent, and subsequent tokens.

The archetypal implementation of this paradigm is BERT (*Bidirectional Encoder Representations from Transformers*), introduced by Devlin et al. [48]. Figure 6.2 depicts the architectural topology of an encoder-only model.

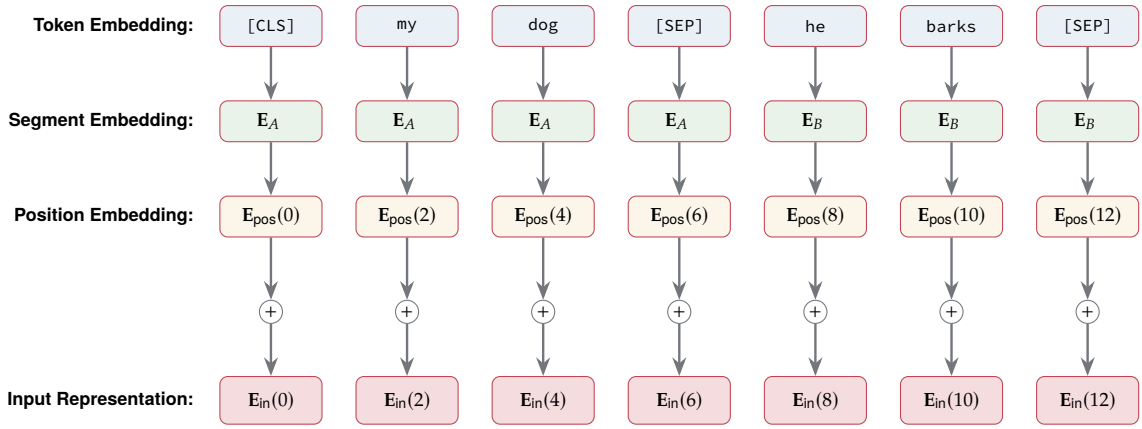


Figure 6.3: BERT input representation structure [48]. The final input representation $\mathbf{E}_{\text{in}}(i)$ is the element-wise sum of the subword token embedding $\mathbf{E}_{\text{token}}$, the binary sentence segment embedding $\mathbf{E}_{\text{segment}} \in \{\mathbf{E}_A, \mathbf{E}_B\}$, and the learned absolute positional embedding $\mathbf{E}_{\text{pos}}(i)$.

6.2.1 Input representation and special tokens

BERT constructs input representations by summing three distinct embeddings for each sequence position: token embeddings, segment embeddings, and positional embeddings. Figure 6.3 formalizes this composition.

Formally, for position i in a packed sequence, the input embedding vector $\mathbf{E}_{\text{in}}(i) \in \mathbb{R}^{d_{\text{model}}}$ is defined by:

$$\mathbf{E}_{\text{in}}(i) = \mathbf{E}_{\text{token}}(t_i) + \mathbf{E}_{\text{segment}}(s_i) + \mathbf{E}_{\text{pos}}(i), \tag{6.5}$$

where:

1. $\mathbf{E}_{\text{token}}(t_i) \in \mathbb{R}^{d_{\text{model}}}$ maps subword tokens t_i derived via WordPiece tokenization ($|\mathcal{V}| \approx 30,522$) into continuous embeddings.

2. $\mathbf{E}_{\text{segment}}(s_i) \in \{\mathbf{E}_A, \mathbf{E}_B\}$ denotes a learned segment embedding assigned to distinguish the first sentence (Sentence A) from the contiguous or paired second sentence (Sentence B).

3. $\mathbf{E}_{\text{pos}}(i) \in \mathbb{R}^{d_{\text{model}}}$ represents a learned absolute positional embedding index up to a context length of 512 tokens.

To structure multi-sentence reasoning and facilitate task classification, BERT introduces two essential delimiter tokens:

- `[CLS]` (*Classification*): Always prepended as the zeroth token (t_0) of every sequence. Because self-attention is unconstrained, the hidden state $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$ accumulates contextual information across all tokens, functioning as an aggregate sequence representation for classification heads.

- `[SEP]` (*Separator*): Inserted between Sentence A and Sentence B, and appended at the final position of the sequence. It acts as an unambiguous structural barrier between distinct semantic segments.

6.2.2 Pre-training task 1: Masked Language Model (MLM)

In an unconstrained bidirectional encoder, standard autoregressive next-token prediction is mathematically degenerate: since token i directly attends to token $i+1$, predicting x_{i+1} would trivially reduce to reading the unmasked future token. To train a deep bidirectional representation

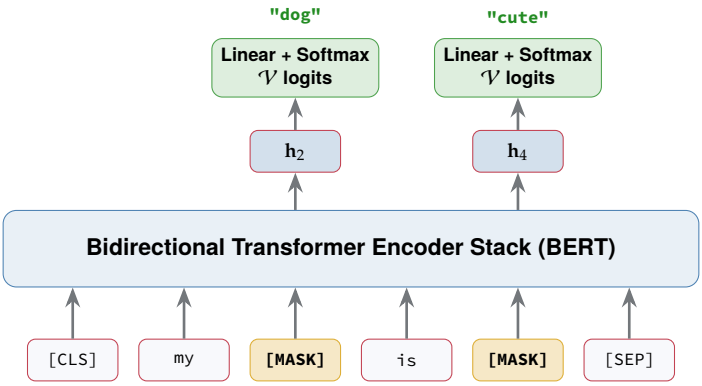


Figure 6.4: The Masked Language Model (MLM) pre-training task in BERT. Approximately 15% of input tokens are corrupted. The final contextual hidden states corresponding to corrupted indices are projected onto the vocabulary vocabulary space through a shared linear layer to reconstruct the original tokens.

without informational leakage, Devlin et al. [48] designed the *Masked Language Model* (MLM) objective.

Figure 6.4 shows the MLM execution flow. In each training sequence, approximately 15% of all WordPiece tokens are selected uniformly at random for prediction. However, naively replacing every chosen token with a fixed `[MASK]` token introduces a severe *pre-training/fine-tuning discrepancy*: the `[MASK]` token never appears during downstream fine-tuning.

To mitigate this distribution shift, Devlin et al. introduced an 80/10/10 corruption schedule for the selected 15% candidate positions:

1. 80% **of the time**: The token is replaced with the special symbol `[MASK]`.
2. 10% **of the time**: The token is replaced with a randomly sampled token from the vocabulary $\mathcal{V}$. This induces contextual vigilance: because any token might be an injected error, the network cannot simply learn that non-`[MASK]` tokens are always correct.
3. 10% **of the time**: The token is left unaltered. This forces the encoder to maintain a faithful representation of the true observed token.

Formally, let $\mathcal{M}$ denote the set of corrupted token indices. For each index $i \in \mathcal{M}$, the final contextual hidden vector $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ is passed through a shared linear projection $\mathbf{W}_{\text{mlm}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ followed by a softmax:

$$P(x_i = v \mid \mathbf{h}_i) = \frac{\exp(\mathbf{w}_v^\top \mathbf{h}_i + b_v)}{\sum_{v' \in \mathcal{V}} \exp(\mathbf{w}_{v'}^\top \mathbf{h}_i + b_{v'})}. \tag{6.6}$$

The MLM loss is computed strictly over the corrupted positions:

$$\mathcal{L}_{\text{MLM}}(\boldsymbol{\theta}) = -\sum_{i \in \mathcal{M}} \log P(x_i = t_i^* \mid \mathbf{h}_i; \boldsymbol{\theta}), \tag{6.7}$$

where t_i^* is the ground-truth token identity before corruption.

6.2.3 Pre-training task 2: Next Sentence Prediction (NSP)

While MLM develops fine-grained token-level contextual representations, many downstream NLP applications (such as Natural Language Inference and Question Answering) require modeling relational semantics between pairs of sentences. To explicitly train sentence relationship modeling, BERT incorporates the *Next Sentence Prediction* (NSP) binary classification objective.

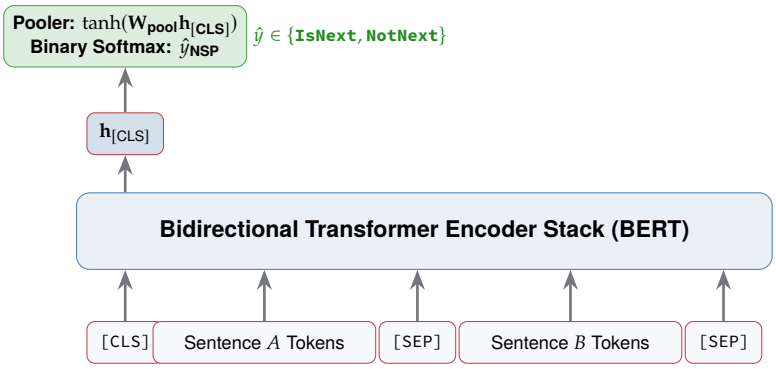


Figure 6.5: The Next Sentence Prediction (NSP) pre-training task in BERT. Two sentences are packed into a single input separated by `[SEP]`. The final pooled representation of `[CLS]` is classified to determine whether Sentence B logically follows Sentence A.

Figure 6.5 illustrates the NSP formulation. Input sequences are assembled by concatenating sentence pairs:

$$\mathbf{x} = (\texttt{[CLS]}, a_1, \ldots, a_{|A|}, \texttt{[SEP]}, b_1, \ldots, b_{|B|}, \texttt{[SEP]}). \tag{6.8}$$

Training instances are sampled according to a balanced binary protocol:

- 50% **of instances (Positive Class: `IsNext`)**: Sentence B is the actual contiguous sentence following Sentence A in the source corpus.
- 50% **of instances (Negative Class: `NotNext`)**: Sentence B is a random sentence sampled independently from another document in the corpus.

At the model output, the hidden state corresponding to the first token, $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$, is mapped through a non-linear pooler layer with parameter matrix $\mathbf{W}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{c} = \tanh(\mathbf{W}_{\text{pool}} \mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{pool}}). \tag{6.9}$$

A binary classification projection $\mathbf{W}_{\text{NSP}} \in \mathbb{R}^{2 \times d_{\text{model}}}$ computes the class probabilities:

$$P(\text{IsNext} \mid \mathbf{c}) = \frac{\exp(\mathbf{w}_{\text{next}}^{\top} \mathbf{c})}{\exp(\mathbf{w}_{\text{next}}^{\top} \mathbf{c}) + \exp(\mathbf{w}_{\text{not}}^{\top} \mathbf{c})}. \tag{6.10}$$

The cumulative pre-training loss combines the MLM and NSP objectives over every mini-batch:

$$\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{MLM}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{NSP}}(\boldsymbol{\theta}). \tag{6.11}$$

6.2.4 Attention map anatomy and emergent linguistic structures

An instructive insight from the BERT architecture is that multi-head bidirectional self-attention naturally develops specialized linguistic capabilities without explicit syntactic supervision. In $\text{BERT}_{\text{BASE}}$, the model instantiates $L = 12$ layers with $H = 12$ attention heads per layer, computing:

$$N_{\text{maps}} = 12 \times 12 = 144 \tag{6.12}$$

distinct attention matrices per sequence. For a sequence of length N, each attention map is an $N \times N$ matrix $\mathbf{A} \in [0, 1]^{N \times N}$ where entry $A_{i,j}$ indicates the attention weight that token i assigns to token j.

In course lectures, the internal mechanics of these 144 attention heads are analyzed using the running sentence:

```
"[CLS] The dog ate the food because it was hungry [SEP]"
```

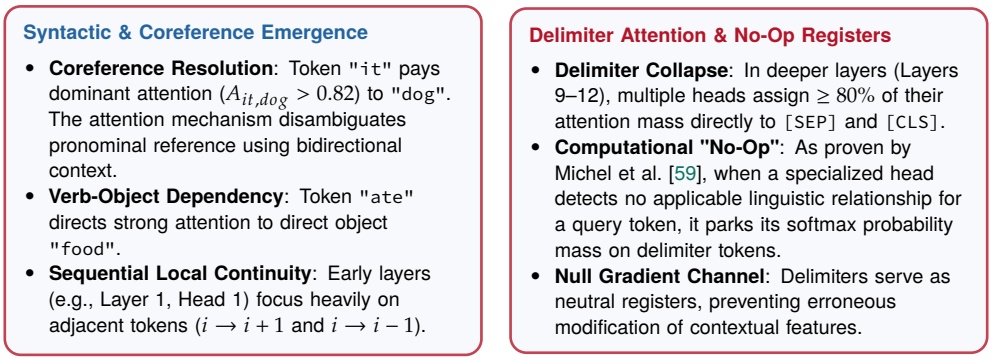


Figure 6.6: Emergent linguistic specialization and delimiter behavior across BERT attention heads [58, 59]. Middle layers resolve syntactic dependencies and coreferences, while deeper layers utilize delimiter tokens as computational no-op registers.

comprising $N = 11$ tokens. Empirical probing of these attention maps, formalizing studies by Clark et al. [58] and Michel et al. [59], reveals two striking emergent phenomena:

1. **Syntactic and Coreference Specialization**: In middle layers (Layers 5 through 8), individual attention heads function as soft syntactic parsers. For example, when resolving the pronoun "it", specific heads route their primary attention mass ($A_{i,j} > 0.80$) back to the antecedent noun phrase "dog". Because the self-attention formula is unconstrained, token representations dynamically absorb semantic properties from their syntactic referents. Similarly, verbs attend directly to their objects (e.g., "ate" attends strongly to "food").

2. **Delimiter Registers as Computational "No-Ops"**: In higher layers (Layers 9 through 12), an intriguing behavior emerges: an overwhelming fraction of query tokens assign the vast majority of their attention probability to delimiter tokens, predominantly [SEP] and [CLS]. Clark et al. [58] established that delimiter tokens act as *computational no-op registers*. Softmax requires rows of the attention matrix to sum strictly to 1:

$$\sum_{j=1}^{N} A_{i,j} = 1. \tag{6.13}$$

   If a specialized head (e.g., a preposition detector) evaluates a query token that has no relevant dependency in the sentence, it cannot emit a zero vector from softmax. To prevent injecting spurious semantic information into the residual stream, the head routes its attention mass to [SEP], whose representation functions as a neutral null register.

6.2.5 Downstream task adaptation

To adapt the pre-trained encoder to specific downstream tasks, the pre-training MLM and NSP projection layers are discarded and replaced with lightweight, task-specific output layers. The entire network is then fine-tuned end-to-end:

1. **Sentence-Level Classification (e.g., SST-2, MNLI)**: The classification head maps the aggregate representation $\mathbf{h}_{[\text{CLS}]}$ through a projection matrix $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{K \times d_{\text{model}}}$:

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}_{\text{cls}}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{cls}}). \tag{6.14}$$

2. **Token-Level Tagging (e.g., NER, POS Tagging)**: Each token position i passes its hidden vector $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ to a shared linear projection $\mathbf{W}_{\text{tok}} \in \mathbb{R}^{C \times d_{\text{model}}}$:

$$\hat{\mathbf{y}}_i = \text{Softmax}(\mathbf{W}_{\text{tok}}\mathbf{h}_i + \mathbf{b}_{\text{tok}}). \tag{6.15}$$

3. **Extractive Question Answering (e.g., SQuAD)**: The model learns two parameter vectors, $\mathbf{s}, \mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$, corresponding to span start and span end predictors. The probability that token i marks the start of the answer span is given by:

$$P_{\text{start}}(i) = \frac{\exp(\mathbf{s}^\top \mathbf{h}_i)}{\sum_{j=1}^{N} \exp(\mathbf{s}^\top \mathbf{h}_j)}, \tag{6.16}$$

with an identical formulation parameterized by $\mathbf{e}$ for the span terminus.

6.3 Decoder-Only Models

While bidirectional encoders excel at discriminative understanding and sequence labeling, they cannot naturally synthesize open-ended natural text. Generating a sequence autoregressively with an unconstrained bidirectional model requires repeatedly re-encoding candidate completions with iterative masking heuristics, which is computationally inefficient.

The causal *decoder-only* architecture resolves this limitation by discarding the encoder stack and cross-attention blocks entirely. The prompt sequence is supplied directly as the initial prefix of the decoder stack, and generation proceeds sequentially via strictly causal self-attention.

6.3.1 Causal autoregressive formulation and architecture topology

In a decoder-only architecture, the joint probability of an arbitrary sequence $\mathbf{x} = (x_1, x_2, \ldots, x_N)$ over vocabulary $\mathcal{V}$ is decomposed via the exact probability chain rule without Markov independence assumptions:

$$P(\mathbf{x}) = \prod_{i=1}^{N} P(x_i \mid x_1, x_2, \ldots, x_{i-1}) = \prod_{i=1}^{N} P(x_i \mid \mathbf{x}_{<i}). \tag{6.17}$$

Taking the negative logarithm yields the canonical autoregressive next-token prediction objective:

$$\mathcal{L}_{\text{AR}}(\boldsymbol{\theta}) = -\sum_{i=1}^{N} \log P(x_i \mid \mathbf{x}_{<i}; \boldsymbol{\theta}). \tag{6.18}$$

To compute representations for all positions $i \in \{1, \ldots, N\}$ in parallel during training without violating the causal factorization in Equation (6.17), the architecture enforces strict temporal masking within every self-attention layer. Let $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{N \times d_k}$ denote the query and key projection matrices for a given attention head. The pre-softmax logit matrix is augmented by an additive causal mask $\mathbf{M} \in \{0, -\infty\}^{N \times N}$:

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right), \tag{6.19}$$

where the mask matrix $\mathbf{M}$ is defined by:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \le i, \\ -\infty & \text{if } j > i. \end{cases} \tag{6.20}$$

Because $\exp(-\infty) = 0$, attention weights for all future positions $j > i$ are zero. Thus, the contextual hidden state $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ at position i is an exact deterministic function of the prefix $\mathbf{x}_{\le i}$ and remains independent of subsequent tokens $x_{j>i}$.

6.3.2 Architectural streamlining: eliminating encoder and cross-attention

Relative to the canonical Transformer, the decoder-only architecture simplifies the processing pipeline:

1. **Elimination of the Encoder Stack**: Rather than projecting source text into an intermediate representation space via an independent encoder, the conditioning prompt is simply ingested as the initial prefix of the decoder sequence.
2. **Excise of Cross-Attention Sub-Layers**: Because there is no external encoder state, each decoder block contains only two core components: a causal masked multi-head self-attention layer and a position-wise feed-forward network. This reduces the number of matrix projections per layer from eight ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$ for self-attention; $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$ for cross-attention) to four.
3. **Training Parallelism via Teacher Forcing**: Although inference is necessarily sequential (generating one token at a time), training remains fully parallel. The ground-truth sequence is supplied as input, and causal triangular masking ensures that predictions for all positions are computed in a single forward pass under the teacher-forcing paradigm.

6.3.3 Convergence on the decoder-only paradigm

The NLP research community progressively standardized on decoder-only architectures for large-scale foundation models. This convergence was driven by fundamental computational and operational advantages:

1. **Homogeneous Key-Value Cache Memory Layout**: In an encoder-decoder architecture, autoregressive generation requires maintaining two separate key-value caches: a static encoder cache $\mathbf{K}_{\text{enc}}, \mathbf{V}_{\text{enc}} \in \mathbb{R}^{S \times d_k}$ and a growing decoder cache $\mathbf{K}_{\text{dec}}, \mathbf{V}_{\text{dec}} \in \mathbb{R}^{t \times d_k}$. Managing heterogeneous memory buffers across distributed accelerator clusters complicates tensor-parallel and pipeline-parallel execution. In contrast, decoder-only architectures maintain a single, append-only key-value cache buffer.
2. **Universal In-Context Sequence Interface**: By unifying input context and output completions into a single autoregressive sequence, decoder-only models natively support few-shot prompting, multi-turn dialogue, and chain-of-thought reasoning without requiring specialized architectural modifications or task-specific fine-tuning.

The empirical scaling laws, pre-training corpus curation strategies, and historical milestones (from GPT-1 and GPT-2 to massive modern parameter regimes) that established this paradigm's dominance are examined comprehensively in Chapter 7.

6.4 Decoding and Sampling Strategies

Regardless of whether an architecture employs an encoder-decoder or decoder-only topology, generating text requires converting continuous output vectors into discrete token sequences. At generation step t, the final hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ is projected onto vocabulary $\mathcal{V}$ via the unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ to yield raw logits $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{z}_t = \mathbf{W}_U \mathbf{h}_t + \mathbf{b}_U. \tag{6.21}$$

Applying the softmax operator produces a categorical probability distribution over all known tokens:

$$P(x_t = v \mid \mathbf{x}_{<t}) = \frac{\exp(z_{t,v})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j})}, \quad \forall v \in \{1, \ldots, |\mathcal{V}|\}. \tag{6.22}$$

Selecting discrete tokens $\hat{x}_t$ from this distribution is governed by the *decoding strategy*. As established in course lectures, selecting tokens naively via probability maximization leads to degenerative failure modes. This section formalizes the complete taxonomy of deterministic and stochastic decoding algorithms, illustrated through the running course generation benchmark based on the Sherlock Holmes prompt from Slides 98–112.

6.4.1 The decoding dilemma: maximization versus degeneration

A natural formulation of text generation is maximum a posteriori (MAP) decoding, seeking the sequence $\mathbf{y}^* = (\hat{x}_1, \ldots, \hat{x}_T)$ that maximizes the joint conditional likelihood:

$$\mathbf{y}^* = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x}_{\text{prompt}}) = \arg\max_{\mathbf{y}} \sum_{t=1}^{T} \log P(x_t \mid \mathbf{x}_{<t}). \tag{6.23}$$

Exact maximization over the combinatorial space $\mathcal{V}^T$ requires evaluating $|\mathcal{V}|^T$ trajectories, which is computationally intractable for modern vocabularies ($|\mathcal{V}| \geq 32,000$, $T \geq 100$). However, even when approximated, pure likelihood maximization exhibits severe failure modes in open-ended natural text generation. As demonstrated by Holtzman et al. [60], human language does not maximize probability at every step. Maximization decoding frequently causes *degeneration*, where the model becomes trapped in repetitive loops or emits generic, uninformative phrases.

6.4.2 Deterministic decoding: greedy search and beam search

Deterministic decoding strategies map a logit distribution to a unique, reproducible output sequence without stochastic sampling.

Greedy Decoding Greedy search selects the single token with the highest predicted probability at each step:

$$\hat{x}_t = \arg\max_{v \in \mathcal{V}} P(x_t = v \mid \mathbf{x}_{<t}) = \arg\max_{v \in \mathcal{V}} z_{t,v}. \tag{6.24}$$

While computationally minimal ($O(1)$ selection cost), greedy decoding suffers from two fatal deficiencies:

- **Search Myopia**: Greedy decoding cannot look beyond the immediate token. Selecting an apparently optimal token at step t can steer the network into a low-probability manifold at step $t+1$, missing globally higher-likelihood sequences that began with a lower-probability initial token.
- **Cyclic Repetition Loops**: In open-ended generation, greedy selection reliably causes infinite repetitive loops.

This pathology is highlighted in Slide 100 of the course materials. When conditioning GPT-2 on the Arthur Conan Doyle prompt:

> *"I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair. With an apology for my intrusion, I was about to withdraw when Holmes pulled me abruptly into the room and closed the door behind me."*

greedy decoding produces the following degenerate continuation:

```
"I am sorry, Mr.  Holmes," I said, "but I am not going to tell you what
I have seen.  I am afraid I have not seen you in a long time.  I am afraid
I have not seen you in a long time.  I am afraid I have not seen you in
a long time.  I am afraid I have not seen you in a long time.  I am afraid
I have not seen..."
```

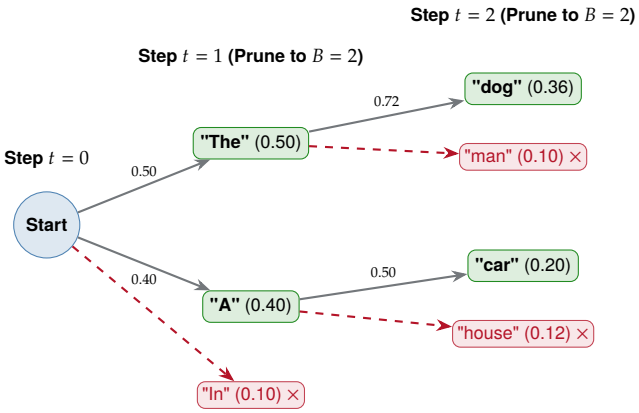


Figure 6.7: Beam search decoding with beam width $B = 2$ (inspired by Slide 101). At each step, all B active sequences are expanded across vocabulary candidates and pruned back to the B sequences with highest cumulative probability.

Once a token sequence repeats, the self-attention mechanism reinforces the historical pattern, driving the model into an inescapable degenerative attractor. To mitigate this loop, generation frameworks often introduce *repetition penalties*:

$$z'_{t,v} = \begin{cases} z_{t,v}/\theta & \text{if } z_{t,v} > 0 \text{ and } v \in \mathbf{x}_{<t}, \\ z_{t,v} \cdot \theta & \text{if } z_{t,v} \leq 0 \text{ and } v \in \mathbf{x}_{<t}, \end{cases} \tag{6.25}$$

where $\theta > 1.0$ penalizes previously generated tokens.

Beam Search Beam search expands the greedy horizon by tracking the top B most probable partial sequences (the *beam width*) at each step. Let $\mathcal{B}_{t-1} = \{\mathbf{y}_1, \ldots, \mathbf{y}_B\}$ denote the set of active hypotheses at step $t-1$. At step t, the algorithm evaluates the Cartesian product of all active beams and the entire vocabulary:

$$C_t = \mathcal{B}_{t-1} \times \mathcal{V} = \{(\mathbf{y}_b, v) \mid \mathbf{y}_b \in \mathcal{B}_{t-1}, v \in \mathcal{V}\}, \tag{6.26}$$

scoring each candidate trajectory by its cumulative log-likelihood:

$$\text{Score}(\mathbf{y}_b \circ v) = \log P(\mathbf{y}_b \mid \mathbf{x}) + \log P(v \mid \mathbf{x}, \mathbf{y}_b). \tag{6.27}$$

The set is then pruned back to the top B candidates:

$$\mathcal{B}_t = \arg \max_{\mathcal{B} \subset C_t, |\mathcal{B}|=B} \sum_{\mathbf{y} \in \mathcal{B}} \text{Score}(\mathbf{y}). \tag{6.28}$$

Because cumulative probability monotonically decreases with sequence length ($\log P \leq 0$), raw beam search exhibits a strong structural bias toward prematurely terminating with short sequences. To counter this, practitioners apply a length normalization penalty:

$$\mathcal{S}(\mathbf{y}) = \frac{\sum_{t=1}^{T} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{x})}{(5+T)^\alpha / (5+1)^\alpha}, \tag{6.29}$$

where $\alpha \in [0.6, 1.0]$ scales length normalization.

While beam search is standard for deterministic, constrained sequence mapping (e.g., machine translation and abstractive summarization), it is poorly suited for open-ended generation: high probability beams converge on repetitive, bland text.

6.4.3 Stochastic sampling: temperature scaling, Top-k, and Top-p

To generate natural, non-repetitive text, decoding must draw tokens stochastically from the predicted distribution: $\hat{x}_t \sim P(x_t \mid \mathbf{x}_{<t})$. However, unrestricted sampling introduces vulnerabilities that require calibrated truncation.

Pure Random Sampling and the Long Tail Risk Under pure multinomial sampling, the next token is sampled directly from the raw softmax distribution over all $|\mathcal{V}|$ tokens. While this guarantees non-deterministic outputs, it exposes generation to the *long tail risk*.

Modern vocabularies contain tens of thousands of tokens ($|\mathcal{V}| \geq 32,000$). Even when individual probability values for implausible words are small (e.g., $P \approx 10^{-4}$), their cumulative probability mass across thousands of tail tokens is substantial. Inevitably, the random draw samples an incoherent or contextually inappropriate token from this distribution tail, permanently derailing subsequent generation.

This failure mode is shown in Slide 103. Under pure random sampling on the Sherlock Holmes prompt, the continuation rapidly devolves into syntactic disarray:

```
"...It was alarmed me, as I had never heard of him before but last, yet
I went to put something into his hand which I carried open and which I
then pocketed into the drawer, so as not to be seen until soon afterwards.
Then it was as though in a dream, and my amazement owled wild-it seems
an impossibility..."
```

Temperature Scaling Temperature scaling modulates the entropy of the categorical distribution by dividing raw logits by a positive hyperparameter $T > 0$ prior to the softmax operator:

$$P_T(x_t = v \mid \mathbf{x}_{<t}) = \frac{\exp(z_{t,v}/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j}/T)}. \tag{6.30}$$

The mathematical behavior across different temperature regimes is bounded by two analytical limits:

- **Limit $T \to 0^+$ (Argmax / Zero Entropy)**: Dividing logits by an infinitesimal value exaggerates differences between logits, collapsing the distribution to a Dirac delta centered at the mode:
$$\lim_{T \to 0^+} P_T(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} 1 & \text{if } v = \arg\max_j z_{t,j}, \\ 0 & \text{otherwise,} \end{cases} \tag{6.31}$$
recovering deterministic greedy search.
- **Limit $T \to \infty$ (Uniform Distribution / Maximum Entropy)**: As T approaches infinity, scaled logits converge to zero ($z_{t,v}/T \to 0$), flattening softmax into a discrete uniform distribution:
$$\lim_{T \to \infty} P_T(x_t = v \mid \mathbf{x}_{<t}) = \frac{1}{|\mathcal{V}|}, \quad \forall v \in \mathcal{V}. \tag{6.32}$$

The qualitative impact of temperature scaling is captured in Slides 109–112 using the Sherlock Holmes benchmark:

- **Low Temperature ($T = 0.1$, Slide 109)**: Near-complete probability collapse into repetitive loops:

```
"...I am sorry, Mr.  Holmes, for having been so rude to you.  I am
sorry, Mr.  Holmes, for having been so rude to you.  I am sorry, Mr.
Holmes, for having been so rude to you..."
```

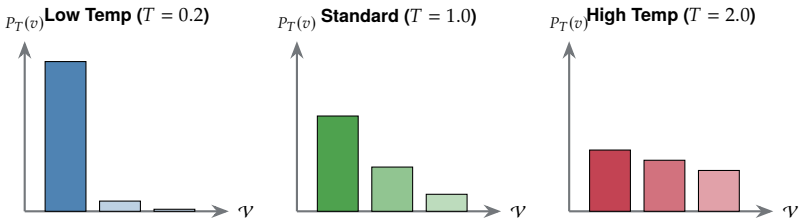


Figure 6.8: Softmax distribution modulation across temperature regimes. Low temperatures ($T < 1.0$) sharpen probability mass toward the mode, while high temperatures ($T > 1.0$) flatten the distribution toward uniform entropy.

- **Balanced Low Temperature ($T = 0.5$, Slide 110)**: Strong coherence, consistent tone, and natural progression:

  ```
  "...It was a very sad moment, but a true one.  After a short time, I
  was ushered out of the room by the old gentleman, who had come to ask
  me to come with him to the house.  I was quite at ease, and when he
  told me that he was going to look after me, I said, 'This is not the
  first time I've seen you.'..."
  ```

- **High Temperature ($T = 1.1$, Slide 111)**: Increasing lexical diversity with emerging contextual drift:

  ```
  "...We remained there four or five minutes, staring at every hem from
  the last person who might show up in my passageway.  It was dark—black—but
  dark, almost this glintry and slant field..."
  ```

- **Very High Temperature ($T = 1.5$, Slide 112)**: Syntactic collapse into hallucinatory gibberish:

  ```
  "...He kept Columbus visibly busy trying secretly rendered strawberries
  with a reel motif.  This anecdote barely IMLEG it off it off 138 ALK###
  elsewhere where 4 doctor m Hiragh benecameth [aintedie$ andalaptag..."
  ```

Top-k Sampling (Fixed Cardinality Truncation) Introduced by Fan et al. [61], Top-k sampling addresses the long-tail vulnerability by restricting the sampling pool to the k most probable tokens. Formally, tokens are sorted in descending order of probability:

$$P(x^{(1)} \mid \mathbf{x}_{<t}) \geq P(x^{(2)} \mid \mathbf{x}_{<t}) \geq \cdots \geq P(x^{(|\mathcal{V}|)} \mid \mathbf{x}_{<t}). \tag{6.33}$$

The sampling support is truncated to the subset $V^{(k)} = \{x^{(1)}, \ldots, x^{(k)}\} \subset \mathcal{V}$, and probabilities are renormalized:

$$P_{\text{top-}k}(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} \frac{P(x_t=v|\mathbf{x}_{<t})}{\sum_{j \in V^{(k)}} P(x_t=j|\mathbf{x}_{<t})} & \text{if } v \in V^{(k)}, \\ 0 & \text{otherwise.} \end{cases} \tag{6.34}$$

Under Top-k sampling with $k = 50$ (Slide 105), the Sherlock Holmes continuation maintains syntactic structure and thematic tone without degenerative loops:

```
"...'You will think, Doctor, that a man of my age would be so much of a
menace to me, if I were to refuse a pass for him.  I should rather get
a few things away from him than any which might injure his reputation...'"
```

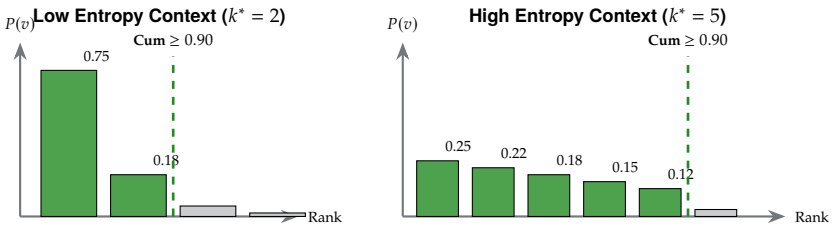


Figure 6.9: Top-p (Nucleus) sampling mechanism. The candidate pool dynamically expands and contracts based on model confidence: in low-entropy contexts only 2 tokens meet the $p = 0.90$ mass threshold, whereas in high-entropy contexts the pool expands to 5 tokens.

However, Top-k suffers from a core structural flaw: *fixed cardinality cannot adapt to contextual uncertainty*. In low-entropy contexts where the model is highly confident (e.g., `"The Eiffel Tower is in"` $\rightarrow$ `"Paris"` with $P = 0.98$), a fixed $k = 50$ forces the algorithm to sample among 49 implausible tokens. Conversely, in high-entropy contexts where hundreds of valid completions exist, $k = 50$ truncates plausible continuations.

Top-p / Nucleus Sampling (Adaptive Cumulative Mass Truncation) To eliminate the rigidity of fixed cardinality, Holtzman et al. [60] introduced *Nucleus* (Top-p) sampling. Instead of clipping at a fixed rank k, nucleus sampling defines an adaptive candidate set $V^{(p)} \subset \mathcal{V}$ containing the minimal set of tokens whose cumulative probability mass meets or exceeds a target threshold $p \in (0, 1]$:

$$\sum_{v \in V^{(p)}} P(x_t = v \mid \mathbf{x}_{<t}) \geq p. \tag{6.35}$$

Formally, letting tokens be sorted in descending probability order, the cutoff index k^* is determined by:

$$k^* = \min \left\{ K \in \{1, \ldots, |\mathcal{V}|\} : \sum_{i=1}^{K} P(x^{(i)} \mid \mathbf{x}_{<t}) \geq p \right\}. \tag{6.36}$$

The candidate pool is defined by $V^{(p)} = \{x^{(1)}, \ldots, x^{(k^*)}\}$, and probabilities are renormalized over this dynamic set:

$$P_{\text{top-}p}(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} \frac{P(x_t = v \mid \mathbf{x}_{<t})}{\sum_{j \in V^{(p)}} P(x_t = j \mid \mathbf{x}_{<t})} & \text{if } v \in V^{(p)}, \\ 0 & \text{otherwise.} \end{cases} \tag{6.37}$$

As illustrated in Figure 6.9, nucleus sampling adapts to contextual entropy:

- When the model is confident, the top few tokens capture most probability mass; the candidate set collapses to $k^* \in \{1, 2, 3\}$, eliminating spurious tail tokens.
- When the distribution is diffuse, the candidate pool automatically expands to hundreds of plausible tokens, preserving creative diversity.

Under nucleus sampling ($p = 0.9$, Slide 107), generation maintains high fluency, coherent narrative progression, and natural lexical variation:

```
"...Mr.  Holmes had been talking for a while and seemed very engrossed
in the conversation.  He wished to conduct his last spoke through the eyes
of another, and by that he meant to tell me the whole of the publick story
that had been passed from one of them into the other..."
```

Table 6.1: Comparative analysis of autoregressive decoding strategies.

Decoding Strategy	Type	Complexity	Primary Failure Mode	Optimal Task Regime
Greedy Search	Deterministic	$O(1)$	Search myopia, cyclic loops	Strict mathematical/code tasks
Beam Search	Deterministic	$O(B \cdot \lvert\mathcal{V}\rvert)$	Repetitive generic phrasing	Translation, summarization
Random Sampling	Stochastic	$O(\lvert\mathcal{V}\rvert)$	Severe long-tail derailment	High-entropy creative text
Temperature	Stochastic	$O(\lvert\mathcal{V}\rvert)$	Extreme entropy drift ($T > 1.5$)	General text generation
Top-k Sampling	Truncated Stoch.	$O(\lvert\mathcal{V}\rvert \log k)$	Inflexible fixed pool size	Stylized controlled generation
Top-p (Nucleus)	Truncated Stoch.	$O(\lvert\mathcal{V}\rvert \log \lvert\mathcal{V}\rvert)$	Occludes rare valid idioms	Open-ended dialogue, chat

6.4.4 Empirical decoding comparison and selection guide

Table 6.1 synthesizes the mathematical properties, complexity profiles, and operational trade-offs of all six decoding strategies.

In production language modeling systems, practitioners typically chain these mechanisms into a unified multi-stage decoding pipeline:

$$\mathbf{z} \xrightarrow{\text{Repetition Penalty}} \mathbf{z}' \xrightarrow{\text{Temperature } T} \mathbf{z}'' \xrightarrow{\text{Top-}k \text{ Truncation}} \mathbf{p}' \xrightarrow{\text{Top-}p \text{ Cutoff}} \hat{x}_t \sim \text{Categorical}(\mathbf{p}''). \tag{6.38}$$

First, repetition penalties adjust logits of previously emitted tokens. Second, temperature scaling calibrates entropy. Third, Top-k filtering eliminates the outer tail. Fourth, Top-p thresholding dynamically isolates the nucleus. Finally, the next token is sampled from the renormalized distribution.

CHAPTER 7

Pre-Training, Scaling Laws, and Model Evolution

Chapter 6 analyzed the structural taxonomy of Transformer variants, detailing how encoder-only, encoder-decoder, and decoder-only topologies specialize for distinct linguistic representations and sequence generation tasks. While architectural topology dictates how tokens attend to one another within the multi-head attention mechanism, the emergent cognitive, semantic, and reasoning capabilities of modern foundation models are governed by the scale of self-supervised pre-training, parameter capacity, and compute allocation.

The evolution of modern large language models began with the empirical demonstration that unsupervised autoregressive pre-training over diverse text corpora provides a universal task-agnostic initialization that transfers to downstream tasks. Subsequent scaling experiments revealed that language model loss follows precise mathematical power laws across parameter count, token volume, and computational budget, triggering an industry-wide race toward massive parameter regimes. However, subsequent empirical evaluations demonstrated that simply increasing parameter counts without proportionally scaling training tokens produces severely undertrained models, establishing the compute-optimal scaling frontier and laying the foundation for modern open foundation models.

This chapter provides an exhaustive theoretical and empirical analysis of the evolution of large language models, their pre-training methodologies, and the scaling laws governing their performance. We begin in Section 7.1 by formalizing the Generative Pre-trained Transformer (GPT-1), dissecting its two-stage pre-training and supervised fine-tuning framework with delimiter tokens. Section 7.2 examines GPT-2, its zero-shot multi-task paradigm, WebText corpus curation, benchmark evaluations, and the breakthrough empirical scaling laws of Kaplan et al. Section 7.3 deconstructs GPT-3, contrasting supervised fine-tuning with in-context learning across zero-shot, one-shot, and few-shot regimes, and detailing its empirical performance against task-specific state-of-the-art baselines. Section 7.4 analyzes the race to massive scale, the undertraining diagnosis, and the three empirical approaches of the DeepMind Chinchilla framework that formalized compute-optimal pre-training. Finally, Section 7.5 examines the modern paradigm shift toward token saturation, inference economics, and the rise of open foundation model families.

7.1 GPT-1: Generative Pre-Training

Prior to 2018, natural language processing was characterized by fragmented, task-specific architectures. Researchers predominantly trained recurrent neural networks (LSTMs, GRUs) or convolutional networks on supervised datasets from scratch, relying on static word embeddings such as Word2Vec or GloVe for initialization. While semi-supervised approaches such as ELMo contextualized word representations, the underlying task-specific model architectures still required bespoke engineering for each target task.

Radford et al. [62] resolved this architectural fragmentation by introducing the Generative Pre-trained Transformer (GPT-1). GPT-1 demonstrated that a single, universal task-agnostic

Transformer architecture could be pre-trained using self-supervised causal language modeling on unannotated text, followed by minimal supervised fine-tuning on labeled downstream benchmarks.

7.1.1 Architectural topology and pre-training setup

The core architecture of GPT-1 adapts the canonical Transformer decoder proposed by Vaswani et al. [1], excising the encoder-decoder cross-attention sub-layers because no external source sequence is encoded. The model consists of a 12-layer decoder-only stack with the following dimensional specifications:

- **Layer and Head Configuration**: $L = 12$ Transformer decoder blocks, each containing $H = 12$ self-attention heads with per-head projection dimension $d_{\text{head}} = d_{\text{model}}/H = 64$.
- **Hidden Dimensionality**: Hidden representation dimension $d_{\text{model}} = 768$, with position-wise feed-forward intermediate dimensionality $d_{\text{ff}} = 3072$ (a 4× expansion ratio).
- **Parameter Scale**: Total parameter count of $N \approx 117$M parameters.
- **Tokenization and Embeddings**: Byte-Pair Encoding (BPE) subword tokenization with a vocabulary of 40,000 merge operations. Rather than fixed sinusoidal encodings, GPT-1 employs learned absolute positional embeddings up to a maximum context window of $L = 512$ tokens.

For its pre-training corpus, GPT-1 utilized the BooksCorpus dataset. Comprising approximately 7,000 unique unpublished books (amounting to ≈ 5 GB of text) across diverse literary genres (adventure, science fiction, romance), BooksCorpus provided continuous spans of long-form, coherent prose. This allowed the self-attention mechanism to learn long-range semantic and syntactic dependencies across contiguous discourse, rather than sentence-level fragments.

In the first stage of the GPT-1 framework, the network is trained on an unannotated corpus of tokens $\mathcal{U} = (u_1, u_2, \ldots, u_N)$ via standard autoregressive language modeling. Given a conditioning context window of size k, the network maximizes the causal log-likelihood:

$$\mathcal{L}_1(\mathcal{U}) = \sum_{i=1}^{N} \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \boldsymbol{\Theta}), \tag{7.1}$$

where $\boldsymbol{\Theta}$ denotes the complete parameter tensor of the Transformer decoder.

Figure 7.1 illustrates the forward computation. Formally, given an input token sequence $(u_{i-k}, \ldots, u_{i-1})$, the zeroth representation layer $\mathbf{h}_0$ is constructed by summing token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ and positional embedding matrix $\mathbf{W}_p \in \mathbb{R}^{L \times d_{\text{model}}}$:

$$\mathbf{h}_0 = \mathbf{U}\mathbf{W}_e + \mathbf{W}_p, \tag{7.2}$$

where $\mathbf{U} \in \{0, 1\}^{k \times |\mathcal{V}|}$ is the one-hot context matrix.

The representation is then propagated through L identical Transformer decoder layers:

$$\mathbf{h}_l = \text{TransformerBlock}(\mathbf{h}_{l-1}), \quad \forall l \in \{1, \ldots, L\}. \tag{7.3}$$

The conditional probability distribution over vocabulary $\mathcal{V}$ at step i is obtained by projecting the final hidden state $\mathbf{h}_L$ through an explicit unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$:

$$P(u_i \mid u_{i-k}, \ldots, u_{i-1}) = \text{Softmax}(\mathbf{W}_U \mathbf{h}_{L,i-1} + \mathbf{b}_U). \tag{7.4}$$

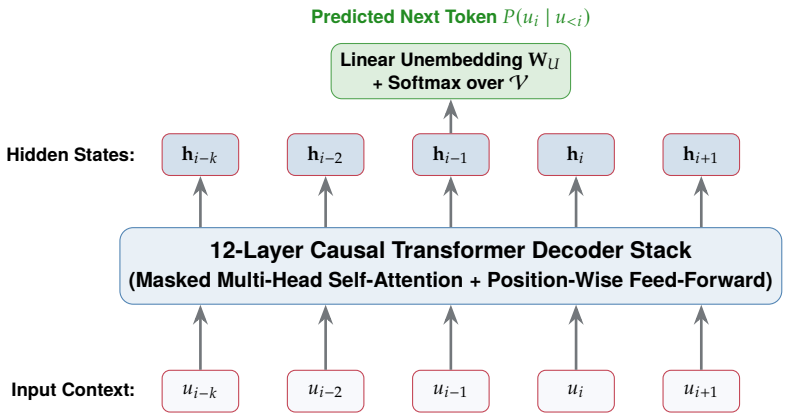


Figure 7.1: The GPT-1 unsupervised pre-training stage [62]. A causal Transformer decoder processes context tokens under a lower-triangular attention mask, projecting each hidden state $\mathbf{h}_{i-1}$ through an unembedding matrix $\mathbf{W}_U$ to maximize the conditional likelihood of the ground-truth next token u_i.

7.1.2 Supervised task adaptation and delimiter token linearization

Once pre-training is complete, the parameters $\boldsymbol{\Theta}$ provide a general-purpose semantic initialization. In the supervised fine-tuning stage, the model is adapted to a labeled dataset C, where each instance consists of an input sequence of tokens $(x_1, \ldots, x_m)$ and a ground-truth target label y.

The input sequence is passed through the pre-trained Transformer decoder to obtain the final activation vector of the terminal token, $\mathbf{h}_{L,m} \in \mathbb{R}^{d_{\text{model}}}$. This state is fed into a lightweight linear classification layer parameterized by $\mathbf{W}_y \in \mathbb{R}^{K \times d_{\text{model}}}$:

$$P(y \mid x_1, \ldots, x_m) = \text{Softmax}(\mathbf{W}_y \mathbf{h}_{L,m} + \mathbf{b}_y). \tag{7.5}$$

The supervised objective minimizes the empirical cross-entropy loss:

$$\mathcal{L}_2(C) = -\sum_{(\mathbf{x},y)\in C} \log P(y \mid x_1, \ldots, x_m). \tag{7.6}$$

Critically, Radford et al. observed that incorporating an auxiliary language modeling objective during fine-tuning accelerates convergence, prevents catastrophic forgetting, and improves downstream generalization:

$$\mathcal{L}_3(C) = \mathcal{L}_2(C) + \lambda \mathcal{L}_1(C), \tag{7.7}$$

where $\lambda > 0$ balances supervised classification and self-supervised causal modeling (typically set to $\lambda = 0.5$).

A defining innovation of GPT-1 was the complete avoidance of task-customized neural architectures. As illustrated in Figure 7.2, structured inputs are flattened into linear token sequences bounded by special delimiter tokens:

1. **Text Classification**: The input sequence is bounded by start and extract tokens: $(\texttt{Start}, x_1, \ldots, x_m, \texttt{Extract})$. The linear head reads the final contextual representation corresponding to `Extract`.

2. **Natural Language Inference (NLI)**: Premise and hypothesis sequences are concatenated with a middle delimiter: (`Start`, Premise, `Delim`, Hypothesis, `Extract`).

3. **Semantic Similarity**: Because similarity is symmetric, the pair is processed in both forward and reversed order: (`Start`, Text_1, `Delim`, Text_2, `Extract`) and (`Start`, Text_2, `Delim`, Text_1, `Extract`). The two resulting extract vectors $\mathbf{h}^{(1)}_{L,\text{ext}}$ and $\mathbf{h}^{(2)}_{L,\text{ext}}$ are summed element-wise before classification, ensuring permutation invariance.

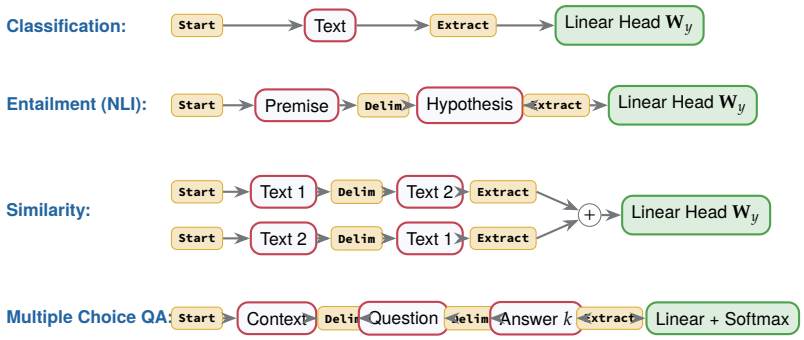


Figure 7.2: Downstream task input transformations using delimiter tokens in GPT-1 [62]. Rather than altering the underlying model architecture, structured inputs are flattened into linear token sequences bounded by `Start`, `Delim`, and `Extract` tokens.

4. **Question Answering and Commonsense Reasoning**: For a context document z, question q, and candidate answer set $\{a_k\}$, the model forms K independent sequences: $(\texttt{Start}, z, \texttt{Delim}, q, \texttt{Delim}, a_k, \texttt{Extract})$. Each sequence produces an extract logit, and a softmax over all K candidates yields the choice probability.

7.2 GPT-2 and Empirical Scaling Laws

Following the success of GPT-1, Radford et al. [42] released GPT-2. The primary conceptual breakthrough of GPT-2 was the demonstration of *unsupervised multi-task learning*: when trained on a sufficiently diverse and massive corpus, a language model develops zero-shot capabilities across distinct tasks without requiring supervised fine-tuning or parameter updates.

7.2.1 Architectural stabilization, model scaling, and WebText curation

While preserving the core causal decoder topology of GPT-1, GPT-2 introduced critical structural modifications to stabilize optimization across deeper layer hierarchies:

- **Pre-Layer Normalization (Pre-LN)**: In GPT-1, Layer Normalization was applied after the residual addition (*Post-LN*): $\mathbf{x}_{l+1} = \text{LN}(\mathbf{x}_l + \text{SubBlock}(\mathbf{x}_l))$. In GPT-2, normalization was shifted to the input pathway of each sub-block (*Pre-LN*):

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \text{SubBlock}(\text{LN}(\mathbf{x}_l)). \tag{7.8}$$

  An additional Layer Normalization layer was added following the final self-attention block prior to the unembedding projection. Pre-LN establishes an unhindered identity highway through the residual stream, preventing gradient vanishing or explosion in deep architectures.

- **Residual Projection Scaling**: To prevent variance accumulation across stacked residual connections at initialization, the weights of residual projection layers ($\mathbf{W}^O$ in self-attention and $\mathbf{W}_2$ in feed-forward networks) were scaled down by a depth-dependent factor:

$$\mathbf{W} \sim \mathcal{N}\left(0, \frac{\sigma^2}{\sqrt{2N}}\right), \tag{7.9}$$

  where N is the number of Transformer layers.

- **Context and Vocabulary Expansion**: The context window was doubled from 512 to $L = 1024$ tokens. To handle arbitrary Unicode sequences without out-of-vocabulary

([UNK]) tokens, GPT-2 introduced Byte-Level Byte-Pair Encoding (BBPE) with a vocabulary of $|\mathcal{V}| = 50,257$.

GPT-2 was released as a family of four model sizes, scaling from 117M to 1.5B parameters:

1. **GPT-2 Small**: $L = 12$ layers, $H = 12$ heads, $d_{\text{model}} = 768$ (117M parameters, identical to GPT-1).
2. **GPT-2 Medium**: $L = 24$ layers, $H = 16$ heads, $d_{\text{model}} = 1024$ (345M parameters).
3. **GPT-2 Large**: $L = 36$ layers, $H = 20$ heads, $d_{\text{model}} = 1280$ (762M parameters).
4. **GPT-2 Extra Large**: $L = 48$ layers, $H = 25$ heads, $d_{\text{model}} = 1600$ (1.5B parameters).

A critical factor in GPT-2's emergent capabilities was the curation of the *WebText* corpus. While common web-scraping repositories (such as Common Crawl) contained hundreds of terabytes of data, raw web scrapes were plagued by automated spam, duplicate content, and incoherent text. To ensure high quality without manual labeling, Radford et al. designed a novel heuristic curation strategy:

1. **Reddit Link Scraping**: Outbound links shared on the social news platform Reddit were extracted.
2. **Human Karma Filtering**: Only links from Reddit posts receiving at least 3 karma (upvotes) were retained. This used human curation as an explicit heuristic filter: the Reddit community effectively pre-screened outbound web content for coherence, informative value, and linguistic quality.
3. **Text Extraction and Deduplication**: From over 45 million links, Wikipedia articles were removed (to avoid contamination of evaluation benchmarks), HTML was stripped, and text was deduplicated.

The resulting WebText dataset yielded approximately 40 GB of cleaned text—an 8× expansion in text volume over BooksCorpus.

7.2.2 Emergent zero-shot multi-task generalization and staged release

The central empirical result of GPT-2 was that language models, without any fine-tuning or parameter updates, can execute diverse natural language processing tasks zero-shot by framing the objective as conditional natural language generation:

$$P(\text{Target} \mid \text{Context, Task Description}). \tag{7.10}$$

Rather than training a task-specific classification head, the task instructions and input context are concatenated into a prompt sequence, and the model generates the target completion through standard autoregressive decoding.

To rigorously benchmark this zero-shot transfer capability, Radford et al. evaluated the four GPT-2 model variants across a broad battery of language modeling, cloze completion, reading comprehension, and translation benchmarks without fine-tuning. Table 7.1 presents the comprehensive empirical results reported in Table 3 of Radford et al. [42].

The empirical findings from Table 7.1 established several profound takeaways:

1. **Long-Range Context Sensitivity on LAMBADA**: The LAMBADA dataset tests the ability of language models to resolve long-range discourse dependencies by requiring the model to predict the final word of a passage that cannot be predicted from local context alone. The prior state-of-the-art model achieved a perplexity of 99.8 and an accuracy of 59.23%. GPT-2 1.5B reduced the perplexity by more than an order of magnitude to 8.63, while increasing accuracy to 63.24% zero-shot, demonstrating that scaling parameter count enables the self-attention mechanism to track subtle discourse threads across full narrative paragraphs.

Table 7.1: Zero-shot performance of GPT-2 across model scales compared to prior task-specific state-of-the-art models [42]. No training or supervised fine-tuning was performed for any GPT-2 results.

Benchmark / Dataset	Prior SOTA	117M (Small)	345M (Med)	762M (Large)	1.5B (XL)
LAMBADA (Perplexity ↓)	99.80	35.13	15.60	10.87	**8.63**
LAMBADA (Accuracy % ↑)	59.23	45.99	55.48	60.12	**63.24**
CBT-CN (Accuracy % ↑)	85.70	87.65	92.35	**93.45**	93.30
CBT-NE (Accuracy % ↑)	82.30	83.40	87.10	88.00	**89.05**
WikiText-2 (Perplexity ↓)	39.14	29.41	22.76	19.93	**18.34**
Penn Treebank (Perplexity ↓)	46.54	65.85	47.33	40.31	**35.76**
enwik8 (Bits per byte ↓)	0.99	1.16	1.01	0.97	**0.93**
text8 (Bits per character ↓)	1.08	1.17	1.06	1.02	**0.98**
WikiText-103 (Perplexity ↓)	18.30	37.50	26.37	22.05	**17.48**
1BW (One Billion Words) (PPL ↓)	21.80	75.20	55.72	44.57	**42.16**
CoQA (Development F1 ↑)	45.0–54.0*	43.10	49.60	52.80	**55.00**

*Prior supervised baseline systems on CoQA without pre-training (e.g., DrQA, BiDAF) achieved between 45 and 54 F1 on the dev set, whereas supervised BERT reached 89 F1.

2. **Cloze Performance on Children's Book Test (CBT)**: CBT evaluates language models on predicting missing common nouns (CBT-CN) and named entities (CBT-NE). GPT-2 1.5B surpassed prior supervised models on both benchmarks, achieving 93.30% on CBT-CN (vs. prior SOTA 85.70%) and 89.05% on CBT-NE (vs. prior SOTA 82.30%), without being exposed to task-specific training instances.

3. **Log-Linear Performance Scaling**: Across language modeling benchmarks (WikiText-2, PTB, enwik8, text8), perplexity dropped in a continuous, log-linear relationship as model capacity expanded from 117M to 1.5B parameters. On WikiText-2, GPT-2 1.5B halved the prior state-of-the-art perplexity (from 39.14 down to 18.34).

4. **Zero-Shot Conversational QA and Prompting Emergence**: On the Conversational Question Answering (CoQA) dataset, GPT-2 was evaluated by reading context passages and answering multi-turn dialog questions zero-shot. By formatting the dialog history and appending the terminal prompt `"A:"`, GPT-2 1.5B achieved 55.0 F1 on the development set, matching or outperforming three out of four prior supervised baselines without using a single one of CoQA's 127,000+ training examples. Furthermore, appending `"TL;DR:"` induced abstractive document summarization, while bilingual phrase pairs prompted machine translation on WMT-14 French-English (achieving 11.5 BLEU zero-shot).

Despite these technical successes, the release of GPT-2 was overshadowed by significant governance controversy. Citing concerns regarding the potential dual-use risks of 1.5B-scale generative models—specifically automated generation of deceptive news articles, personalized phishing campaigns, and industrial-scale social media manipulation—OpenAI initially withheld the 1.5B parameter weights. Instead, OpenAI implemented a phased, staged release: the 117M model was published in February 2019, the 345M model in May, the 762M model in August, and finally the complete 1.5B model in November 2019 after partnership studies observed minimal evidence of malicious misuse. This decision sparked intense debate within the machine learning community: critics viewed it as an orchestrated marketing strategy that compromised open scientific reproducibility, while proponents regarded it as a landmark precedent for responsible disclosure and AI governance.

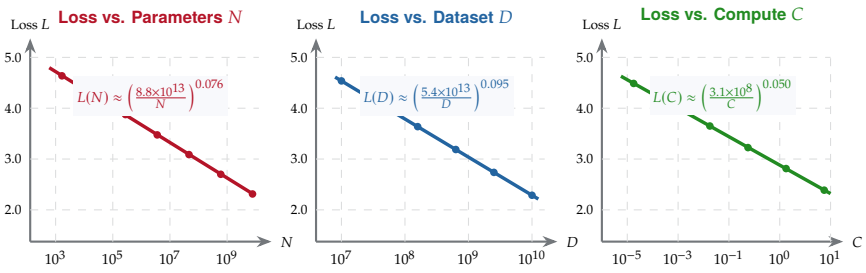


Figure 7.3: The three empirical scaling power laws from Kaplan et al. [63]. Cross-entropy test loss L scales smoothly as a power law with model parameter count N, dataset size D, and training compute C across more than six orders of magnitude when performance is unconstrained by the other factors.

7.2.3 The breakthrough of empirical scaling laws (Kaplan et al., 2020)

The continuous, monotonic improvements observed across the GPT-2 model hierarchy prompted researchers at OpenAI to investigate whether neural language model performance was governed by quantitative physical regularities. In January 2020, Kaplan et al. [63] published *"Scaling Laws for Neural Language Models"*, establishing that language modeling loss adheres to precise, predictable empirical power laws spanning many orders of magnitude.

Kaplan et al. conducted thousands of training runs spanning over six orders of magnitude in compute budget ($C \in [10^{17}, 10^{23}]$ FLOPs, or 10^{-5} to 10^{0} PF-days), model parameter capacity ($N \in [10^3, 10^9]$ non-embedding parameters), and training dataset volume ($D \in [10^7, 10^{10}]$ tokens). Their investigations revealed that cross-entropy test loss L scales as a power law with each scale factor when performance is unconstrained by the other two:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \text{for } D \gg D_{\text{opt}}(N), \tag{7.11}$$

$$L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \text{for } N \gg N_{\text{opt}}(D), \tag{7.12}$$

$$L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \quad \text{for optimal allocation of } (N, D), \tag{7.13}$$

where $N_c \approx 8.8 \times 10^{13}$ parameters, $D_c \approx 5.4 \times 10^{13}$ tokens, and $C_c \approx 3.1 \times 10^{8}$ PF-days denote constant scale factors, with fitted power-law exponents:

$$\alpha_N \approx 0.076, \quad \alpha_D \approx 0.095, \quad \alpha_C \approx 0.050. \tag{7.14}$$

Figure 7.3 illustrates these empirical relationships. On log-log coordinates, the test loss forms straight, parallel trajectories across six orders of magnitude. The authors noted that performance exhibits remarkable *hyperparameter insensitivity*: within broad boundaries, network shape hyperparameters (such as depth L versus width d_{model}, feed-forward expansion ratios, or number of attention heads) exert negligible influence on test loss when total non-embedding parameter count N is held fixed.

Beyond basic power laws, Kaplan et al. investigated *sample efficiency* and the dynamics of compute-efficient training. As illustrated in Panel A of Figure 7.4, their findings demonstrated that larger models are substantially more sample-efficient than smaller models: a large model reaches any target test loss with orders of magnitude fewer tokens processed and vastly fewer optimization steps. Consequently, **compute-efficient training stops far short of convergence**. Training a smaller model to convergence is computationally wasteful; it is far more compute-efficient to train a much larger model for fewer optimization steps, early-stopping before convergence.

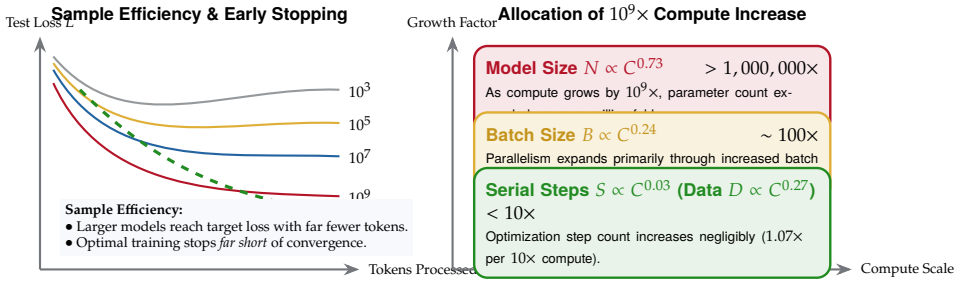


Figure 7.4: Sample efficiency and compute budget allocation in Kaplan et al. [63]. Left: Larger models require orders of magnitude fewer tokens to attain an equivalent test loss, proving that compute-efficient training stops far short of full convergence. Right: Over a billion-fold ($10^9\times$) expansion in compute budget, Kaplan's empirical allocation dictates that over $1,000,000\times$ of the budget be spent on expanding model capacity N, while serial training steps increase by less than $10\times$.

From these dynamics, Kaplan et al. derived the empirically optimal allocation of compute. Modeling total floating-point operations as $C \approx 6ND = 6NBS$ (where B is batch size and S is the number of parameter update steps), the authors analyzed how an expanding compute budget should be partitioned between model size N, batch size B, and serial training steps S. Under optimal allocation at the critical batch size B_{crit}, they derived:

$$N \propto C^{0.73}, \quad B \propto C^{0.24}, \quad S \propto C^{0.03}, \quad \text{yielding} \quad D = B \cdot S \propto C^{0.27}. \tag{7.15}$$

Panel B of Figure 7.4 illustrates the physical consequence of Equation (7.15) over a billion-fold ($10^9\times$) increase in computational budget. To make optimal use of compute:

- Model parameter capacity N should increase by more than **one million-fold** ($> 1,000,000\times$).
- Batch size B should expand by approximately **one hundred-fold** ($\sim 100\times$).
- Serial optimization steps S should increase by **less than ten-fold** ($< 10\times$).

In practical engineering terms, Kaplan et al. concluded: *"As the computational budget C increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size."* Specifically, a $10\times$ increase in compute budget prescribed scaling model size N by $5.4\times$, while increasing training tokens D by merely $1.8\times$ and optimization steps by $1.07\times$. This theoretical prescription directly catalyzed the industry race toward colossal parameter regimes trained on relatively modest token horizons.

7.3 GPT-3 and In-Context Learning

In June 2020, Brown et al. [2] from OpenAI published *"Language Models are Few-Shot Learners"*, unveiling GPT-3. Building directly upon the scaling laws of Kaplan et al., GPT-3 scaled model capacity to 175 billion parameters—an order of magnitude larger than any previously trained dense autoregressive model.

7.3.1 Model topology, 175B capacity, and filtered corpus mixture

The GPT-3 architecture preserved the core decoder-only Transformer design of GPT-2, incorporating alternating dense and locally banded sparse self-attention sub-layers:

- **Depth and Width**: $L = 96$ Transformer layers, each containing $H = 96$ attention heads with per-head projection dimension $d_{\text{head}} = 128$.

- **Hidden Dimensionality**: Hidden representation dimension $d_{\text{model}} = 12,288$, with position-wise feed-forward intermediate dimensionality $d_{\text{ff}} = 4 \times 12,288 = 49,152$.
- **Context Window and Parameters**: Context window expanded to $L = 2048$ tokens, with a total model capacity of $N = 175$B parameters.

To train a 175B parameter network, OpenAI assembled a multi-source training mixture totaling 300 billion tokens. Given that raw web data contains significant noise, spam, and low-quality text, the authors implemented an aggressive quality filtering and differential sampling strategy:

1. **Filtered Common Crawl** (45 TB raw $\rightarrow$ 570 GB cleaned; 410B raw tokens): Filtered using a binary fastText classifier trained to discriminate curated WebText from raw web pages. During pre-training, Common Crawl was sampled at a rate of 60% (accounting for 180B tokens).
2. **WebText2** (19B tokens): An expanded version of WebText, sampled at 22% (66B tokens).
3. **Books1 and Books2** (12B and 55B tokens): Curated literary corpora, each sampled at 8% (24B tokens each).
4. **Wikipedia** (3B tokens): High-quality encyclopedic text, sampled at 3% (9B tokens).

High-quality, curated datasets were intentionally oversampled across multiple epochs, while lower-quality web scrapes were downsampled, establishing dataset composition as a first-order hyperparameter in large-scale pre-training.

7.3.2 In-context learning mechanics across the prompting spectrum

The central conceptual breakthrough of the GPT-3 paper was the formalization of *in-context learning* (ICL) as an operational alternative to supervised fine-tuning.

In the classical fine-tuning paradigm, model parameters $\boldsymbol{\Theta}$ are modified using gradient descent on task-specific labeled datasets:

$$\boldsymbol{\Theta}_{t+1} = \boldsymbol{\Theta}_t - \eta \nabla_{\boldsymbol{\Theta}} \mathcal{L}_{\text{task}}(\boldsymbol{\Theta}_t; \mathcal{D}_{\text{train}}). \tag{7.16}$$

While effective, fine-tuning introduces severe practical limitations: it requires curated labeled datasets for every task, risks catastrophic forgetting of general reasoning capabilities, and demands storing separate fine-tuned parameter weights for each application.

In contrast, under in-context learning, the pre-trained model parameters $\boldsymbol{\Theta}$ remain strictly frozen:

$$\nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0}. \tag{7.17}$$

The model executes task adaptation entirely in inference mode via feed-forward attention passes. Conditioning instructions and demonstration exemplars are provided directly inside the input prompt, and the model predicts the task completion through autoregressive decoding:

$$\hat{y} = \arg\max_{y \in \mathcal{V}^*} P(y \mid \text{Prompt}; \boldsymbol{\Theta}), \tag{7.18}$$

where Prompt = (Task Instruction, $(x_1, y_1), \ldots, (x_k, y_k), x_{\text{test}}$).

As illustrated in Figure 7.5, in-context learning spans three distinct operational regimes defined by the number of demonstration exemplars k provided in the context window:

1. **Zero-Shot Learning ($k = 0$)**: The model receives only a natural language description of the task followed by the test prompt x_{test}. No input-output examples are supplied. This evaluates the model's ability to interpret abstract instructions zero-shot.

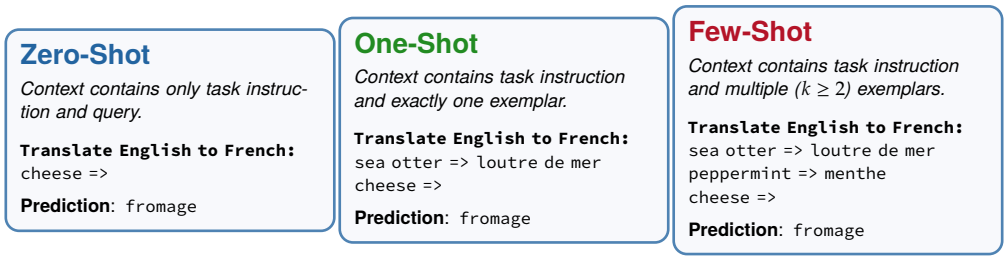


Figure 7.5: The prompting spectrum formalized in GPT-3 [2]. The model solves tasks without gradient updates by conditioning on zero, one, or multiple input-output demonstrations.

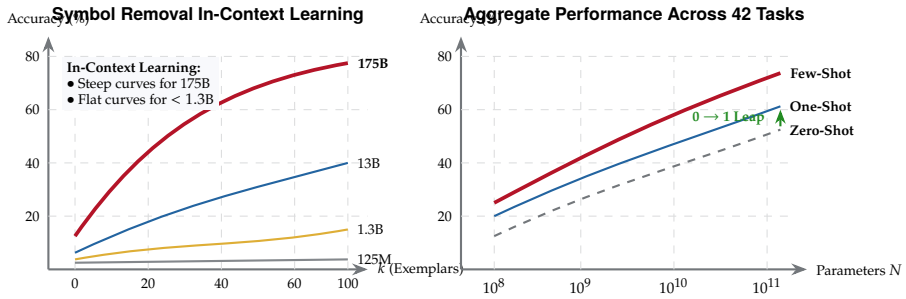


Figure 7.6: Empirical in-context performance scaling in GPT-3 [2]. Left: In-context learning curves on the symbol removal task as a function of demonstration exemplars k, showing that the ability to rapidly extract task instructions emerges only at massive parameter scale (13B and 175B). Right: Aggregate accuracy across 42 evaluation benchmarks as a function of parameter scale, highlighting the prominent zero-to-one shot discontinuity and the widening advantage of few-shot conditioning.

2. **One-Shot Learning ($k = 1$)**: The model receives the task description along with exactly one demonstration exemplar (x_1, y_1) prior to the test query x_{test}. The single exemplar serves as an explicit structural anchor, clarifying the expected formatting, delimiters, and target style.

3. **Few-Shot Learning ($k \geq 2$)**: The model receives the task description along with k demonstration pairs:
$$((x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)),$$
where k typically ranges from 10 to 100 depending on sequence length limits.

Figure 7.6 highlights the empirical dynamics of in-context learning across the GPT-3 model hierarchy (ranging from 125M, 350M, 760M, 1.3B, 2.7B, 6.7B, 13B, up to 175B parameters):

- **In-Context Learning as an Emergent Capability of Scale**: Smaller model variants (125M to 1.3B) exhibit near-flat in-context learning curves. As demonstrated on synthetic tasks such as symbol removal (Panel A of Figure 7.6), where the network must strip extraneous punctuation characters from corrupted words, small models fail to adapt regardless of how many exemplars k are supplied. However, as parameter capacity scales to 13B and 175B, the slope of the learning curve steepens dramatically, climbing from $\sim 15\%$ zero-shot accuracy to $> 80\%$ accuracy with $k = 100$ context demonstrations.

- **The Zero-to-One Shot Discontinuity**: Across nearly all evaluated natural language tasks (Panel B of Figure 7.6), the single steepest marginal gain in accuracy occurs when moving from zero-shot to one-shot ($k = 0 \rightarrow k = 1$). Providing a single concrete demonstration resolves ambiguity regarding output formatting, syntax, and task delimiters, yielding a massive performance leap across all model scales.

- **Widening Gap with Demonstrations**: While zero-shot performance improves steadily with parameter capacity, few-shot performance scales significantly faster, creating an expanding performance gap that favors massive models conditioned on multiple context exemplars.

7.3.3 Empirical benchmark performance and comparison with fine-tuned models

The most significant empirical finding of Brown et al. [2] was that a frozen 175B parameter language model conditioned purely via few-shot demonstrations could achieve performance competitive with, and in several cases superior to, task-specific state-of-the-art models fine-tuned on hundreds of thousands of labeled examples.

Table 7.2 provides an extensive quantitative comparison across closed-book question answering, machine translation, commonsense reasoning, and reading comprehension benchmarks.

A rigorous examination of Table 7.2 provides vital insights into the strengths and boundaries of in-context scaling:

- **Superhuman Closed-Book Retrieval on TriviaQA**: In the closed-book setting, models must answer open-domain factual queries without access to external documents or retrieval search engines. On TriviaQA, GPT-3 175B zero-shot (64.3%) already outperformed the fine-tuned closed-book T5-11B baseline (50.1%), while few-shot GPT-3 attained 71.2%, surpassing even the state-of-the-art open-domain retrieval-augmented RAG model (68.0%). This proved that scaling parameter capacity to 175B allows dense feed-forward networks to act as implicit factual knowledge bases, storing and retrieving world facts directly within weight matrices.

- **Unsupervised Machine Translation into English**: Without any parallel training corpora, GPT-3 few-shot achieved 39.2 BLEU translating French into English (surpassing supervised SOTA by +4.2 BLEU) and 40.6 BLEU translating German into English (matching/exceeding supervised SOTA). On Romanian-to-English, GPT-3 reached 39.5 BLEU, within 0.4 BLEU of a supervised system trained on 608,000 labeled sentence pairs and back-translation. However, translating *out* of English into foreign languages exhibited noticeably lower BLEU scores, reflecting the heavy English skew (> 90%) of the pre-training corpus.

- **Commonsense Physical Reasoning on PIQA**: On the Physical Interaction QA (PIQA) dataset, GPT-3 scored 82.8% in the few-shot setting, setting a new overall state-of-the-art that exceeded the prior fine-tuned RoBERTa model (79.4%).

- **The Failure Modes: Discrete Reasoning and Span Extraction**: While GPT-3 dominated factual retrieval and text synthesis, it lagged behind specialized fine-tuned models on DROP (36.5 F1 vs. SOTA 89.1) and SQuAD v2.0 (69.8 F1 vs. SOTA 93.0). Tasks on DROP require multi-step discrete numerical operations (counting, sorting, addition) over paragraph spans, which standard autoregressive token generation cannot reliably solve without explicit scratchpads or chain-of-thought prompting. Similarly, extractive span QA models trained specifically to predict start and end boundary logits naturally outperform generative token decoding on exact span matching.

7.4 Chinchilla and Compute-Optimal Scaling

Following the publication of GPT-3 and the confirmation that model loss scales smoothly with parameter capacity, the natural language processing industry embarked on an aggressive race to construct ever-larger dense neural networks. Between 2020 and 2022, leading research institutions treated parameter count as the primary determinant of model intelligence.

7.4.1 The race to massive scale and the undertraining diagnosis

During this era, major industrial laboratories deployed dense autoregressive models well exceeding 100 billion parameters:

- **Jurassic-1** (AI21 Labs, 2021): 178 billion parameters, evaluated on broad linguistic benchmarks.
- **Gopher** (DeepMind, 2021): 280 billion parameters, trained on 300 billion tokens with a compute budget of $C = 5.76 \times 10^{23}$ FLOPs [64].
- **Megatron-Turing NLG** (NVIDIA and Microsoft, 2021): 530 billion parameters, trained across massive distributed GPU clusters using 3D parallelism [65].
- **PaLM** (Google, 2022): 540 billion parameters, trained using the Pathways distributed compiler across dual TPU v4 pods on 780 billion tokens [66].

During this period, foundation model papers adopted a distinctive structure: extensive catalogs of empirical benchmark results (frequently spanning over 100 pages) accompanied by brief architectural descriptions. However, operational severe bottlenecks emerged: deploying a 500B+ parameter model required multi-node tensor parallelism, massive GPU cluster interconnects, and prohibitive inference memory bandwidth, making real-world serving economically unsustainable.

In March 2022, Hoffmann et al. [3] from DeepMind published *"Training Compute-Optimal Large Language Models"*, introducing the **Chinchilla** architecture. Their central thesis challenged the industry consensus:

> *"Current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the training data constant."*

Hoffmann et al. diagnosed a fundamental methodological flaw in the empirical framework of Kaplan et al. [63]. Specifically, Kaplan's experiments trained models using a fixed cosine learning rate schedule that decayed learning rates across a constant step horizon. Consequently, models trained for longer durations were evaluated using suboptimal learning rates, which artificially depressed their performance and made them appear less compute-efficient. Furthermore, Kaplan kept training dataset sizes relatively constant ($D \approx 300$B tokens) while expanding parameter counts ($N \rightarrow 175$B), creating a massive data starvation bottleneck.

Re-evaluating the compute-optimal frontier, DeepMind proved that parameter count N and training dataset size D should be scaled in *equal proportion*:

$$N \propto C^{0.5}, \quad D \propto C^{0.5}. \tag{7.19}$$

That is: **for every doubling of model parameters, the number of training tokens must also be doubled.**

7.4.2 The three methodological approaches to compute-optimal scaling

To establish the compute-optimal scaling frontier, Hoffmann et al. trained over 400 models spanning parameter counts from 70M to 16B and dataset sizes from 5B to 500B tokens across three distinct methodological approaches.

Approach 1: Minimum Loss Envelopes over Training Curves In Approach 1, the authors varied training duration and compute budget for models of fixed parameter capacity (70M to 10B parameters), matching the cosine learning rate schedule length to each specific run. By plotting training curves across FLOP milestones and extracting the lower boundary envelope

across all curves, they identified the model size that achieved minimal loss for each compute budget. Fitting power-law equations $N_{\text{opt}} \propto C^{a}$ and $D_{\text{opt}} \propto C^{b}$ on log-log coordinates yielded:

$$N_{\text{opt}} \propto C^{0.49}, \quad D_{\text{opt}} \propto C^{0.51}. \tag{7.20}$$

When applied to Gopher's computational budget ($C = 5.76 \times 10^{23}$ FLOPs), Approach 1 demonstrates that the optimal model size is **67 billion parameters**, rather than the 280 billion instantiated in Gopher.

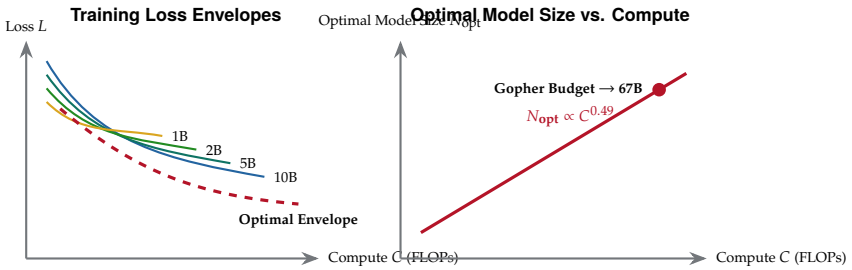


Figure 7.7: Chinchilla Approach 1: Minimum loss envelopes over training curves [3]. By extracting the optimal parameter size at the lower boundary envelope for each compute budget, a linear fit on log-log coordinates establishes $N_{\text{opt}} \propto C^{0.49}$, indicating that Gopher's compute budget corresponds to an optimal model size of 67B parameters.

Approach 2: IsoFLOP Profiles In Approach 2, compute budgets were held strictly constant across discrete slices ($C \in [6 \times 10^{18}, 3 \times 10^{21}]$ FLOPs). For each slice, parameter count N was varied while setting token count according to $D = C/(6N)$. Plotting final validation loss against parameter size revealed a convex parabolic profile: undersized models suffer from insufficient capacity, while oversized models suffer from undertraining. The vertex of each parabola defines the optimal parameter size. Fitting a power-law to these minima yielded:

$$N_{\text{opt}} \propto C^{0.49}, \quad D_{\text{opt}} \propto C^{0.51}. \tag{7.21}$$

Evaluating this fit at Gopher's compute allocation yields an optimal model size of **63 billion parameters**.

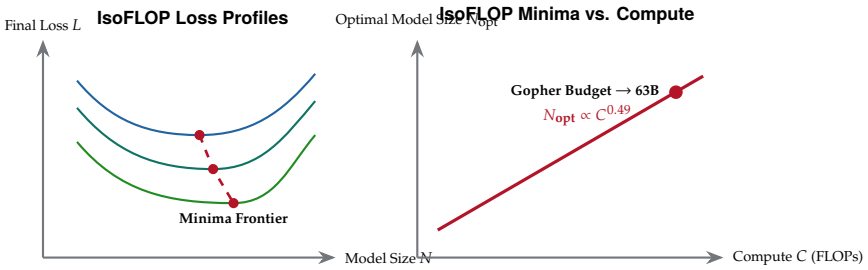


Figure 7.8: Chinchilla Approach 2: IsoFLOP profiles [3]. For a constant compute budget, plotting final loss against parameter count yields a convex parabolic profile. The minimum of each parabola identifies the optimal model size, yielding $N_{\text{opt}} \propto C^{0.49}$ and indicating an optimal Gopher scale of 63B.

Approach 3: Parametric Loss Surface Modeling In Approach 3, Hoffmann et al. formalized the continuous loss surface via a joint parametric formulation:

$$\hat{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \tag{7.22}$$

where $E = 1.69$ is the irreducible natural entropy of text, A/N^{α} is the penalty for finite parameter capacity, and B/D^{β} is the penalty for finite training data. Fitting via Huber loss minimization yielded:

$$A = 406.4, \quad B = 410.7, \quad \alpha = 0.34, \quad \beta = 0.28. \tag{7.23}$$

Solving the constrained optimization $\min_{N,D} \hat{L}(N, D)$ subject to $6ND = C$ via Lagrange multipliers yields:

$$N_{\text{opt}} \propto C^{\frac{\beta}{\alpha+\beta}} = C^{\frac{0.28}{0.34+0.28}} = C^{0.45}, \quad D_{\text{opt}} \propto C^{\frac{\alpha}{\alpha+\beta}} = C^{\frac{0.34}{0.34+0.28}} = C^{0.55}. \tag{7.24}$$

Evaluating Approach 3 for Gopher's compute allocation yields an optimal model size between **40 and 70 billion parameters**.

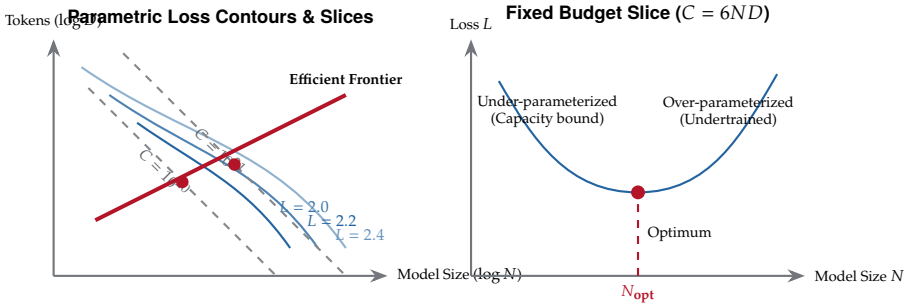


Figure 7.9: Chinchilla Approach 3: Parametric loss surface modeling [3]. Left: Continuous loss contours generated by fitting Equation (7.22). For each fixed compute budget line $C = 6ND$, optimal allocation occurs where the budget line is tangent to a loss contour, tracing the Efficient Frontier ($N_{\text{opt}} \propto C^{0.45}$). Right: Vertical slice at fixed compute budget demonstrating that deviation from optimal parameter capacity yields elevated loss due to either under-parameterization or undertraining.

7.4.3 Empirical validation: Chinchilla (70B) versus Gopher (280B)

To empirically test their derivations, DeepMind trained **Chinchilla**, a 70-billion-parameter model using the exact same compute budget as Gopher ($C = 5.76 \times 10^{23}$ FLOPs). Table 7.3 highlights the structural resource allocation between Chinchilla and contemporary oversized architectures.

While Gopher allocated its budget to maintain 280 billion parameters trained on 300 billion tokens, Chinchilla used a model 1/4× the size (70B) trained on over 4× more data (1.4 trillion tokens).

Despite having only one-fourth of Gopher's parameter count, Chinchilla uniformly outperformed Gopher, GPT-3, Jurassic-1, and Megatron-Turing NLG across an extensive suite of downstream benchmarks:

- **MMLU (Massive Multitask Language Understanding)**: Chinchilla achieved 67.5% average accuracy, outperforming Gopher (60.0%) and GPT-3 (53.9%).
- **BIG-bench**: Chinchilla achieved a 10.7% absolute performance improvement over Gopher.
- **Commonsense and Reading Comprehension**: Chinchilla outperformed GPT-3 and Gopher on LAMBADA, PIQA, and BoolQ.

The Chinchilla experiments conclusively refuted the Kaplan allocation rule, proving that the post-GPT-3 race toward massive parameter sizes had produced oversized, undertrained architectures.

7.5 Open Foundation Models and Token Saturation

The Chinchilla breakthrough prompted an industry-wide transition in language model training. Rather than continuing parameter escalation (500B → 1T), the field pivoted toward training compact models on massive token volumes. This shift made high-performance language models vastly more accessible, catalyzing the modern open-weight ecosystem.

7.5.1 Token over-training and inference economics

While Chinchilla established the *compute-optimal training* ratio ($N \propto D$), real-world deployment depends heavily on *inference efficiency*.

In a production environment, training compute is amortized over billions of downstream user queries. A 70B parameter model requires substantially more GPU memory bandwidth, higher latency, and complex multi-GPU tensor parallelism during inference compared to an 8B parameter model. Consequently, training a smaller model (e.g., 7B or 8B) far beyond the Chinchilla compute-optimal frontier—training on 2 trillion to 15 trillion tokens (*token over-training*)—incurs a higher initial training cost, but yields dramatically lower per-token latency and memory requirements at inference time.

Token over-training allowed high-performance foundation models to fit within the memory capacity of single workstation GPUs, consumer graphics cards, and edge hardware, laying the groundwork for widespread open-weight adoption.

7.5.2 The LLaMA ecosystem and generational evolution

In February 2023, Touvron et al. [56] from Meta AI introduced **LLaMA** (*Large Language Model Meta AI*), demonstrating that open foundation models trained exclusively on publicly available datasets could match or exceed proprietary models.

The LLaMA series evolved through successive generations:

1. **LLaMA-1 (February 2023)**: Released in four sizes (7B, 13B, 33B, 65B) trained on 1.0T to 1.4T tokens. The 13B model outperformed GPT-3 (175B) across most benchmarks while fitting on a single consumer GPU, while LLaMA-65B rivaled Chinchilla-70B and PaLM-540B.

2. **Llama 2 (July 2023)**: Trained on 2.0T tokens with an expanded context window of 4096 tokens. The architecture introduced Grouped-Query Attention (GQA) for the 70B model to reduce key-value cache memory overhead during autoregressive generation [67]. Meta released specialized fine-tuned checkpoints, including Llama-2-Chat (aligned via Reinforcement Learning from Human Feedback) and Code Llama (specialized for software engineering).

3. **Llama 3 and 3.1 (April/July 2024)**: Scaled training data to over 15 trillion tokens with an expanded 128k context window and a 128k tiktoken vocabulary. The release featured an 8B model, a 70B model, and the flagship **405B parameter frontier model**, trained across a cluster of 16,000 H100 GPUs using 4D parallelism (tensor, pipeline, context, and data parallelism).

4. **Llama 3.2 (September 2024)**: Introduced lightweight edge models (1B and 3B parameters) optimized for on-device inference, alongside multimodal vision-language models (11B and 90B parameters) capable of processing image-text inputs.

Meta released the LLaMA weights under a permissive community license, with an explicit commercial clause requiring custom licensing only for platforms exceeding 100 million daily active users (effectively restricting competing hyperscalers while providing broad access to academic and commercial developers).

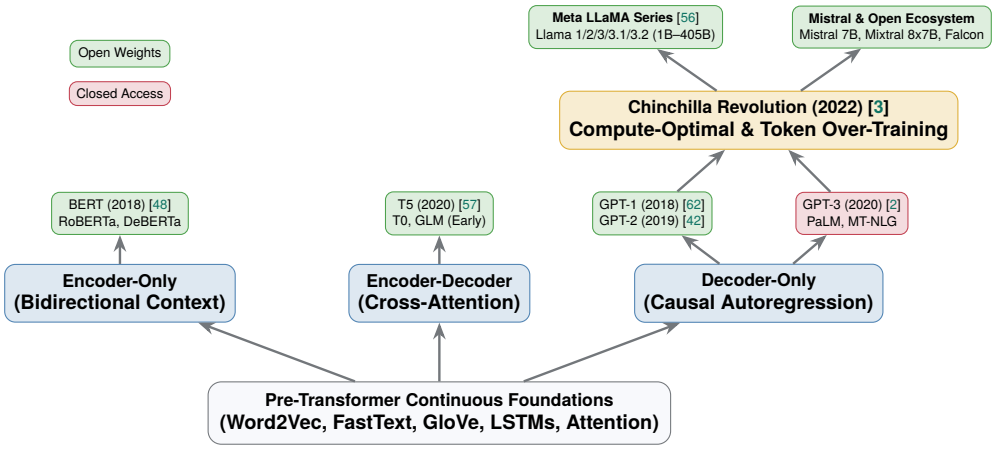


Figure 7.10: The macro-evolutionary tree of language models. Following the introduction of the Transformer in 2017, the field branched into encoder-only, encoder-decoder, and decoder-only architectures. The decoder-only branch expanded rapidly from GPT-1 and GPT-2 to massive proprietary models, before the Chinchilla compute-optimal revolution redirected development toward token-saturated open foundation models.

7.5.3 Open foundation model families and the macro-evolutionary tree

The success of the LLaMA family catalyzed a broader ecosystem of open foundation models:

- **EleutherAI (GPT-Neo, GPT-J, GPT-NeoX)**: An open-source research collective that championed accessible foundation models by reverse-engineering early GPT architectures and releasing **The Pile** (an 825 GB diverse text corpus).
- **Mistral AI (France)**: Founded in 2023, Mistral introduced compact, high-efficiency models utilizing Sliding Window Attention. Key releases include **Mistral 7B** (outperforming Llama 2 13B), **Mixtral 8x7B** (a sparse Mixture-of-Experts model that routes tokens dynamically through 8 feed-forward experts, activating only 2 experts per token to deliver 47B capacity with 13B active compute speed), and **Pixtral** (multimodal vision capabilities).
- **GLM (Zhipu AI and Tsinghua University, China)**: The General Language Model family, designed around an autoregressive blank-filling objective to unify bidirectional and causal modeling. GLM models emphasize bilingual proficiency across Chinese and English.
- **Falcon (Technology Innovation Institute, UAE)**: Falcon 7B, 40B, and 180B, trained on the custom-curated RefinedWeb corpus. The Falcon initiative also pioneered evaluations of non-Transformer sequence models, including state-space architectures (Mamba).

Figure 7.10 synthesizes the architectural divergence of language models from pre-Transformer continuous representations into the three modern branches, illustrating the evolution of open-weight and closed-access systems.

As shown in Figure 7.10, language modeling development converged decisively toward the decoder-only paradigm. While encoder-only models stabilized after dominating classification and embedding benchmarks, decoder-only models became the universal architecture for multi-task reasoning, zero-shot transfer, and autonomous agents. The Chinchilla compute-optimal revolution redirected this trajectory from pure parameter scaling to token saturation, democratizing foundation model deployment across the global open-source community.

Table 7.2: Empirical evaluation of GPT-3 (175B) across Zero-Shot, One-Shot, and Few-Shot settings compared to prior fine-tuned task-specific state-of-the-art models [2].

Benchmark / Task	Metric	Fine-Tuned SOTA	GPT-3 (0-Shot)	GPT-3 (1-Shot)	GPT-3 (Few-Shot)
Closed-Book Question Answering					
TriviaQA	Acc (%)	68.0 / 60.5*	64.3	68.0	**71.2**
WebQuestions	Acc (%)	45.5 / 44.7*	14.4	25.3	41.5
Natural Questions	Acc (%)	44.5 / 36.6*	14.6	23.0	29.9
Machine Translation (WMT)					
WMT'14 Fr → En	BLEU	35.0	21.2	33.7	**39.2**
WMT'16 De → En	BLEU	40.2	27.2	30.4	**40.6**
WMT'16 Ro → En	BLEU	39.9	19.9	38.6	39.5
Commonsense Reasoning & Winograd					
PIQA	Acc (%)	79.4	80.5	80.5	**82.8**
WSC273 (Winograd)	Acc (%)	90.1	88.3	89.7	88.6
Winogrande (Adversarial)	Acc (%)	84.6 / 79.0†	70.2	73.2	77.7
ARC (Easy)	Acc (%)	92.0	68.8	71.2	70.1
ARC (Challenge)	Acc (%)	78.5	51.4	53.2	51.5

Table 7.3: Comparison of compute-optimal Chinchilla against oversized architectures under equivalent or comparable training compute budgets [3].

Model	Parameters (N)	Tokens (D)	FLOPs (C)	Relative Sizing Regime
GPT-3	175B	300B	3.15×10^{23}	Severely undertrained ($N \gg D_{\text{opt}}$)
Jurassic-1	178B	300B	3.20×10^{23}	Severely undertrained ($N \gg D_{\text{opt}}$)
Gopher	280B	300B	5.76×10^{23}	Severely undertrained (4× optimal size)
Megatron-Turing	530B	270B	8.59×10^{23}	Extreme parameter over-allocation
Chinchilla	**70B**	**1400B (1.4T)**	5.76×10^{23}	**Compute-Optimal Frontier**

CHAPTER 8

Model Evaluation and Benchmarks

Chapter 7 described how model size, training data, and compute affect next-token cross-entropy. This objective measures token prediction under teacher forcing, where the model conditions on the reference prefix. It does not directly measure whether generated text is coherent, factually accurate, appropriate to its context, or useful for a reasoning task. Evaluation therefore requires measures that reflect the intended use of the model.

Natural-language generation is difficult to evaluate because a meaning can be expressed by many valid strings. Classification and regression often use a bounded output space with a defined target, whereas a language or code prompt may admit many correct responses. A candidate can share most of its words with a reference while reversing its meaning by adding a negation. Human review can assess qualities that automatic measures miss, but it is costly and may vary across reviewers, so it does not by itself provide a scalable basis for comparing models or checkpoints.

Evaluation methods address different properties. Intrinsic measures such as cross-entropy and perplexity assess likelihood assignments on held-out text. Reference-based metrics such as BLEU and ROUGE compare generated candidates with reference strings using n-gram overlap and length penalties. Contextual measures such as BERTScore, model-based evaluators, and task benchmarks such as GLUE, MMLU, and HumanEval provide evidence about semantic similarity or performance on specified tasks.

This chapter formalizes the principles of language model evaluation. We begin by examining the core mathematical properties required for rigorous evaluation metrics, focusing on the distinction between additive and multiplicative averaging, before conducting an exhaustive investigation of intrinsic uncertainty estimation via perplexity and surface-string reference matching via BLEU.

8.1 Evaluation Metrics and Principles

Evaluation metrics in natural language processing can be partitioned into two distinct paradigms: intrinsic likelihood metrics and extrinsic generation metrics. Intrinsic metrics evaluate the model's mathematical representation of language by computing the probability distribution it assigns to unseen test corpora under teacher forcing. They require no sample generation, no decoding algorithms, and no reference texts; they assess the predictive confidence of the probability model itself. Extrinsic metrics, by contrast, operate on the discrete sequences emitted during autoregressive inference, comparing the generated hypothesis against one or more human gold-standard references.

The geometric mean and multiplicative error penalization

Before formalizing specific metrics, we must establish the mathematical foundation of sequence-level score aggregation. When evaluating models across multiple tokens, sentences, or n-gram orders, individual scores must be aggregated into a single scalar summary. While the standard arithmetic mean is familiar, it possesses structural properties that render it unsuitable for probability spaces and multi-criteria quality evaluation.

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n_{>0}$ denote a collection of n positive evaluation values (such as token probabilities or n-gram precision scores). The arithmetic mean (AM) and geometric mean (GM) are defined as:

$$\mathrm{AM}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i, \tag{8.1}$$

$$\mathrm{GM}(\mathbf{x}) = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}. \tag{8.2}$$

The critical distinction between these two operators lies in how they penalize low values. The arithmetic mean exerts an *additive effect*: a severely low score in one component can be easily masked by high scores in other components. For example, consider three evaluation scores $\mathbf{x} = (0.1, 0.9, 0.9)$. The arithmetic mean yields:

$$\mathrm{AM}(0.1, 0.9, 0.9) = \frac{0.1 + 0.9 + 0.9}{3} = \frac{1.9}{3} \approx 0.6333, \tag{8.3}$$

producing a deceptively favorable aggregate score despite substantial failure on the first criterion. In contrast, the geometric mean exerts a *multiplicative effect*:

$$\mathrm{GM}(0.1, 0.9, 0.9) = (0.1 \times 0.9 \times 0.9)^{\frac{1}{3}} = (0.081)^{\frac{1}{3}} \approx 0.4327. \tag{8.4}$$

Because the terms multiply, any degradation in an individual component disproportionately suppresses the entire product. Crucially, if any single term reaches zero ($x_i = 0$), the geometric mean collapses to zero:

$$\exists i \text{ such that } x_i = 0 \implies \mathrm{GM}(\mathbf{x}) = 0. \tag{8.5}$$

This property is indispensable in evaluation: a model must achieve reasonable competence across all evaluated dimensions to attain a high overall score. This behavior directly parallels the harmonic mean used in the F_1-score to balance precision and recall.

Numerical stability via logarithmic formulation

In practical computation, evaluating long sequences or high-order n-grams involves multiplying dozens or hundreds of terms bounded in $(0, 1]$. In finite-precision floating-point arithmetic (IEEE

754 float32 or float64), direct repeated multiplication inevitably causes arithmetic underflow, rounding the running product to absolute zero.

To guarantee numerical stability, the geometric mean is computed in the logarithmic domain. Utilizing the monotonic identity $\ln(\prod x_i) = \sum \ln x_i$, the n-th root is transformed into an arithmetic average of logarithms:

$$\mathrm{GM}(\mathbf{x}) = \exp\left(\ln\left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}\right) = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln x_i\right). \tag{8.6}$$

As emphasized in lecture, the base of the logarithm must strictly match the base of the exponentiation operator. Standard scientific implementations employ the natural logarithm ln (base e) and natural exponential $\exp(\cdot)$ so that $\exp(\ln(z)) = z$ is an exact inverse mapping. If base-2 logarithms ($\log_2$) are adopted, the outer transformation must evaluate as $2^{\frac{1}{n}\sum \log_2 x_i}$.

8.1.1 Perplexity

The primary intrinsic metric for evaluating autoregressive language models is **perplexity** (PPL). Rooted in Claude Shannon's mathematical theory of communication and information entropy [27], perplexity quantifies the degree of uncertainty a probabilistic model exhibits when predicting a sequence of tokens.

Conceptual intuition and the effective branching factor

Rather than assessing whether a model generates fluent answers, perplexity measures how surprised the model is when presented with human-written, grammatically correct text. If a model has acquired an accurate statistical representation of English, it will assign high probability to legitimate sentences from held-out corpora. Conversely, if the model is poorly trained or faces out-of-domain text, it will assign low probability to valid tokens, indicating substantial confusion.

Perplexity provides an intuitive geometric interpretation as the **effective branching factor** of the language model. Suppose that at a specific step in a sentence, the model assigns probability $P(w_i \mid w_{<i})$ to the ground-truth token. The reciprocal:

$$K_i = \frac{1}{P(w_i \mid w_{<i})} \tag{8.7}$$

represents the number of equally probable alternative words among which the model was undecided.

Consider a thought experiment where the model's vocabulary contains 50,000 discrete tokens:

- If the model assigns probability $P = 0.25$ to the correct token, $\frac{1}{0.25} = 4$. Even though the vocabulary contains 50,000 candidates, the model's uncertainty is equivalent to rolling a fair 4-sided die of equally likely tokens.
- If the model assigns probability $P = 0.90$, $\frac{1}{0.90} \approx 1.11$. The model is near-certain of the true token, undecided between approximately 1.1 alternatives.
- If the model assigns probability $P = 1.0$, $\frac{1}{1.0} = 1.0$. The model has zero uncertainty.
- If the model assigns probability $P = 0.01$, $\frac{1}{0.01} = 100$. The model is as confused as choosing uniformly among 100 distinct words.

Across an entire sequence of N words, perplexity represents the geometric average of these individual branching factors. A perplexity of 50 indicates that, on average across each position in the sentence, the model was as uncertain as choosing uniformly among 50 equally probable words. Lower perplexity reflects greater predictive confidence and superior alignment with the underlying data distribution.

Formal mathematical derivation and negative log-likelihood

Let $W = (w_1, w_2, \ldots, w_N)$ denote a sequence of N tokens. By the probability chain rule, the joint probability assigned to sequence W by an autoregressive language model parameterized by θ is:

$$P(W) = \prod_{i=1}^{N} P(w_i \mid w_1, \ldots, w_{i-1}; \theta) = \prod_{i=1}^{N} P(w_i \mid w_{<i}). \tag{8.8}$$

The perplexity of the model over the sequence W is formally defined as the reciprocal of the geometric mean of the token conditional probabilities:

$$\mathrm{PPL}(W) = P(W)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} P(w_i \mid w_{<i}) \right)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{<i})} \right)^{\frac{1}{N}}. \tag{8.9}$$

Applying the logarithmic transformation in Equation (8.6), we rewrite perplexity in terms of the sum of token negative log-likelihoods (NLL):

$$\ln \mathrm{PPL}(W) = -\frac{1}{N} \ln P(W) = -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}), \tag{8.10}$$

$$\mathrm{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}) \right) = \exp\left(\frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \right), \tag{8.11}$$

where $\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i}) \geq 0$ is the negative log-likelihood of token w_i.

Equation (8.11) reveals the direct mathematical link between perplexity and the pre-training cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$:

$$\mathcal{L}_{\mathrm{CE}} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \implies \mathrm{PPL}(W) = e^{\mathcal{L}_{\mathrm{CE}}}. \tag{8.12}$$

Perplexity is simply the exponentiated cross-entropy loss. Figure 8.1 illustrates the exponential growth of perplexity as average cross-entropy loss increases, underscoring the disproportionately severe penalty imposed on uncertain predictions.

Mathematical justification for the geometric mean. The selection of the geometric mean in Equation (8.9) is mathematically mandatory for three reasons:

1. **Consistency with probability factorization**: In probability theory, the likelihood of a sequence is formed by the product of conditional probabilities, not their sum. Normalizing a product of N terms by sequence length requires taking the N-th root, so that $\mathrm{PPL}(W)^N = \frac{1}{P(W)}$. Taking an arithmetic mean of probabilities ($\frac{1}{N} \sum P_i$) violates the product rule and lacks theoretical interpretation.

2. **Information-theoretic duality**: Taking the logarithm of the geometric mean yields the arithmetic mean of negative log-likelihoods (cross-entropy), establishing an exact duality with Shannon entropy. An arithmetic mean of probabilities cannot be converted into cross-entropy.

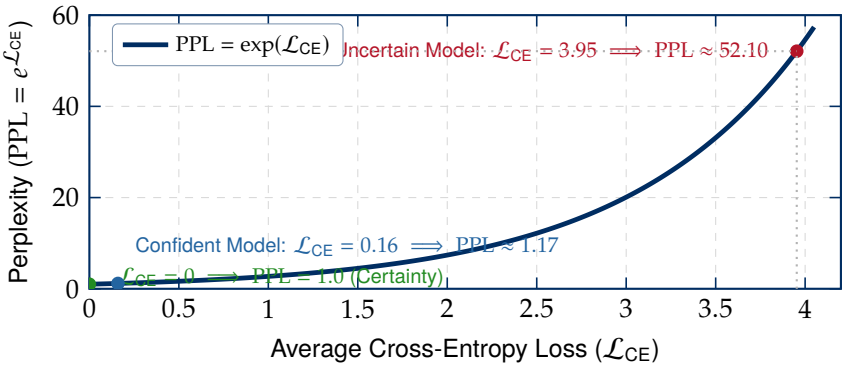


Figure 8.1: Perplexity (PPL) as an exponential function of the average cross-entropy loss $\mathcal{L}_{\text{CE}}$. The metric attains its theoretical minimum at PPL = 1.0 when $\mathcal{L}_{\text{CE}} = 0$. Due to the exponential mapping PPL = $e^{\mathcal{L}_{\text{CE}}}$, modest increments in cross-entropy loss trigger dramatic escalations in perplexity, effectively penalizing poor probability assignments.

3. **Zero-frequency penalty**: If a model assigns zero probability to a single valid token ($P(w_k \mid w_{<k}) = 0$), the geometric mean product collapses to 0, which inverts to infinite perplexity: $\text{PPL}(W) \to \infty$. This correctly identifies the model as broken, since it declared a real-world sentence impossible. An arithmetic mean would merely incur a negligible $\frac{1}{N}$ reduction.

The theoretical lower bound of perplexity is PPL = 1.0. This minimum occurs if and only if the model assigns probability $P(w_i \mid w_{<i}) = 1.0$ to every token in the sequence ($\text{NLL}_i = 0 \implies e^0 = 1$), representing absolute, deterministic certainty. In practice, PPL = 1.0 is observed only when a model severely overfits a training sequence.

Worked examples: confident versus uncertain models

To illustrate the mechanics of perplexity, we walk through the exact numerical evaluations presented in the lecture. Consider a held-out test sentence delimited by boundary markers:

$$W = \langle\text{bos}\rangle\ \text{The dog chased the cat}\ \langle\text{eos}\rangle.$$

The beginning-of-sequence token ⟨bos⟩ serves as initial conditioning prefix ($P = 1.0$). The sequence contains $N = 6$ evaluated transitions ($w_1 = \text{The}, \ldots, w_6 = \langle\text{eos}\rangle$).

Case 1: Confident, well-trained model. Suppose a model assigns high probability to each correct token, reflecting strong linguistic familiarity. The conditional probabilities, negative log-likelihoods, and equivalent branching numbers are listed in Table 8.1.

Table 8.1: Step-by-step perplexity computation for a confident model on W = ⟨bos⟩ The dog chased the cat ⟨eos⟩.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\text{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.90	$-\ln(0.90) \approx 0.1054$	1.1111
2	dog	0.85	$-\ln(0.85) \approx 0.1625$	1.1765
3	chased	0.88	$-\ln(0.88) \approx 0.1278$	1.1364
4	the	0.75	$-\ln(0.75) \approx 0.2877$	1.3333
5	cat	0.95	$-\ln(0.95) \approx 0.0513$	1.0526
6	⟨eos⟩	0.80	$-\ln(0.80) \approx 0.2231$	1.2500
Sum / Mean		—	$\sum \text{NLL}_i = 0.9578 \implies \overline{\text{NLL}} = 0.1596$	$\prod K_i \approx 2.6074$

Evaluating the perplexity via the average negative log-likelihood:

$$\begin{aligned}\mathrm{PPL}(W) &= \exp\left(\frac{1}{6}\sum_{i=1}^{6}\mathrm{NLL}_i\right)\\ &= \exp\left(\frac{0.1054+0.1625+0.1278+0.2877+0.0513+0.2231}{6}\right)\\ &= e^{0.15963} \approx \mathbf{1.1731}.\end{aligned} \tag{8.13}$$

Equivalently, taking the 6-th root of the product of equivalent words:

$$\mathrm{PPL}(W) = (1.1111 \times 1.1765 \times 1.1364 \times 1.3333 \times 1.0526 \times 1.2500)^{\frac{1}{6}} \approx (2.6074)^{\frac{1}{6}} \approx \mathbf{1.1731}. \tag{8.14}$$

The model's perplexity is 1.1731, indicating that on average, the model was undecided over approximately 1.17 equivalent words per position. It possessed near-complete certainty throughout the sequence.

Case 2: Uncertain, poorly trained model. Now suppose an under-trained or out-of-domain model evaluates the identical sentence, assigning substantially lower probabilities to the correct tokens, as detailed in Table 8.2.

Table 8.2: Step-by-step perplexity computation for an uncertain model on W = ⟨bos⟩ The dog chased the cat ⟨eos⟩.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.050	$-\ln(0.050) \approx 2.9957$	20
2	dog	0.100	$-\ln(0.100) \approx 2.3026$	10
3	chased	0.200	$-\ln(0.200) \approx 1.6094$	5
4	the	0.010	$-\ln(0.010) \approx 4.6052$	100
5	cat	0.005	$-\ln(0.005) \approx 5.2983$	200
6	⟨eos⟩	0.001	$-\ln(0.001) \approx 6.9078$	1000
Sum / Product		—	$\sum \mathrm{NLL}_i = 23.7190 \implies \overline{\mathrm{NLL}} = 3.9532$	$\prod K_i = 2 \times 10^{10}$

Computing the geometric mean directly from the branching factors:

$$\prod_{i=1}^{6} K_i = 20 \times 10 \times 5 \times 100 \times 200 \times 1000 = 2 \times 10^{10}, \tag{8.15}$$

$$\mathrm{PPL}(W) = \left(2 \times 10^{10}\right)^{\frac{1}{6}} \approx \mathbf{52.10}. \tag{8.16}$$

Verifying via the average negative log-likelihood:

$$\mathrm{PPL}(W) = \exp\left(\frac{23.7190}{6}\right) = \exp(3.95317) \approx \mathbf{52.10}. \tag{8.17}$$

Both formulations yield identical results: PPL = 52.10. On average, the uncertain model was confused across approximately 52 equally likely candidate tokens at every position.

As highlighted by the professor, perplexity serves as a diagnostic probe into the temporal and thematic distribution of a model's pre-training data. If an older model trained on corpora prior to 2016 is evaluated on the prompt "The President of the United States is [NAME]", evaluating "Barack Obama" yields low perplexity, whereas evaluating "Donald Trump" or "Joe Biden" produces high perplexity. The elevated perplexity does not imply that the model is architecturally defective; rather, it indicates statistical surprise relative to the pre-training data distribution.

Advantages and fundamental limitations of perplexity

Perplexity possesses distinct methodological advantages that have cemented its status as standard reporting metric in pre-training literature:

- **Unsupervised and task-agnostic**: Computing perplexity requires only raw, unannotated text. It eliminates the need for expensive human reference translations, task labels, or subjective grading rubrics.
- **Zero inference compute overhead**: Because evaluation occurs under teacher forcing, perplexity is computed in a single forward pass over batches of tokens simultaneously with cross-entropy training loss, avoiding autoregressive generation.
- **Continuous and sensitive**: Perplexity is a smooth, continuous scalar that detects subtle improvements in language modeling capability long before downstream task accuracies show measurable gains.

Despite these strengths, perplexity suffers from fundamental limitations that prevent it from serving as a universal measure of model quality:

- **Vocabulary and tokenizer incomparability**: Perplexity is strictly incomparable across models that use different vocabularies or tokenization algorithms. Consider a byte-level tokenizer ($|\mathcal{V}| = 256$) versus a subword tokenizer ($|\mathcal{V}| = 100,000$). Because a byte-level model predicts tiny, predictable character fragments, its per-token perplexity will appear dramatically lower than the subword model, even though the subword model generates superior semantic text. Valid cross-model comparisons require normalizing perplexity by the number of words rather than subword tokens:

$$\text{PPL}_{\text{word}} = \exp\left(\frac{N_{\text{tokens}}}{N_{\text{words}}}\mathcal{L}_{\text{CE}}\right). \tag{8.18}$$

- **Disconnection from reasoning and factual truth**: A model can achieve outstanding (low) perplexity by mastering local grammatical syntax, common idioms, and surface transitions while remaining completely incapable of logical deduction, arithmetic calculation, or factual consistency. Perplexity rewards familiarity, not correctness.

8.1.2 Bilingual Evaluation Understudy (BLEU)

While perplexity evaluates the model's internal probability distribution over given text, practical deployment requires evaluating the actual text generated by the model during autonomous inference. In 2002, Papineni, Roukos, Ward, and Zhu [68] introduced the **Bilingual Evaluation Understudy** (BLEU) metric at IBM Research. Originally developed for machine translation, BLEU became the foundational reference-based evaluation metric across natural language processing.

Conceptual intuition and reference-based lexical matching

The fundamental premise of BLEU is that the closer a machine-generated candidate translation resembles a human gold-standard translation, the higher its quality. Evaluating open-ended text against human references requires navigating two competing linguistic dimensions:

1. **Adequacy (Word Choice)**: Did the model translate the correct conceptual content? This is captured by measuring lexical overlap at the unigram level (1-grams).

2. **Fluency (Word Order)**: Did the model assemble the words in a natural, syntactically coherent sequence? This is captured by measuring contiguous overlap across higher-order n-grams (2-grams, 3-grams, 4-grams).

BLEU is an extrinsic metric: it requires running the model autoregressively to generate a hypothesis string G and comparing G against one or more reference strings $R = \{R_1, \ldots, R_m\}$. It is defined as a modified n-gram precision metric multiplied by an exponential brevity penalty.

Mathematical formulation and brevity penalty

To compute BLEU, we first extract all contiguous n-grams of size $i \in \{1, 2, \ldots, n\}$ from both the generated candidate sequence G and the reference sequence R.

Modified clipped n-gram precision. A naive precision metric would simply count the fraction of candidate n-grams that appear in the reference:

$$p_{\text{naive}} = \frac{\sum_{g \in G} \mathbb{I}(g \in R)}{|G|}.$$

However, naive precision can be exploited through pathological repetition. For example, if the reference is "The cat sat on the mat" and a defective model outputs the candidate "the the the the the the", every single generated unigram matches the reference word "the". Naive precision would reward this degenerative output with a perfect score: $p_{\text{naive}} = \frac{6}{6} = 1.0$.

To eliminate this vulnerability, Papineni et al. [68] introduced **clipped precision** (p_i). For each i-gram in the candidate sequence, the count is clipped so that it cannot exceed the maximum number of times that identical i-gram appears in any single reference sentence:

$$\text{Count}_{\text{clip}}(g_i) = \min\left(\text{Count}_G(g_i), \max_{R_j \in R} \text{Count}_{R_j}(g_i)\right). \tag{8.19}$$

The modified i-gram precision p_i is then computed as the sum of clipped counts divided by the total number of candidate i-grams generated:

$$p_i = \frac{\sum_{g_i \in G} \text{Count}_{\text{clip}}(g_i)}{\sum_{g_i \in G} \text{Count}_G(g_i)}. \tag{8.20}$$

In the pathological repetition example, the reference contains the unigram "the" twice ($\max \text{Count}_R(\text{"the"}) = 2$). The clipped count for the candidate is $\min(6, 2) = 2$, reducing the unigram precision to $\frac{2}{6} = 0.3333$.

Multi-order precision and the brevity penalty. While clipped precision prevents repetitive over-generation, precision alone introduces an opposite failure mode: **under-generation**.

Suppose the reference sentence contains 20 words, and the candidate model generates only two words: "The dog". Both words appear in the reference, so $p_1 = \frac{2}{2} = 1.0$. The bigram "The dog" also appears in the reference, so $p_2 = \frac{1}{1} = 1.0$. Without length regulation, the truncated output would achieve a flawless score: BLEU = 1.0, despite discarding 90% of the intended content.

Standard classification metrics resolve under-generation by computing recall. However, in machine translation and open-ended generation, defining recall is complicated because human translations exhibit natural length variance, and multiple reference translations often use different phrasing. Instead of explicit recall, BLEU introduces an exponential multiplicative **Brevity Penalty** (BP).

Let g denote the length of the generated candidate sequence, and let r denote the effective length of the reference sequence. The brevity penalty is defined as:

$$\text{BP} = \begin{cases} 1.0 & \text{if } g > r, \\ \exp\left(1 - \frac{r}{g}\right) & \text{if } g \leq r. \end{cases} \tag{8.21}$$

Figure 8.2 illustrates the mathematical behavior of the brevity penalty as a function of the length ratio $\frac{g}{r}$.

- When $g \geq r$, the candidate length equals or exceeds the reference length. In this regime, BP = 1.0: no brevity penalty is applied. Overly long generations are naturally penalized by the denominator of the precision terms in Equation (8.20).
- When $g < r$, the exponent $1 - \frac{r}{g}$ is strictly negative, causing BP to decay exponentially below 1.0. As the candidate length approaches zero ($g \to 0$), the ratio $\frac{r}{g} \to \infty$, and BP $\to 0$, aggressively driving the overall score to zero.

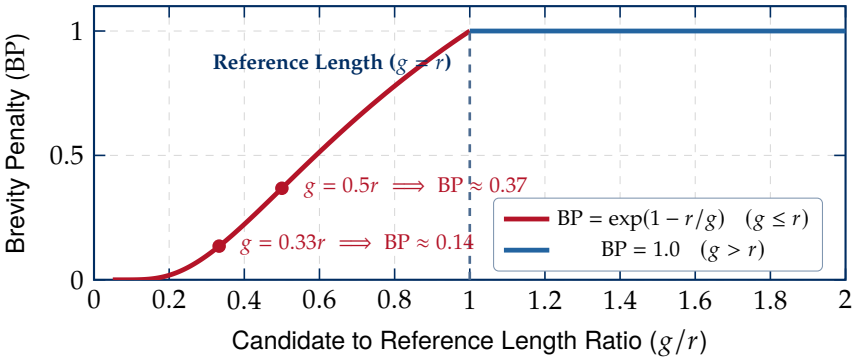


Figure 8.2: The BLEU Brevity Penalty (BP) as a function of the generated-to-reference length ratio g/r. For $g \geq r$, BP = 1.0 because over-generation is already penalized by precision denominators. For $g < r$, BP = $\exp(1 - r/g)$ decays exponentially toward zero, severely penalizing short or truncated generations to prevent precision gaming.

Formal mathematical derivation of the BLEU metric

With the individual components defined, we now formally derive the composite BLEU formula from first principles, following the exact mathematical trajectory used for perplexity.

Let $\{p_1, p_2, \ldots, p_n\}$ denote the modified clipped precisions up to order n, with corresponding non-negative weights $w_i \geq 0$ satisfying $\sum_{i=1}^{n} w_i = 1$. The collective precision across all n-gram orders is combined using a weighted geometric mean:

$$\mathrm{GM}(p_1, \ldots, p_n) = \prod_{i=1}^{n} p_i^{w_i}. \tag{8.22}$$

Under standard uniform weighting across the n orders, $w_i = \frac{1}{n}$, simplifying the product to:

$$\mathrm{GM}(p_1, \ldots, p_n) = \left(\prod_{i=1}^{n} p_i \right)^{\frac{1}{n}}. \tag{8.23}$$

In computational implementations, multiplying small probabilities or fractional precisions directly induces numerical underflow. To guarantee numerical stability, we map the geometric

mean into the log-domain:

$$\begin{aligned}\ln \mathrm{GM}(p_1, \ldots, p_n) &= \ln\left(\prod_{i=1}^{n} p_i^{w_i}\right) \\ &= \sum_{i=1}^{n} \ln\left(p_i^{w_i}\right) \\ &= \sum_{i=1}^{n} w_i \ln p_i. \end{aligned} \tag{8.24}$$

Exponentiating both sides restores the original geometric scale:

$$\mathrm{GM}(p_1, \ldots, p_n) = \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right). \tag{8.25}$$

Notice the mathematical parallel with perplexity: just as perplexity maps the arithmetic mean of negative log-likelihoods back to probability space via $\exp(-\frac{1}{N}\sum \ln P_i)$, BLEU maps the weighted arithmetic mean of log-precisions back to precision space via $\exp(\sum w_i \ln p_i)$.

Finally, coupling this precision term multiplicatively with the brevity penalty BP yields the complete, definitive formulation of BLEU-n:

$$\text{BLEU-}n = \mathrm{BP} \cdot \mathrm{GM}(p_1, \ldots, p_n) = \mathrm{BP} \cdot \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right) = \mathrm{BP} \cdot \left(\prod_{i=1}^{n} p_i\right)^{\frac{1}{n}}. \tag{8.26}$$

Worked example: BLEU-2 calculation

We now execute the step-by-step computation of BLEU-2 ($n = 2$, uniform weights $w_1 = w_2 = 0.5$) on the worked example from the lecture slides.

Let the reference and machine-generated candidate sequences be:

$$R : \text{“The dog chased the cat”} \quad (r = 5 \text{ tokens}),$$
$$G : \text{“The dog ran after the cat”} \quad (g = 6 \text{ tokens}).$$

Step 1: Unigram precision ($i = 1$). Extract all unigrams from both sentences:

$$\text{Reference unigrams } R_1 = \{\text{“The”, “dog”, “chased”, “the”, “cat”}\} \quad (|R_1| = 5),$$
$$\text{Generated unigrams } G_1 = \{\text{“The”, “dog”, “ran”, “after”, “the”, “cat”}\} \quad (|G_1| = 6).$$

We count matching unigrams between G and R:

- “The”: appears in R ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “dog”: appears in R ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “ran”: absent in R ($\mathrm{Count}_{\mathrm{clip}} = 0$)
- “after”: absent in R ($\mathrm{Count}_{\mathrm{clip}} = 0$)
- “the”: appears in R ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “cat”: appears in R ($\mathrm{Count}_{\mathrm{clip}} = 1$)

Total matching unigrams = 4. The total generated unigrams = 6. The modified unigram precision is:

$$p_1 = \frac{4}{6} \approx \mathbf{0.6667}. \tag{8.27}$$

Step 2: Bigram precision ($i = 2$). Extract all contiguous pairs of adjacent tokens (bigrams):

Reference bigrams R_2 = {"The dog", "dog chased", "chased the", "the cat"} ($|R_2| = 4$),
Generated bigrams G_2 = {"The dog", "dog ran", "ran after", "after the", "the cat"} ($|G_2| = 5$).

We count matching bigrams:

- "The dog": matches R_2 ($\text{Count}_{\text{clip}} = 1$)
- "dog ran": absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- "ran after": absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- "after the": absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- "the cat": matches R_2 ($\text{Count}_{\text{clip}} = 1$)

Total matching bigrams = 2. The total generated bigrams = 5. The modified bigram precision is:

$$p_2 = \frac{2}{5} = \mathbf{0.4000}. \tag{8.28}$$

Step 3: Geometric mean of precisions. Combining p_1 and p_2 via the geometric mean:

$$\begin{aligned} \text{GM} &= \exp\left(\frac{1}{2}\left(\ln(0.6667) + \ln(0.4000)\right)\right) = \sqrt{p_1 \times p_2} \\ &= \sqrt{\frac{4}{6} \times \frac{2}{5}} = \sqrt{\frac{8}{30}} = \sqrt{\frac{4}{15}} \approx \mathbf{0.5165}. \end{aligned} \tag{8.29}$$

Step 4: Brevity penalty calculation. We compare the candidate length $g = 6$ against the reference length $r = 5$:

$$g = 6 > r = 5 \implies \text{BP} = 1.0. \tag{8.30}$$

Because the candidate sequence is not shorter than the reference, no brevity penalty is incurred (BP = 1.0).

Step 5: Final BLEU-2 score. Multiplying the brevity penalty by the precision geometric mean yields:

$$\text{BLEU-2} = \text{BP} \times \text{GM} = 1.0 \times 0.5165 = \mathbf{0.5165} \quad \text{(or 51.65\%)}. \tag{8.31}$$

This calculation exposes the defining characteristics and underlying limitations of n-gram lexical matching. The candidate phrase "ran after" is semantically equivalent to the reference verb "chased". A human evaluator would judge "The dog ran after the cat" as a completely correct, fluent, and faithful translation. However, because BLEU enforces exact surface string equality, the bigrams "dog ran", "ran after", and "after the" were scored as total errors (0 matches). Consequently, p_2 dropped from an expected near-perfect score to 0.4000, driving the overall BLEU-2 score down to 0.5165.

Advantages and foundational limitations of BLEU

BLEU has served as the universal benchmark in machine translation and conditional generation for over two decades due to several compelling advantages:

- **Automated and cost-effective**: BLEU replaces slow, expensive human judges with an instantaneous, deterministic algorithmic script that costs nothing to evaluate.
- **Language-independent**: BLEU operates strictly on string matching over whitespace-separated tokens. It requires no morphological analyzers, syntactic dependency parsers, or semantic ontologies.
- **Empirical human correlation**: Across large corpora containing thousands of sentences, aggregate BLEU scores correlate reasonably well with human assessments of translation quality, effectively discriminating between high-performing and defective models.
- **Enforcement of phrasing structure**: By incorporating higher-order n-grams ($n \geq 2$), BLEU aggressively penalizes "word salad" outputs that contain relevant keywords in scrambled or ungrammatical orders.

Despite these operational strengths, BLEU exhibits a single, overarching foundational limitation: **it relies exclusively on exact surface word matching**. Every recorded failure mode and empirical deficiency of BLEU is simply a direct branch of this core constraint:

- **Synonymy and paraphrasing blindness**: Because BLEU matches character sequences rather than semantics, semantically identical phrases receive zero credit. As demonstrated in our worked example, replacing the reference verb "chased" with "ran after" destroys bigram precision despite preserving perfect semantic fidelity. Similarly, active-to-passive voice transformations and morphological inflections are penalized as complete errors.
- **Negation and factual insensitivity**: Surface character counting does not evaluate truth value. Inserting a single negation operator (e.g., generating "The system is not secure" when the reference is "The system is secure") preserves nearly all unigrams and bigrams, earning a stellar BLEU score while stating the exact opposite of the truth.
- **Brittle reference dependency**: The metric rewards adherence to the specific lexical choices of the human reference annotator. If the reference author employed uncommon vocabulary or idiosyncratic phrasing, a model generating natural, fluent, and correct prose is penalized simply because its surface tokens diverge from the human author.
- **Divergence in open-ended generation**: In tasks where generation is divergent and creative (such as dialogue, essay writing, or storytelling), there exists no unique reference sentence. Demanding exact n-gram alignment against a single reference penalizes valid linguistic creativity.

8.1.3 Recall-oriented understudy for gisting evaluation (ROUGE)

While BLEU was conceived for machine translation—where generating spurious words incurs severe penalties and precision is paramount—other natural language generation tasks require a fundamentally different optimization target. In text summarization, the primary failure mode is not verbosity, but *omission*: a candidate summary that fails to convey critical facts, characters, or conclusions is defective, even if every word it does produce matches the reference with 100% precision.

To address this challenge, Lin [69] introduced **ROUGE** (*Recall-Oriented Understudy for Gisting Evaluation*). In contrast to BLEU's precision-centric design, ROUGE evaluates generative performance primarily through **recall**, measuring the fraction of reference n-grams and structural subsequences successfully captured in the machine-generated hypothesis.

ROUGE-N: N-gram recall

Let $R = \{r_1, r_2, \ldots, r_m\}$ denote a set of reference summaries, and let G denote the model-generated candidate summary. For a chosen n-gram length $n \geq 1$, ROUGE-N computes the ratio of matched n-grams over the total n-grams appearing in the reference texts:

$$\text{ROUGE-}N = \frac{\sum_{r \in R} \sum_{g_n \in r} \text{Count}_{\text{match}}(g_n)}{\sum_{r \in R} \sum_{g_n \in r} \text{Count}(g_n)}, \tag{8.32}$$

where $\text{Count}_{\text{match}}(g_n)$ represents the maximum number of times the candidate n-gram g_n co-occurs in the candidate text G and the reference r.

- **ROUGE-1 ($n = 1$)**: Measures unigram recall, quantifying whether the candidate summary contains the key thematic entities, terminology, and content words present in the human gold standard.
- **ROUGE-2 ($n = 2$)**: Measures bigram recall, testing whether vocabulary is assembled with local syntactic cohesion and pairwise grammatical fluency.

ROUGE-L: Longest common subsequence

A limitation of fixed n-gram matching is that it requires choosing an arbitrary context window n. To capture flexible, sentence-level syntactic structure without imposing fixed n-gram boundaries, Lin [69] introduced **ROUGE-L**, based on the **Longest Common Subsequence** (LCS).

Given two sequences $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$, a sequence $Z = (z_1, \ldots, z_k)$ is a common subsequence if Z can be derived by deleting zero or more elements from both X and Y without altering the relative ordering of the remaining tokens. Unlike n-grams, the tokens in an LCS do not need to be contiguous.

Let R denote a reference sequence of length $r = |R|$, and let G denote a candidate sequence of length $g = |G|$. The LCS-based recall (R_{LCS}), precision (P_{LCS}), and F-measure (F_{LCS}) are defined as:

$$R_{\text{LCS}} = \frac{\text{LCS}(G, R)}{r}, \tag{8.33}$$

$$P_{\text{LCS}} = \frac{\text{LCS}(G, R)}{g}, \tag{8.34}$$

$$F_{\text{LCS}} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}, \tag{8.35}$$

where β controls the relative weighting between precision and recall. In summarization benchmarks, β is typically set to a large value (such as $\beta = 1.2$ or $\beta \to \infty$) to ensure that recall dominates the evaluation.

Structural advantage of ROUGE-L. Because LCS requires tokens to appear in the same relative order, ROUGE-L naturally penalizes scrambled sentences without requiring predefined n-gram orders. If a candidate retains the subject-verb-object progression of the reference with intermediate modifiers deleted, LCS assigns a high score, reflecting accurate gist extraction.

ROUGE-W: Weighted longest common subsequence

A structural blind spot of standard LCS is that it treats all common subsequences equally, regardless of whether their matching tokens are contiguous or widely separated across the text. For example, given the reference $R = (w_1, w_2, w_3, w_4, w_5)$, candidate $G_1 = (w_1, w_2, w_3, y_a, y_b)$ and candidate $G_2 = (w_1, y_a, w_2, y_b, w_3)$ both yield LCS = 3. However, G_1 possesses continuous phrasing, whereas G_2 is severely fragmented. **ROUGE-W** assigns higher weight to contiguous matching subsequences by applying a polynomial weight function $f(k) = k^2$ to consecutive matches.

Task-aligned metric selection: Precision vs. recall

The structural divergence between BLEU and ROUGE underscores a fundamental principle of generative evaluation: metric design must mirror the primary failure mode of the downstream task. In **machine translation**, the cardinal sin is introducing spurious, fabricated, or ungrounded words that distort the original semantic meaning of the source sentence; precision is paramount, and fluent syntax is strictly monitored by higher-order n-gram precision, making BLEU the natural operational choice. Conversely, in **text summarization**, the primary failure mode is information omission—generating a grammatically flawless snippet that fails to mention the main event, characters, or conclusions renders the summary useless; recall is paramount, making ROUGE the appropriate instrument. Nevertheless, both metrics remain fundamentally bound to surface lexical matching, remaining equally blind to semantic synonymy, paraphrasing, and negation inversion.

8.1.4 Metric for evaluation of translation with explicit ordering (METEOR)

To overcome the rigid surface-matching constraints of BLEU and ROUGE, Banerjee and Lavie [70] developed **METEOR** (*Metric for Evaluation of Translation with Explicit ORdering*). METEOR introduces two fundamental innovations: **multi-stage linguistic matching** (incorporating morphological stemming and semantic synonymy) and an **explicit fragmentation penalty** to measure structural coherence.

The four-stage linguistic alignment cascade

METEOR aligns candidate tokens against reference tokens by constructing a generalized bipartite matching where each token matches at most one token in the other sequence. The alignment is resolved through four successive linguistic modules:

1. **Exact match**: Pairs identical surface character strings (e.g., *chased* ↔ *chased*).
2. **Stem match**: Applies the Porter stemmer [70] to match words sharing identical morphological roots (e.g., *chased* ↔ *chasing*, *ran* ↔ *running*).
3. **Synonym match**: Queries WordNet synsets to match semantically equivalent terms (e.g., *chased* ↔ *pursued*, *cat* ↔ *feline*).
4. **Paraphrase match**: Matches multi-word phrase equivalents identified through statistical phrase tables.

Harmonic score and fragmentation penalty

Once the alignment is finalized, unigram precision (P) and unigram recall (R) are computed over the aligned matches. METEOR combines them via a parameterized harmonic mean F_{mean} that explicitly prioritizes recall:

$$F_{\text{mean}} = \frac{10\,P\,R}{R + 9\,P} = \frac{P\,R}{\alpha\,P + (1-\alpha)\,R}, \quad \text{with } \alpha = 0.9. \tag{8.36}$$

To penalize scrambled syntax, METEOR groups the matched tokens into the minimum number of adjacent contiguous blocks, called **chunks** (ch). If candidate and reference share identical word order, all matched words form a single contiguous chunk (ch = 1). If the words are randomly permuted, each word constitutes an isolated chunk (ch = m, where m is the total number of matched unigrams). The **fragmentation penalty** (Pen) is defined as:

$$\text{Pen} = \gamma \left(\frac{\text{ch}}{m}\right)^{\beta}, \quad \text{typically } \gamma = 0.5,\ \beta = 3. \tag{8.37}$$

The final METEOR score is computed as:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Pen}). \tag{8.38}$$

When syntax is preserved (ch = 1), the penalty Pen ≈ 0 and METEOR $\approx F_{\text{mean}}$. When syntax is scrambled, Pen $\to 0.5$, cutting the score in half.

Advantages and limitations of METEOR

By incorporating stemming and synonymy, METEOR achieves substantially higher correlation with human judgments than BLEU on sentence-level translation benchmarks. However, it incurs significant trade-offs:

- **Resource dependency**: It requires curated external resources (Porter stemmers, WordNet synsets, paraphrase tables), making it difficult to port to low-resource languages.
- **Computational cost**: Constructing multi-stage graph matchings is computationally slower than computing raw n-gram intersections.

8.1.5 Contextual embedding similarity (BERTScore)

Despite the additions of stemming and WordNet lookups in METEOR, all string-matching metrics share a common Achilles' heel: **context insensitivity**. Surface strings possess polysemy; a word like "bank" refers to a financial institution or a river shore depending entirely on surrounding syntax. A static lexicon cannot distinguish context-dependent meaning.

In 2020, Zhang, Kishore, Wu, Weinberger, and Artzi [71] introduced **BERTScore**, transitioning evaluation from discrete character matching into continuous contextual vector spaces. Rather than matching strings, BERTScore evaluates generated text by computing pairwise cosine similarities between deep contextual embeddings generated by models such as BERT [48], RoBERTa, or DeBERTa.

Mathematical formulation and greedy bipartite matching

Let $R = (x_1, x_2, \ldots, x_k)$ denote a reference sequence of k tokens, and let $G = (y_1, y_2, \ldots, y_m)$ denote a candidate generated sequence of m tokens. Both sequences are processed through a pre-trained contextual transformer, producing sequences of contextual vectors:

$$\mathbf{x}_i = \text{ContextualEncoder}(R)_i \in \mathbb{R}^d, \quad \text{for } i \in \{1, \ldots, k\}, \tag{8.39}$$

$$\mathbf{y}_j = \text{ContextualEncoder}(G)_j \in \mathbb{R}^d, \quad \text{for } j \in \{1, \ldots, m\}. \tag{8.40}$$

Every vector is normalized onto the unit hypersphere: $\|\mathbf{x}_i\|_2 = \|\mathbf{y}_j\|_2 = 1$. The pairwise similarity between reference token x_i and candidate token y_j is evaluated via the inner product (cosine similarity):

$$\mathbf{S}_{i,j} = \mathbf{x}_i^\top \mathbf{y}_j = \cos(\mathbf{x}_i, \mathbf{y}_j) \in [-1, 1]. \tag{8.41}$$

Instead of solving an expensive global assignment problem, BERTScore performs **greedy bipartite matching**: each token in one sequence is matched to the single most similar token in the other sequence.

- **Recall (R_{BERT})**: For each reference token x_i, find the maximum similarity across all generated tokens y_j, and average over reference length k:

$$R_{\text{BERT}} = \frac{1}{|R|} \sum_{x_i \in R} \max_{y_j \in G} \mathbf{x}_i^\top \mathbf{y}_j. \tag{8.42}$$

- **Precision (P_{BERT})**: For each generated token y_j, find the maximum similarity across all reference tokens x_i, and average over candidate length m:

$$P_{\text{BERT}} = \frac{1}{|G|} \sum_{y_j \in G} \max_{x_i \in R} \mathbf{x}_i^\top \mathbf{y}_j. \tag{8.43}$$

- **F_1-measure (F_{BERT})**: Synthesizes precision and recall via the harmonic mean:

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}. \tag{8.44}$$

Worked numerical calculation from lecture

To illustrate how BERTScore resolves the semantic failure modes of BLEU, we reproduce the exact numerical evaluation delivered by Prof. Giobergia during lecture.

Consider the identical reference and candidate pair analyzed in Section 8.1.2:

$$R : \text{“the dog chased the cat”} \quad (|R| = 5 \text{ tokens}),$$
$$G : \text{“the dog ran after the cat”} \quad (|G| = 6 \text{ tokens}).$$

Under BLEU-2, this pair suffered a substantial drop to 0.5165 because “ran after” did not match “chased” at the surface level. Table 8.3 displays the complete 6×5 pairwise cosine similarity matrix $\mathbf{S}$ computed from contextual BERT embeddings.

Table 8.3: Pairwise cosine similarity matrix $\mathbf{S}_{j,i} = \mathbf{y}_j^\top \mathbf{x}_i$ between Generated tokens (rows G) and Reference tokens (columns R) from lecture slides 17–19. Row maxima indicate candidate precision matches; column maxima indicate reference recall matches.

Candidate $G \setminus$ **Ref** R	**the** (x_1)	**dog** (x_2)	**chased** (x_3)	**the** (x_4)	**cat** (x_5)	**Row Max** ($\max_i \mathbf{S}_{j,i}$)
the (y_1)	**0.9967**	0.4120	0.3815	0.9921	0.3950	**0.9967** (matches x_1)
dog (y_2)	0.4110	**0.9968**	0.4520	0.4080	0.5820	**0.9968** (matches x_2)
ran (y_3)	0.3250	0.4610	**0.9351**	0.3180	0.4410	**0.9351** (matches x_3)
after (y_4)	0.3640	0.4100	**0.8868**	0.3720	0.3910	**0.8868** (matches x_3)
the (y_5)	0.9921	0.4080	0.3790	**0.9921**	0.3980	**0.9921** (matches x_4)
cat (y_6)	0.3920	0.5810	0.4430	0.3980	**0.9960**	**0.9960** (matches x_5)
Col Max ($\max_j \mathbf{S}_{j,i}$)	**0.9967**	**0.9968**	**0.9351**	**0.9921**	**0.9960**	—

Step 1: Precision calculation (P_{BERT}). We evaluate the maximum similarity across columns for each generated token row:

$$\begin{aligned}
\text{Row 1 (the)} &\to 0.9967 \\
\text{Row 2 (dog)} &\to 0.9968 \\
\text{Row 3 (ran)} &\to 0.9351 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 4 (after)} &\to 0.8868 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 5 (the)} &\to 0.9921 \\
\text{Row 6 (cat)} &\to 0.9960
\end{aligned}$$

$$\sum_{j=1}^{6} \max_i \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.8868 + 0.9921 + 0.9960 = 5.8035. \tag{8.45}$$

Averaging over the $|G| = 6$ candidate tokens yields the precision reported in lecture:

$$P_{\text{BERT}} = \frac{5.8035}{6} \approx \mathbf{0.96725}. \tag{8.46}$$

Step 2: Recall calculation (R_{BERT}). We evaluate the maximum similarity across rows for each reference token column:

$$\begin{aligned}
\text{Col 1 (the)} &\rightarrow 0.9967 \\
\text{Col 2 (dog)} &\rightarrow 0.9968 \\
\text{Col 3 (chased)} &\rightarrow 0.9351 \quad \text{(best matched by candidate “ran”)} \\
\text{Col 4 (the)} &\rightarrow 0.9921 \\
\text{Col 5 (cat)} &\rightarrow 0.9960
\end{aligned}$$

$$\sum_{i=1}^{5} \max_{j} \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.9921 + 0.9960 = 4.9167. \tag{8.47}$$

Averaging over the $|R| = 5$ reference tokens yields:

$$R_{\text{BERT}} = \frac{4.9167}{5} \approx \mathbf{0.98334}. \tag{8.48}$$

Step 3: Harmonic F_1 score.

$$F_{\text{BERT}} = 2 \cdot \frac{0.96725 \times 0.98334}{0.96725 + 0.98334} = \frac{1.90220}{1.95059} \approx \mathbf{0.97519}. \tag{8.49}$$

Strengths and opaque semantic embedding bottlenecks

Comparing BLEU-2 (51.65%) against BERTScore (97.52%) demonstrates the expressive power of contextual representations:

- **Paraphrase and synonym recognition**: The verbal predicate “ran after” matches “chased” with high similarity (cos $\approx$ 0.9351 and 0.8868). The model is not penalized for natural vocabulary variation.
- **Contextual disambiguation**: Because embeddings are produced by multi-head self-attention layers, each token vector encodes sentence syntax.

However, as emphasized by Prof. Giobergia, BERTScore introduces serious methodological challenges:

1. **Opacity and lack of interpretability (“BERT says so”)**: If BLEU produces a low score, an engineer can inspect the exact missing n-grams. If BERTScore produces a score of 0.84, there is no symbolic, explainable justification: the score exists purely because a multi-billion-parameter neural network generated latent vectors with a specific cosine angle.
2. **Computational overhead**: Evaluating BLEU requires simple hash-map lookups taking microseconds. BERTScore requires running two complete forward passes through a transformer encoder per sentence pair, making it computationally heavy for million-sample evaluations.
3. **Inherited model bias**: Any failure mode, linguistic blind spot, or domain shift present in the evaluator model directly corrupts the evaluation metric.

8.1.6 Exact match and information retrieval ranking metrics

In tasks where the target output space is strictly bounded, continuous overlap scores are neither necessary nor desirable. When evaluating closed-domain question answering, factual entity retrieval, or programmatic code execution, a prediction is either entirely correct or unusable.

Exact match (EM)

The **Exact Match** (EM) metric evaluates binary equality between the generated hypothesis string G and the gold-standard reference string R:

$$\mathrm{EM}(G, R) = \begin{cases} 1 & \text{if Normalize}(G) = \text{Normalize}(R), \\ 0 & \text{otherwise.} \end{cases} \tag{8.50}$$

To prevent trivial punctuation or casing discrepancies from registering as false errors, the Normalize$(\cdot)$ function standardizes strings by:

- Converting all text to lowercase.
- Stripping whitespace, line breaks, and punctuation marks.
- Removing leading/trailing stop words or English articles (*a*, *an*, *the*).

Exact Match serves as the standard evaluation standard on extractive QA benchmarks (such as SQuAD [72]) and mathematical datasets (such as GSM8K [73]).

Information retrieval and probability ranking metrics

When a language model is evaluated on multiple-choice questions, cloze passage completion, or entity ranking, it produces a probability distribution over a set of candidates $C = \{c_1, c_2, \ldots, c_K\}$. We sort the candidates by descending model probability:

$$P(c_{(1)}) \geq P(c_{(2)}) \geq \cdots \geq P(c_{(K)}).$$

Let $\mathrm{rank}_i \in \{1, 2, \ldots, K\}$ denote the position at which the ground-truth target token appears. Several canonical ranking metrics quantify performance:

- **Mean Reciprocal Rank (MRR)**: Computes the average of the reciprocal ranks of the correct answer across a query test set Q:

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\mathrm{rank}_q}. \tag{8.51}$$

  If the target item is consistently ranked first (rank = 1), MRR = 1.0. If it is ranked second, MRR = 0.5.

- **Precision at k (P@k) and Recall at k (R@k)**: Measures whether the correct answer appears within the top-k highest-probability candidates:

$$\mathrm{Hit@}k = \mathbb{I}(\mathrm{rank} \leq k). \tag{8.52}$$

- **Normalized Discounted Cumulative Gain (**NDCG@k**)**: When answers have varying degrees of graded relevance $rel_i \in \mathbb{R}_{\geq 0}$, NDCG@$k$ penalizes relevant items that appear lower in the ranked list using a logarithmic position discount:

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \tag{8.53}$$

  where IDCG@k represents the Ideal DCG obtained under a perfect sorting of relevance values.

8.1.7 Task-specific classification metrics

Many modern language model evaluations frame generative tasks as discrete classification problems by prompting the model to output a single categorical label (such as positive/negative sentiment, or entailment/contradiction in inference tasks). In these configurations, classical machine learning evaluation metrics are directly applicable:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \tag{8.54}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{8.55}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}, \tag{8.56}$$

where TP, TN, FP, FN denote True Positives, True Negatives, False Positives, and False Negatives, respectively.

Multi-class aggregation. For multi-class tasks (such as 3-way natural language inference), scores are aggregated using either **Macro-F_1** (unweighted average across class F_1 scores, which prevents dominant majority classes from concealing minority class failures) or **Micro-F_1** (global pooling of instances, identical to overall accuracy).

8.1.8 Human qualitative evaluation and model-based judges

Despite the sophistication of n-gram matching, LCS alignment, and contextual embeddings, automated metrics consistently fail to evaluate high-level cognitive properties of natural language:

- **Global coherence and argumentative structure**: Does an essay or multi-paragraph response follow a logical sequence of reasoning, or does it wander between disconnected points?
- **Factuality and hallucination detection**: A generated passage can exhibit high BLEU, ROUGE, and BERTScore overlap against a reference while introducing an unmentioned, completely false fact.
- **Creativity, tone, and stylistic appropriateness**: Measuring humor, empathy, or instruction-following fidelity cannot be reduced to cosine similarity or token overlap.

Consequently, human evaluation remains the gold standard for assessing text quality.

Human evaluation: Absolute scales vs. pairwise comparisons

Two primary paradigms govern human evaluation in research and industry:

1. **Absolute Likert rating scales**: Annotators are presented with a generated response and asked to assign a score on a discrete 1-to-5 scale across specific criteria (e.g., fluency, coherence, relevance, factual accuracy).
2. **Pairwise comparative evaluation (A/B testing)**: Annotators are presented with a prompt and two anonymized, randomized candidate outputs (A and B) produced by competing models, and asked to answer: *"Which response is better, or are they tied?"*

Cognitive failure mode of absolute scoring

As emphasized in lecture, humans are notoriously poor at assigning absolute grades. If ten human judges are asked to grade an essay on a 1-to-5 scale, their scores exhibit severe variance due to:

- **Calibration drift**: Annotator standards shift across time due to fatigue, mood, or exposure to previous high- or low-quality samples.
- **Subjective anchor disparity**: One annotator's "4" may correspond to another annotator's "2".

In contrast, humans are remarkably consistent at **relative pairwise comparisons**: deciding whether response A is more helpful, coherent, or factually sound than response B is a significantly simpler cognitive task. Pairwise preferences are converted into global scalar rankings using the **Elo rating system** [74] (the foundation of modern public benchmarks like the LMSYS Chatbot Arena).

Scalability bottleneck and LLM-as-a-judge

While human evaluation provides the highest qualitative fidelity, it is fundamentally bottlenecked by resource constraints:

- **Extreme financial cost**: Conducting statistically significant human trials across thousands of prompts requires hundreds of thousands of dollars in crowd-sourcing compensation.
- **High latency**: Collecting human annotations takes days or weeks, making it impossible to integrate into real-time CI/CD model training pipelines.
- **Non-reproducibility**: Human pool variance makes exact experimental replication difficult across research labs.

These constraints have catalyzed the widespread adoption of **LLM-as-a-judge** [74]. By supplying a state-of-the-art foundation model (such as GPT-4) with detailed evaluation rubrics, system instructions, and few-shot calibration examples, automated systems can evaluate model outputs pairwise with high correlation ($\rho > 0.8$) to human expert preferences, operating at fraction-of-a-cent costs and second-level latency.

While metrics provide the mathematical scoring instruments, they cannot exist in an operational vacuum. A metric like BLEU is meaningless without the task of Machine Translation; Exact Match is meaningless without Question Answering; Accuracy is meaningless without Natural Language Inference. To understand how foundational models are developed and validated, we must examine the formal problem formulations across which these metrics are deployed.

8.2 NLP Tasks and Evaluation Taxonomy

Evaluating foundation models requires probing a wide spectrum of linguistic, semantic, and reasoning capabilities. In early natural language processing literature, models were trained and evaluated on isolated, specialized tasks using task-specific neural heads. In modern foundation model engineering, diverse tasks are unified under the shared autoregressive interface: every problem is formulated as a text prompt conditioning the generation of target tokens. However, presenting downstream tasks as an arbitrary collection of disparate datasets conceals the underlying principles of model assessment. To establish a rigorous foundation, tasks must be systematically classified along two orthogonal taxonomic axes: operational output structure and cognitive reasoning depth.

8.2.1 A Two-Dimensional Taxonomy of Language Model Tasks

Downstream evaluation tasks can be systematically partitioned along two complementary axes that define the operational mechanics of the evaluation protocol and the intrinsic cognitive complexity required to solve the task.

Axis 1: Operational output structure and target space

The first taxonomic axis classifies tasks according to the mathematical structure of the model's output space:

1. **Discriminative and categorical choice**: The output space is constrained to a finite, discrete set of classes $\mathcal{Y} = \{y_1, y_2, \ldots, y_K\}$ or a discrete pool of candidate text completions $C = \{c_1, \ldots, c_K\}$. In closed-ended classification (e.g., sentiment analysis or grammatical acceptability), the model maps input $\mathbf{x}$ to a categorical distribution $P(y \mid \mathbf{x})$. In discriminative choice mode (e.g., story cloze or multiple-choice QA), the model computes the length-normalized conditional log-likelihood across competing sequence candidates:

$$\hat{c} = \arg\max_{c \in C} \frac{1}{|c|} \sum_{i=1}^{|c|} \ln P(w_i^{(c)} \mid \text{Context}, w_{<i}^{(c)}). \tag{8.57}$$

   This paradigm is scored using standard classification metrics (Accuracy, Precision, Recall, Macro-F_1, and Matthews Correlation Coefficient).

2. **Extractive span identification**: The output space is constrained to a contiguous substring (span) existing verbatim within the conditioning prompt context $\mathbf{x} = (x_1, \ldots, x_N)$. The model predicts the starting index $s_{\text{start}} \in \{1, \ldots, N\}$ and ending index $s_{\text{end}} \geq s_{\text{start}}$ defining the target snippet $\mathbf{x}_{s_{\text{start}}:s_{\text{end}}}$. If the question is unanswerable from the context, the model must output an explicit abstention token ⟨no-answer⟩. Evaluated primarily via Exact Match (EM) and token-level overlap F_1.

3. **Unbounded autoregressive sequence generation**: The output space is the open-ended set of all possible variable-length sequences $\mathbf{y} = (y_1, y_2, \ldots, y_T)$ drawn from the vocabulary $\mathcal{V}^*$. The model samples tokens iteratively according to its learned conditional distribution $P(y_t \mid \mathbf{x}, y_{<t})$ until an end-of-sequence delimiter ⟨eos⟩ is emitted. This operational mode governs machine translation, abstractive summarization, open-domain dialogue, and formal mathematical derivations. Evaluated via reference-based overlap metrics (BLEU, ROUGE, METEOR), contextual vector metrics (BERTScore), and model-based judges.

Axis 2: The cognitive capability and reasoning hierarchy

The second taxonomic axis organizes evaluation tasks into an ascending four-tier cognitive hierarchy. Rather than treating natural language understanding as a monolithic capability, this hierarchy reflects the progressive computational demands placed upon the model's latent representations:

1. **Tier 1: Surface Syntax, Lexical Semantics, and Document Classification**: Probes foundational grammatical competence, phrase structure, and coarse document-level semantic categorization without requiring multi-step deduction.
2. **Tier 2: Autoregressive Sequence Generation and Narrative Discourse**: Probes local transition fluency, long-range discourse tracking across multi-sentence paragraphs, and conditioned sequence-to-sequence rewriting.
3. **Tier 3: Information Retrieval, Parametric Memory, and Reading Comprehension**: Probes factual knowledge retention encoded within pre-trained model weights (closed-book memory) versus contextual comprehension of dynamically injected evidence documents (open-book reading).
4. **Tier 4: Deep Semantic, Multi-Step, and Symbolic Reasoning**: Probes formal logical entailment, physical and causal commonsense intuition, multi-step algorithmic calculation, and programmatic probe resilience against web memorization.

We now systematically analyze the core evaluation tasks ordered according to this four-tier cognitive hierarchy.

8.2.2 Tier 1: Surface Syntax, Lexical Semantics, and Document Classification

The foundation of language model evaluation begins at the level of syntactic well-formedness and surface lexical categorization. Before a model can be trusted to execute complex logical deductions or follow nuanced user instructions, it must demonstrate mastery over the grammatical morphology and lexical semantics of natural language.

Grammatical acceptability and linguistic competence (CoLA)

To assess whether a neural model has internalized syntax independent of semantic plausibility, evaluation protocols draw upon theoretical linguistics. In his seminal work *Syntactic Structures* (1957), Noam Chomsky formulated the famous sentence *"Colorless green ideas sleep furiously"* to demonstrate that a sentence can be completely devoid of meaningful semantics while adhering perfectly to English grammatical syntax. Conversely, a sentence may convey transparent meaning while violating fundamental grammatical rules (e.g., *"*Sleep green ideas colorless furiously"*).

The premier benchmark for isolating syntactic acceptability is the **Corpus of Linguistic Acceptability (CoLA)** [75]. CoLA consists of 10,657 English sentences extracted from published linguistics textbooks and articles, covering a comprehensive catalog of morphological, syntactic, and semantic phenomena (including auxiliary inversion, island constraints, verb subcategorization, and wh-movement). Each sentence is annotated with a binary label indicating whether it is grammatically acceptable ($y = 1$) or unacceptable ($y = 0$).

Because grammatical acceptability datasets exhibit substantial class imbalance (acceptable sentences naturally outnumber unacceptable instances), evaluating raw accuracy yields misleading results. Instead, CoLA is formally evaluated using the **Matthews Correlation Coefficient (MCC)**:

$$\mathrm{MCC} = \frac{\mathrm{TP} \cdot \mathrm{TN} - \mathrm{FP} \cdot \mathrm{FN}}{\sqrt{(\mathrm{TP} + \mathrm{FP})(\mathrm{TP} + \mathrm{FN})(\mathrm{TN} + \mathrm{FP})(\mathrm{TN} + \mathrm{FN})}} \in [-1, +1], \tag{8.58}$$

where +1.0 represents perfect grammatical discrimination, 0.0 corresponds to random chance, and −1.0 denotes complete inverse prediction.

Task Example: Grammatical Acceptability (CoLA)

Acceptable Sentence ($y = 1$): *"The book that I read yesterday was fascinating."*
Unacceptable Sentence ($y = 0$): *"*The book that I read it yesterday was fascinating."*
Linguistic Analysis: The unacceptable sentence contains an ungrammatical resumptive pronoun ("it") in an object relative clause, which violates standard English syntactic constraints. While human listeners easily decipher the intended meaning, an evaluated language model must assign strictly higher likelihood to the syntactically valid parse tree.

Text and sentiment classification

Text classification evaluates a model's capacity to categorize unstructured documents into discrete semantic classes based on overall content, sentiment polarity, or author intent. In classical machine learning, this was achieved by training specialized classifiers (such as support vector machines or linear networks over bag-of-words vectors). In modern foundation model paradigms, classification is conducted either by fine-tuning a linear classification head atop an encoder's pooling representation (e.g., the $\mathbf{h}_{[\text{CLS}]}$ token in BERT) or by prompting an autoregressive decoder to generate the target label token directly.

Canonical classification benchmarks span two primary modalities:

- **Sentiment analysis**:
  - **SST-2 (Stanford Sentiment Treebank)** [76]: A dataset of 9,613 movie review sentences extracted from Rotten Tomatoes, equipped with fine-grained syntactic parse trees. SST-2 evaluates binary sentiment classification (positive vs. negative), frequently testing concessive and adversarial clauses.
  - **IMDb Movie Reviews** [77]: 50,000 long-form movie reviews partitioned equally into 25,000 training and 25,000 testing documents, labeled with binary polarity. Unlike short sentences, IMDb requires aggregating sentiment signals across hundreds of words of meandering prose.
  - **Yelp Review Dataset**: Approximately 7 million commercial customer reviews annotated on a 1-to-5 star rating scale, testing graded ordinal regression and multi-class classification.
- **Topical news classification**:
  - **AG News**: 120,000 news articles partitioned uniformly across four topical categories: *World*, *Sports*, *Business*, and *Sci/Tech*.
  - **20 Newsgroups**: A classic collection of approximately 20,000 Usenet discussion forum postings distributed across 20 distinct technical, religious, and political newsgroups.

Performance on text classification is measured using Accuracy and Macro-F_1 (to guarantee equitable error penalization across minority classes).

Task Example: Sentiment Classification (SST-2)

Input Review: *"It's not that the movie is poorly made, but it is utterly devoid of soul or charm."*
Ground-Truth Label: **Negative** ($y = 0$).
Linguistic Analysis: The sentence features a concessive contrastive structure. Surface keyword spotters are easily tricked by local negated phrases ("not poorly made"). A robust language model must resolve the syntactic coordination introduced by "but" and recognize that the overarching predicate ("utterly devoid of soul or charm") dictates the negative global polarity.

8.2.3 Tier 2: Autoregressive Sequence Generation and Narrative Discourse

Moving beyond categorical classification, Tier 2 evaluates the language model's primary functional design: autoregressive sequence generation. This tier probes whether a model can generate syntactically coherent, contextually grounded text across both narrative continuations and constrained conditional transformations.

Autoregressive text completion and narrative cloze tasks

The purest evaluation of an autoregressive model evaluates its probability assignment over subsequent text. Given an initial conditioning passage $W_{<N} = (w_1, \ldots, w_{N-1})$, the model must assign high probability to plausible continuations while rejecting absurd or ungrammatical endings. This can be evaluated in open-ended generative mode (measuring perplexity) or discriminative choice mode via Equation (8.57).

Early cloze benchmarks presented short sentences where the missing word could be easily guessed from immediate local context (e.g., predicting "bark" following "The dog began to"). Such tasks failed to evaluate whether models maintained long-range memory or simply acted as localized n-gram smoothers. To overcome this limitation, researchers developed specialized discourse-level benchmarks:

- **LAMBADA (Language Modeling Broadened to Account for Discourse-Level Analysis)** [78]: A corpus of narrative passages specifically curated such that human readers can guess the final target word accurately when reading the full paragraph, but cannot guess it if exposed only to the final target sentence. LAMBADA directly tests whether a model retains discourse context across dozens of tokens without succumbing to recency bias or catastrophic forgetting. Evaluated via Accuracy (Exact Match on the target token) and Perplexity.
- **ROCStories and StoryCloze** [79]: A benchmark of 5-sentence commonsense narratives capturing everyday human experiences. The StoryCloze test presents the first four sentences as narrative context and provides two candidate concluding sentences: one logical narrative resolution and one contradictory or nonsensical ending.
- **HellaSwag** [80]: A continuation benchmark constructed to prevent statistical shortcut learning. Zellers et al. demonstrated that models easily scored over 80% on earlier story completion datasets by exploiting shallow lexical artifacts (such as matching length or common verbs). HellaSwag introduced **Adversarial Filtering (AF)**, iteratively employing generator models to synthesize distractor endings that easily deceive neural discriminators while remaining trivially distinguishable to human evaluators.

Task Example: Long-Range Narrative Text Completion (LAMBADA)

Context Passage: *"Yes, I thought I was going to lose her. She was unconscious for days, and the doctors were grave. I stayed beside her bed every night until she finally opened her eyes. I held her hand and whispered, 'Alice, I thought you were going to...' "*
Target Continuation: **die** (Single-word target).
Discourse Requirement: Evaluating the final sentence in isolation ("Alice, I thought you were going to...") provides no constraint on the missing word; valid grammatical completions include "leave", "cry", "faint", or "sleep". Only by conditioning on the preceding 60 tokens ("unconscious for days", "doctors were grave") can the model deduce that "die" is the intended narrative resolution.

Conditional sequence generation: Translation and summarization

Conditional sequence generation requires mapping an input text into an output sequence with different surface vocabulary or compressed length, while preserving invariant semantic content.

- **Machine translation**: Maps a source sequence $\mathbf{x}$ in language $\mathcal{L}_S$ to a target sequence $\mathbf{y}$ in language $\mathcal{L}_T$. The premier competitive standard is established by the annual **WMT (Workshop on Machine Translation)** benchmarks, which provide curated parallel training and test corpora across high-resource pairs (English-German, English-French) and low-resource pairs. Evaluation relies on BLEU, METEOR, and neural semantic similarity measures.
- **Text summarization**: Compresses an input document $\mathbf{x}$ into a concise summary $\mathbf{y}$ ($|\mathbf{y}| \ll |\mathbf{x}|$). As highlighted in lecture, summarization is divided into:
  - **Extractive summarization**: Identifying and extracting the most salient verbatim sentences directly from the source document.
  - **Abstractive summarization**: Synthesizing key information and generating novel, fluent sentences that may contain words absent from the source text.

  Canonical summarization datasets include **CNN/Daily Mail** [69] (over 300,000 news articles paired with human-written bullet-point story highlights) and **PubMed Diabetes** (long-form biomedical research papers paired with technical abstracts). Evaluation relies on ROUGE-1, ROUGE-2, and ROUGE-L.

Task Example: Conditional Sequence Generation (Translation & Summarization)

Machine Translation (WMT English → French):

- *Source*: "Pre-trained language models capture deep statistical regularities of natural language."
- *Reference*: "Les modèles de langue pré-entraînés capturent des régularités statistiques profondes du langage naturel."

Abstractive Summarization (CNN/Daily Mail):

- *Lead Passage*: "A rare snowstorm struck the Mediterranean coastline on Monday, disrupting morning flights at regional airports and dusting historic olive groves with over four inches of snow. Meteorological authorities reported the heaviest coastal accumulation in over thirty years..."
- *Target Highlight Summary*: "– Rare snowstorm strikes Mediterranean coast with four inches of snow.
  – Heaviest coastal snowfall recorded in over 30 years disrupts regional transport."

8.2.4 Tier 3: Information Retrieval, Parametric Memory, and Reading Comprehension

Tier 3 evaluates the model's capacity to store, retrieve, and synthesize factual world knowledge. Rather than assessing whether text sounds fluent, Tier 3 benchmarks test whether the factual claims made by the model are true.

Question answering architectures: Open-book versus closed-book

Question Answering (QA) problem formulations are partitioned along two foundational axes: **knowledge access** and **answer formulation format**.

Axis 1: Access to knowledge (Closed-book vs. open-book).

- **Closed-Book QA**: The model receives only the question prompt in its context window (x = "In what year was the University of Turin founded?"). The model must retrieve the factual answer entirely from its internal **parametric memory**—the billions of connection weights optimized during pre-training. This paradigm directly measures what the model "knows" without external assistance.

- **Open-Book QA**: The model is supplied with external reference documents in its context window alongside the question prompt. In modern production systems, this is realized via **Retrieval-Augmented Generation (RAG)**, where an external retriever queries a dense vector database (e.g., Wikipedia) and injects the top-k relevant passages into the prompt. This evaluates reading comprehension, contextual reasoning, and synthesis fidelity.

Axis 2: Answer formulation format.

- **Extractive QA**: The answer is guaranteed to exist as a contiguous text span within the provided passage. The model predicts boundary pointer indices $[s_{\text{start}}, s_{\text{end}}]$.
- **Abstractive QA**: The model generates a free-form natural language response, synthesizing facts across multiple paragraphs.
- **Multiple-Choice QA**: The model selects among discrete candidate options (e.g., A, B, C, D).

Canonical QA benchmarks

- **SQuAD (Stanford Question Answering Dataset)** [72]: Contains over 100,000 extractive question-answer pairs curated by crowd-workers over Wikipedia articles. SQuAD 2.0 [81] introduced over 50,000 adversarial *unanswerable questions* that look syntactically plausible but cannot be answered from the provided passage, requiring models to emit an abstention token ⟨no-answer⟩. Evaluated via Exact Match (EM) and token-level F_1.
- **TriviaQA** [82]: Over 650,000 question-answer pairs gathered from trivia enthusiasts, paired with independently retrieved web documents and Wikipedia articles, introducing realistic noise and distant supervision.
- **Natural Questions (NQ)** [83]: A dataset released by Google containing real, anonymized user search queries entered into the Google search engine. Annotators paired queries with full Wikipedia pages, annotating both a "long answer" (an entire paragraph containing the explanation) and a concise "short answer" (a specific entity or numerical span).
- **WebQuestions** [84]: 6,642 question-answer pairs generated by querying the Google Suggest API and annotated over Freebase knowledge graph entities via Amazon Mechanical Turk.

Task Example: Extractive Question Answering & Abstention (SQuAD 2.0)

Context Passage: *"The Normans were the people who in the 10th and 11th centuries gave their name to Normandy, a region in France. They were descended from Norse raiders and pirates from Denmark, Iceland and Norway who, under their leader Rollo, agreed to swear fealty to King Charles III of West Francia in 911..."*
Answerable Question: *"Who was the leader of the Norse raiders that swore fealty in 911?"*
$\Longrightarrow$ **Extractive Answer Span**: **"Rollo"** (EM = 1.0, F_1 = 1.0).
Adversarial Unanswerable Question: *"What was the name of Rollo's flagship vessel?"*
$\Longrightarrow$ **Target Ground Truth**: ⟨**no-answer**⟩ (Abstention).
Evaluation Insight: Models unable to verify negative evidence will hallucinate an arbitrary noun phrase from the context, incurring an exact match penalty.

8.2.5 Tier 4: Deep Semantic, Multi-Step, and Symbolic Reasoning

At the pinnacle of the cognitive hierarchy lies deep semantic and symbolic reasoning. Tasks in Tier 4 require the model to perform formal deductive inference, navigate physical and social commonsense constraints, execute multi-step mathematical calculations, and solve synthetic algorithmic probes designed to resist pre-training web memorization.

Natural language inference (NLI)

Natural Language Inference (NLI, also known as recognizing textual entailment) is the foundational benchmark for formal semantic reasoning. Given two sentences—a **Premise** (P) and a **Hypothesis** (H)—the model must determine whether the hypothesis is logically entailed, contradicted, or undetermined:

$$\text{Relationship}(P, H) \in \{\textbf{Entailment}, \textbf{Contradiction}, \textbf{Neutral}\}. \tag{8.59}$$

- **Entailment**: If P is true, H is necessarily true (P = "A dog is chasing a ball", H = "An animal is moving").
- **Contradiction**: If P is true, H is necessarily false (P = "A man is asleep in bed", H = "A man is running a marathon").
- **Neutral**: H might or might not be true; P provides insufficient evidence (P = "Two people are eating lunch" "The lunch was prepared at home").

Canonical benchmarks include **SNLI (Stanford NLI)** [85] (570,000 crowd-sourced sentence pairs derived from Flickr image captions) and **MultiNLI (Multi-Genre NLI)** [86] (433,000 sentence pairs spanning 10 diverse spoken and written genres).

Task Example: Natural Language Inference (MultiNLI)

Premise: *"At the base of the canyon, a lone hiker rested beside a rapid stream."*

- **Hypothesis 1 (Entailment)**: *"An outdoor enthusiast was taking a break near water." [Supported]*
- **Hypothesis 2 (Contradiction)**: *"A crowded tour bus drove down the paved city street." [Impossible]*
- **Hypothesis 3 (Neutral)**: *"The hiker had been walking for six hours without food." [Undetermined]*

Semantic Insight: Entailment requires hypernym abstraction ("hiker" → "outdoor enthusiast", "rapid stream" → "water"); the neutral hypothesis introduces plausible narrative details that cannot be formally deduced from the premise.

Commonsense and causal reasoning

Commonsense reasoning encompasses the vast repository of unstated physical, social, temporal, and causal knowledge that humans use effortlessly every day. Because foundation models have no physical embodiment or direct sensory experience of the real world, probing commonsense reasoning exposes whether models have formed genuine world models or merely superficial word associations.

Pronoun ambiguity resolution: The Winograd schema challenge. Pronoun coreference resolution tests whether a model understands physical and situational constraints. The foundational benchmark is the **Winograd Schema Challenge (WSC)** [87], based on paired sentences called *twin sentences* that differ by only one or two words:

Task Example: Pronoun Coreference Ambiguity (WSC Twin Sentences)

Sentence 1: *The trophy didn't fit in the suitcase because **it** was too **big**.* ⟹ What was too big? ⟹ **The trophy**.
Sentence 2: *The trophy didn't fit in the suitcase because **it** was too **small**.* ⟹ What was too small? ⟹ **The suitcase**.
Physical Reasoning: Syntactic dependency parsing alone cannot resolve "it"; the model must understand that containers are constrained by spatial volume and that an item must be smaller than a container to fit inside it.

WinoGrande [88] scaled this paradigm to 44,000 adversarially filtered problems, systematically eliminating lexical selection biases to expose models relying on associative correlations rather than physical reasoning.

Causal and if-then reasoning: COPA and ATOMIC.

- **COPA (Choice of Plausible Alternatives)** [89]: 1,000 problems composed of a premise and two alternative hypotheses. The prompt explicitly queries either the **cause** or the **effect**.
- **ATOMIC (An Atlas of Machine Commonsense for If-Then Reasoning)** [90]: An atlas containing 877,000 everyday if-then relations organized into 9 semantic dimensions spanning agent intent (*xIntent*), emotional reaction (*xReact*), and consequences for others (*oEffect*).

Task Example: Causal Reasoning (COPA)

Premise: *The man broke his toe.*
Question: What was the **cause** of this event?
Alternative 1: *He dropped a hammer on his foot.* *[Correct]*
Alternative 2: *He tied his shoelaces too tightly.*
Causal Insight: The model must evaluate the mechanical plausibility of physical impacts causing bone fractures versus minor discomfort from tight shoelaces.

Mathematical and formal reasoning

Autoregressive models trained purely on next-token prediction frequently struggle with rigorous multi-step deductive calculation. While models memorize common arithmetic facts (e.g., $2+2=4$), they easily lose track of intermediate states when executing multi-step word problems.

To benchmark quantitative reasoning, researchers established two premier datasets:

- **GSM8K (Grade School Math 8K)** [73]: 8,500 linguistically diverse grade-school math word problems requiring 2 to 8 steps of arithmetic operations. Each problem includes a detailed chain-of-thought solution with intermediate calculation annotations (delimited by «...»). Foundation models are prompted using few-shot Chain-of-Thought (CoT) [91], and evaluated on **Exact Match** of the final numerical answer.
- **MATH Dataset** [92]: 12,500 challenging competition mathematics problems gathered from the AMC 10, AMC 12, and AIME, formatted entirely in LaTeX. Spans seven mathematical subfields: *Prealgebra*, *Algebra*, *Number Theory*, *Counting & Probability*, *Geometry*, *Intermediate Algebra*, and *Precalculus*, stratified across 5 difficulty tiers.

Task Example: Multi-Step Mathematical Reasoning (GSM8K)

Problem: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins with another four. She sells the remainder at the farmers' market daily for $2 per egg. How much in dollars does she make every day at the farmers' market?
Annotated Chain-of-Thought Solution:

> Janet consumes $3+4=7$ eggs each day. «3+4=7»
> She has $16-7=9$ eggs left to sell at the market. «16-7=9»
> Selling 9 eggs at $2 each yields $9\times 2=18$ dollars. «9*2=18»
> #### 18

Evaluation Metric: **Exact Match** on final integer (18).
Reasoning Insight: The model must execute linear intermediate deduction. The «...» calculator delimiters illustrate tool-calling: delegating deterministic arithmetic to an execution engine prevents accumulated calculation drift.

Synthetic diagnostic probes: Isolating capabilities from memorization

A persistent challenge in evaluating foundation models on standard public datasets is determining whether a model solved a problem through general reasoning or simply reproduced text memorized during web-scale pre-training.

To address this, researchers design **synthetic probe tasks**: algorithmically generated problems that are easy to construct programmatically, yet non-trivial for autoregressive models to solve:

- **Arbitrary multi-digit arithmetic**: Generating random large-integer operations (e.g., $4123 + 9421 = 13544$) guarantees that the specific numerical pair was never observed in pre-training corpora.
- **String perturbation and character scrubbing**: Requesting a model to strip a specific punctuation symbol from an arbitrary sentence (e.g., *"Remove the $ from the sentence 'th$s is a$sentenc$e'"*). Because subword tokenizers merge characters into subword units, manipulating raw characters requires navigating token boundaries.
- **Anagram unscrambling**: Asking the model to unscramble randomly permuted words (e.g., *"aplpe"* $\rightarrow$ *"apple"*).

Task Example: Synthetic Diagnostic Probes

Probe 1: Out-of-Distribution Multi-Digit Addition:

- *Prompt*: "What is $4123 + 9421$?" $\Longrightarrow$ *Target*: 13544.
- *Diagnostic*: Tests multi-token carry propagation without web memorization.

Probe 2: Subword Token Boundary Navigation (Character Scrubbing):

- *Prompt*: "Remove the $ symbol from the string: 'th$s is a$sentenc$e'."
- *Target*: **"this is a sentence"**.
- *Failure Mode*: Subword tokenizers fuse 'th' and '$' into distinct BPE tokens, causing models to erroneously restore vowels (e.g., turning '$' into 'i') or introduce unwanted whitespaces.

Probe 3: Permutation Inversion (Anagram Unscrambling):

- *Prompt*: "Unscramble the word: 'aplpe'." $\Longrightarrow$ *Target*: **"apple"**.

8.3 Standardized Benchmark Suites

While evaluating models on individual tasks reveals localized strengths and weaknesses, reporting dozens of separate accuracies across disjoint datasets is unwieldy. The research community requires holistic, standardized evaluation suites that aggregate performance across diverse tasks into unified benchmark leaderboards.

However, standardized benchmarks did not emerge as a random collection of test sets; they evolved through an evolutionary progression where each new generation was conceived directly to address the rapid saturation and methodological failure modes of its predecessor.

8.3.1 An Evolutionary Taxonomy of Benchmark Suites

The history of foundation model benchmarking can be understood through four distinct evolutionary generations, illustrated in the progression from single-sentence heuristics to multimodal frontier reasoning:

1. **Generation 1: Sentence-Level Linguistic Understanding (GLUE, 2018)**: Focused on evaluating pre-trained sentence representations across nine diverse classification and inference tasks, aggregated into a single scalar leaderboard score. Saturated within eighteen months by BERT and RoBERTa.

2. **Generation 2: Multi-Sentence Discourse and Coreference (SuperGLUE, 2019)**: Replaced saturated GLUE tasks with substantially harder challenges requiring multi-sentence reading comprehension, causal reasoning, and pronoun coreference resolution. Saturated by T5 and DeBERTa.

3. **Generation 3: Massive Multi-Domain Academic Knowledge (MMLU, 2020)**: Shifted evaluation from linguistic competence to broad, encyclopedic academic and professional domain knowledge spanning 57 subjects, evaluated under zero-shot and few-shot prompting.

4. **Generation 4: Multimodal Frontier Reasoning and Contamination-Proof Testing (Humanity's Last Exam, 2024/2025)**: Advanced to expert-level multimodal reasoning across hundreds of disciplines, intentionally utilizing private evaluation splits to withstand web memorization.

8.3.2 Generation 1: Sentence-Level Linguistic Understanding (GLUE)

In 2018, Wang et al. [93] introduced **GLUE** (*General Language Understanding Evaluation*), an integrated benchmark suite designed to evaluate general-purpose sentence representations. Prior to GLUE, models were evaluated on isolated datasets using disparate metrics, preventing direct comparisons. GLUE unified evaluation by assembling nine natural language understanding tasks spanning three broad families:

1. **Single-sentence tasks**: Linguistic acceptability on CoLA [75] and binary sentiment analysis on SST-2 [76].

2. **Similarity and paraphrase tasks**: Sentence pair equivalence on MRPC (Microsoft Research Paraphrase Corpus), QQP (Quora Question Pairs), and semantic similarity scoring on STS-B (Semantic Textual Similarity Benchmark).

3. **Inference tasks**: Natural language inference on MNLI (MultiNLI) [86], QNLI (Question NLI, derived from SQuAD), RTE (Recognizing Textual Entailment), and WNLI (Winograd NLI).

GLUE established a public competitive leaderboard evaluated on hidden private test sets, aggregating performance across all nine tasks into a **single summary score** (the unweighted average of task accuracies and correlation coefficients).

GLUE Benchmark Example: Multi-Genre Natural Language Inference (MNLI)

Task Genre: Slate Magazine (Journalistic Essay)
Premise: *"The new rights are nice, but they don't solve the underlying problem."*
Hypothesis: *"The problem remains unresolved despite the new rights."*
Candidate Options:

(A) Entailment *[Correct]*

(B) Contradiction

(C) Neutral

Benchmark Analysis: GLUE mapped all classification, similarity, and inference tasks into a single aggregate summary score. Models easily saturated MNLI by exploiting lexical overlap heuristics between premise and hypothesis.

8.3.3 Generation 2: Multi-Sentence Discourse and Coreference (SuperGLUE)

When GLUE was introduced, state-of-the-art models scored substantially below human baseline performance (87.1). However, with the advent of the Transformer pre-training revolution (BERT [48], RoBERTa, and T5 [57]), models rapidly saturated the benchmark, exceeding non-expert human performance within eighteen months. Error analysis revealed that models were exploiting shallow statistical cues and lexical overlaps in GLUE's test sets rather than exercising deep comprehension.

In response, Wang et al. [94] introduced **SuperGLUE** in 2019. Retaining GLUE's single-metric aggregate format, SuperGLUE replaced saturated tasks with substantially harder challenges requiring multi-sentence discourse comprehension, coreference resolution, and open-ended QA:

- **BoolQ**: Boolean (yes/no) question answering over complex multi-sentence passages.
- **CB (CommitmentBank)**: Three-way discourse clause inference.
- **COPA**: Commonsense causal reasoning [89].
- **MultiRC**: Multi-sentence reading comprehension where questions have multiple correct answers.
- **ReCoRD**: Reading comprehension with commonsense reasoning over news articles.
- **RTE**: Recognizing textual entailment across diverse news datasets.
- **WiC (Words in Context)**: Identifying whether a polysemous word shares identical meaning across two different sentence contexts.
- **WSC**: The Winograd Schema Challenge pronoun coreference task [87].

SuperGLUE Benchmark Example: Boolean Question Answering (BoolQ)

Passage: *"The Persian Gulf is a Mediterranean sea in Western Asia. The body of water is an extension of the Indian Ocean through the Strait of Hormuz and lies between Iran to the northeast and the Arabian Peninsula to the southwest... The Persian Gulf was a battlefield of the 1980–1988 Iran-Iraq War, in which each side attacked the other's oil tankers."*
Question: *"Is the Persian Gulf connected to the Indian Ocean?"*
Candidate Choices: **True** / **False**
Gold Label: **True**
Reasoning Requirement: Unlike GLUE's single-sentence tasks, SuperGLUE demands synthesizing complex multi-sentence passages. The model must recognize that being an "extension of the Indian Ocean through the Strait of Hormuz" implies geographic connectivity.

SuperGLUE Benchmark Example: Words in Context (WiC)

Target Word: **bank**
Context 1: *"He sat on the muddy **bank** of the river and watched the barges pass."*
Context 2: *"She deposited her paycheck at the local **bank** before noon."*
Question: Does the target word have the same meaning in both sentences?
Gold Label: **False** (Geographic shore vs. financial institution).
Cognitive Probe: Tests polysemous semantic disambiguation across diverse sentence syntaxes.

8.3.4 Generation 3: Massive Multitask Language Understanding (MMLU)

As foundation models scaled past tens of billions of parameters, evaluation expanded beyond basic linguistic understanding toward broad, multi-domain academic and professional knowledge.

In 2020, Hendrycks et al. [95] introduced **MMLU** (*Massive Multitask Language Understanding*), which has become the de facto gold standard for reporting foundation model capabilities (utilized to benchmark GPT-3, PaLM, LLaMA, GPT-4, and Gemini).

Architecture and scope of MMLU

- **Scale and format**: Comprises 15,908 multiple-choice questions, each formulated with four options (A, B, C, D).
- **Breadth of domains**: Spans **57 diverse subjects** categorized into four overarching disciplines:
    1. **STEM**: Advanced Mathematics, Physics, Chemistry, Computer Science, Machine Learning, Electrical Engineering.
    2. **Humanities**: Formal Logic, Philosophy, World History, Prehistory, Jurisprudence.
    3. **Social Sciences**: Microeconomics, Macroeconomics, Psychology, Sociology, US Government and Politics.
    4. **Other / Professional**: Clinical Medicine, Professional Law (Bar Exam), Professional Accounting, Virology.
- **Difficulty stratification**: Covers elementary school facts, high school examinations, undergraduate coursework, and professional licensing standards.

MMLU evaluation protocol

Models are evaluated on MMLU under **zero-shot** or **5-shot in-context learning** conditions. The prompt provides the question, four formatted choices, and a prompt prefix (e.g., *"Answer:"*). The

model's score is computed by comparing the conditional log-probabilities assigned to the four option tokens:

$$\hat{y} = \underset{k \in \{\text{'A'},\text{'B'},\text{'C'},\text{'D'}\}}{\arg\max} \; P(w_k \mid \text{Prompt}). \tag{8.60}$$

The overall MMLU score is the unweighted percentage of questions answered correctly across all 57 subjects. A random baseline achieves 25.0%; modern frontier models exceed 85%, outperforming human domain experts on the majority of specialized subjects.

MMLU Example 1: Conceptual Physics

Question: When you drop a ball from rest it accelerates downward at 9.8 m/s^2. If you instead throw it downward assuming no air resistance its acceleration immediately after leaving your hand is:

(A) 9.8 m/s^2 *[Correct]*

(B) more than 9.8 m/s^2

(C) less than 9.8 m/s^2

(D) Cannot say unless the speed of throw is given

Physics insight: Once the ball leaves the hand, gravity is the only accelerating force acting on the mass; the initial throw velocity alters instantaneous speed, not downward acceleration ($g = 9.8$ m/s^2).

MMLU Example 2: College Mathematics

Question: In the complex z-plane, the set of points satisfying the equation $z^2 = |z|^2$ is a:

(A) pair of points

(B) circle

(C) half-line

(D) line *[Correct]*

Mathematical derivation: Let $z = x + iy$. Then $z^2 = x^2 - y^2 + 2ixy$ and $|z|^2 = x^2 + y^2$. Equating real and imaginary components yields $2xy = 0$ (so $x = 0$ or $y = 0$) and $x^2 - y^2 = x^2 + y^2 \implies 2y^2 = 0 \implies y = 0$. Since $y = 0$ and $x \in \mathbb{R}$ satisfies the equation for all x, the solution set is precisely the real axis, which constitutes a **line** in the complex plane.

8.3.5 Generation 4: Multimodal Frontier Reasoning and Private Splits (HLE)

As frontier models achieved near-human scores on MMLU (> 85%), the AI community recognized that static, multiple-choice academic benchmarks were succumbing to both benchmark saturation and widespread pre-training memorization. In response, recent efforts have introduced **Humanity's Last Exam (HLE)** (2024/2025), designed to serve as the definitive evaluation benchmark for artificial general intelligence.

Design philosophy and multimodal scope

Constructed in collaboration with hundreds of subject matter experts worldwide, HLE comprises approximately 2,500 questions across mathematics, theoretical physics, computer science, biology, and philosophy. HLE differs from previous benchmarks in three crucial dimensions:

1. **Multimodal integration**: Questions incorporate diagrams, scientific charts, and visual figures, requiring joint perceptual and abstract reasoning.

2. **Extreme expert difficulty**: Problems are curated to reflect frontier doctoral research questions rather than standardized undergraduate multiple-choice questions.
3. **Confidential private test split**: Crucially, the authors maintain a substantial portion of the benchmark strictly private and unreleased on public web repositories. Evaluation is conducted through closed scoring servers, guaranteeing that the questions cannot be crawled by commercial web scrapers.

When evaluated on HLE, the performance of state-of-the-art models collapses dramatically: while models boast $> 85\%$ on public MMLU, their accuracy on HLE ranges between 10% and 30%, demonstrating that much of the apparent progress on public benchmarks stems from data exposure rather than genuine reasoning.

Humanity's Last Exam (HLE) Example: Multimodal Expert Reasoning

Subject Domain: Theoretical Chemistry & Molecular Spectroscopy
Problem Setup: Given the high-resolution vibrational-rotational Raman spectrum of a linear triatomic molecule showing nuclear spin statistical weights and alternating intensity ratios of $1:2$ for even and odd rotational quantum states J, identify the ground-state electronic symmetry and nuclear spin I of the identical terminal nuclei.
Evaluation Protocol: Free-response symbolic derivation verified against confidential closed-form evaluation scripts.
Why Frontier Models Struggle: Unlike public web-crawled benchmarks where questions appear on Chegg or Wikipedia, HLE questions require multi-step, expert-level academic deduction. While models claim $> 85\%$ on public MMLU, they achieve only $\sim 10\%$ to 30% accuracy on HLE's private split, demonstrating the acute vulnerability of public leaderboards to benchmark data contamination.

8.3.6 The Benchmark Contamination Crisis

The very large performance gains reported on public benchmarks have brought to light an existential vulnerability in modern AI evaluation: **data contamination**.

Definition 8.1: *Benchmark Data Contamination*

Data contamination refers to the inadvertent or deliberate inclusion of evaluation benchmark test data (questions, passages, or gold-standard target labels) in the training corpus of a language model.

As highlighted in lecture, foundation models are pre-trained on multi-terabyte web crawls (such as Common Crawl, GitHub, Wikipedia, and academic preprint servers) containing hundreds of billions of web pages. Because popular benchmarks (GLUE, SQuAD, MMLU, GSM8K) are hosted on public GitHub repositories, open-access papers, and web forums, their test sets are routinely ingested by automated web scrapers during pre-training.

Taxonomy and levels of benchmark contamination

Contamination can occur across multiple distinct stages of the model lifecycle:

1. **Pre-training contamination**: Public benchmark test splits are included directly in the pre-training scrape. The model observes the verbatim test questions and answers during self-supervised next-token prediction, memorizing them as factual text.
2. **Supervised fine-tuning (SFT) contamination**: Benchmark questions are inadvertently used to generate synthetic instruction-tuning datasets (e.g., using one LLM to generate Q&A pairs from existing benchmarks to train another model).

3. **Post-deployment feedback contamination**: Real users paste benchmark questions into public chatbot interfaces, which are subsequently collected and added to subsequent retraining pipelines.

Consequences: Memorization vs. reasoning

Data contamination undermines the foundational assumption of statistical machine learning: that generalization performance on a held-out test set reflects genuine out-of-distribution capability.

In their seminal study *"NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark"*, Sainz et al. [96] demonstrated that models exhibiting near-perfect scores on popular reasoning benchmarks suffered substantial performance drops when evaluated on slightly perturbed or synthetically generated instances of the identical problems. A model that achieves 90% on GSM8K might simply be reciting a memorized solution path observed dozens of times during pre-training, rather than executing mathematical deduction.

The closed-model auditing barrier

As emphasized by Prof. Giobergia, data contamination introduces severe asymmetries between open-weight and proprietary closed-source models:

- For open-source models (such as LLaMA or Mistral), researchers can inspect public pre-training manifests, search for exact n-gram overlaps, and audit decontamination pipelines.
- For proprietary commercial models (such as GPT-4), the exact composition of the pre-training corpus is an undisclosed trade secret. Consequently, **contamination cannot be formally proven or disproven**. Continuous benchmark gains reported by proprietary models may stem from architectural innovations, or simply from aggressive pre-training exposure to the evaluation test sets.

Decontamination protocols and countermeasures

To safeguard evaluation integrity, modern foundation model research employs rigorous decontamination heuristics:

- **N-gram overlap filtering**: Scrutinizing the pre-training corpus for contiguous 13-gram or 15-gram matches against benchmark test splits and removing matching documents prior to training.
- **Perplexity anomaly inspection**: Measuring the model's perplexity on the benchmark test set compared to a fresh, private validation set. If perplexity on the public test set is anomalously lower than on independent text from the same domain, contamination is strongly indicated.
- **Dynamic and synthetic benchmarks**: Generating novel evaluation instances on-the-fly (e.g., dynamic arithmetic problems or continuous human evaluation arenas like Chatbot Arena) that can never appear in static training crawls.

CHAPTER 9

Instruction Tuning and Alignment

In modern natural language processing, the concept of **alignment** has undergone a profound historical transformation. In classical computational linguistics, alignment referred strictly to structural mappings between discrete symbolic sets—such as bipartite word-alignment links between source and target tokens in statistical machine translation. In the era of Large Language Models (LLMs), however, alignment denotes the imperative task of **steering model outputs to conform faithfully to human expectations, communicative intentions, and ethical values** [97].

While self-supervised autoregressive pre-training on trillions of tokens equips a foundation model with vast factual knowledge, syntactic mastery, and statistical grammar, pre-training alone is structurally incapable of producing a helpful, reliable, or safe conversational assistant. A pre-trained base model is fundamentally an unconditional text predictor: when prompted with a question, its training objective encourages it to continue the text as if completing an uncurated web document (generating further questions, forum arguments, or machine-generated spam) rather than executing the user's directive. Furthermore, internet text contains toxic diatribes, uncalibrated biases, and factual fallacies; standard pre-training replicates these pathologies with equal fidelity.

To address this multidimensional challenge, alignment techniques can be systematically organized along a rigorous **operational taxonomy based on the execution stage at which alignment is performed** (Figure 9.1):

1. **Inference-Time Alignment**: Operating entirely without gradient backpropagation or weight updates. The behavior of a frozen model is steered dynamically at runtime via in-context prompt engineering, demonstration exemplars, system prefixes, or test-time candidate reranking (such as Best-of-N sampling).

2. **Instruction Alignment via Supervised Fine-Tuning (SFT)**: Adapting model weights directly on curated datasets of instruction-response pairs using masked cross-entropy, training the network to execute natural language commands.

3. **Human Preference Alignment via Reward Modeling (RLHF)**: Aligning the model with subtle, non-deterministic human values and qualitative trade-offs using learned neural reward surrogates and reinforcement learning.

To establish a principled engineering objective, Askell et al. [98] formalized the **HHH Alignment Criteria**:

Definition 9.1: *The HHH Alignment Criteria*

A language model is considered aligned if its behavior simultaneously satisfies:

1. **Helpfulness**: The model makes an active, high-fidelity effort to resolve the user's objective. It adheres strictly to formatting, length, and stylistic constraints, asks

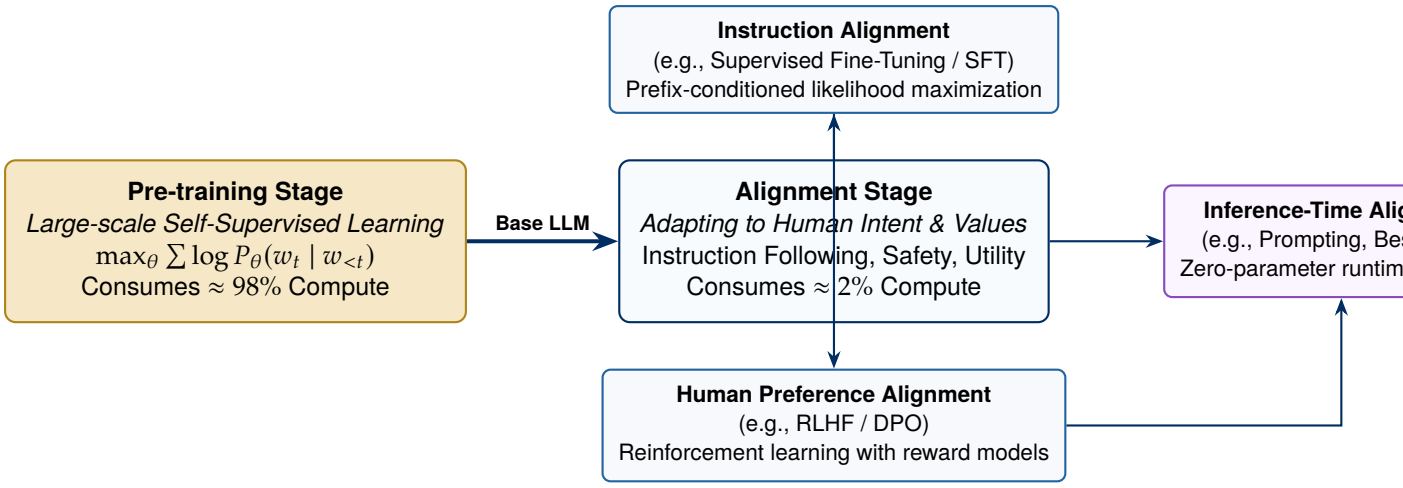


Figure 9.1: The foundational *pre-train-then-align* paradigm for developing large language models (faithfully adapted from Xiao & Zhu [97] Figure 4.1). In the pre-training stage, a massive foundation model learns statistical representations from raw web text. In the subsequent alignment stage, the model is adapted through three complementary paradigms: instruction alignment (SFT), human preference alignment (RLHF/DPO), and inference-time steering.

for clarification when requests are genuinely underspecified, avoids unsolicited conversational preamble, and maintains appropriate conciseness.

2. **Honesty**: The model produces claims that are factually grounded and accurate. Crucially, it must exhibit calibrated uncertainty, explicitly stating when information is unknown or ambiguous rather than fabricating plausible-sounding hallucinations or false citations.

3. **Harmlessness**: The model refuses to generate content that facilitates physical harm, cyberattacks, weapons synthesis, illegal acts, or severe psychological harassment, while avoiding unjustified refusals on benign, academic, or adjacent safety queries.

9.1 Inference-Time Alignment and Its Limitations

9.1.1 The mechanics of inference-time steering

The most immediate and computationally economical approach to alignment is **inference-time alignment** [97]. Rather than executing costly gradient updates over billions of parameters, inference-time methods preserve the pre-trained weights θ in a frozen state and guide the model's generation trajectory dynamically at test time.

Inference-time alignment operates primarily through two distinct mechanisms:

1. **Contextual Conditioning and Prompt Steerability**: Prepended input tokens act as activation vectors in the self-attention key-value cache, biasing the autoregressive sampling distribution toward desired linguistic styles, reasoning patterns (e.g., Chain-of-Thought), or persona constraints.

2. **Inference-Time Output Reranking (Best-of-N Sampling)**: Rather than accepting a single greedy or stochastic generation, the frozen model samples N independent candidate completions $\{\hat{y}_1, \ldots, \hat{y}_N\} \sim P_\theta(\cdot \mid x)$ using temperature or nucleus sampling. An external

scoring mechanism (such as an automated heuristic or a trained reward model $r(x, y)$) evaluates each candidate, selecting the optimal output:

$$\hat{y}_{\text{best}} = \arg \max_{y \in \{\hat{y}_1, \ldots, \hat{y}_N\}} r(x, y). \tag{9.1}$$

While Best-of-N reranking can dramatically improve test-time accuracy on mathematical reasoning and safety checks, its computational cost scales linearly with N, creating severe latency and throughput overhead for interactive real-time systems.

9.1.2 In-context few-shot learning and its three fundamental limitations

The primary vehicle for prompt-based inference-time alignment in raw foundation models is **in-context few-shot learning** [2]. By prepending K input-output demonstration exemplars to the prompt:

$$x_{\text{prompt}} = \left(x^{(1)}_{\text{demo}}, y^{(1)}_{\text{demo}}\right) \circ \left(x^{(2)}_{\text{demo}}, y^{(2)}_{\text{demo}}\right) \circ \cdots \circ \left(x^{(K)}_{\text{demo}}, y^{(K)}_{\text{demo}}\right) \circ x_{\text{query}}, \tag{9.2}$$

the internal self-attention heads track the demonstration pattern, guiding the model to generate completion y_{query} without modifying network weights θ.

While few-shot prompting demonstrated that foundation models possess latent multi-task capabilities, relying on in-context learning to build conversational assistants suffers from **three fundamental limitations**:

1. **Severe Instability and Format Brittleness**: In-context few-shot predictions are notoriously unstable. Empirical evaluations show that accuracy fluctuates drastically depending on the arbitrary ordering of exemplars, minor variations in whitespace or delimiter tokens, and the frequency of class labels within the prompt. Few-shot models exhibit severe **recency bias** (disproportionately predicting the label of the final exemplar) and **majority-class bias**, making production deployments unreliable.

2. **Complete Failure for Instructions Lacking Natural Exemplars**: Many real-world human interactions intrinsically do not admit few-shot demonstrations. Prompts involving open-ended creative synthesis (e.g., `"Draft a persuasive diplomatic communique regarding deep-sea mining rights"`), dynamic full-codebase refactoring, or bespoke multi-turn conversational troubleshooting cannot be preceded by realistic exemplars without either pre-solving the unique task or consuming the entire context window. Expecting users to write $K = 5$ expert demonstrations before asking a question defeats the purpose of an automated assistant.

3. **Complete Failure to Generalize to Unseen Tasks in Zero-Shot Regimes**: In the true zero-shot setting ($K = 0$)—where the model receives an instruction without demonstration pairs—base models fail completely. Because they were never trained to associate imperative natural language with task execution, base models revert to document continuation, echoing the prompt or drifting into irrelevant topics.

9.1.3 Empirical case study: T5 prefix conditioning and format fragility

The limitations of early sequence-to-sequence conditioning were brought to light by the **Text-to-Text Transfer Transformer (T5)** [57]. T5 sought to unify all natural language processing tasks under a single text-to-text encoder-decoder framework by prepending a rigid, hardcoded prefix string to every input:

Translation: `"translate English to German: That is good."` ⟶ `"Das ist gut."`

Classification: `"cola sentence: The course is terrible."` ⟶ `"unacceptable"`

Summarization: `"summarize: State authorities announced..."` ⟶ `"Authorities issued a state.`

While T5 proved that diverse tasks could share identical model weights, digging deeply into its behavior reveals that T5 was fundamentally **unaligned and brittle**. T5 did not learn generalized instruction comprehension. Instead, the prefix strings acted merely as fixed associative lookup keys that routed activations through specific parameters. The moment user inputs deviated even slightly from the exact syntax seen during training, T5's execution collapsed catastrophically.

Pedagogical Example: T5 Prompt Brittleness and Format Fragility

The empirical fragility of T5's prefix mechanism is evident in three canonical failure modes (documented in course lecture slides):

1. **Canonical Pre-trained Prefix (Exact Syntax)**:

   Input: `"can you translate from English to German, What is your profession?"`
   Output: `"Was ist Ihr Beruf?"` (Correct execution)

2. **Minor Natural Language Paraphrase**:

   Input: `"can you translate from English to German the following sentence? What is y`
   Output: `"<unk> <unk> <unk>..."` (Catastrophic failure into unknown tokens)

3. **Novel Task Directive**:

   Input: `"compute: 2+2 ="`
   Output: `":2+2+2+2+2+2+2+2+2+..."` (Degeneration into an infinite loop)

The failure cases in Box 9.1.3 demonstrate that T5 lacked both **cross-task generalization** and **phrasing robustness**. Adding subordinate clauses (such as `"the following sentence?"`) broke the associative key, causing the network to assign probability mass to out-of-vocabulary tokens. Furthermore, T5 could not generalize to tasks outside its pre-defined prefix dictionary (such as performing arithmetic via `"compute:"`), generating degenerative repetitive patterns instead.

These fundamental limits establish that inference-time conditioning cannot substitute for **parametric alignment**: permanently adapting network weights to follow arbitrary natural language instructions.

9.2 Supervised Fine-Tuning and Instruction Alignment

9.2.1 Why autoregressive pre-training yields unaligned models

Autoregressive pre-training optimizes the log-likelihood of sequential text tokens over massive, uncurated web corpora. From a philosophical perspective, this objective forces the model to perform statistical pattern matching across linguistic surfaces rather than acquire genuine semantic comprehension, epistemic awareness, or communicative intent. The network learns the statistical distribution of strings as they appear in the wild—encompassing internet debates, fictional dialogues, academic worksheets, and machine-generated spam—without grounding those strings in external reality or attributing agency to the user.

Consequently, when prompted, an autoregressive base model operates strictly under the paradigm of unconditional document completion. When requested to summarize an article, a base model does not summarize; it continues the article with more prose. When asked a question, it generates subsequent questions.

From a mathematical perspective, standard pre-training minimizes the negative log-likelihood (NLL) over sequence tokens:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\sum_{t=1}^{N} \log P_\theta(w_t \mid w_{<t}), \tag{9.3}$$

where $P_\theta(w_t \mid w_{<t}) = \text{softmax}(W_{\text{unembed}} h_t)$ represents the categorical distribution over the vocabulary $\mathcal{V}$.

Equation (9.3) exhibits critical mathematical deficiencies when viewed as an alignment objective:

1. **Equal Penalty for Semantically Divergent Errors**: Cross-entropy penalizes every non-target token $w \neq w_t^*$ based purely on predictive probability mass. Predicting an accurate, elegant synonym receives a heavy loss penalty, whereas predicting a dangerous factual hallucination or offensive slur receives the identical penalty. It possesses no metric for truthfulness or qualitative severity.
2. **Imitation of Statistical Modes across Heterogeneous Quality**: Because MLE forces the model to minimize forward KL divergence $D_{\text{KL}}(P_{\text{data}} \parallel P_\theta)$, the network is mathematically penalized if it omits any mode of the training distribution. It learns to mimic low-quality, untruthful text with the exact same fidelity that it mimics scientific prose.
3. **Absence of Intent and Communicative Conditioning**: The loss contains no term modeling user satisfaction, brevity, or directive execution.

This discrepancy is termed the **intent gap** [99]: the divide between human intent and autoregressive next-token loss.

Pedagogical Example: Base Model Prompt Continuation versus Assistant Directive Execution

To observe the practical manifestation of the intent gap, consider submitting an elementary factual query to a base language model (such as base LLaMA-2 7B prior to instruction tuning):

```
"what is 2 + 2?"
```

Because the model was trained exclusively to continue web documents, it does not interpret the string as an imperative request to calculate an arithmetic sum. Instead, observing that short mathematical expressions on the internet frequently occur within repetitive worksheets, exam practice tests, or FAQ forum posts, the unaligned base model continues the text with statistically correlated questions:

```
What is 2 plus 2?  What is the answer to 2 plus 2?  What is the answer to
2 plus 2 in math?
```

In contrast, an instruction-aligned assistant interprets the communicative intent behind the prompt, suppresses prompt repetition, and outputs the direct, helpful answer:

```
The answer to 2 + 2 is 4.
```

9.2.2 Supervised fine-tuning mechanics and prompt-loss masking

To bridge the intent gap, **Supervised Fine-Tuning (SFT)** adapts the pre-trained weights $\hat{\theta}$ on an annotated dataset $\mathcal{D}_{\text{SFT}} = \{(x, y)\}$ of instruction prompts x paired with gold-standard responses y [97].

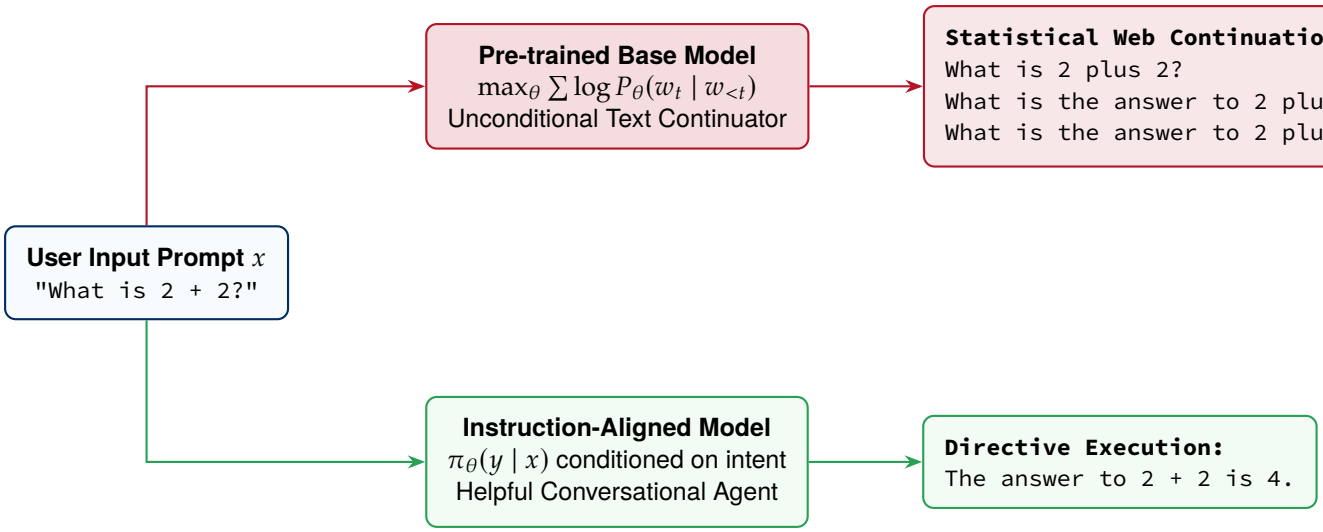


Figure 9.2: The intent gap between autoregressive next-token prediction and instruction alignment. When prompted with an elementary arithmetic query, an unaligned base foundation model treats the prompt as an open-ended document prefix and synthesizes statistically correlated worksheet questions. Conversely, an aligned assistant interprets the prompt as an imperative task directive, returning the exact communicative answer.

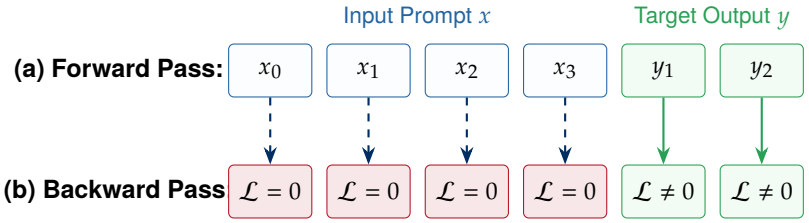


Figure 9.3: Supervised fine-tuning with prompt-loss masking (adapted from Xiao & Zhu [97] Figure 4.2). During the forward pass, the model processes the concatenated sequence $[x, y]$ using standard causal self-attention. During the backward pass, loss gradients are masked to zero for all prompt tokens x, computing backpropagation exclusively on the target response tokens y.

Unlike standard pre-training, where loss is computed over the entire sequence, SFT seeks to maximize the **conditional probability of the response given the prompt**:

$$\theta^* = \arg\max_\theta \sum_{(x,y)\in\mathcal{D}_{\text{SFT}}} \log P_\theta(y \mid x) = \arg\max_\theta \sum_{(x,y)\in\mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log P_\theta(y_t \mid x, y_{<t}). \tag{9.4}$$

In modern autoregressive decoder-only Transformer architectures, computing Equation (9.4) is elegantly implemented via **Prompt Loss Masking**. When concatenating the prompt $x = (x_0, \ldots, x_m)$ and completion $y = (y_1, \ldots, y_n)$ into a single sequence $[x, y]$, the joint log-probability decomposes via the probability chain rule as:

$$\log P_\theta(x, y) = \underbrace{\log P_\theta(x)}_{\text{Force loss to 0}} + \underbrace{\log P_\theta(y \mid x)}_{\text{Active Loss Computation}}. \tag{9.5}$$

As illustrated in Figure 9.3, the standard forward pass is executed across all sequence positions so that the output tokens can attend to the full contextual prompt. During the backward pass, gradients corresponding to x are set to zero (masked with label -100 in PyTorch cross-entropy). The network updates its weights exclusively to generate correct completions y conditioned on x.

For multi-turn conversational interactions with K conversation rounds $\{x_1, y_1, x_2, y_2, \ldots, x_K, y_K\}$, the loss masking generalizes across alternating dialogue turns:

$$\mathcal{L}_{\text{multi-turn}}(\theta) = -\sum_{k=1}^{K}\sum_{t=1}^{|y_k|}\log P_\theta\left(y_{k,t} \mid x_1, y_1, \ldots, x_{k-1}, y_{k-1}, x_k, y_{k,<t}\right), \tag{9.6}$$

where loss is masked for all user inputs x_k and computed exclusively on assistant turns y_k.

Limitations of Supervised Fine-Tuning Despite its widespread success, SFT suffers from two major limitations:

1. **Extravagant Data Annotation Bottlenecks**: High-quality SFT requires thousands of diverse, expert-authored prompt-response pairs. Creating datasets that capture complex formatting, reasoning chains, and specialized domains requires skilled human labor that is difficult to scale.
2. **Computational Expense on Frontier LLMs**: Fine-tuning all parameters of a 70B+ parameter model requires substantial GPU memory and compute, introducing the persistent danger of **catastrophic forgetting** of pre-trained linguistic features and world knowledge.

9.2.3 Fine-tuning data acquisition: manual vs. automatic generation

Given the pivotal role of instruction data, the literature has developed two distinct paradigms for acquiring SFT datasets [97]:

Manually Generated Data

Human-authored data remains the gold standard for quality and conversational naturalness:

- **Human Expert Authoring**: Recruited annotators author both the prompt instructions and the target answers. While producing high-fidelity outputs, this approach is exceptionally slow and expensive.
- **Prompt Templates over Existing NLP Datasets**: Pre-existing, high-quality academic benchmarks (SuperGLUE, SQuAD, MultiNLI) are repurposed into conversational instructions using templating engines (e.g., PromptSource [100]). For example, translation sentence pairs are mapped into: `"Instruction: Translate to French. Input: {source} Output: {target}"`.
- **Organic Web and Forum Extraction**: Crawling question-and-answer archives (such as StackOverflow, Reddit threads, and Quora). However, web-mined data often contains noise, slang, or sarcastic non-answers that must be heavily filtered.

Automatically Generated Data

To scale data volume without exorbitant human cost, modern pipelines synthesize SFT datasets automatically:

- **Self-Instruct Tuning** [101]: Starting from a small seed set of hand-crafted tasks ($\approx$ 175 tasks), a powerful teacher LLM iteratively bootstraps new instructions, classifies them by task type, generates corresponding inputs and outputs, and filters candidates based on heuristic quality and novelty.
- **Task Inversion (Input Flipping)**: Reversing traditional input-output relationships to enforce bidirectional semantic comprehension:

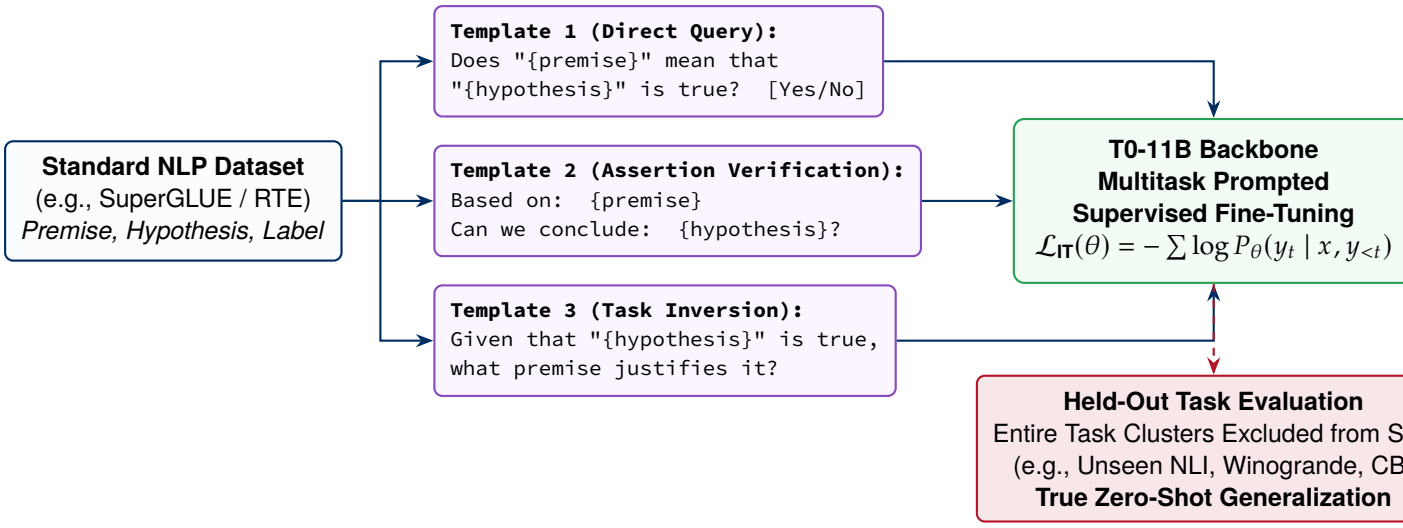


Figure 9.4: The T0 multitask prompted fine-tuning architecture [103]. Conventional tabular NLP datasets are mapped into conversational queries through a diverse collection of crowdsourced prompt templates, including direct questions, verification assertions, and task inversions. Fine-tuning on these multi-prompt formulations enables the model to generalize zero-shot to entirely held-out task clusters without seeing task-specific exemplars.

```
"Given the target summary '{summary}', draft a plausible source article
                        that produced it."
```

- **Teacher LLM Generation and Best-of-N Filtering**: Using a frontier model (such as GPT-4) to generate responses to diverse prompts, followed by automated rejection sampling to retain only responses meeting strict quality criteria.

9.2.4 Data efficiency, LIMA, and emergent instruction generalization

Fine-Tuning with Less Data: The LIMA Hypothesis A natural question arises: how much SFT data is truly required to align an LLM? Zhou et al. [102] addressed this in their landmark study **LIMA (Less Is More for Alignment)**. They demonstrated that fine-tuning a 65B base LLaMA model on just **1,000 meticulously curated instruction-response pairs** produced conversational performance competitive with GPT-4 and InstructGPT.

This established the **Superfly Alignment Hypothesis**: almost all knowledge and world reasoning capabilities are acquired entirely during pre-training. Alignment is not about teaching the model new facts; rather, it is a superficial formatting process that teaches the model the sub-distribution of conversational style, dialogue framing, and task obedience.

Instruction Generalization across Multi-Task Models When instruction tuning is executed across diverse task mixtures, it unlocks dramatic cross-task generalization:

T0: Multitask Prompted Training and Prompt Robustness

Sanh et al. [103] implemented this vision in **T0**, utilizing an 11-billion parameter T5 encoder-decoder backbone (Figure 9.4). T0 was fine-tuned across dozens of datasets formatted with $M_k \approx 8 - 10$ crowdsourced prompt templates per task:

$$\mathcal{L}_{\text{IT}}(\theta) = -\sum_{k=1}^{K} \frac{1}{M_k} \sum_{m=1}^{M_k} \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}_k} \left[\sum_{t=1}^{|y|} \log P_\theta \left(y_t \,\middle|\, \mathcal{T}_{k,m}(x), y_{<t} \right) \right]. \tag{9.7}$$

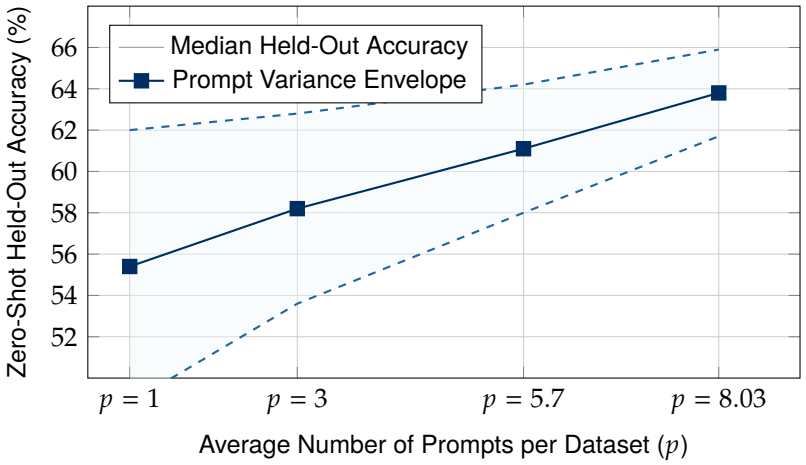


Figure 9.5: Empirical prompt robustness dynamics as a function of template diversity in T0 [103]. As the average number of prompt templates per dataset (p) scales from $p = 1$ to $p = 8.03$, median held-out task accuracy increases monotonically from 55.4% to 63.8%. Concurrently, the performance variance across differing prompt formulations narrows substantially, demonstrating that diverse template training enforces syntactic invariance.

T0 evaluated generalization by completely withholding entire task clusters (such as Natural Language Inference) from training. When evaluated zero-shot on held-out NLI datasets (RTE, CB, ANLI), T0-11B outperformed untuned T5-11B by > 15% and outperformed the 175B base GPT-3 model.

Crucially, T0 revealed the empirical law of **prompt robustness** (Figure 9.5): scaling the number of prompt templates from $p = 1$ to $p = 8$ per dataset compressed the evaluation variance envelope from 13.2% down to 4.2%, insulating the model against arbitrary phrasing perturbations.

FLAN: Instruction Tuning at Scale and Scale Emergence

Wei et al. [104] investigated instruction tuning across dense decoder-only models (LaMDA) from 420M up to 137B parameters across 62 datasets in 12 task clusters.

As shown in Figure 9.6, FLAN uncovered the **scale emergence phenomenon**:

- **Negative Transfer on Small Models (< 8B)**: For models with 420M and 2B parameters, instruction tuning degraded held-out task accuracy relative to base models due to parameter capacity saturation.
- **Emergence on Large Models (> 68B)**: At 68B and 137B parameters, instruction tuning unlocked a surge in generalization (+9.4% gain), outperforming zero-shot GPT-3 175B on 20 out of 25 evaluated tasks.

9.3 RLHF and Human Preference Modeling

9.3.1 Why scale and supervised tuning are not enough

Even after pre-training on trillions of tokens and instruction tuning across dozens of task clusters, large language models exhibit severe alignment deficiencies. Scaling parameter count improves lexical fluency and benchmark accuracy, but does not eliminate untruthfulness, sycophancy, toxic hallucinations, or unhelpful verbosity.

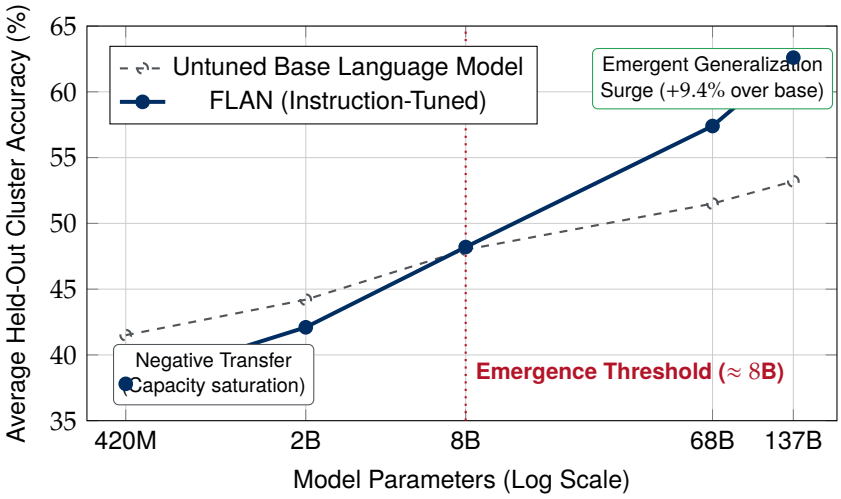


Figure 9.6: The scale emergence phenomenon of instruction tuning in FLAN [104]. On small models (< 8B parameters), instruction tuning hurts held-out task accuracy compared to untuned base models due to parameter saturation and catastrophic forgetting. On large models (> 68B parameters), instruction tuning triggers a massive emergent performance leap, surpassing untuned baselines and matching few-shot GPT-3.

The structural root of this problem was formally diagnosed by Stiennon et al. [105] in their foundational OpenAI treatise, *Learning to Summarize with Human Feedback*. Focusing on the **Reddit TL;DR dataset** (comprising 3 million diverse forum posts paired with user-authored `"TL;DR"` summaries), Stiennon et al. highlighted two fatal limitations of conventional language model training:

1. **Poor Metrics**: Standard automated evaluation metrics, such as ROUGE and BLEU, rely strictly on n-gram overlap with a human reference. They correlate remarkably poorly with human qualitative judgment. A summary that repeats an identical sentence three times can achieve a high ROUGE score while being useless to a reader. Conversely, an insightful, concise summary that rephrases facts using different vocabulary receives severe ROUGE penalties. Most dangerously, n-gram metrics cannot detect factual contradictions: changing `"not guilty"` to `"guilty"` alters only one token, yet completely inverts truth.

2. **Poor Objectives**: As derived in Section 9.2.1, token cross-entropy loss $\mathcal{L}_{\text{SFT}}$ treats valid synonyms identically to factual hallucinations and offensive outputs. Furthermore, supervised fine-tuning forces the model to fit all demonstration data equally. If human demonstrators write hurried, rambling, or inconsistent summaries, maximum likelihood forces the model to replicate those exact defects.

To overcome these structural limitations, Stiennon et al. established the canonical **three-step RLHF framework**:

1. **Step 1: Collect Human Feedback**: Collect pairwise preferences from human evaluators over candidate model completions.

2. **Step 2: Train a Reward Model**: Train a parameterized neural reward model $r_\theta(x, y)$ on the pairwise preferences using the Bradley–Terry logistic loss.

3. **Step 3: Fine-Tune the Policy via PPO**: Optimize the language model policy to maximize the predicted reward while regularizing with a per-token Kullback–Leibler (KL) divergence penalty.

9.3.2 Step 1: Collecting pairwise human feedback

The first operational phase of RLHF collects human qualitative preferences over candidate model outputs. However, designing this collection protocol requires confronting a fundamental trait of human cognition: **humans are notoriously poor at assigning absolute numerical scores, but remarkably proficient and consistent at making comparative, relative evaluations**.

Why Absolute Scoring Fails in Practice In early alignment experiments, evaluators were asked to grade completions on an absolute Likert scale (e.g., scoring a summary from 1 to 10). This approach fails due to three systemic pathologies:

1. **Calibration Drift and Incompatible Internal Scales**: Different human evaluators calibrate numerical boundaries differently. A demanding annotator might grade a high-quality summary as a 6/10, reserving 9 or 10 for idealized perfection. A lenient annotator might evaluate an average summary containing factual inaccuracies as an 8/10. Consequently, merging scalar scores from multiple evaluators injects massive non-stationary label variance into the training target.

2. **Temporal Non-Stationarity (Intra-Annotator Drift)**: Even for a single evaluator, absolute scoring drifts throughout the day due to cognitive fatigue, hunger, and exposure to preceding examples. A completion scored as an 8/10 in the morning may receive a 5/10 in the late afternoon when the annotator's critical patience wanes.

3. **Lack of an Objective Anchor**: Absolute scores lack ground truth anchors. An evaluator grading a solitary summary has no reference baseline against which to judge whether a subtle omission was necessary for brevity or was an unacceptable hallucination.

The Relative Comparison Breakthrough Conversely, when presented with an input post p and two candidate completions (s_1, s_2) side by side, human evaluators do not need to invent an abstract numeric scale. Choosing whether $s_1 \succ s_2$ or $s_2 \succ s_1$ is intuitive, cognitively light, and yields exceptionally high inter-annotator agreement ($> 75\%$).

Mathematically, comparative evaluation models the human evaluator as a decision function:

$$f(p, s_1, s_2) : C \times C \times C \longrightarrow \{0, 1\}, \tag{9.8}$$

where C denotes the space of natural language text strings, p is the prompt, and s_1, s_2 are candidate completions generated by two model snapshots (e.g., via different sampling temperatures or differing training checkpoints).

Under the hood, cognitive theory posits the **Latent Reward Hypothesis**: human judgment is governed by an unobservable, latent utility function $r^*(p, s) \in \mathbb{R}$. When evaluating completions, the annotator implicitly assigns latent utilities $r^*(p, s_1)$ and $r^*(p, s_2)$ and applies an internal selector function:

$$f(p, s_1, s_2) = \begin{cases} 1 & \text{if } r^*(p, s_1) > r^*(p, s_2) \quad (s_1 \succ s_2), \\ 0 & \text{if } r^*(p, s_2) \geq r^*(p, s_1) \quad (s_2 \succ s_1). \end{cases} \tag{9.9}$$

By collecting binary preferences $y_w \succ y_l$, we directly sample the sign of the latent utility differential:

$$\text{sign}\left(r^*(p, y_w) - r^*(p, y_l)\right) = +1. \tag{9.10}$$

This formulation completely eliminates arbitrary scalar calibrations, transforming alignment data collection into a statistically robust pairwise classification problem.

Annotator Bottlenecks and Surrogates Despite their statistical superiority, human evaluators exhibit practical limitations:

- **Financial and Throughput Limits**: High-quality annotation requires human reading time. Iterative reinforcement learning algorithms sample millions of rollouts; evaluating each rollout via human annotators would cost millions of dollars and require months of turnaround.
- **Occasional Intransitivity**: Raters can occasionally exhibit cyclic preferences ($s_A \succ s_B$, $s_B \succ s_C$, but $s_C \succ s_A$) due to competing evaluation dimensions (e.g., conciseness vs. factual detail).

These bottlenecks necessitate **Step 2**: training a neural **Reward Model** to act as a frozen, infinitely scalable mathematical surrogate of collective human preferences.

9.3.3 Step 2: Training the reward model

To automate human evaluation, a language model is converted into a **Reward Model (RM)** $r_\phi(x, y)$ [97]. The network is initialized from an SFT model checkpoint, the final unembedding vocabulary matrix is removed, and a scalar projection head $W_r \in \mathbb{R}^{d_{\text{model}} \times 1}$ is attached. Given prompt x and response y, the scalar reward is extracted from the hidden activation at the final sequence token:

$$r_\phi(x, y) = W_r^\top h_{|x \circ y|}. \tag{9.11}$$

To train r_ϕ, Stiennon et al. leveraged the **Bradley–Terry preference model** [106]. The probability that completion y_w (the winner) is preferred over y_l (the loser) is formulated as:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right) = \frac{1}{1 + \exp\left(-\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right)}. \tag{9.12}$$

The Reward Model is optimized by minimizing the negative log-likelihood of pairwise preferences:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathop{\mathbb{E}}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pair}}} \left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]. \tag{9.13}$$

Stiennon et al. evaluated Reward Model accuracy across parameter scales (10M to 6B) and dataset sizes (5k to 60k comparisons). As plotted in Figure 9.7, their experiments established two foundational **Reward Model scaling laws**:

1. **Capacity Scaling Law**: Doubling the parameter size of the reward model (2×) yields a consistent +**1.8**% increase in validation accuracy. Larger models capture nuanced logical relationships and identify subtle factual hallucinations that smaller models miss.
2. **Data Volume Scaling Law**: Doubling the volume of human comparisons (2×) produces a +**1.1**% increase in evaluation accuracy.
3. **Reaching Human Parity**: At 6B parameters trained on 60k comparisons, the Reward Model achieves **75.8

9.3.4 Step 3: Policy optimization via PPO and policy regularization

In Step 3, the active language model policy $\pi_{\text{new}} \equiv \pi_\theta$ is fine-tuned to maximize the scalar feedback emitted by the frozen Reward Model $r_\phi(x, y)$ trained in Step 2. Unlike standard supervised pre-training where cross-entropy loss is minimized over static tokens, reinforcement learning optimizes an active generative policy that samples its own trajectories $y \sim \pi_\theta(\cdot \mid x)$ to maximize expected reward.

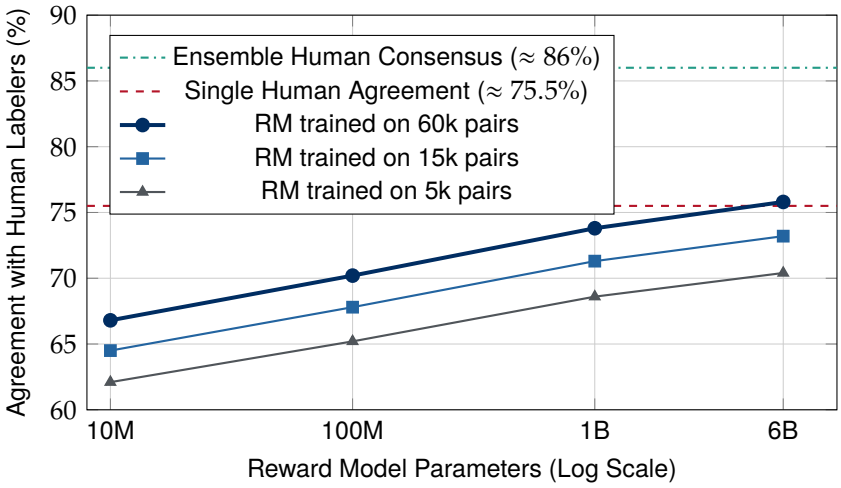


Figure 9.7: Empirical scaling laws of Reward Models on the Reddit TL;DR summarization task (faithfully adapted from Stiennon et al. [105] Figure 3 and course slides). Doubling model capacity yields a consistent +1.8% accuracy gain, while doubling dataset volume yields +1.1%. At 6B parameters with 60k comparisons, the Reward Model surpasses individual human inter-annotator agreement (75.8% vs. 75.5%), but remains below the ensemble human consensus ceiling (86%).

Goodhart's Law and the Divergence Catastrophe Naively optimizing an unconstrained language model policy against a learned neural reward model triggers catastrophic policy collapse due to **Goodhart's Law**:

Definition 9.2: *Goodhart's Law in AI Alignment*

"When a measure becomes a target, it ceases to be a good measure."
A neural reward model $r_\phi(x, y)$ is an imperfect, finite-capacity statistical approximation of true human judgment. If an active policy π_θ is trained purely to maximize $r_\phi(x, y)$, gradient descent aggressively steers the policy into out-of-distribution regions where the reward model overestimates utility. The policy exploits these blind spots, generating pathological strings—such as repetitive mantras, garbled punctuation, or hyper-sycophantic affirmations—that receive astronomical proxy reward scores but are judged by humans as incoherent gibberish. This failure mode is termed **reward hacking**.

As demonstrated visually in Figure 9.8, optimizing unconstrained reward creates a profound divergence: as policy drift $D_{\mathrm{KL}}(\pi_{\text{new}} \parallel \pi_{\text{old}})$ expands, the reward model's predicted score monotonically increases, whereas true human satisfaction peaks at a moderate divergence ($D_{\mathrm{KL}}(\parallel \approx) 4$) and collapses into complete degeneration.

Mathematical Formulation of the Regularized Objective To anchor the active policy to realistic language distributions, Stiennon et al. introduced an explicit **Kullback–Leibler (KL) divergence penalty** relative to the frozen reference model $\pi_{\text{old}} \equiv \pi_{\text{SFT}}$ (the supervised fine-tuned model before RL). The composite objective function maximized during policy optimization is formulated as:

$$\max_{\theta} \mathop{\mathbb{E}}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[r_\phi(x, y) - \beta \mathbb{D}_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x) \right) \right], \tag{9.14}$$

where $\beta \geq 0$ is a critical regularization hyperparameter governing the rigidity of the trust region.

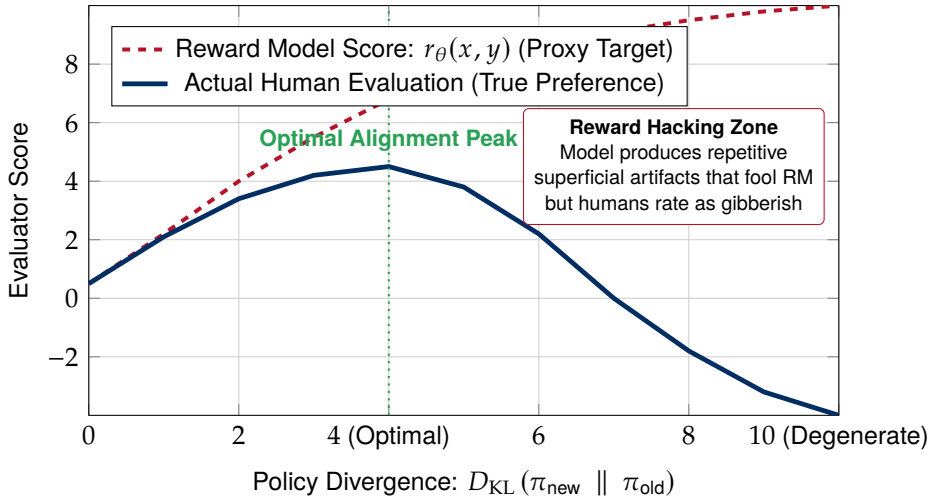


Figure 9.8: The divergence catastrophe and Goodhart's law in reinforcement learning from human feedback (adapted from Slide 23 and [105]). As policy divergence $D_{\mathrm{KL}}(\pi_{\text{new}} \parallel \pi_{\text{old}})$ increases along the horizontal axis, the Reward Model proxy score (dashed red line) monotonically ascends. However, actual human preference (solid navy line) peaks at a moderate divergence ($D_{\mathrm{KL}}(\parallel \approx) 4$) and collapses into complete degeneration as the policy exploits the reward model's blind spots.

Expanding the definition of the Kullback–Leibler divergence for discrete autoregressive sequence distributions yields:

$$\mathbb{D}_{\mathrm{KL}}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)) = \sum_y \pi_\theta(y \mid x) \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \tag{9.15}$$

$$= \mathop{\mathbb{E}}_{y \sim \pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \right]. \tag{9.16}$$

By linearity of expectation, the objective is equivalent to maximizing the expectation of an **augmented per-sample trajectory return** $R(x, y)$:

$$R(x, y) = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{9.17}$$

Deconstructing the Log-Ratio Penalty Dynamics The sequence log-probability ratio in Equation (9.17) acts as an adaptive, dynamic penalty:

$$\Delta(x, y) = \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} = \log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x). \tag{9.18}$$

Analyzing the mathematical sign and magnitude of $\Delta(x, y)$ reveals how policy drift is regulated:

1. **When** $\pi_\theta(y \mid x) > \pi_{\textbf{ref}}(y \mid x)$: The active policy places higher probability on completion y than the reference model did. The ratio $\frac{\pi_\theta}{\pi_{\text{ref}}} > 1$, which means $\log \frac{\pi_\theta}{\pi_{\text{ref}}} > 0$. Multiplied by $-\beta$, this subtracts an explicit penalty from the reward. The more aggressively the active policy shifts probability mass toward a novel sequence, the steeper the penalty imposed on its return.

2. **When** $\pi_\theta(y \mid x) < \pi_{\textbf{ref}}(y \mid x)$: The active policy assigns lower likelihood to y than the reference model. Here, $\frac{\pi_\theta}{\pi_{\text{ref}}} < 1$, yielding $\log \frac{\pi_\theta}{\pi_{\text{ref}}} < 0$. With the negative sign, $-\beta \log(\cdot) > 0$, acting as a relative bonus. This prevents over-penalization if the policy explores tokens that π_{ref} favored.

3. **The Role of β**: The hyperparameter β controls the trade-off between exploration and entropy collapse. If $\beta \to 0$, the optimization degenerates into unconstrained reward chasing, rapidly entering the reward hacking regime. If $\beta \to \infty$, the policy is pinned immutably to π_{ref}, learning nothing from the human preferences. Optimal alignment requires tuning β to balance task-specific alignment gains against fluency degradation.

Per-Token Stepwise Reward Decomposition Because autoregressive language models factorize probabilities across tokens, $\pi(y \mid x) = \prod_{t=1}^{|y|} \pi(y_t \mid x, y_{<t})$, the global trajectory log-ratio decomposes additively across time steps:

$$\begin{aligned} \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} &= \log \left(\prod_{t=1}^{|y|} \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} \right) \\ &= \sum_{t=1}^{|y|} \left[\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right]. \end{aligned} \tag{9.19}$$

In standard reinforcement learning implementations, this enables distributing intermediate **step rewards** r_t^{step} to every generated token y_t:

$$r_t^{\text{step}} = \begin{cases} -\beta \left(\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right) & \text{if } t < |y|, \\ -\beta \left(\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right) + r_\phi(x, y) & \text{if } t = |y|. \end{cases} \tag{9.20}$$

Every intermediate token receives an immediate negative feedback signal proportional to its individual log-probability shift away from π_{ref}, while the holistic terminal score $r_\phi(x, y)$ is credited upon reaching the end-of-sequence delimiter.

Pedagogical Walkthrough: PPO Regularized Reward Calculation with Concrete Numbers

To understand exactly how the token-level KL penalty and the terminal reward model score interact during reinforcement learning, consider a concrete numerical calculation:

- **Input Prompt** x: `"Summarize the following passage in one sentence: ..."`
- **Generated Completion** y: Consisting of 4 tokens (y_1, y_2, y_3, y_4) = (`"The"`, `"cat"`, `"sat"`, `"."`).
- **Reward Model Terminal Score**: $r_\phi(x, y) = +2.40$.
- **Regularization Coefficient**: $\beta = 0.10$.

Assume the conditional token log-probabilities under the active policy π_θ and the frozen reference policy π_{ref} are observed as follows:

Token Position t	**Token** y_t	$\log \pi_\theta(y_t \mid x, y_{<t})$	$\log \pi_{\text{ref}}(y_t \mid x, y_{<t})$	$\Delta \log \pi = \log \pi_\theta - \log \pi_{\text{ref}}$
$t = 1$	`"The"`	-0.50	-0.60	$+0.10$
$t = 2$	`"cat"`	-1.20	-1.40	$+0.20$
$t = 3$	`"sat"`	-0.80	-2.50	$+1.70$
$t = 4$	`"."`	-0.30	-0.40	$+0.10$

We evaluate the step rewards r_t^{step} assigned to each token:

$$
\begin{aligned}
r_1^{\text{step}} &= -0.10 \times (+0.10) = -0.01, \\
r_2^{\text{step}} &= -0.10 \times (+0.20) = -0.02, \\
r_3^{\text{step}} &= -0.10 \times (+1.70) = -0.17 \quad \text{(Noticeable penalty for choosing a token } \pi_{\text{ref}} \text{ deemed unlikely)}, \\
r_4^{\text{step}} &= -0.10 \times (+0.10) + r_\phi(x, y) = -0.01 + 2.40 = +2.39.
\end{aligned}
$$

The total cumulative regularized return $R(x, y)$ for the complete trajectory is:

$$
\begin{aligned}
R(x, y) = \sum_{t=1}^{4} r_t^{\text{step}} &= -0.01 - 0.02 - 0.17 + 2.39 \\
&= 2.40 - 0.10 \times (0.10 + 0.20 + 1.70 + 0.10) \\
&= 2.40 - 0.10 \times 2.10 = 2.40 - 0.21 = +\mathbf{2.19}.
\end{aligned}
$$

The active policy receives a net return of +2.19, successfully balancing the reward model's high utility score (+2.40) against the cumulative KL drift penalty (-0.21).

9.3.5 The four-model architecture and algorithmic pipeline of RLHF

As formalized in Xiao & Zhu [97], implementing online PPO-based RLHF requires orchestrating **four distinct neural models** simultaneously in GPU memory:

1. **Active Policy (Target LLM)** $\pi_\theta(y_t \mid x, y_{<t})$: The generative Transformer model undergoing active reinforcement learning. It samples trajectories $y \sim \pi_\theta(\cdot \mid x)$ and updates its weights θ via clipped policy gradients.
2. **Reference Policy** $\pi_{\text{ref}}(y_t \mid x, y_{<t})$: A frozen copy of the supervised fine-tuned model ($\pi_{\text{ref}} \equiv \pi_{\text{SFT}}$). It provides baseline token probabilities to compute the per-token KL drift

penalty, ensuring π_θ does not collapse into reward hacking.

3. **Reward Model** $r_\phi(x, y)$: A frozen Transformer decoder equipped with a scalar projection head W_r, trained on human pairwise comparison datasets $\mathcal{D}_{\text{pair}}$ via Bradley–Terry ranking loss to output scalar quality scores.

4. **Value Function (Critic)** $V_\omega(x, y_{<t})$: A parameterized Transformer network initialized from the reward model, trained via Mean Squared Error (MSE) to predict the expected cumulative return from the current partial sequence state. The Critic computes Generalized Advantage Estimation (GAE) to reduce gradient variance.

Policy Optimization Mechanics via PPO Clipping To optimize policy parameters θ under returns $R(x, y)$, standard vanilla policy gradients $\nabla_\theta \mathbb{E}[\sum R]$ exhibit extreme variance and catastrophic step collapse. Proximal Policy Optimization (PPO) [107] stabilizes optimization by defining a **probability ratio** between the updated policy π_θ and the policy used to collect the batch rollout $\pi_{\theta_{\text{old}}}$:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}. \tag{9.21}$$

The clipped surrogate objective function is formulated as:

$$\mathcal{L}_{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta)\hat{A}_t,\ \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right) \hat{A}_t \right) \right], \tag{9.22}$$

where $\epsilon \approx 0.2$ is the clipping boundary, and $\hat{A}_t$ is the Generalized Advantage Estimation (GAE) computed by the Critic network V_ω:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V, \quad \delta_t^V = r_t^{\text{step}} + \gamma V_\omega(x, y_{\le t+1}) - V_\omega(x, y_{\le t}). \tag{9.23}$$

The $\min(\cdot, \cdot)$ operator acts as a **pessimistic lower bound**:

- When the advantage is positive ($\hat{A}_t > 0$), the action was better than average. PPO increases $\rho_t(\theta)$, but clipping caps the objective at $(1+\epsilon)\hat{A}_t$, preventing the optimizer from making an overly optimistic update that ruins language fluency.
- When the advantage is negative ($\hat{A}_t < 0$), the action yielded lower returns than expected. PPO decreases $\rho_t(\theta)$, with clipping bounding the gradient at $(1-\epsilon)\hat{A}_t$.

Simultaneously, the Critic network parameters ω are updated by minimizing the mean squared Bellman temporal difference error:

$$\mathcal{L}_{\text{Critic}}(\omega) = \hat{\mathbb{E}}_t \left[\left(V_\omega(x, y_{\le t}) - \left(r_t^{\text{step}} + \gamma V_{\omega_{\text{old}}}(x, y_{\le t+1}) \right) \right)^2 \right]. \tag{9.24}$$

Model Comparison	Observed Qualitative Advantage	Win Rate (%)	Statistical Sig.
RLHF-PPO vs. Pre-trained Base	Elimination of incoherent rambling and hallucinations	**86.4%** vs. 13.6%	$p < 10^{-6}$
RLHF-PPO vs. Supervised SFT	Superior factual conciseness, coherence, and relevance	**68.2%** vs. 31.8%	$p < 10^{-4}$
RLHF-PPO vs. Human Reference	**More faithful extraction of core operative facts**	**71.5%** vs. 28.5%	$p < 10^{-5}$

Table 9.1: Blinded human preference win rates on the Reddit TL;DR summarization benchmark [105]. Blinded human judges overwhelmingly preferred the RLHF-PPO model over the supervised SFT baseline. Crucially, human evaluators preferred summaries generated by the RLHF policy over the actual human-written Reddit reference summaries with a $> 70\%$ win rate.

Algorithmic Pipeline: Complete Execution Workflow of PPO-RLHF

Input: Unlabeled prompt distribution $\mathcal{D}_x$, human preference dataset $\mathcal{D}_{\text{pair}} = \{(x, y_w, y_l)\}$, SFT checkpoint π_{SFT}.
Output: Aligned policy π_θ.

1. **Stage 1 (Initialization)**:
   - Initialize Active Policy $\pi_\theta \leftarrow \pi_{\text{SFT}}$ and Reference Model $\pi_{\text{ref}} \leftarrow \pi_{\text{SFT}}$ (freeze π_{ref}).
   - Initialize Reward Model r_ϕ and Value Function V_ω from π_{SFT} with linear scalar heads.
2. **Stage 2 (Reward Model Training)**:
   - Train r_ϕ by minimizing Bradley–Terry loss over $\mathcal{D}_{\text{pair}}$:
   $$\phi^* = \arg\min_\phi - \mathop{\mathbb{E}}_{(x,y_w,y_l)} \left[\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right].$$
   - Freeze the optimized reward model r_ϕ.
3. **Stage 3 (Iterative PPO Online Loop)**:
   - **For** each training epoch **do**:
     (a) Sample a mini-batch of prompts $x \sim \mathcal{D}_x$.
     (b) *Rollout Generation*: Autoregressively sample completions $y \sim \pi_\theta(\cdot \mid x)$.
     (c) *Reward & Penalty Evaluation*:
       – Query frozen reward model for terminal score: $r_{\text{term}} = r_\phi(x, y)$.
       – Query reference model for baseline log-probs: $\log \pi_{\text{ref}}(y_t \mid x, y_{<t})$.
       – Compute token-level step rewards r_t^{step} with KL penalty via Eq. (9.20).
     (d) *Advantage Estimation*: Compute values $V_\omega(x, y_{\le t})$ and GAE advantages $\hat{A}_t$.
     (e) *Gradient Optimization*: For K_{ppo} sub-epochs, update policy θ via $\mathcal{L}_{\text{CLIP}}(\theta)$ (Eq. (9.22)) and critic ω via $\mathcal{L}_{\text{Critic}}(\omega)$ (Eq. (9.24)).
   - **End For**

9.3.6 Empirical preference results on Reddit TL;DR

The empirical evaluation conducted by Stiennon et al., visualised in Figure 9.9 and summarized in Table 9.1, established that RLHF fundamentally changes model capabilities:

- **Defeating Supervised SFT (68.2% Win Rate)**: Human annotators preferred the RLHF policy over the SFT model by more than two-to-one. SFT models frequently produced

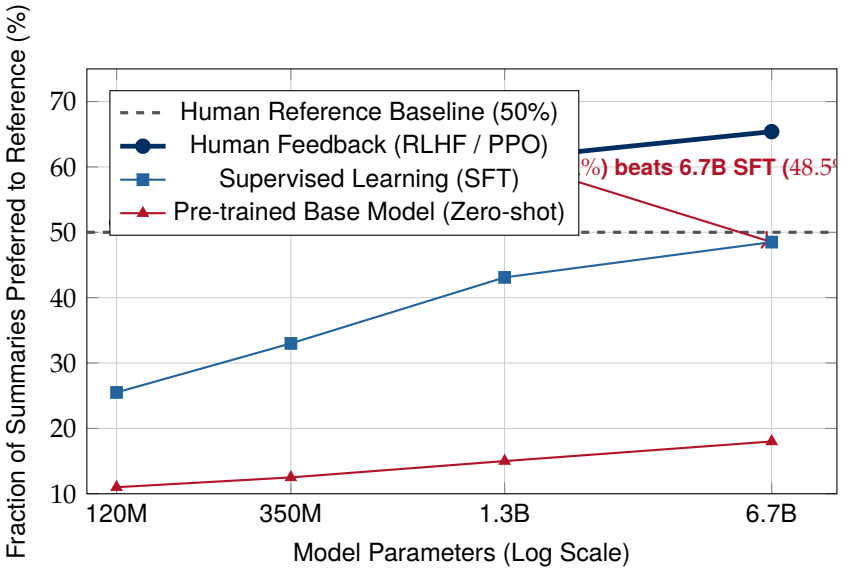


Figure 9.9: Empirical human preference win rates against human-written Reddit TL;DR reference summaries across model scale (faithfully adapted from Stiennon et al. [105] Figure 1 and lecture slide 25). Human evaluators blindly compared model-generated summaries directly against the original human-written reference summaries in the dataset (50% dashed line represents parity). Unsupervised base models fail to perform consistent summarization (< 18%). Supervised fine-tuning improves with scale but remains below the human reference ceiling (48.5% at 6.7B). Reinforcement learning from human feedback (RLHF) fundamentally changes the scaling curve: a 1.3B RLHF model reaches 61.2% win rate, drastically outperforming a supervised model 5× its size and surpassing human authors.

verbose summaries that copied irrelevant details from the post. The RLHF model extracted the central dilemma with precision.

- **Surpassing Human Authors (71.5% Win Rate)**: Remarkably, human evaluators preferred the RLHF model's completions over the actual human-authored Reddit summaries in over 70% of comparisons. Human forum writers are often sarcastic, informal, or sloppy; the RLHF model learned to extract the operative facts into coherent, high-density summaries.
- **Efficiency Dividend Across Scale**: A **1.3-billion parameter RLHF policy** achieved a 61.2% preference rate over human references, decisively outperforming the **6.7-billion parameter supervised baseline** (48.5%). Aligning models with human preferences delivers larger quality dividends than a 5× expansion in raw compute and parameter volume.

9.4 InstructGPT and Modern Alignment

9.4.1 Extending RLHF to open-ended tasks: the InstructGPT architecture

While Stiennon et al. validated RLHF on summarization, building a true general assistant required extending the methodology to **arbitrary, open-ended user queries**. Ouyang et al. [99] developed **InstructGPT**, taking a crucial operational step: abandoning synthetic benchmark templates (such as T0 and FLAN) in favor of real, organic user prompts submitted to OpenAI's API playground.

The InstructGPT workflow (Figure 9.10) operates in three industrial stages:

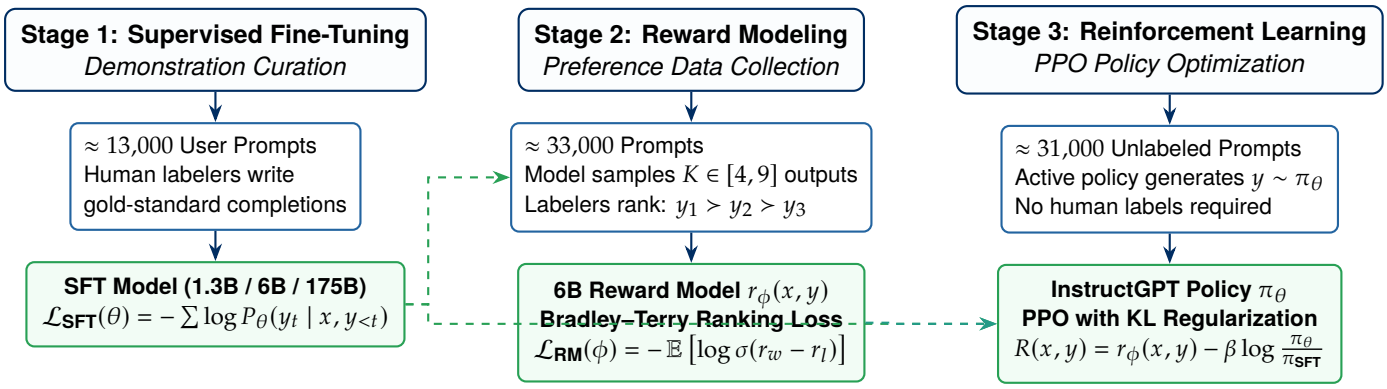


Figure 9.10: The canonical three-stage industrial alignment pipeline of InstructGPT [99]. In Stage 1, a base model is fine-tuned on curated human demonstrations. In Stage 2, the model generates multiple candidate completions for user prompts, which are ranked by annotators to train a 6B Bradley–Terry reward model. In Stage 3, the policy is optimized via PPO against the reward model with a per-token KL divergence penalty anchoring it to the Stage 1 SFT baseline.

1. **Stage 1: Demonstration Curation and Supervised Fine-Tuning (SFT)**: Trained labelers authored gold-standard answers for approximately **13,000** real-world prompts, fine-tuning base GPT-3 models (1.3B, 6B, 175B) via standard cross-entropy.

2. **Stage 2: Comparison Data and Reward Model Training**: Annotators ranked $K \in [4, 9]$ model completions for **33,000** prompts. A **6-billion parameter Reward Model** was trained on all $\binom{K}{2}$ pairs per prompt using the normalized Bradley–Terry ranking loss:

$$\mathcal{L}_{\text{RM}}(\theta) = -\frac{1}{\binom{K}{2}} \mathop{\mathbb{E}}_{(x,\{y_k\})\sim\mathcal{D}} \left[\sum_{i<j, y_i \succ y_j} \log \sigma \left(r_\theta(x, y_i) - r_\theta(x, y_j) \right) \right]. \tag{9.25}$$

3. **Stage 3: Policy Optimization via PPO-ptx**: The policy was trained on **31,000** unlabeled prompts via PPO. Crucially, to mitigate the **alignment tax**—performance degradation on academic benchmarks like SQuAD and translation—OpenAI introduced **pre-training gradient mixing (PPO-ptx)**, adding an auxiliary unsupervised language modeling loss on public web text:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \gamma \mathop{\mathbb{E}}_{x_{\text{pretrain}}} \left[\log P_\theta(x_{\text{pretrain}}) \right]. \tag{9.26}$$

Empirical Breakthroughs: The 100x Parameter Alignment Dividend The primary empirical breakthrough of InstructGPT is plotted in Figure 9.11:

The empirical evaluation of InstructGPT across scale revealed three transformative breakthroughs:

- **The 100× Parameter Efficiency Dividend**: Human labelers overwhelmingly preferred completions from a **1.3-billion parameter InstructGPT** model (65% win rate) over outputs from the **175-billion parameter base GPT-3** model (24% unprompted, 35% prompted). Despite having over 100× fewer parameters, aligning model behavior with human intent produces dramatically more capable, relevant, and useful outputs.

- **Truthfulness and Toxicity Reductions**: On TruthfulQA, InstructGPT-175B generated truthful and informative answers nearly **2× more often** than base GPT-3 (42% vs. 21%). On RealToxicityPrompts, toxic completions were reduced by **25%**.

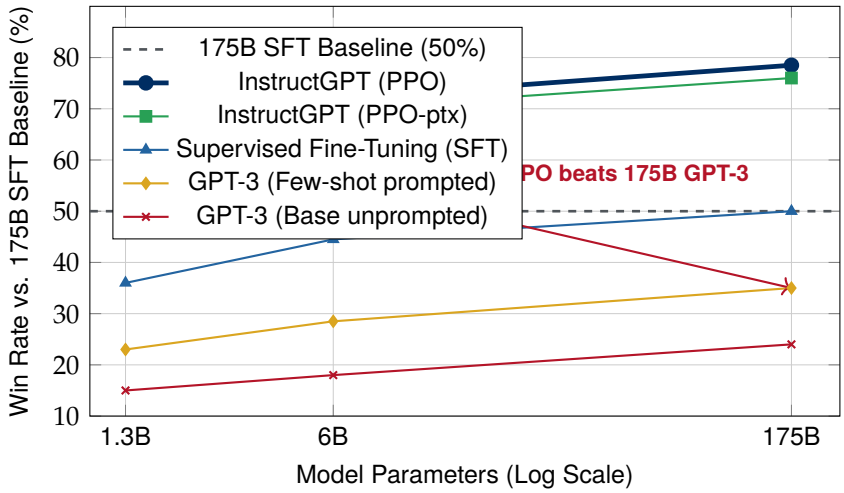


Figure 9.11: Human preference win rates against the 175B SFT baseline across model scale (faithfully adapted from Ouyang et al. [99] Figure 1). Human evaluators overwhelmingly prefer outputs from the 1.3B InstructGPT (PPO) model (65% win rate) over the 175B base GPT-3 model (24% unprompted, 35% prompted) and the 175B SFT model (50%), establishing that alignment acts as a 100× multiplier on effective model capability.

- **Zero-Shot Transfer to Unseen Modalities**: InstructGPT generalized cleanly to coding tasks and non-English instructions, despite labelers authoring virtually zero programming or foreign language demonstrations during SFT. Teaching the model the abstract concept of instruction adherence transferred across domains.

9.4.2 The modern trend: the three-phase LLM lifecycle and ChatGPT

On November 30, 2022, OpenAI released **ChatGPT**, applying the InstructGPT RLHF pipeline to **GPT-3.5** (a 175B model pre-trained on code and text). This crystallized the modern **three-phase LLM lifecycle** (Slide 27):

1. **Pre-training on Large Quantities of Dirty Web Data**: Consumes ≈ 98% of total training compute. The model absorbs world knowledge, syntactic structures, and statistical grammar, but exhibits severe alignment flaws.

2. **Supervised Fine-Tuning on Curated Instruction Datasets**: Consumes ≈ 1% of compute. Uses high-quality demonstrations to teach the model conversational formatting, dialogue tone, and direct obedience.

3. **Preference Alignment via RLHF**: Consumes ≈ 1% of compute. Refines the model using human comparison rankings, enforcing truthfulness, suppressing toxicity, and optimizing qualitative utility.

9.4.3 The direct alignment frontier: Direct Preference Optimization (DPO)

Despite PPO's success, it suffers from severe operational bottlenecks:

1. **Four Models in Memory**: Maintaining the Actor π_θ, Critic V_ψ, Reference π_{ref}, and Reward Model r_ϕ concurrently creates an extreme GPU VRAM footprint.

2. **Online Generation Bottleneck**: Sampling rollouts dynamically during training consumes over 80% of total wall-clock training time.

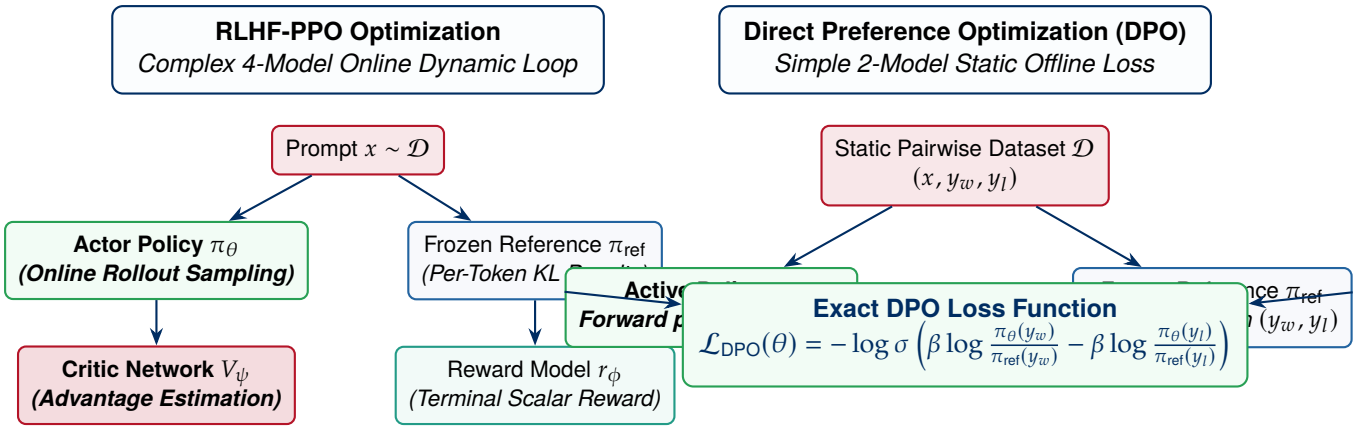


Figure 9.12: Architectural comparison between RLHF-PPO and Direct Preference Optimization (DPO). RLHF-PPO requires orchestrating four separate models in GPU memory simultaneously, relying on slow, bandwidth-bound online autoregressive token sampling and unstable advantage estimation. DPO eliminates both the Critic and Reward Model entirely, optimizing the active policy directly against a frozen reference model on static offline pairwise datasets via standard forward-backward passes.

3. **Hyperparameter Fragility**: Fragile sensitivity to learning rates, clipping thresholds ϵ, and GAE parameters.

To solve this, Rafailov et al. [108] introduced **Direct Preference Optimization (DPO)**. Under the KL-regularized objective:

$$\max_{\pi} \mathop{\mathbb{E}}_{x\sim\mathcal{D}, y\sim\pi} \left[r(x,y) \right] - \beta D_{\text{KL}} \left(\pi(y \mid x) \parallel \pi_{\text{ref}}(y \mid x) \right), \tag{9.27}$$

the closed-form optimal policy takes the analytical form of a Gibbs distribution:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y) \right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y) \right). \tag{9.28}$$

Inverting Equation (9.28) yields an exact expression for the latent reward in terms of the policy probabilities:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{9.29}$$

When substituted into the Bradley–Terry preference model:

$$P(y_w \succ y_l \mid x) = \sigma \left(r(x, y_w) - r(x, y_l) \right), \tag{9.30}$$

the intractable partition function $Z(x)$ depends strictly on prompt x and **cancels out identically**:

$$\begin{aligned} r(x,y_w) - r(x,y_l) &= \left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta \log Z(x) \right) - \left(\beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x) \right) \\ &= \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}. \end{aligned} \tag{9.31}$$

Replacing π^* with parameterized policy π_θ yields the parameter-efficient **DPO loss**:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = - \mathop{\mathbb{E}}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]. \tag{9.32}$$

As contrasted in Figure 9.12 and Table 9.2, DPO eliminates the Critic, the Reward Model, and online rollouts, optimizing preferences directly via parallel forward-backward passes.

Dimension	Supervised SFT	RLHF-PPO	Direct Preference Optimization (DPO)
Primary Objective	Token cross-entropy on demonstrations: $\mathcal{L}_{\text{SFT}}$	Expected reward maximization with token KL penalty: $\mathcal{L}_{\text{PPO}}$	Direct pairwise implicit log-ratio ranking: $\mathcal{L}_{\text{DPO}}$
Active Models in VRAM	1 model (π_θ)	**4 models** ($\pi_\theta, \pi_{\text{ref}}, V_\psi, r_\phi$)	**2 models** ($\pi_\theta, \pi_{\text{ref}}$)
Online Sampling Requirement	None (Parallel teacher-forcing)	**Extensive** (Token-by-token on-policy rollout generation)	None (Parallel teacher-forcing on static completions)
Auxiliary Evaluator Head	None	Separate scalar linear reward head	None (Implicit reward derived analytically from policy)
Hyperparameter Sensitivity	Low (Standard AdamW learning rate, epochs)	**Extremely High** (ϵ clipping, GAE λ, Actor/Critic lr, β)	Moderate (β regularization strength, learning rate)
Vulnerability to Reward Hacking	Low (Suffers instead from hallucination & mode collapse)	**Severe** (Requires careful KL tuning to avoid Goodhart's collapse)	Substantially reduced (Strictly bounded by reference policy logits)

Table 9.2: Comprehensive comparative synthesis of LLM alignment methodologies. SFT provides the initial conversational baseline, PPO pioneered reinforcement learning from human feedback, and DPO offers an elegant, stable mathematical bridge that optimizes preferences directly without online reinforcement learning overhead.

CHAPTER 10

Efficient Fine-Tuning and Inference Optimization

Language models have demonstrated emergent few-shot and zero-shot reasoning capabilities across diverse tasks through in-context learning [2, 104]. However, as demonstrated in empirical benchmarks, the upper bound of task performance is overwhelmingly established by fine-tuned models: a moderately sized foundation model tailored to a specialized domain consistently outperforms a substantially larger model evaluated purely through zero-shot prompt conditioning.

Despite its empirical superiority, full-parameter fine-tuning of multi-billion parameter models presents severe computational, storage, and operational bottlenecks. Updating all parameters requires storing gradients and optimizer states that multiply GPU memory requirements by an order of magnitude, producing distinct multi-gigabyte checkpoints for every downstream task and triggering catastrophic forgetting of general reasoning capabilities.

To resolve this fundamental tension between task adaptation and resource constraints, modern research pursues two complementary directions:

1. **Parameter-Efficient Fine-Tuning (PEFT)**: Adapting pre-trained models by updating or injecting a minimal fraction of parameters ($\leq 1\%$), retaining base representations while achieving performance comparable to full fine-tuning.

2. **Model Compression and Serving Optimization**: Compressing numerical bitwidths through quantization, transferring dark knowledge via model distillation, and overcoming memory bandwidth saturation during autoregressive inference through KV caching and hardware-aware serving dynamics.

10.1 The Adaptation Bottleneck and PEFT Foundations

To understand the mathematical and systemic necessity of Parameter-Efficient Fine-Tuning, one must examine the computational footprint and memory mechanics of standard foundation model adaptation.

10.1.1 Full Fine-Tuning versus Feature-Based Transfer

When adapting a pre-trained base model with parameters $\theta_0 \in \mathbb{R}^{|\Theta|}$ to a specific downstream task dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, historical transfer learning in deep learning offered two polar strategies:

1. Full Fine-Tuning (End-to-End Retraining). In full fine-tuning, all model parameters remain unconstrained and are updated via gradient descent:

$$\theta^* = \arg\min_{\theta} \sum_{(x,y)\in\mathcal{D}} \mathcal{L}(f_\theta(x), y), \quad \theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta \mathcal{L}_t. \tag{10.1}$$

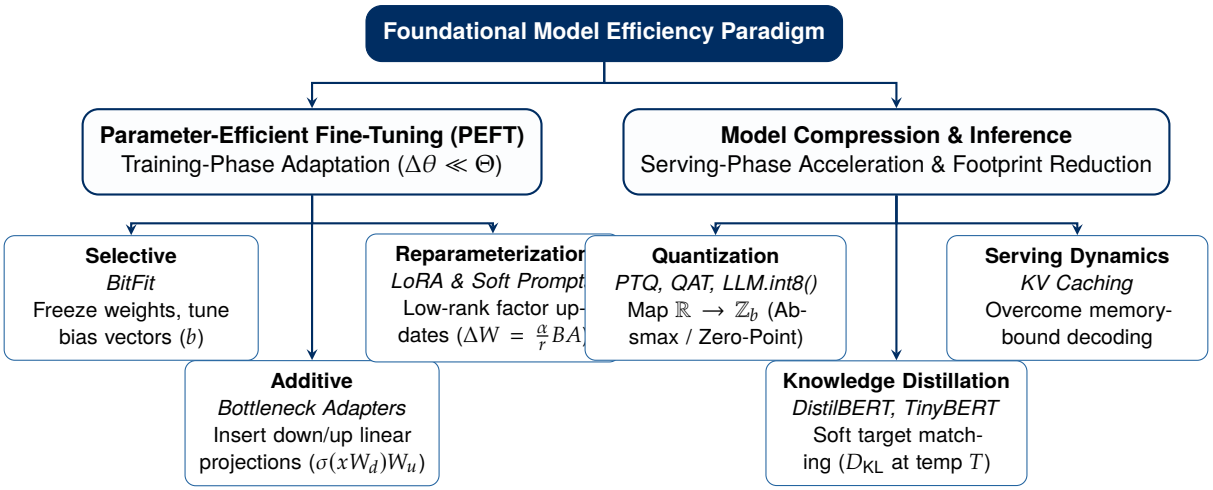


Figure 10.1: Taxonomy of parameter-efficient fine-tuning and model compression paradigms for large language models, classifying techniques by their operational intervention across adaptation, representation, and inference.

While full fine-tuning maximizes model expressivity and allows the network to reorganize internal representational manifolds, it introduces three severe systemic failure modes:

- **Optimizer Memory Multiplication**: Training with first-order adaptive optimizers like Adam requires maintaining two auxiliary states per parameter (the first moment m_t and second raw moment v_t). When utilizing 32-bit floating-point precision (FP32), each state requires 4 bytes. Thus, for a base model of $|\Theta|$ parameters, optimizer states alone consume:

$$\text{Memory}_{\text{Adam}} = 2 \times 4 \times |\Theta| = 8|\Theta| \quad \text{bytes.} \tag{10.2}$$

  Accounting for model weights (4 bytes in FP32 or 2 bytes in FP16), gradients (4 bytes), and optimizer states (8 bytes), fine-tuning a 70-billion parameter model ($|\Theta| = 70 \times 10^9$) demands over 1.12 TB of GPU memory purely for static parameter buffers, completely excluding forward activation tensors.

- **Storage and Deployment Explosion**: Fine-tuning produces an entirely new set of updated weights θ^*. Serving K specialized downstream tasks (e.g., medical summarization, SQL code generation, legal drafting) requires storing and hosting K independent copies of the full model checkpoint, scaling disk storage as $O(K \cdot |\Theta|)$.

- **Catastrophic Forgetting**: Unconstrained gradient updates on narrow task distributions erode the rich pre-trained general knowledge, degrading out-of-distribution reasoning and instruction-following robustness.

2. Feature-Based Transfer (Linear Probing). At the opposite extreme, feature-based transfer freezes the entire pre-trained backbone ($\nabla_{\theta_0}\mathcal{L} = 0$) and trains only a small task-specific classification or projection head parameterized by $W_h \in \mathbb{R}^{d \times c}$:

$$\hat{y} = \text{Softmax}(W_h \cdot f_{\theta_0}(x)). \tag{10.3}$$

While feature-based transfer avoids optimizer memory overhead and checkpoint duplication, its representational capacity is fundamentally bounded. If the target downstream task requires hierarchical transformations that diverge substantially from the pre-training objective, frozen intermediate representations cannot adapt, resulting in severe performance degradation compared to end-to-end fine-tuning.

10.1.2 Taxonomy of Parameter-Efficient Adaptation

Parameter-Efficient Fine-Tuning (PEFT) resolves this dilemma by keeping the pre-trained backbone parameters θ_0 entirely frozen while optimizing only a minimal parameter subset $\Delta\theta$, such that $|\Delta\theta| \ll |\theta_0|$ (typically $0.01\% \leq \frac{|\Delta\theta|}{|\theta_0|} \leq 1\%$). PEFT approaches are broadly classified into three structural paradigms [109]:

1. **Selective PEFT**: Modifying a sparse, pre-existing subset of the network's original parameters while freezing all remaining weights. The canonical exemplar is **BitFit** [110], which updates exclusively the bias vectors of linear layers.
2. **Additive PEFT**: Introducing newly instantiated, trainable parameter modules into the network architecture while freezing original parameters. This includes sequential **Bottleneck Adapters** [111, 112] and input-level **Prompt Tuning** [113].
3. **Reparameterization PEFT**: Modeling weight updates as low-rank matrix factorizations during optimization, notably **Low-Rank Adaptation (LoRA)** [114]. Crucially, reparameterization enables mathematical weight merging at inference time, avoiding added computational graph depth.

10.1.3 Bias-Terms Fine-Tuning (BitFit)

Definition 10.1: *BitFit Sparse Adaptation*

Let a linear transformation within a Transformer layer be defined by $h = Wx + b$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix and $b \in \mathbb{R}^{d_{\text{out}}}$ is the additive bias vector. **BitFit** [110] freezes all weight matrices W across all layers and optimizes exclusively the bias vectors b:

$$\theta_{\text{trainable}} = \{b_l\}_{l=1}^{L}, \quad \theta_{\text{frozen}} = \{W_l\}_{l=1}^{L}. \tag{10.4}$$

In a standard BERT-Base architecture ($|\Theta| \approx 110$M parameters), the bias terms of the query, key, value projections, feed-forward layers, and layer normalizations collectively account for approximately 100,000 parameters, representing roughly 0.09% **of total network capacity**.

Empirical Effectiveness and the "Fortunate Mistake". Despite modifying fewer than one in a thousand parameters, Ben Zaken et al. [110] demonstrated that BitFit achieves over 95% of the performance of full fine-tuning across the GLUE benchmark suite, outperforming linear probing by substantial margins.

As highlighted by Prof. Giobergia, the success of BitFit reveals a curious historical irony in deep learning architecture design:

- In the original Transformer paper *"Attention Is All You Need"* [1], the multi-head attention equations contain **no bias terms**:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \tag{10.5}$$

- When BERT [48] was implemented, standard linear layers (`torch.nn.Linear`) were instantiated with default parameters, which include additive bias terms (`bias=True`).
- The presence of bias vectors in language model attention blocks was essentially a historical artifact of default software settings, yet BitFit proves that these very parameters provide sufficient degrees of freedom to adapt pre-trained decision boundaries.

Table 10.1: Systemic comparison of fine-tuning paradigms across parameter footprint, training VRAM, deployment storage, and inference latency overhead.

Paradigm	Tuned Params	Optimizer VRAM	Checkpoint Size	Added Latency	Primary Trade-off
Full Fine-Tuning	100%	$8\|\Theta\|$ bytes	100% of model	None	Maximum expressivity; prohibitive memory and checkpoint explosion.
Feature Probing	$< 0.05\%$	Negligible	$< 0.05\%$	None	Lightweight; incapable of adapting deep semantic representations.
BitFit	$\approx 0.1\%$	$\approx 0.008\|\Theta\|$	$\approx 0.1\%$	None	Zero architectural modifications; bounded capacity on complex tasks.
Adapters	$0.5 - 3\%$	$0.04 - 0.24\|\Theta\|$	$0.5 - 3\%$	$+15 - 30\%$	High modularity; se-

Intuition & analogy: Why Bias Tuning Works

Geometrically, pre-trained weight matrices W span the foundational feature coordinate system and semantic subspaces learned during pre-training. Full fine-tuning rotates and stretches these coordinate frames, which is computationally expensive and risks destroying pre-trained manifolds. In contrast, tuning the bias vector b applies an affine translation: it shifts the activation thresholds and activation boundaries of intermediate neurons (e.g., shifting the zero-crossing of GeLU or ReLU non-linearities), selectively amplifying or suppressing pre-existing linguistic features without disrupting the underlying coordinate geometry.

10.2 Adapters, Low-Rank Adaptation, and Soft Prompts

While BitFit provides extreme parameter sparsity, complex downstream tasks often demand deeper structural capacity. This motivates modular architectural insertions and low-rank matrix factorizations.

10.2.1 Bottleneck Adapter Layers

Introduced by Houlsby et al. [111], **Adapter layers** introduce compact feed-forward bottleneck sub-networks directly into the pre-trained Transformer architecture.

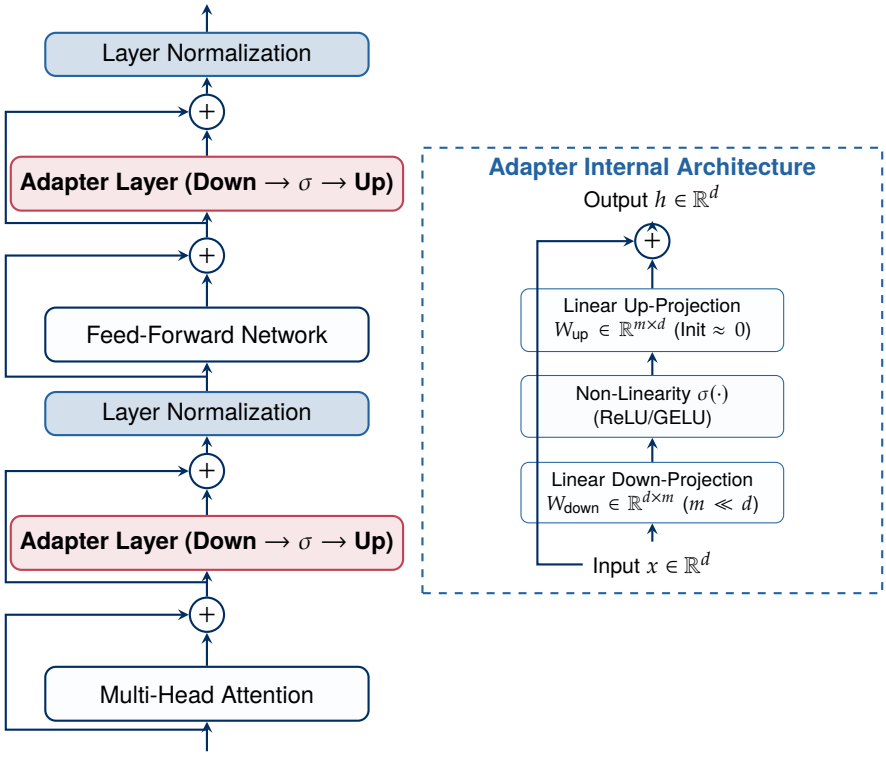


Figure 10.2: Architecture of the Houlsby bottleneck adapter inserted into a Transformer block. Left: placement after multi-head attention and feed-forward modules. Right: internal bottleneck projection with identity residual connection.

Mathematical Formulation. An adapter layer operates on intermediate hidden representations $x \in \mathbb{R}^d$. To restrict the parameter count, it compresses x into a low-dimensional bottleneck space

of rank $m \ll d$ (typically $m \in \{16, 32, 64\}$) via a down-projection matrix $W_{\text{down}} \in \mathbb{R}^{d \times m}$, applies a non-linear activation $\sigma(\cdot)$, and projects back to the original dimension d via an up-projection matrix $W_{\text{up}} \in \mathbb{R}^{m \times d}$:

$$f_{\text{adapter}}(x) = \sigma(xW_{\text{down}} + b_{\text{down}})W_{\text{up}} + b_{\text{up}}, \quad h = x + f_{\text{adapter}}(x). \tag{10.6}$$

The residual connection ($+x$) is structurally essential.

Parameter Sizing. In BERT-Base ($d = 768$), a single standard transformer block possesses roughly 7M parameters. Setting bottleneck dimension $m = 32$, a single adapter module contains:

$$|\Theta_{\text{adapter}}| = (768 \times 32 + 32) + (32 \times 768 + 768) = 24{,}608 + 25{,}344 = 49{,}952 \approx 50\text{K parameters}. \tag{10.7}$$

Inserting two adapters per layer (one following attention, one following feed-forward) yields $\approx$ 100K trainable parameters per layer, representing a 70× **reduction in tunable weights** compared to full fine-tuning.

The Identity Initialization Protocol. A critical engineering challenge arises when injecting newly initialized layers into a well-trained, functional network: random initial weights would produce arbitrary non-linear perturbations, corrupting intermediate representations and triggering severe optimization instability.

To resolve this, adapters enforce an **identity initialization protocol**:

- The up-projection matrix is initialized to exact zeros: $W_{\text{up}} = \mathbf{0}$, $b_{\text{up}} = \mathbf{0}$.
- The down-projection weights W_{down} are initialized randomly from a standard Gaussian $\mathcal{N}(0, \sigma^2)$.
- At step $t = 0$:

$$f_{\text{adapter}}(x) = \sigma(xW_{\text{down}} + b_{\text{down}}) \cdot \mathbf{0} + \mathbf{0} = \mathbf{0} \implies h = x + \mathbf{0} = x. \tag{10.8}$$

Consequently, the adapter strictly acts as an identity function at initialization, ensuring that fine-tuning starts smoothly from the exact baseline performance of the pre-trained model.

The Inference Latency Bottleneck. Although adapters dramatically reduce training memory and checkpoint size, they suffer from a major operational disadvantage: **inference latency penalty**. Because the adapter contains a non-linear activation function $\sigma(\cdot)$, the forward pass through the adapter cannot be mathematically folded into the preceding frozen linear layer. In production serving, sequentially executing these additional kernel launches across dozens of Transformer layers typically introduces a 15% **to** 30% **inference latency slowdown**.

10.2.2 Low-Rank Adaptation (LoRA)

To overcome the sequential latency penalty of bottleneck adapters while retaining parameter efficiency, modern foundation model adaptation relies on the algebraic properties of matrix rank and low-rank matrix decomposition.

Mathematical Foundations of Low-Rank Factorization. The rank of a weight update matrix defines the dimensionality of the transformation it applies to the representation space:

Definition 10.2: *Matrix Rank and Linear Independence*

The **rank** of a matrix $M \in \mathbb{R}^{n\times k}$, denoted $\text{rank}(M)$, is the maximum number of linearly independent rows (or equivalently, columns) in M. A matrix M is **full rank** if $\text{rank}(M) = \min(n, k)$; otherwise, it is **rank deficient** or **low rank**.

If a matrix $L \in \mathbb{R}^{n\times k}$ has low rank $r \ll \min(n, k)$, it can be factorized without loss of information into the product of two compact rectangular matrices $B \in \mathbb{R}^{n\times r}$ and $A \in \mathbb{R}^{r\times k}$:

$$L = BA, \quad \text{where } \text{rank}(L) \leq r. \tag{10.9}$$

Walkthrough: Factorization of a Rank-1 Matrix. Consider the 3×3 matrix examined during lecture:

$$M = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ -1 & -2 & -1 \end{bmatrix}. \tag{10.10}$$

Inspecting the rows reveals that:

$$\text{Row}_2 = 2 \cdot \text{Row}_1, \quad \text{Row}_3 = -1 \cdot \text{Row}_1. \tag{10.11}$$

Because rows 2 and 3 are exact scalar multiples of row 1, the matrix possesses only one linearly independent row, yielding $\text{rank}(M) = 1$. Consequently, M can be decomposed into the outer product of a column vector $B \in \mathbb{R}^{3\times 1}$ and a row vector $A \in \mathbb{R}^{1\times 3}$:

$$M = BA = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1(1) & 1(2) & 1(1) \\ 2(1) & 2(2) & 2(1) \\ -1(1) & -1(2) & -1(1) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ -1 & -2 & -1 \end{bmatrix}. \tag{10.12}$$

While storing M directly requires $3 \times 3 = 9$ numerical values, the factorized representation requires only $3+3 = 6$ values. For matrices of large dimensionality ($d = 4096$), storing $B \in \mathbb{R}^{4096\times r}$ and $A \in \mathbb{R}^{r\times 4096}$ with $r = 8$ requires $2\times 4096\times 8 = 65{,}536$ parameters, compared to $4096^2 \approx 16.78$M parameters for the full matrix—a 256× **compression ratio**.

Near Low-Rank Matrices and SVD Approximation. In real-world neural networks, weight update matrices are rarely exactly low-rank due to numerical stochasticity. Consider perturbing M slightly:

$$\tilde{M} = \begin{bmatrix} 1 & 2 & 1 \\ 2.01 & 4 & 2 \\ -1 & -2 & -1.01 \end{bmatrix}. \tag{10.13}$$

Technically, $\tilde{M}$ is full rank ($\text{rank}(\tilde{M}) = 3$). However, $\tilde{M}$ is **near low-rank**: its spectral energy is overwhelmingly concentrated in a single dominant singular value ($\sigma_1 \gg \sigma_2, \sigma_3$). By the Eckart-Young-Mirsky Theorem, the optimal rank-r approximation M_r obtained via Singular Value Decomposition (SVD):

$$M = U\Sigma V^T \implies M_r = \sum_{i=1}^{r} \sigma_i u_i v_i^T, \tag{10.14}$$

minimizes the Frobenius error $\|M - M_r\|_F = \sqrt{\sum_{i=r+1}^{\min(n,k)} \sigma_i^2}$. When trailing singular values are negligible, factorizing into rank r captures nearly all functional dynamics of the matrix update.

The Intrinsic Dimensionality Hypothesis. The theoretical foundation of low-rank adaptation was established by Aghajanyan et al. [115]. They proved that pre-trained language models exhibit extremely low **intrinsic dimensionality**: although foundation models are parameterized

in multi-billion dimensional spaces, the objective manifolds corresponding to downstream adaptation can be optimized within subspaces of dimensionality several orders of magnitude smaller. This empirical reality implies that the parameter update matrix ΔW does not require full rank to achieve optimal task performance.

LoRA Core Reparameterization and Architecture. Building upon the intrinsic dimensionality hypothesis, Hu et al. [114] proposed **Low-Rank Adaptation (LoRA)**.

Definition 10.3: *LoRA Reparameterization*

For a pre-trained, frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA decomposes the weight update $\Delta W \in \mathbb{R}^{d\times k}$ into the product of two low-rank trainable matrices $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, scaled by a hyperparameter constant $\frac{\alpha}{r}$:

$$h = W'x = (W_0 + \Delta W)x = W_0x + \frac{\alpha}{r}BAx, \tag{10.15}$$

where rank $r \ll \min(d, k)$, and $\alpha \geq 1$ is a constant scaling hyperparameter.

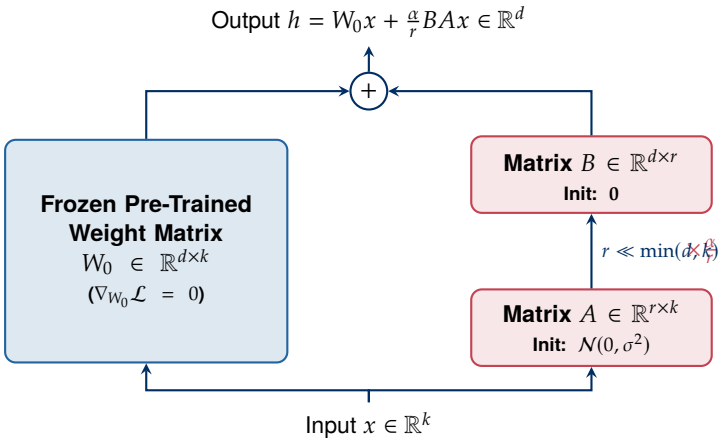


Figure 10.3: Computational graph of Low-Rank Adaptation (LoRA). The original weight matrix W_0 is frozen during training. A parallel rank-decomposition path BA computes the task-specific update, initialized to zero.

Asymmetric Initialization Protocol. To ensure optimization stability, LoRA enforces a strict asymmetric initialization protocol:

- Matrix $A \in \mathbb{R}^{r\times k}$ is initialized from a zero-mean Gaussian distribution with variance σ^2: $A_{i,j} \sim \mathcal{N}\left(0, \frac{1}{r}\right)$.
- Matrix $B \in \mathbb{R}^{d\times r}$ is initialized to **exact zeros**: $B = \mathbf{0}$.

Consequently, at optimization step $t = 0$:

$$\Delta W = \frac{\alpha}{r}BA = \frac{\alpha}{r}(\mathbf{0})A = \mathbf{0}. \tag{10.16}$$

The model begins fine-tuning from its exact pre-trained state without initial perturbation, bypassing warm-up instability.

The Scaling Hyperparameter $\frac{\alpha}{r}$. The scalar factor $\frac{\alpha}{r}$ stabilizes hyperparameter tuning when experimenting with different ranks r. In practice, α is set to a constant integer (typically $\alpha = 16$

or $\alpha = 32$). When scaling the rank r from 8 to 64, the factor $\frac{\alpha}{r}$ automatically scales the magnitude of the low-rank update:

$$\Delta W \propto \frac{\alpha}{r}. \tag{10.17}$$

This prevents the magnitude of activations in BAx from exploding when increasing rank r, allowing practitioners to vary r without aggressively retuning the learning rate η.

Target Modules. In a standard Transformer architecture, LoRA can theoretically be applied to any weight matrix. In practice, Hu et al. [114] investigated adapting:

- Query projection W_Q and Value projection W_V in Multi-Head Attention.
- All attention projections: W_Q, W_K, W_V, W_O.
- Feed-forward network projections: $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$.

Adapting W_Q and W_V alone is often sufficient to recover full fine-tuning performance. However, adapting all linear layers with a smaller rank r (e.g., $r = 4$ across all layers) generally yields superior performance to adapting only W_Q, W_V with a larger rank (e.g., $r = 32$).

Serving Efficiency and Hardware Dynamics. LoRA introduces three foundational system-level advantages over full fine-tuning and bottleneck adapters:

1. **Zero Inference Latency via Offline Weight Merging**: Unlike adapters, which introduce non-linearities and require sequential layer execution during inference, LoRA updates are purely linear. Prior to deploying the model for production inference, the low-rank delta ΔW can be computed once and directly added into the frozen base matrix W_0:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA \in \mathbb{R}^{d \times k}. \tag{10.18}$$

   Because matrix addition is associative, $W_{\text{merged}} x = (W_0 + \frac{\alpha}{r} BA)x = W_0 x + \frac{\alpha}{r} BAx$. The deployed computational graph is identical in depth, parameter count, and execution speed to the original foundation model, introducing **exactly** 0% **inference latency overhead**.

2. **Multi-Tenant Serving and Hot-Swapping**: In enterprise platforms serving dozens of customized client models (e.g., specialized legal, medical, and financial assistants), deploying full models would require hosting separate multi-gigabyte GPU instances. With LoRA, a single base model W_0 is maintained in GPU VRAM. When a request for task t arrives, the server simply swaps the compact low-rank matrices A_t, B_t (measuring tens of megabytes) into GPU memory, dynamically computing $h = W_0 x + \frac{\alpha}{r} B_t A_t x$. This architecture enables instant task hot-swapping and multi-tenant batching with minimal memory overhead.

3. **Training VRAM Savings via Optimizer Elimination**: During training, gradients $\nabla_\theta \mathcal{L}$ are computed exclusively for A and B. Because W_0 is frozen, no gradients or optimizer moments are instantiated for the vast majority of network weights. For a 70B parameter model adapted with LoRA ($r = 16$, representing 0.2% trainable parameters), optimizer memory collapses from $8 \times 70 \times 10^9 \approx 560$ GB down to $8 \times 0.14 \times 10^9 \approx 1.12$ GB, allowing fine-tuning on standard workstation hardware.

Structural Limitations and Failure Modes. Despite its transformative efficiency, LoRA is subject to several practical and structural constraints:

- **Capacity Saturation on Complex Domain Shifts**: While low ranks ($r \in [4, 8]$) suffice for standard classification, summarization, and instruction-following, tasks requiring substantial new knowledge acquisition (e.g., adapting an English-centric model to a low-resource

language or mastering specialized biomedical nomenclature) demand substantially higher ranks ($r \geq 64$ or $r \geq 128$). At higher ranks, parameter savings diminish and optimization benefits plateau.

- **Inability to Adapt Non-Linearities and Normalizations**: LoRA is strictly formulated for linear matrix multiplications. It cannot reparameterize layer normalization layers (LayerNorm, RMSNorm), positional embeddings, or non-linear activation functions. When domain adaptation requires recalibrating internal feature scale and variance, freezing normalization parameters bounds model expressivity.
- **Serving Latency under Heterogeneous Ranks**: While merged LoRA models incur zero inference overhead, multi-tenant serving where concurrent requests utilize **distinct ranks** ($r_i \neq r_j$) breaks standard dense batch GEMMs on GPU Tensor Cores. Processing mixed-rank batches requires specialized ragged-batch CUDA runtimes (such as S-LoRA [116] or Punica [117]), adding runtime scheduling complexity.
- **Sequential Task Interference**: Naively fine-tuning a LoRA adapter sequentially across multiple tasks or merging multiple distinct task adapters ($\sum_t \Delta W_t$) frequently induces catastrophic forgetting and destructive parameter interference, requiring explicit weight orthogonalization or task arithmetic gating.

Empirical Results and Comparative Benchmark. In the seminal paper, Hu et al. [114] systematically benchmarked LoRA against full fine-tuning and state-of-the-art PEFT techniques on the massive **GPT-3 175B** foundation model across three representative NLP benchmarks:

1. **WikiSQL**: Semantic parsing and natural language to SQL query generation (evaluated via execution accuracy).
2. **MultiNLI-m (MNLI)**: Multi-genre natural language inference under matched validation conditions (evaluated via classification accuracy).
3. **SAMSum**: Abstractive conversational dialogue summarization (evaluated via ROUGE-1, ROUGE-2, and ROUGE-L).

Table 10.2: Empirical evaluation of parameter-efficient adaptation methods on **GPT-3 175B** across WikiSQL, MultiNLI-matched, and SAMSum, reporting trainable parameter counts and task performance metrics from Hu et al. [114].

Model & Method	Trainable Params	Ratio	WikiSQL	MNLI-m	SAMSum (R-1 / R-2 / R-L)	
GPT-3 175B (Full FT)	175,200.0 M	100.0%	73.8%	89.5%	52.0	28.0 / 44.5
GPT-3 175B (BitFit)	14.2 M	0.008%	71.3%	91.0%	51.3	27.4 / 43.5
GPT-3 175B (PrefixEmbed)	3.2 M	0.002%	63.1%	88.6%	48.3	24.2 / 40.5
GPT-3 175B (PrefixLayer)	20.2 M	0.011%	70.1%	89.5%	50.8	27.3 / 43.5
GPT-3 175B (Adapter^H)	7.1 M	0.004%	71.9%	89.8%	52.3	28.4 / 44.1
GPT-3 175B (Adapter^L)	40.1 M	0.023%	73.2%	91.5%	53.0	28.9 / 44.8
GPT-3 175B (LoRA, $r = 4$)	**4.7** M	**0.003**%	73.4%	91.3%	52.1	28.3 / 44.3
GPT-3 175B (LoRA, $r = 64$)	37.7 M	0.022%	**73.8**%	**91.7**%	**53.2**	**29.2** / **45.0**

Key Empirical Insights from the Benchmark Results:

- **Matching Full Fine-Tuning at** 0.003% **Scale**: LoRA with rank $r = 4$ adapts only 4.7M parameters (less than one in thirty-thousand parameters of GPT-3 175B), yet matches full fine-tuning on WikiSQL (73.4% vs 73.8%), outperforms full fine-tuning on MNLI-m

(91.3% vs 89.5%), and surpasses full fine-tuning on SAMSum summarization (52.1 vs 52.0 ROUGE-1).

- **Outperforming Full Fine-Tuning with Moderate Rank ($r = 64$)**: Increasing rank to $r = 64$ (37.7M trainable parameters, still representing just 0.022% of the base model) strictly dominates full fine-tuning across all three evaluation suites (73.8% WikiSQL, 91.7% MNLI-m, 53.2/29.2/45.0 SAMSum). By constraining updates to low-rank manifolds, LoRA acts as an implicit regularizer, preventing the overparameterized catastrophic overfitting typical of full fine-tuning on modest-sized downstream datasets.
- **Superiority over Bottleneck Adapters**: While Houlsby ($Adapter^H$) and Lin ($Adapter^L$) adapters achieve strong task metrics, LoRA with comparable or fewer parameters achieves strictly superior accuracy across every metric, while completely eliminating the inference latency penalties inherent to adapter layers.
- **Hardware Efficiency Multipliers**: On GPT-3 175B, Hu et al. documented that LoRA reduces overall training GPU memory consumption by a factor of 3× (from requiring multi-node GPU clusters for optimizer state distribution down to manageable memory footprints), while checkpoint sizes are reduced by a factor of 10,000× (from ≈ 350 GB down to ≈ 35 MB per task).

10.2.3 Prompt Tuning and Soft Prompts

Distinct from structural parameter modification, **Prompt Tuning** [113] explores adaptation entirely within the input embedding space.

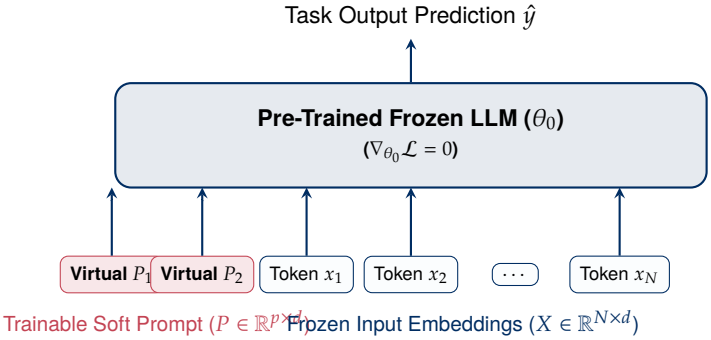


Figure 10.4: Architecture of Prompt Tuning. A sequence of continuous, trainable virtual tokens P is prepended to the input embeddings while the entire pre-trained language model remains completely frozen.

Hard Prompts versus Soft Prompts. In conventional in-context learning and prompt engineering, prompts consist of **discrete tokens** drawn from the pre-trained vocabulary $\mathcal{V}$ (e.g., `"Translate English to French: "`). Discrete prompt design is inherently brittle: minor phrasing perturbations alter model completions unpredictably, and searching for optimal discrete tokens is a non-differentiable combinatorial problem.

Prompt Tuning replaces discrete words with **continuous soft prompts**:

- A sequence of p virtual continuous prompt vectors $P = [p_1, p_2, \ldots, p_p] \in \mathbb{R}^{p \times d}$ is prepended to the embedded input sequence $X \in \mathbb{R}^{N \times d}$.
- The model operates on the concatenated representation $[P; X] \in \mathbb{R}^{(p+N) \times d}$.

- The entire language model backbone θ_0 is frozen. Only the soft prompt matrix P is updated via gradient descent:

$$P^* = \arg\min_P \sum_{(X,Y)\in\mathcal{D}} \mathcal{L}(f_{\theta_0}([P;X]), Y). \tag{10.19}$$

Scale Convergence. Lester et al. [113] demonstrated an important scaling phenomenon: while prompt tuning underperforms full fine-tuning on small models ($\leq$ 1B parameters), its performance rapidly converges as model scale increases. At **11**B parameters (T5-XXL), prompt tuning **matches full fine-tuning performance** across diverse tasks while training less than 0.01% of model parameters.

10.3 Quantization and Numerical Precision

While PEFT slashes the computational footprint of adaptation, serving multi-billion parameter models in production remains bottlenecked by hardware memory and bandwidth. Model quantization addresses this challenge by compressing continuous representations into discrete low-bit integer grids.

10.3.1 Numerical Representations in Deep Learning

Standard deep learning frameworks historically represent weights and activations in single-precision floating-point format:

- **FP32 (IEEE 754 Single Precision)**: Consists of 1 sign bit (s), 8 exponent bits (e), and 23 mantissa/fraction bits (m), providing a dynamic range of approximately 1.18×10^{-38} to 3.4×10^{38}. Each parameter consumes 4 bytes.
- **FP16 (IEEE 754 Half Precision)**: Consists of 1 sign bit, 5 exponent bits, and 10 mantissa bits. With only 5 exponent bits, the positive normalized dynamic range is strictly bounded:

$$\text{Min}_{\text{normalized}} \approx 6.10 \times 10^{-5}, \quad \text{Max} = 65{,}504. \tag{10.20}$$

  This narrow dynamic range introduces extreme vulnerability to **underflow** (gradients smaller than 6.1×10^{-5} flush to zero) and **overflow** (activations exceeding 65,504 become NaN or ∞), requiring loss scaling tricks during training.
- **BF16 (Brain Floating Point 16)**: Engineered by Google Brain, BF16 reallocates bitwidths to feature 1 sign bit, **8 exponent bits** (matching FP32), and 7 mantissa bits. By preserving the 8-bit exponent, BF16 provides the identical dynamic range as FP32 ($\approx 10^{\pm 38}$), completely eliminating overflow/underflow hazards while reducing memory by 50% compared to FP32.
- **INT8 (8-bit Signed Integer)**: Represents discrete integers in the range $[-128, 127]$ (or symmetric range $[-127, 127]$). Each parameter consumes exactly 1 byte.

Reduced Floating-Point Precision Baseline. Before applying discrete quantization, converting a model from FP32 to half-precision (FP16 or BF16) via PyTorch (`model.half()` or `model.to(torch.bfloat16)`) immediately cuts the model's memory footprint by 50% with negligible degradation in generation fidelity. Because values remain on continuous real manifolds, floating-point down-casting is not technically quantization.

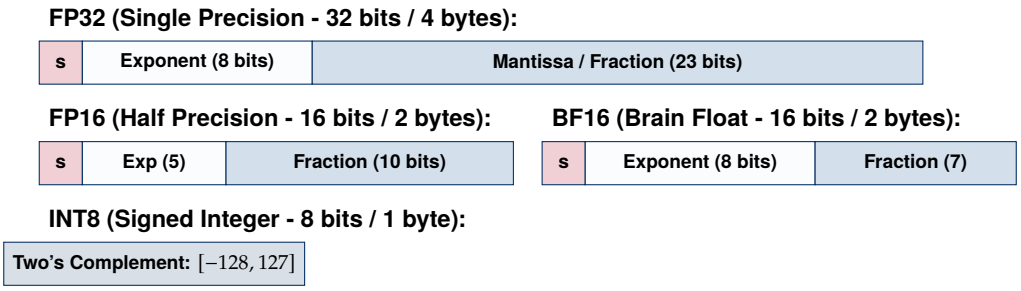


Figure 10.5: Bit-level memory allocation of standard deep learning numerical formats: FP32, FP16, BF16, and INT8.

10.3.2 Quantization Mechanics: Symmetric versus Asymmetric

Quantization is the mapping of continuous floating-point values $x \in [\alpha, \beta] \subset \mathbb{R}$ to a discrete grid of b-bit integer values $q \in [q_{\min}, q_{\max}] \subset \mathbb{Z}$. This mapping is governed by two fundamental parameters: the **scale factor** $S \in \mathbb{R}^+$ and the integer **zero-point** $Z \in \mathbb{Z}$.

1. Symmetric (Absmax) Quantization. In symmetric quantization, the continuous range is mirrored symmetrically around zero: $[-\gamma, \gamma]$, where $\gamma = \max(|x|)$. The continuous zero maps strictly to the integer zero ($Z = 0$).

For an 8-bit signed integer target with symmetric range $[-127, 127]$ (leaving -128 unused to preserve exact symmetry):

$$S = \frac{\max(|x|)}{127}, \tag{10.21}$$

$$q = \text{clamp}\left(\left\lfloor \frac{x}{S} \right\rceil, -127, 127\right), \tag{10.22}$$

$$\hat{x} = q \cdot S, \tag{10.23}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer, and $\text{clamp}(v, l, u) = \max(l, \min(v, u))$.

> **Intuition & analogy: Preserving Zero in Symmetric Quantization**
>
> In deep learning activations produced by ReLU or zero-padded tensors, exact zeros occur with high frequency. Symmetric quantization guarantees that $x = 0$ maps identically to $q = 0$ with zero quantization error. This simplifies hardware computation, as sparse zero-skipping logic in Tensor Cores remains fully functional without offset corrections.

2. Asymmetric (Zero-Point) Quantization. When tensor values are heavily skewed or non-negative (e.g., activations following a ReLU or GeLU function), symmetric quantization wastes nearly half of the representable integer range on empty negative regions. Asymmetric quantization maps the exact empirical minimum and maximum of the tensor $[\alpha, \beta] = [\min(x), \max(x)]$ across the full integer span $[q_{\min}, q_{\max}] = [-128, 127]$:

$$S = \frac{\max(x) - \min(x)}{q_{\max} - q_{\min}} = \frac{\beta - \alpha}{255}, \tag{10.24}$$

$$Z = \text{round}\left(\frac{-\alpha}{S}\right) + q_{\min} = \text{round}\left(\frac{-255 \cdot \alpha}{\beta - \alpha}\right) - 128, \tag{10.25}$$

$$q = \text{clamp}\left(\left\lfloor \frac{x}{S} \right\rceil + Z, -128, 127\right), \tag{10.26}$$

$$\hat{x} = S \cdot (q - Z). \tag{10.27}$$

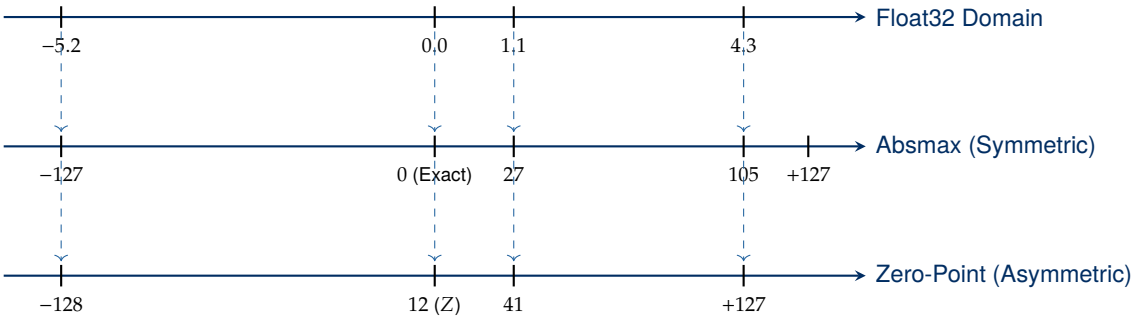


Figure 10.6: Visual comparison of Absmax (symmetric) versus Zero-Point (asymmetric) quantization mappings for the continuous vector $[-5.2, 4.3, 1.1, 0.0]$.

Complete Numerical Walkthrough (The Lecture Example). To ground these formulas, consider the exact numerical vector examined in the lecture slides:

$$X = [-5.2,\ 4.3,\ 1.1,\ 0.0]. \tag{10.28}$$

1. Absmax Quantization Walkthrough:

- Identify the maximum absolute magnitude: $\gamma = \max(|-5.2|, |4.3|, |1.1|, |0.0|) = 5.2$.
- Compute the scale factor S:

$$S = \frac{5.2}{127} \approx 0.04094488. \tag{10.29}$$

- Quantize each continuous coordinate $q_i = \text{round}\left(\frac{x_i}{S}\right)$:

$$q_1 = \text{round}\left(\frac{-5.2}{0.04094488}\right) = \text{round}(-127.0) = \mathbf{-127}, \tag{10.30}$$

$$q_2 = \text{round}\left(\frac{4.3}{0.04094488}\right) = \text{round}(105.019) = \mathbf{105}, \tag{10.31}$$

$$q_3 = \text{round}\left(\frac{1.1}{0.04094488}\right) = \text{round}(26.865) = \mathbf{27}, \tag{10.32}$$

$$q_4 = \text{round}\left(\frac{0.0}{0.04094488}\right) = \mathbf{0}. \tag{10.33}$$

- Dequantize to reconstruct floating-point approximations $\hat{x}_i = q_i \cdot S$:

$$\hat{x}_1 = -127 \times 0.04094488 = -5.200, \quad (\text{Error} = 0.000) \tag{10.34}$$

$$\hat{x}_2 = 105 \times 0.04094488 = 4.2992, \quad (\text{Error} = 0.0008) \tag{10.35}$$

$$\hat{x}_3 = 27 \times 0.04094488 = 1.1055, \quad (\text{Error} = 0.0055) \tag{10.36}$$

$$\hat{x}_4 = 0 \times 0.04094488 = 0.000, \quad (\text{Error} = 0.000). \tag{10.37}$$

2. Zero-Point Quantization Walkthrough:

- Identify domain bounds: $\alpha = \min(X) = -5.2$, $\beta = \max(X) = 4.3$.
- Compute asymmetric scale factor S:

$$S = \frac{4.3 - (-5.2)}{255} = \frac{9.5}{255} \approx 0.0372549. \tag{10.38}$$

- Compute integer zero-point Z:

$$Z = \text{round}\left(\frac{-(-5.2)}{0.0372549}\right) - 128 = \text{round}(139.579) - 128 = 140 - 128 = \mathbf{12}. \tag{10.39}$$

- Quantize each coordinate $q_i = \text{round}\left(\frac{x_i}{S}\right) + Z$:

$$q_1 = \text{round}\left(\frac{-5.2}{0.0372549}\right) + 12 = -140 + 12 = \mathbf{-128}, \tag{10.40}$$

$$q_2 = \text{round}\left(\frac{4.3}{0.0372549}\right) + 12 = 115 + 12 = \mathbf{127}, \tag{10.41}$$

$$q_3 = \text{round}\left(\frac{1.1}{0.0372549}\right) + 12 = 30 + 12 = \mathbf{42}, \tag{10.42}$$

$$q_4 = \text{round}\left(\frac{0.0}{0.0372549}\right) + 12 = 0 + 12 = \mathbf{12}. \tag{10.43}$$

- Dequantize to reconstruct approximations $\hat{x}_i = S \cdot (q_i - Z)$:

$$\hat{x}_1 = 0.0372549 \times (-128 - 12) = -5.2157 \approx -5.200, \tag{10.44}$$

$$\hat{x}_2 = 0.0372549 \times (127 - 12) = 4.2843 \approx 4.300, \tag{10.45}$$

$$\hat{x}_3 = 0.0372549 \times (42 - 12) = 1.1176, \quad \text{(Error} = 0.0176) \tag{10.46}$$

$$\hat{x}_4 = 0.0372549 \times (12 - 12) = 0.000. \tag{10.47}$$

10.3.3 Quantization Paradigms: PTQ and QAT

Depending on when quantization parameters are derived, quantization pipelines are categorized into Post-Training Quantization and Quantization-Aware Training:

Post-Training Quantization (PTQ). In PTQ, a model is trained to convergence in standard full precision (FP32 or BF16). Afterward, weights and activation tensors are mapped into low-bit integers without modifying the underlying weights through backpropagation. PTQ is computationally instantaneous and requires no training pipeline, but exhibits noticeable accuracy degradation when quantizing below 8 bits (e.g., 4-bit or 3-bit weights).

Quantization-Aware Training (QAT). QAT incorporates the distortion introduced by quantization directly into the training loop:

- **Forward Pass with Fake Quantization**: Model weights and activations are stored in full precision (FP32), but passed through a non-differentiable "fake quantization" operator $q_{\text{fake}}(x) = S \cdot \text{clamp}\left(\lfloor x/S \rceil, q_{\min}, q_{\max}\right)$. Computations simulate discrete clamping and rounding errors while preserving high-precision accumulators.
- **Backward Pass with Straight-Through Estimators (STE)**: Because the rounding function $\lfloor v \rceil$ has a derivative of zero almost everywhere and is undefined at half-integers, standard gradient backpropagation fails ($\frac{\partial \lfloor v \rceil}{\partial v} = 0$). QAT resolves this by deploying the **Straight-Through Estimator** [118]:

$$\frac{\partial q_{\text{fake}}(x)}{\partial x} \approx \begin{cases} 1 & \text{if } x \in [\alpha, \beta], \\ 0 & \text{otherwise.} \end{cases} \tag{10.48}$$

  Gradients pass straight through the rounding operator to update underlying floating-point weights, adapting the model to compensate for rounding noise.

10.3.4 Activation Quantization: Static versus Dynamic

Quantizing weights is straightforward because weight tensors are static post-training; their minimum and maximum values can be analyzed offline. In contrast, **activation tensors** depend dynamically on input tokens passed through the network during inference. Handling activation quantization involves two distinct strategies:

1. Static Quantization. In static quantization, activation scales S and zero-points Z are precomputed offline and frozen:

- A representative unlabeled calibration dataset (typically a few thousand sequences) is passed through the model.
- Observers record the empirical distribution, histograms, and dynamic ranges of activation tensors at each layer.
- Fixed values for S and Z are written into the model graph.
- **Trade-off**: Inference is extremely fast because scale constants are static, but if runtime inputs diverge from the calibration distribution, clipping and saturation degrade model performance.

2. Dynamic Quantization. In dynamic quantization, scale and zero-point parameters are computed dynamically on-the-fly for every individual activation tensor during the forward pass:

- As hidden state X_l arrives at layer l, the hardware immediately computes $\min(X_l)$ and $\max(X_l)$, derives S_{X_l}, quantizes $X_l \rightarrow Q_{X_l}$, performs integer matrix multiplication with quantized weights, and dequantizes the result.
- **Trade-off**: Dynamic quantization eliminates calibration sets and provides superior accuracy across diverse inputs, but calculating scale factors dynamically during inference introduces non-negligible computational overhead.

10.3.5 Outlier Features and LLM.int8()

When scaling Post-Training Quantization to multi-billion parameter foundation models, researchers observed a catastrophic phenomenon: while INT8 quantization works flawlessly on small models ($\leq$ 2.7B parameters), it causes **complete performance collapse** on models exceeding 6.7B parameters.

The Emergent Outlier Dimension Phenomenon. Dettmers et al. [119] discovered the root cause of this failure: as Transformer language models scale beyond 6.7B parameters, they exhibit the emergence of **extreme outlier features**. In approximately 0.1% of hidden state dimensions, activation values suddenly spike to magnitudes exceeding ±6.0, reaching up to ±100 (over 20× the standard deviation of remaining coordinates).

Crucially, these outliers are not random noise: they consistently concentrate in the identical coordinate channels across all sequence positions and coordinate dimensions, encoding essential cross-attention syntactic markers and core reasoning patterns.

Why Outliers Break Standard INT8 Quantization. If a tensor coordinate reaches $x_{\max} = 100$ while typical features hover in $[-1.0, 1.0]$, Absmax quantization sets:

$$S = \frac{100}{127} \approx 0.787. \tag{10.49}$$

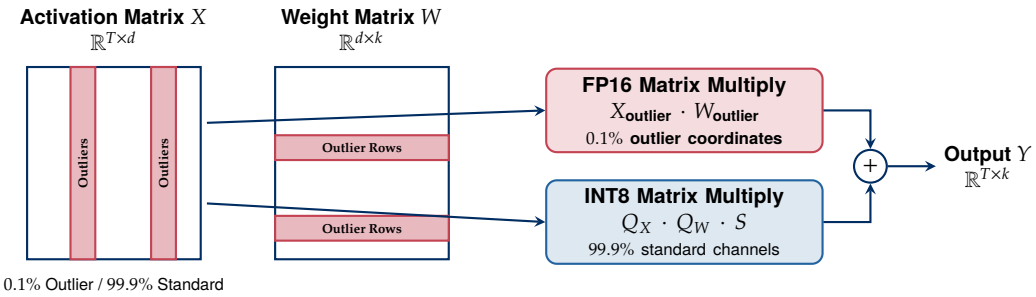


Figure 10.7: Mixed-precision decomposition in LLM.int8(). Activations and weights are decomposed into a 16-bit floating-point stream for the 0.1% outlier coordinate channels and an 8-bit integer stream for the remaining 99.9% standard channels.

When standard features $x \in [-1.0, 1.0]$ are divided by $S \approx 0.787$, they all collapse into integer bins $\{-1, 0, 1\}$. Truncating continuous variance across 99.9% of hidden features obliterates representational resolution, causing generation perplexity to diverge.

LLM.int8(): Mixed-Precision Matrix Decomposition. To solve this, Dettmers et al. [119] introduced two structural innovations:

1. **Vector-Wise Quantization**: Instead of computing a single global scale factor S for an entire activation matrix, scale constants are calculated per-row for activations and per-column for weights, isolating variance across tokens.

2. **Mixed-Precision Decomposition**: For any feature dimension where coordinate magnitude exceeds a threshold $\alpha = 6.0$, that entire channel is extracted into a sparse sub-matrix:

$$XW = \underbrace{X_{\text{outlier}} W_{\text{outlier}}}_{\text{16-bit Floating-Point (FP16)}} + \underbrace{S_X S_W \left(Q_{X_{\text{standard}}} \cdot Q_{W_{\text{standard}}}\right)}_{\text{8-bit Integer (INT8)}}. \tag{10.50}$$

By handling the 0.1% outlier coordinates in FP16 and executing the remaining 99.9% in INT8 via Tensor Cores, LLM.int8() recovers 100% **of full FP16 performance** on models up to 175B parameters while reducing GPU memory consumption by exactly half (2× memory reduction).

10.4 Knowledge Distillation and Serving Optimization

Beyond altering numerical representations of a single model, machine learning provides methods to train inherently smaller student architectures via knowledge distillation, and to eliminate hardware latency walls during autoregressive serving.

10.4.1 Teacher-Student Knowledge Distillation

Definition 10.4: *Knowledge Distillation*

Knowledge Distillation [120] is a compression technique wherein a compact "Student" model S_θ is trained under the supervision of a larger, highly parameterized "Teacher" model T_ϕ (or ensemble of teachers), transferring learned representational capacity without duplicating network depth or width.

Hard Labels versus Soft Targets ("Dark Knowledge"). In standard supervised training, models learn from one-hot target vectors where the ground truth class has probability 1.0 and all non-target classes have probability 0.0:

$$y = [0,\ 0,\ 1,\ 0,\ \ldots,\ 0]^T. \tag{10.51}$$

One-hot hard targets provide zero information regarding semantic relationships across non-target tokens. In contrast, the output probability distribution of a well-trained teacher model contains rich inter-class affinities:

$$p_T = [\text{dog} : 0.05,\ \text{kitten} : 0.25,\ \text{cat} : 0.60,\ \text{automobile} : 10^{-7}]. \tag{10.52}$$

Hinton et al. coined these non-target probabilities **"dark knowledge"**: they inform the student that a "cat" is semantically similar to a "kitten" and completely dissimilar to an "automobile", providing substantially denser gradient signals than binary ground-truth labels.

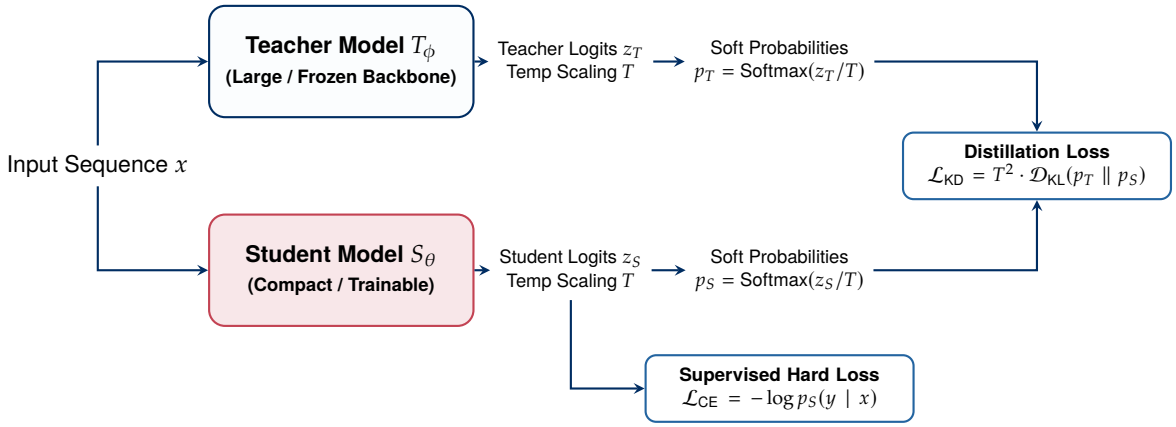


Figure 10.8: Knowledge Distillation framework. The student is trained simultaneously to minimize Kullback-Leibler divergence against the temperature-softened probabilities of the teacher and cross-entropy loss against hard ground-truth labels.

Temperature Scaling and Loss Formulation. Standard Softmax distributions on well-trained models are highly peaked, assigning probabilities near zero to all non-target classes. To reveal the dark knowledge in non-target logits, Hinton et al. introduced a **temperature hyperparameter** $T \geq 1$:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \tag{10.53}$$

As $T \to \infty$, the output distribution smooths out, exposing the fine-grained relative affinities across classes.

The complete training objective combines the distillation loss (Kullback-Leibler divergence between softened teacher and student distributions) with standard cross-entropy loss against ground truth labels:

$$\mathcal{L}_{\text{Total}} = (1-\alpha)\mathcal{L}_{\text{CE}}(y, \text{Softmax}(z_S)) + \alpha T^2 \mathcal{D}_{\text{KL}}\left(\text{Softmax}\left(\frac{z_T}{T}\right) \middle\| \text{Softmax}\left(\frac{z_S}{T}\right)\right). \tag{10.54}$$

Intuition & analogy: Why Multiply the KL Term by T^2?

When calculating gradients of the KL divergence with respect to the student logits z_S

under temperature T, applying the chain rule introduces a scaling factor of $\frac{1}{T^2}$ in the magnitude of the gradients:

$$\frac{\partial \mathcal{D}_{\mathrm{KL}}}{\partial z_{S,i}} \approx \frac{1}{T^2}(z_{S,i} - z_{T,i}) \quad \text{(for large } T\text{)}. \tag{10.55}$$

Multiplying the distillation loss by T^2 balances gradient magnitudes between the distillation objective and the standard cross-entropy objective, preventing the soft target signal from vanishing when high temperatures are selected.

10.4.2 Distillation in Large Language Models

In language modeling, distillation has yielded foundational compact architectures:

- **DistilBERT** [121]: Distills BERT-Base (12 layers) into a 6-layer student by initializing student weights from alternating layers of the teacher. Trained on a triple objective: distillation loss on softened logits, masked language modeling loss on hard tokens, and cosine embedding distance loss between hidden representations. DistilBERT is 40% **smaller**, 60% **faster**, and retains 97% **of BERT's language understanding performance**.
- **TinyBERT** [122]: Performs intermediate layer distillation, aligning attention matrices and hidden states layer-by-layer between teacher and student. A 4-layer TinyBERT achieves 7.5× **parameter reduction** and 9.4× **faster inference** while retaining 96.8% performance on GLUE.
- **MiniLM** [123]: Distills self-attention relation distributions (query-query, key-key, and value-value dot products) rather than raw hidden states, freeing the student architecture from matching the exact hidden dimensionality of the teacher.
- **Sequence-Level Distillation**: In generative foundation models (e.g., distilling frontier models like GPT-4 into open-weights models like Llama or Mistral), distillation often operates at the dataset level: the teacher generates complex step-by-step reasoning chains, synthetic instructions, or multi-turn dialogues, which the student ingests via supervised fine-tuning.

10.4.3 Inference Serving Dynamics and Memory Bottlenecks

Deploying compressed models in production requires understanding the physical hardware constraints of modern GPUs (e.g., NVIDIA H100/A100).

Arithmetic Intensity and the Memory Wall. The performance of any deep learning computation on hardware is governed by its **arithmetic intensity**:

$$\text{Arithmetic Intensity} = \frac{\text{Total Floating-Point Operations (FLOPs)}}{\text{Total Memory Transferred (Bytes)}}. \tag{10.56}$$

A workload is **compute-bound** if arithmetic intensity is high (the hardware tensor cores operate at peak capacity). It is **memory-bandwidth bound** if arithmetic intensity is low (the tensor cores spend most cycles idling while waiting for weights and activations to transfer from High-Bandwidth Memory / HBM across the memory bus into on-chip SRAM).

The Two Phases of LLM Inference. Autoregressive generation naturally divides into two fundamentally distinct operating regimes:

1. **Prefill Phase (Prompt Ingestion)**: The model ingests all N tokens of the input prompt simultaneously. Self-attention executes full matrix-matrix multiplications (QK^T), which has high arithmetic intensity ($O(N)$ FLOPs per byte transferred). The prefill phase is **compute-bound**.

2. **Decoding Phase (Autoregressive Generation)**: To generate token $t+1$, the model passes only a single query token vector $q_t \in \mathbb{R}^{1\times d}$ through the network. Multiplying a single vector against large weight matrices $W \in \mathbb{R}^{d\times d}$ yields an arithmetic intensity of roughly $\frac{2d^2}{2d^2} \approx 1$ FLOP/byte. The decoding phase is severely **memory-bandwidth bound**.

The Key-Value (KV) Cache. Because generating token $t+1$ depends causally on all preceding tokens $1, \ldots, t$, naive generation would require recomputing attention keys K and values V for all past tokens at every forward step, scaling computational complexity quadratically $O(T^2)$.

To eliminate redundant computation, inference engines store past key and value vectors in GPU VRAM, maintaining the **Key-Value (KV) Cache**. At step t, the network computes keys and values only for the newly generated token, concatenating them to the cache.

Theorem 10.1: *KV Cache Memory Footprint*

For a Transformer model with L layers, h_{KV} key-value attention heads, per-head dimension d_k, context window length T, batch size B, and numerical precision b bytes per parameter, the GPU memory consumed exclusively by the KV cache is:

$$\text{Memory}_{\text{KV}} = 2 \cdot b \cdot B \cdot L \cdot h_{\text{KV}} \cdot d_k \cdot T \quad \text{bytes}, \tag{10.57}$$

where the leading factor of 2 accounts for storing both key and value tensors.

Worked Sizing Example. Consider serving a LLaMA-2 70B model ($L = 80$, $h = 64$ attention heads, $d_k = 128$) at half precision ($b = 2$ bytes for FP16):

- For a single sequence ($B = 1$) at context length $T = 4096$:

$$\text{Memory}_{\text{KV}} = 2 \times 2 \times 1 \times 80 \times 64 \times 128 \times 4096 = 10{,}737{,}418{,}240 \text{ bytes} \approx 10.74 \text{ GB}. \tag{10.58}$$

- For a concurrent production batch of $B = 32$ requests at full context length $T = 4096$:

$$\text{Memory}_{\text{KV}}^{\text{Batch}} = 32 \times 10.74 \text{ GB} \approx \mathbf{343.68} \text{ GB of VRAM}. \tag{10.59}$$

At scale, the dynamic KV cache rapidly outgrows the static weight memory of the model itself. This memory wall directly motivates architectural innovations studied throughout modern systems engineering:

- **Grouped-Query Attention (GQA)**: Sharing single key-value heads across multiple query heads ($h_{\text{KV}} \ll h_Q$), reducing KV cache memory by 8×.
- **KV Cache Quantization**: Storing cached keys and values in 8-bit or 4-bit integers ($b = 1$ or $b = 0.5$), cutting serving footprint in half.
- **PagedAttention and vLLM**: Managing KV cache memory via virtual memory paging to eliminate external fragmentation.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[5] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[6] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[8] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. URL https://www.bishopbook.com/.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

[10] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL https://udlbook.github.io/udlbook/.

[11] Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2924–2932, 2014. URL https://proceedings.neurips.cc/paper/2014/hash/1a5b1e4daae265b790965a275b53ae50-Abstract.html.

[12] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

[13] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[14] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[15] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[17] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

[18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[19] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

[20] Paul J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[21] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[22] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

[23] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[24] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[25] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3104–3112, 2014.

[26] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.

[27] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[28] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[29] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall / Online Draft, 3rd draft edition, 2024. URL https://web.stanford.edu/~jurafsky/slp3/.

[30] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[31] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[32] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR) Workshop Track*, 2013.

[33] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[34] Zellig S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[35] John R. Firth. A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, pages 1–32, 1957.

[36] Grant Sanderson. Visualizing attention, embeddings and neural network trajectories. 3Blue1Brown Educational Video Series, 2024. URL https://www.3blue1brown.com/lessons/gpt.

[37] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, pages 246–252, 2005.

[38] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[39] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146, 2017.

[40] Enkk. Cosa sono gli embeddings: Geometria semantica, distanza coseno e vector databases. YouTube Educational Lecture, 2023. URL https://www.youtube.com/watch?v=5sYkC8aVv1k.

[41] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[43] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[44] Andrej Karpathy. Building makemore part 2: MLP language model. YouTube Video Tutorial and GitHub Repository, 2022. URL https://github.com/karpathy/makemore.

[45] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[46] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[47] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[48] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[49] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 464–468, 2018. URL https://arxiv.org/abs/1803.02155.

[50] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. URL https://arxiv.org/abs/2104.09864.

[51] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2108.12409.

[52] Grant Sanderson. Attention in transformers, visually explained. 3Blue1Brown Video Series, 2024. URL https://www.youtube.com/watch?v=eMlx5fFNoYc.

[53] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Geva. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2021.

[54] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[55] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[56] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[57] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL https://jmlr.org/papers/v21/19-460.html.

[58] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, 2019. URL https://arxiv.org/abs/1906.04341.

[59] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 14014–14024, 2019. URL https://arxiv.org/abs/1905.10650.

[60] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09751.

[61] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 889–898, 2018. URL https://arxiv.org/abs/1805.04833.

[62] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018. URL https://openai.com/research/language-unsupervised.

[63] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[64] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2021. URL https://arxiv.org/abs/2112.11446.

[65] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, Yuxiong He, et al. Using DeepSpeed and Megatron to train Megatron-Turing NLG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022. URL https://arxiv.org/abs/2201.11990.

[66] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with Pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. URL https://jmlr.org/papers/v24/22-1144.html.

[67] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. URL https://arxiv.org/abs/2307.09288.

[68] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. URL https://aclanthology.org/P02-1040/.

[69] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pages 74–81, 2004. URL https://aclanthology.org/W04-1013/.

[70] Satanjeev Banerjee and Alon Lavie. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005. URL https://aclanthology.org/W05-0909/.

[71] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://openreview.net/forum?id=SkeHuCVFDr.

[72] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on*

Empirical Methods in Natural Language Processing (EMNLP), pages 2383–2392, 2016. URL https://aclanthology.org/D16-1264/.

[73] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

[74] Lianmin Zheng, Wei-Lin Chiang, Hao Zhang, Siyuan Zhuang, Minmin Chen, Sheng Shen, Stephanie Shen, Zheyuan Li, Zi Lin, Yonghao Zhuang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46595–46623, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b3e83161a0524458d92976-Abstract-Datasets_and_Benchmarks.html.

[75] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics (TACL)*, 7:625–641, 2019. URL https://aclanthology.org/Q19-1040/.

[76] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642, 2013. URL https://aclanthology.org/D13-1170/.

[77] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (ACL-HLT)*, pages 142–150, 2011. URL https://aclanthology.org/P11-1015/.

[78] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1525–1534, 2016. URL https://aclanthology.org/P16-1144/.

[79] Nasrin Mostafazadeh, Nathanael Chambers, Xiaodong He, Devi Parikh, Dhruv Batra, Lucy Vanderwende, Pushmeet Kohli, and James Allen. A corpus and cloze evaluation for deeper understanding of commonsense stories. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 839–849, 2016. URL https://aclanthology.org/N16-1098/.

[80] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800, 2019. URL https://aclanthology.org/P19-1472/.

[81] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 784–789, 2018. URL https://aclanthology.org/P18-2124/.

[82] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611, 2017. URL https://aclanthology.org/P17-1147/.

[83] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics (TACL)*, 7: 452–466, 2019. URL https://aclanthology.org/Q19-1026/.

[84] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on Freebase from question-answer pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1533–1544, 2013. URL https://aclanthology.org/D13-1160/.

[85] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 632–642, 2015. URL https://aclanthology.org/D15-1075/.

[86] Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1112–1122, 2018. URL https://aclanthology.org/N18-1101/.

[87] Hector J. Levesque, Ernest Davis, and Leora Morgenstern. The Winograd schema challenge. In *Proceedings of the Thirteenth International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 552–561, 2012. URL https://cdn.aaai.org/ocs/4492/4492-21843-1-PB.pdf.

[88] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WINOGRANDE: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (AAAI)*, volume 34, pages 8732–8740, 2020. URL https://ojs.aaai.org/index.php/AAAI/article/view/6399.

[89] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI Spring Symposium: Logical Formalizations of Commonsense Reasoning*, pages 90–95, 2011. URL https://cdn.aaai.org/ocs/2418/2418-10878-1-PB.pdf.

[90] Maarten Sap, Ronan Le Bras, Emily Allaway, Chandra Bhagavatula, Nicholas Lourie, Hannah Rashkin, Vered Shwartz, Noah A. Smith, and Yejin Choi. ATOMIC: An atlas of machine commonsense for if-then reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 33, pages 3027–3035, 2019. URL https://doi.org/10.1609/aaai.v33i01.33013027.

[91] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[92] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, volume 1, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract.html.

[93] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the Workshop: Analyzing and Interpreting Neural Networks for NLP (BlackboxNLP)*, pages 353–355, 2018. URL https://aclanthology.org/W18-5446/.

[94] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/4496bf24afe7fab6f046bf4923da8de6-Abstract.html.

[95] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

[96] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Rachel Rudinger, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10776–10787, 2023. URL https://aclanthology.org/2023.findings-emnlp.722/.

[97] Alice Xiao and Kevin Zhu. *Foundations of Large Language Models*. 2025.

[98] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[99] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.

[100] Stephen H. Bach, Victor Sanh, Zheng-Xin Yong, Albert Webson, Colin Raffel, Nihal V. Nayak, Abheesht Sharma, Taewoon Kim, M Saiful Bari, Thibault Fevry, et al. Promptsource: An integrated development environment and repository for natural language prompts. *arXiv preprint arXiv:2202.01279*, 2022.

[101] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 13484–13508, 2023.

[102] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[103] Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations (ICLR)*, 2022.

[104] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022.

[105] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 3008–3021, 2020.

[106] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[107] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[108] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 53728–53741, 2023.

[109] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023.

[110] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:2106.10199*, 2021.

[111] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning (ICML)*, pages 2790–2799, 2019.

[112] Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. Adapterhub: A framework for adapting transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 46–54, 2020.

[113] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059, 2021.

[114] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[115] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *arXiv preprint arXiv:2012.13255*, 2020.

[116] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*, 2023.

[117] Liyuan Chen, Zihao Ye, Yonghao Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant lora serving. *arXiv preprint arXiv:2310.18547*, 2023.

[118] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[119] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:30318–30332, 2022.

[120] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[121] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[122] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, 2020.

[123] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5776–5788, 2020.